MDN JavaScript Documentation

Volume 7 of 9 — Built-in objects: RangeError – SyntaxError

Unofficial print compilation. This volume contains 160 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

RangeError 5
RangeError() constructor 7
ReferenceError 10
ReferenceError() constructor 11
Reflect 14
Reflect.apply() 17
Reflect.construct() 19
Reflect.defineProperty() 24
Reflect.deleteProperty() 26
Reflect.get() 29
Reflect.getOwnPropertyDescriptor() 31
Reflect.getPrototypeOf() 33
Reflect.has() 35
Reflect.isExtensible() 37
Reflect.ownKeys() 40
Reflect.preventExtensions() 42
Reflect.set() 44
Reflect.setPrototypeOf() 46
RegExp 49
RegExp() constructor 57
RegExp.escape() 60
RegExp.$1, …, RegExp.$9 63
RegExp.input ($_) 64
RegExp.lastMatch ($&) 65
RegExp.lastParen ($+) 67
RegExp.leftContext ($`) 68
RegExp.rightContext ($') 69
RegExp[Symbol.species] 71
RegExp.prototype.compile() 73
RegExp.prototype.exec() 75
RegExp.prototype.test() 78
RegExp.prototype.toString() 81
RegExp.prototype[Symbol.match]() 83
RegExp.prototype[Symbol.matchAll]() 87
RegExp.prototype[Symbol.replace]() 90
RegExp.prototype[Symbol.search]() 93
RegExp.prototype[Symbol.split]() 96
RegExp.prototype.dotAll 99
RegExp.prototype.flags 101
RegExp.prototype.global 102
RegExp.prototype.hasIndices 104
RegExp.prototype.ignoreCase 106
RegExp.prototype.multiline 108
RegExp.prototype.source 109
RegExp.prototype.sticky 111
RegExp.prototype.unicode 114
RegExp.prototype.unicodeSets 116
RegExp: lastIndex 118
Set 123
Set() constructor 132
Set[Symbol.species] 134
Set.prototype.add() 136
Set.prototype.clear() 137
Set.prototype.delete() 139
Set.prototype.difference() 141
Set.prototype.entries() 143
Set.prototype.forEach() 144
Set.prototype.has() 147
Set.prototype.intersection() 148
Set.prototype.isDisjointFrom() 151
Set.prototype.isSubsetOf() 153
Set.prototype.isSupersetOf() 155
Set.prototype.keys() 157
Set.prototype.symmetricDifference() 158
Set.prototype.union() 160
Set.prototype.values() 162
Set.prototype[Symbol.iterator]() 164
Set.prototype.size 166
SharedArrayBuffer 168
SharedArrayBuffer() constructor 172
SharedArrayBuffer[Symbol.species] 174
SharedArrayBuffer.prototype.grow() 176
SharedArrayBuffer.prototype.slice() 177
SharedArrayBuffer.prototype.byteLength 179
SharedArrayBuffer.prototype.growable 180
SharedArrayBuffer.prototype.maxByteLength 181
String 183
String() constructor 193
String.fromCharCode() 195
String.fromCodePoint() 197
String.raw() 199
String.prototype.anchor() 204
String.prototype.at() 205
String.prototype.big() 208
String.prototype.blink() 209
String.prototype.bold() 211
String.prototype.charAt() 212
String.prototype.charCodeAt() 215
String.prototype.codePointAt() 217
String.prototype.concat() 220
String.prototype.endsWith() 222
String.prototype.fixed() 224
String.prototype.fontcolor() 225
String.prototype.fontsize() 227
String.prototype.includes() 229
String.prototype.indexOf() 231
String.prototype.isWellFormed() 234
String.prototype.italics() 236
String.prototype.lastIndexOf() 238
String.prototype.link() 240
String.prototype.localeCompare() 242
String.prototype.match() 245
String.prototype.matchAll() 249
String.prototype.normalize() 253
String.prototype.padEnd() 257
String.prototype.padStart() 259
String.prototype.repeat() 261
String.prototype.replace() 262
String.prototype.replaceAll() 268
String.prototype.search() 271
String.prototype.slice() 274
String.prototype.small() 276
String.prototype.split() 278
String.prototype.startsWith() 284
String.prototype.strike() 286

String.prototype.sub() 288
String.prototype.substr() 289
String.prototype.substring() 292
String.prototype.sup() 295
String.prototype.toLocaleLowerCase() 297
String.prototype.toLocaleUpperCase() 299
String.prototype.toLowerCase() 301
String.prototype.toString() 302
String.prototype.toUpperCase() 304
String.prototype.toWellFormed() 306
String.prototype.trim() 308
String.prototype.trimEnd() 309
String.prototype.trimStart() 311
String.prototype.valueOf() 313
String.prototype[Symbol.iterator]() 314
String: length 316

SuppressedError 320
SuppressedError() constructor 322
SuppressedError: error 324
SuppressedError: suppressed 325

Symbol 327
Symbol() constructor 333
Symbol.for() 335
Symbol.keyFor() 337
Symbol.asyncDispose 338
Symbol.asyncIterator 340
Symbol.dispose 342
Symbol.hasInstance 344
Symbol.isConcatSpreadable 347
Symbol.iterator 349
Symbol.match 352
Symbol.matchAll 354
Symbol.replace 355
Symbol.search 357
Symbol.species 359
Symbol.split 361
Symbol.toPrimitive 363
Symbol.toStringTag 365
Symbol.unscopables 367
Symbol.prototype.toString() 372
Symbol.prototype.valueOf() 374
Symbol.prototype[Symbol.toPrimitive]() 375
Symbol.prototype.description 376

SyntaxError 379
SyntaxError() constructor 380

JavaScript › Reference › Standard built-in objects

RangeError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RangeError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`RangeError`** object indicates an error when a value is not in the set or range of allowed values.

Description

A `RangeError` is thrown when trying to pass a value as an argument to a function that does not allow a range that includes the value.

This can be encountered when:

- passing a value that is not one of the allowed string values to `String.prototype.normalize()`, or
- when attempting to create an array of an illegal length with the `Array` constructor, or
- when passing bad values to the numeric methods `Number.prototype.toExponential()`, `Number.prototype.toFixed()` or `Number.prototype.toPrecision()`.

`RangeError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`RangeError` is a subclass of `Error`.

Constructor

`RangeError()`
: Creates a new `RangeError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `RangeError.prototype` and shared by all `RangeError` instances.

`RangeError.prototype.constructor`
: The constructor function that created the instance object. For `RangeError` instances, the initial value is the `RangeError` constructor.

`RangeError.prototype.name`
: Represents the name for the type of error. For `RangeError.prototype.name`, the initial value is `"RangeError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Using RangeError (for numeric values)

```js
function check(n) {
  if (!(n >= -500 && n <= 500)) {
    throw new RangeError("The argument must be between -500 and 500.");
  }
}

try {
  check(2000);
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

Using RangeError (for non-numeric values)

```js
function check(value) {
  if (!["apple", "banana", "carrot"].includes(value)) {
    throw new RangeError(
      'The argument must be an "apple", "banana", or "carrot".',
    );
  }
}

try {
  check("cabbage");
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-native-error-types-used-in-this-standard-rangeerror

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
RangeError	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
RangeError() constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
RangeError is serializable	77	79	103[1,2,3]	64	**No**	77	103[1,2,3]	55	**No**	12.0	77	**No**	?	**No**	**No**

1. Version 103 serializable properties: `name`, `message`, `cause`, `fileName`, `lineNumber` and `columnNumber`.

2. Version 104 also serializes `stack` in the main thread (`window.postMessage()` and `structuredClone()`).
3. Version 110 also serializes `stack` in workers (`worker.postMessage()` and `structuredClone()`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RangeError`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Error`
- `Array`
- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`
- `String.prototype.normalize()`

RangeError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RangeError/RangeError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`RangeError()`** constructor creates `RangeError` objects.

Syntax

```
new RangeError()
new RangeError(message)
new RangeError(message, options)
new RangeError(message, fileName)
new RangeError(message, fileName, lineNumber)

RangeError()
RangeError(message)
RangeError(message, options)
RangeError(message, fileName)
RangeError(message, fileName, lineNumber)
```

> **Note:** `RangeError()` can be called with or without `new`. Both create a new `RangeError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The name of the file containing the code that caused the exception

`lineNumber` Optional Non-standard

The line number of the code that caused the exception

Examples

Using RangeError (for numeric values)

```js
function check(n) {
  if (!(n >= -500 && n <= 500)) {
    throw new RangeError("The argument must be between -500 and 500.");
  }
}

try {
  check(2000);
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

Using RangeError (for non-numeric values)

```js
function check(value) {
  if (!["apple", "banana", "carrot"].includes(value)) {
    throw new RangeError(
      'The argument must be an "apple", "banana", or "carrot".',
    );
  }
}

try {
  check("cabbage");
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-nativeerror-constructors

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RangeError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RangeError.RangeError`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`
- `Array`
- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`
- `String.prototype.normalize()`

JavaScript › Reference › Standard built-in objects

ReferenceError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ReferenceError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`ReferenceError`** object represents an error when a variable that doesn't exist (or hasn't yet been initialized) in the current scope is referenced.

`ReferenceError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`ReferenceError` is a subclass of `Error`.

Constructor

`ReferenceError()`

Creates a new `ReferenceError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `ReferenceError.prototype` and shared by all `ReferenceError` instances.

`ReferenceError.prototype.constructor`

The constructor function that created the instance object. For `ReferenceError` instances, the initial value is the `ReferenceError` constructor.

`ReferenceError.prototype.name`

Represents the name for the type of error. For `ReferenceError.prototype.name`, the initial value is `"ReferenceError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a ReferenceError

js

```
try {
  let a = undefinedVariable;
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "undefinedVariable is not defined"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

Creating a ReferenceError

```js
try {
  throw new ReferenceError("Hello");
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-native-error-types-used-in-this-standard-referenceerror

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ReferenceError`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`ReferenceError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`ReferenceError` is serializable	77	79	103[1,2,3]	64	**No**	77	103[1,2,3]	55	**No**	12.0	77	**No**	?	**No**	**No**

1. Version 103 serializable properties: `name`, `message`, `cause`, `fileName`, `lineNumber` and `columnNumber`.
2. Version 104 also serializes `stack` in the main thread (`window.postMessage()` and `structuredClone()`).
3. Version 110 also serializes `stack` in workers (`worker.postMessage()` and `structuredClone()`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ReferenceError`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Error`

JavaScript › Reference › Standard built-in objects › ReferenceError

ReferenceError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ReferenceError/ReferenceError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`ReferenceError()`** constructor creates `ReferenceError` objects.

Syntax

```js
new ReferenceError()
new ReferenceError(message)
new ReferenceError(message, options)
new ReferenceError(message, fileName)
new ReferenceError(message, fileName, lineNumber)

ReferenceError()
ReferenceError(message)
ReferenceError(message, options)
ReferenceError(message, fileName)
ReferenceError(message, fileName, lineNumber)
```

> **Note:** `ReferenceError()` can be called with or without `new`. Both create a new `ReferenceError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The name of the file containing the code that caused the exception.

`lineNumber` Optional Non-standard

The line number of the code that caused the exception

Examples

Catching a ReferenceError

```js
try {
  let a = undefinedVariable;
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "undefinedVariable is not defined"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

Creating a ReferenceError

```js
try {
  throw new ReferenceError("Hello");
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-nativeerror-constructors

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ReferenceError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ReferenceError.ReferenceError`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`

JavaScript › Reference › Standard built-in objects

Reflect

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect`** namespace object contains static methods for invoking interceptable JavaScript object internal methods. The methods are the same as those of proxy handlers.

Description

Unlike most global objects, `Reflect` is not a constructor. You cannot use it with the `new` operator or invoke the `Reflect` object as a function. All properties and methods of `Reflect` are static (just like the `Math` object).

The `Reflect` object provides a collection of static functions which have the same names as the proxy handler methods.

The major use case of `Reflect` is to provide default forwarding behavior in `Proxy` handler traps. A trap is used to intercept an operation on an object — it provides a custom implementation for an object internal method. The `Reflect` API is used to invoke the corresponding internal method. For example, the code below creates a proxy `p` with a `deleteProperty` trap that intercepts the `[[Delete]]` internal method. `Reflect.deleteProperty()` is used to invoke the default `[[Delete]]` behavior on `targetObject` directly. You can replace it with `delete`, but using `Reflect` saves you from having to remember the syntax that each internal method corresponds to.

```js
const p = new Proxy(
  {},
  {
    deleteProperty(targetObject, property) {
      // Custom functionality: log the deletion
      console.log("Deleting property:", property);

      // Execute the default introspection behavior
      return Reflect.deleteProperty(targetObject, property);
    },
  },
);
```

The `Reflect` methods also allow finer control of how the internal method is invoked. For example, `Reflect.construct()` is the only way to construct a target function with a specific `new.target` value. If you use the `new` operator to invoke a function, the `new.target` value is always the function itself. This has important effects with subclassing. For another example, `Reflect.get()` allows you to run a getter with a custom `this` value, while property accessors always use the current object as the `this` value.

Nearly every `Reflect` method's behavior can be done with some other syntax or method. Some of these methods have corresponding static methods of the same name on `Object`, although they do have some subtle differences. For the exact differences, see the description for each `Reflect` method.

Static properties

`Reflect[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Reflect"`. This property is used in `Object.prototype.toString()`.

Static methods

`Reflect.apply()`

Calls a `target` function with arguments as specified by the `argumentsList` parameter. See also `Function.prototype.apply()`.

`Reflect.construct()`

The `new` operator as a function. Equivalent to calling `new target(...argumentsList)`. Also provides the option to specify a different prototype.

`Reflect.defineProperty()`

Similar to `Object.defineProperty()`. Returns a boolean that is `true` if the property was successfully defined.

`Reflect.deleteProperty()`

The `delete` operator as a function. Equivalent to calling `delete target[propertyKey]`.

`Reflect.get()`

Returns the value of the property. Works like getting a property from an object (`target[propertyKey]`) as a function.

`Reflect.getOwnPropertyDescriptor()`

Similar to `Object.getOwnPropertyDescriptor()`. Returns a property descriptor of the given property if it exists on the object, `undefined` otherwise.

`Reflect.getPrototypeOf()`

Same as `Object.getPrototypeOf()`.

`Reflect.has()`

Returns a boolean indicating whether the target has the property. Either as own or inherited. Works like the `in` operator as a function.

`Reflect.isExtensible()`

Same as `Object.isExtensible()`. Returns a boolean that is `true` if the target is extensible.

`Reflect.ownKeys()`

Returns an array of the target object's own (not inherited) property keys.

`Reflect.preventExtensions()`

Similar to `Object.preventExtensions()`. Returns a boolean that is `true` if the update was successful.

`Reflect.set()`

A function that assigns values to properties. Returns a boolean that is `true` if the update was successful.

`Reflect.setPrototypeOf()`

A function that sets the prototype of an object. Returns a boolean that is `true` if the update was successful.

Examples

Detecting whether an object contains certain properties

```js
const duck = {
  name: "Maurice",
  color: "white",
  greeting() {
    console.log(`Quaaaack! My name is ${this.name}`);
  },
};

Reflect.has(duck, "color");
// true
Reflect.has(duck, "haircut");
// false
```

Returning the object's own keys

```js
Reflect.ownKeys(duck);
// [ "name", "color", "greeting" ]
```

Adding a new property to the object

```js
Reflect.set(duck, "eyes", "black");
// returns "true" if successful
// "duck" now contains the property "eyes: 'black'"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect-object

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Reflect	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
apply	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
construct	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
defineProperty	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
deleteProperty	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
get	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
getOwnPropertyDescriptor	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
getPrototypeOf	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
has	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
isExtensible	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ownKeys`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`preventExtensions`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`set`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`setPrototypeOf`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Proxy

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.apply()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/apply · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.apply()`** static method calls a target function with arguments as specified.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Reflect.apply()". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(Reflect.apply(Math.floor, undefined, [1.75]));
// Expected output: 1

console.log(
  Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]),
);
// Expected output: "hello"

console.log(
  Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index,
);
// Expected output: 4

console.log(Reflect.apply("".charAt, "ponies", [3]));
// Expected output: "i"
```

Syntax

js

```
Reflect.apply(target, thisArgument, argumentsList)
```

Parameters

`target`

The target function to call.

`thisArgument`

The value of `this` provided for the call to `target`.

`argumentsList`

An array-like object specifying the arguments with which `target` should be called.

Return value

The result of calling the given `target` function with the specified `this` value and arguments.

Exceptions

`TypeError`

Thrown if `target` is not a function or `argumentsList` is not an object.

Description

`Reflect.apply()` provides the reflective semantic of a function call. That is, `Reflect.apply(target, thisArgument, argumentsList)` is semantically equivalent to:

```js
Math.floor.apply(null, [1.75]);
Reflect.apply(Math.floor, null, [1.75]);
```

The only differences are:

- `Reflect.apply()` takes the function to call as the `target` parameter instead of the `this` context.
- `Reflect.apply()` throws if `argumentsList` is omitted instead of defaulting to calling with no parameters.

`Reflect.apply()` invokes the `[[Call]]` object internal method of `target`.

Examples

Using Reflect.apply()

```js
Reflect.apply(Math.floor, undefined, [1.75]);
// 1;

Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]);
// "hello"

Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index;
// 4

Reflect.apply("".charAt, "ponies", [3]);
// "i"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.apply`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`apply`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.apply`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.apply` in `core-js`
- es-shims polyfill of `Reflect.apply`
- `Reflect`
- `Function.prototype.apply()`
- `handler.apply()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.construct()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/construct · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.construct()`** static method is like the `new` operator, but as a function. It is equivalent to calling `new target(...args)`. It additionally allows you to specify a different `new.target` value.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.construct()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function func1(a, b, c) {
  this.sum = a + b + c;
}

const args = [1, 2, 3];
const object1 = new func1(...args);
const object2 = Reflect.construct(func1, args);

console.log(object2.sum);
// Expected output: 6

console.log(object1.sum);
// Expected output: 6
```

Syntax

js

```
Reflect.construct(target, argumentsList)
Reflect.construct(target, argumentsList, newTarget)
```

Parameters

`target`

The target function to call.

`argumentsList`

An array-like object specifying the arguments with which `target` should be called.

`newTarget` Optional

The value of the `new.target` expression inside `target`. Defaults to `target`. Generally (see example), `target` specifies the *logic* to initialize the object, while `newTarget.prototype` specifies the *prototype* of the constructed object.

Return value

A new instance of `target` (or `newTarget`, if present), initialized by `target` as a constructor with the given `argumentsList`.

Exceptions

`TypeError`

Thrown if `target` or `newTarget` is not a constructor, or if `argumentsList` is not an object.

Description

`Reflect.construct()` provides the reflective semantic of a constructor call. That is, `Reflect.construct(target, argumentsList, newTarget)` is semantically equivalent to:

js

```
new target(...argumentsList);
```

Note that when using the `new` operator, `target` and `newTarget` are always the same constructor — but `Reflect.construct()` allows you to pass a different `new.target` value. Conceptually, `newTarget` is the function on which `new` was called, and `newTarget.prototype` will become the constructed object's prototype, while `target` is the constructor that is actually executed to initialize the object. For example, `new.target` may also be different from the currently executed constructor in class inheritance.

```js
class A {
  constructor() {
    console.log(new.target.name);
  }
}
class B extends A {}

new B(); // "B"
```

`Reflect.construct()` allows you to invoke a constructor with a variable number of arguments. (This is also possible with the spread syntax in a normal constructor call.)

```js
const obj = new Foo(...args);
const obj = Reflect.construct(Foo, args);
```

`Reflect.construct()` invokes the `[[Construct]]` object internal method of `target`.

Examples

Using Reflect.construct()

```js
const d = Reflect.construct(Date, [1776, 6, 4]);
d instanceof Date; // true
d.getFullYear(); // 1776
```

Changing new.target

If `newTarget` is passed, it changes the value of `new.target` inside the constructor. The constructed object will be an instance of `newTarget`, not `target`.

```js
function OneClass() {
  console.log("OneClass executed");
  console.log(`new.target is ${new.target.name}`);
}

function OtherClass() {
  console.log("OtherClass executed");
  console.log(`new.target is ${new.target.name}`);
}

const obj1 = Reflect.construct(OneClass, []);
// Logs:
// OneClass executed
// new.target is OneClass
console.log(obj1 instanceof OneClass); // true
```

(code continues on the next page)

(code continued from the previous page)

```
const obj2 = Reflect.construct(OneClass, [], OtherClass);
// Logs:
// OneClass executed
// new.target is OtherClass
console.log(obj2 instanceof OtherClass); // true
console.log(obj2 instanceof OneClass); // false
```

Of course, there's no strong guarantee about the prototype chain of the constructed object, as it depends on the constructor's implementation. For example, if the `target` constructor returns an object, then that object will be the constructed object, regardless of the `newTarget` value. If `target` is a proxy with a `construct` trap, then the trap fully controls the construction process.

js

```
function OneClass() {
  return { name: "one" };
}

function OtherClass() {
  return { name: "other" };
}

const obj1 = Reflect.construct(OneClass, [], OtherClass);
console.log(obj1.name); // 'one'
console.log(obj1 instanceof OneClass); // false
console.log(obj1 instanceof OtherClass); // false
```

A valid `new.target` should be a constructor function with a `prototype` property, but the latter is not enforced. If the `prototype` property's value is not an object, the initialized object will inherit from `Object.prototype`.

js

```
function OneClass() {
  console.log("OneClass executed");
  console.log(`new.target is ${new.target.name}`);
}

function OtherClass() {
  console.log("OtherClass executed");
  console.log(`new.target is ${new.target.name}`);
}

OtherClass.prototype = null;

const obj = Reflect.construct(OneClass, [], OtherClass);
// Logs:
// OneClass executed
// new.target is OtherClass
console.log(Object.getPrototypeOf(obj) === Object.prototype); // true
```

Reflect.construct() vs. Object.create()

Prior to the introduction of `Reflect`, objects could be constructed using an arbitrary combination of constructors and prototypes using `Object.create()`.

```js
function OneClass() {
  this.name = "one";
}

function OtherClass() {
  this.name = "other";
}

const args = [];
const obj1 = Reflect.construct(OneClass, args, OtherClass);
const obj2 = Object.create(OtherClass.prototype);
OneClass.apply(obj2, args);

console.log(obj1.name); // 'one'
console.log(obj2.name); // 'one'

console.log(obj1 instanceof OneClass); // false
console.log(obj2 instanceof OneClass); // false

console.log(obj1 instanceof OtherClass); // true
console.log(obj2 instanceof OtherClass); // true
```

However, while the end result is the same, there is one important difference in the process. When using `Object.create()` and `Function.prototype.apply()`, the `new.target` operator will point to `undefined` within the function used as the constructor, since the `new` keyword is not being used to create the object. (In fact, it uses the `apply` semantic, not `construct`, although normal functions happen to operate nearly the same.)

When invoking `Reflect.construct()`, on the other hand, the `new.target` operator will point to the `newTarget` parameter if supplied, or `target` if not.

```js
function OneClass() {
  console.log("OneClass");
  console.log(new.target);
}
function OtherClass() {
  console.log("OtherClass");
  console.log(new.target);
}

const obj1 = Reflect.construct(OneClass, args);
// Logs:
// OneClass
// function OneClass { ... }

const obj2 = Reflect.construct(OneClass, args, OtherClass);
// Logs:
// OneClass
// function OtherClass { ... }

const obj3 = Object.create(OtherClass.prototype);
OneClass.apply(obj3, args);
// Output:
//     OneClass
//     undefined
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.construct`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`construct`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.construct`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.construct` in `core-js`
- `Reflect`
- `new`
- `new.target`
- `handler.construct()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.defineProperty()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/defineProperty · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.defineProperty()`** static method is like `Object.defineProperty()` but returns a `Boolean`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.defineProperty()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};

if (Reflect.defineProperty(object, "foo", { value: 42 })) {
  console.log("foo created!");
  // Expected output: "foo created!"
} else {
  console.log("problem creating foo");
}

console.log(object.foo);
// Expected output: 42
```

Syntax

```js
Reflect.defineProperty(target, propertyKey, attributes)
```

Parameters

`target`

The target object on which to define the property.

`propertyKey`

The name of the property to be defined or modified.

`attributes`

The attributes for the property being defined or modified.

Return value

A boolean indicating whether or not the property was successfully defined.

Exceptions

`TypeError`

Thrown if `target` or `attributes` is not an object.

Description

`Reflect.defineProperty()` provides the reflective semantic of defining an own property on an object. At the very low level, defining a property returns a boolean (as is the case with the proxy handler). `Object.defineProperty()` provides nearly the same semantic, but it throws a `TypeError` if the status is `false` (the operation was unsuccessful), while `Reflect.defineProperty()` directly returns the status.

Many built-in operations would also define own properties on objects. The most significant difference between defining properties and setting them is that setters aren't invoked. For example, class fields directly define properties on the instance without invoking setters.

```js
class B extends class A {
  set a(v) {
    console.log("Setter called");
  }
} {
  a = 1; // Nothing logged
}
```

`Reflect.defineProperty()` invokes the `[[DefineOwnProperty]]` object internal method of `target`.

Examples

Using Reflect.defineProperty()

```js
const obj = {};
Reflect.defineProperty(obj, "x", { value: 7 }); // true
console.log(obj.x); // 7
```

Checking if property definition has been successful

With `Object.defineProperty()`, which returns an object if successful, or throws a `TypeError` otherwise, you would use a `try...catch` block to catch any error that occurred while defining a property.

Because `Reflect.defineProperty()` returns a Boolean success status, you can just use an `if...else` block here:

```js
if (Reflect.defineProperty(target, property, attributes)) {
  // success
} else {
  // failure
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.defineproperty

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`defineProperty`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.defineProperty`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.defineProperty` in `core-js`
- `Reflect`
- `Object.defineProperty()`
- `handler.defineProperty()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.deleteProperty()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/deleteProperty · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.deleteProperty()`** static method is like the `delete` operator, but as a function. It deletes a property from an object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.deleteProperty()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  foo: 42,
};

Reflect.deleteProperty(object, "foo");

console.log(object.foo);
// Expected output: undefined

const array = [1, 2, 3, 4, 5];
Reflect.deleteProperty(array, "3");

console.log(array);
// Expected output: Array [1, 2, 3, <1 empty slot>, 5]
```

Syntax

js

```
Reflect.deleteProperty(target, propertyKey)
```

Parameters

`target`

The target object on which to delete the property.

`propertyKey`

The name of the property to be deleted.

Return value

A boolean indicating whether or not the property was successfully deleted.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.deleteProperty()` provides the reflective semantic of the `delete` operator. That is, `Reflect.deleteProperty(target, propertyKey)` is semantically equivalent to:

js

```
delete target.propertyKey;
```

At the very low level, deleting a property returns a boolean (as is the case with the proxy handler). `Reflect.deleteProperty()` directly returns the status, while `delete` would throw a `TypeError` in strict mode if the status is `false`. In non-strict mode, `delete` and `Reflect.deleteProperty()` have the same behavior.

`Reflect.deleteProperty()` invokes the `[[Delete]]` object internal method of `target`.

Examples

Using Reflect.deleteProperty()

```js
const obj = { x: 1, y: 2 };
Reflect.deleteProperty(obj, "x"); // true
console.log(obj); // { y: 2 }

const arr = [1, 2, 3, 4, 5];
Reflect.deleteProperty(arr, "3"); // true
console.log(arr); // [1, 2, 3, <1 empty slot>, 5]

// Returns true if no such property exists
Reflect.deleteProperty({}, "foo"); // true

// Returns false if a property is unconfigurable
Reflect.deleteProperty(Object.freeze({ foo: 1 }), "foo"); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.deleteproperty

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
deleteProperty	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.deleteProperty`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.deleteProperty` in `core-js`
- `Reflect`
- `delete`
- `handler.deleteProperty()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.get()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/get · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.get()`** static method is like the property accessor syntax, but as a function.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.get()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  x: 1,
  y: 2,
};

console.log(Reflect.get(object, "x"));
// Expected output: 1

const array = ["zero", "one"];

console.log(Reflect.get(array, 1));
// Expected output: "one"
```

Syntax

js

```
Reflect.get(target, propertyKey)
Reflect.get(target, propertyKey, receiver)
```

Parameters

`target`

The target object on which to get the property.

`propertyKey`

The name of the property to get.

`receiver` Optional

The value of `this` provided for the call to `target` if a getter is encountered. Defaults to `target`.

Return value

The value of the property.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.get()` provides the reflective semantic of a property access. That is, `Reflect.get(target, propertyKey, receiver)` is semantically equivalent to:

```js
target[propertyKey];
```

Note that in a normal property access, `target` and `receiver` would observably be the same object.

`Reflect.get()` invokes the `[[Get]]` object internal method of `target`.

Examples

Using Reflect.get()

```js
// Object
const obj1 = { x: 1, y: 2 };
Reflect.get(obj1, "x"); // 1

// Array
Reflect.get(["zero", "one"], 1); // "one"

// Proxy with a get handler
const obj2 = new Proxy(
  { p: 1 },
  {
    get(t, k, r) {
      return `${k}bar`;
    },
  },
);
Reflect.get(obj2, "foo"); // "foobar"

// Proxy with get handler and receiver
const obj3 = new Proxy(
  { p: 1, foo: 2 },
  {
    get(t, prop, receiver) {
      return `${receiver[prop]}bar`;
    },
  },
);
Reflect.get(obj3, "foo", { foo: 3 }); // "3bar"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.get

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
get	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.get`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.get` in `core-js`
- `Reflect`
- Property accessors
- `handler.get()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.getOwnPropertyDescriptor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/getOwnPropertyDescriptor · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.getOwnPropertyDescriptor()`** static method is like `Object.getOwnPropertyDescriptor()`. It returns a property descriptor of the given property if it exists on the object, `undefined` otherwise.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Reflect.getOwnPropertyDescriptor()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const object = {
  property1: 42,
};

console.log(Reflect.getOwnPropertyDescriptor(object, "property1").value);
// Expected output: 42

console.log(Reflect.getOwnPropertyDescriptor(object, "property2"));
// Expected output: undefined

console.log(Reflect.getOwnPropertyDescriptor(object, "property1").writable);
// Expected output: true
```

Syntax

js

```
Reflect.getOwnPropertyDescriptor(target, propertyKey)
```

Parameters

`target`

The target object in which to look for the property.

`propertyKey`

The name of the property to get an own property descriptor for.

Return value

A property descriptor object if the property exists as an own property of `target`; otherwise, `undefined`.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.getOwnPropertyDescriptor()` provides the reflective semantic of retrieving the property descriptor of an object. The only difference with `Object.getOwnPropertyDescriptor()` is how non-object targets are handled. `Reflect.getOwnPropertyDescriptor()` throws a `TypeError` if the target is not an object, while `Object.getOwnPropertyDescriptor()` coerces it to an object.

`Reflect.getOwnPropertyDescriptor()` invokes the `[[GetOwnProperty]]` object internal method of `target`.

Examples

Using Reflect.getOwnPropertyDescriptor()

```js
Reflect.getOwnPropertyDescriptor({ x: "hello" }, "x");
// {value: "hello", writable: true, enumerable: true, configurable: true}

Reflect.getOwnPropertyDescriptor({ x: "hello" }, "y");
// undefined

Reflect.getOwnPropertyDescriptor([], "length");
// {value: 0, writable: true, enumerable: false, configurable: false}
```

Difference with Object.getOwnPropertyDescriptor()

If the `target` argument to this method is not an object (a primitive), then it will cause a `TypeError`. With `Object.getOwnPropertyDescriptor`, a non-object first argument will be coerced to an object at first.

```js
Reflect.getOwnPropertyDescriptor("foo", 0);
// TypeError: "foo" is not non-null object

Object.getOwnPropertyDescriptor("foo", 0);
// { value: "f", writable: false, enumerable: true, configurable: false }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.getownpropertydescriptor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getOwnPropertyDescriptor	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.getOwnPropertyDescriptor`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Reflect.getOwnPropertyDescriptor` in `core-js`
- `Reflect`
- `Object.getOwnPropertyDescriptor()`
- `handler.getOwnPropertyDescriptor()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.getPrototypeOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/getPrototypeOf · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.getPrototypeOf()`** static method is like `Object.getPrototypeOf()`. It returns the prototype of the specified object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Reflect.getPrototypeOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const object = {
  foo: 42,
};

const proto = Reflect.getPrototypeOf(object);

console.log(proto);
// Expected output: Object {  }

console.log(Reflect.getPrototypeOf(proto));
// Expected output: null
```

Syntax

```js
Reflect.getPrototypeOf(target)
```

Parameters

`target`

The target object of which to get the prototype.

Return value

The prototype of the given object, which may be an object or `null`.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.getPrototypeOf()` provides the reflective semantic of retrieving the prototype of an object. The only difference with `Object.getPrototypeOf()` is how non-object targets are handled. `Reflect.getPrototypeOf()` throws a `TypeError` if the target is not an object, while `Object.getPrototypeOf()` coerces it to an object.

`Reflect.getPrototypeOf()` invokes the `[[GetPrototypeOf]]` object internal method of `target`.

Examples

Using Reflect.getPrototypeOf()

```js
Reflect.getPrototypeOf({}); // Object.prototype
Reflect.getPrototypeOf(Object.prototype); // null
Reflect.getPrototypeOf(Object.create(null)); // null
```

Difference with Object.getPrototypeOf()

```js
// Same result for Objects
Object.getPrototypeOf({}); // Object.prototype
Reflect.getPrototypeOf({}); // Object.prototype

// Both throw in ES5 for non-Objects
Object.getPrototypeOf("foo"); // Throws TypeError
Reflect.getPrototypeOf("foo"); // Throws TypeError

// In ES2015 only Reflect throws, Object coerces non-Objects
Object.getPrototypeOf("foo"); // String.prototype
Reflect.getPrototypeOf("foo"); // Throws TypeError

// To mimic the Object ES2015 behavior you need to coerce
Reflect.getPrototypeOf(Object("foo")); // String.prototype
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.getprototypeof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getPrototypeOf	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.getPrototypeOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.getPrototypeOf` in `core-js`
- es-shims polyfill of `Reflect.getPrototypeOf`
- `Reflect`
- `Object.getPrototypeOf()`
- `handler.getPrototypeOf()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.has()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/has · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.has()`** static method is like the `in` operator, but as a function.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.has()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  property1: 42,
};

console.log(Reflect.has(object, "property1"));
// Expected output: true

console.log(Reflect.has(object, "property2"));
// Expected output: false

console.log(Reflect.has(object, "toString"));
// Expected output: true
```

Syntax

```js
Reflect.has(target, propertyKey)
```

Parameters

`target`

The target object in which to look for the property.

`propertyKey`

The name of the property to check.

Return value

A `Boolean` indicating whether or not the `target` has the property.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.has()` provides the reflective semantic of checking if a property is in an object. That is, `Reflect.has(target, propertyKey)` is semantically equivalent to:

```js
propertyKey in target;
```

`Reflect.has()` invokes the `[[HasProperty]]` object internal method of `target`.

Examples

Using Reflect.has()

```js
Reflect.has({ x: 0 }, "x"); // true
Reflect.has({ x: 0 }, "y"); // false

// returns true for properties in the prototype chain
Reflect.has({ x: 0 }, "toString");

// Proxy with .has() handler method
obj = new Proxy(
  {},
  {
    has(t, k) {
      return k.startsWith("door");
    },
  },
);
Reflect.has(obj, "doorbell"); // true
Reflect.has(obj, "dormitory"); // false
```

`Reflect.has` returns `true` for any inherited properties, like the `in` operator:

```js
const a = { foo: 123 };
const b = { __proto__: a };
const c = { __proto__: b };
// The prototype chain is: c -> b -> a
Reflect.has(c, "foo"); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.has

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
has	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.has`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.has` in `core-js`
- `Reflect`
- `in`
- `handler.has()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.isExtensible()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/isExtensible · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.isExtensible()`** static method is like `Object.isExtensible()`. It determines if an object is extensible (whether it can have new properties added to it).

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.isExtensible()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object1 = {};

console.log(Reflect.isExtensible(object1));
// Expected output: true

Reflect.preventExtensions(object1);

console.log(Reflect.isExtensible(object1));
// Expected output: false

const object2 = Object.seal({});

console.log(Reflect.isExtensible(object2));
// Expected output: false
```

Syntax

js

```
Reflect.isExtensible(target)
```

Parameters

`target`

The target object which to check if it is extensible.

Return value

A `Boolean` indicating whether or not the target is extensible.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.isExtensible()` provides the reflective semantic of checking if an object is extensible. The only difference with `Object.isExtensible()` is how non-object targets are handled. `Reflect.isExtensible()` throws a `TypeError` if the target is not an object, while `Object.isExtensible()` always returns `false` for non-object targets.

`Reflect.isExtensible()` invokes the `[[IsExtensible]]` object internal method of `target`.

Examples

Using Reflect.isExtensible()

See also `Object.isExtensible()`.

```js
// New objects are extensible.
const empty = {};
Reflect.isExtensible(empty); // true

// … but that can be changed.
Reflect.preventExtensions(empty);
Reflect.isExtensible(empty); // false

// Sealed objects are by definition non-extensible.
const sealed = Object.seal({});
Reflect.isExtensible(sealed); // false

// Frozen objects are also by definition non-extensible.
const frozen = Object.freeze({});
Reflect.isExtensible(frozen); // false
```

Difference with Object.isExtensible()

If the `target` argument to this method is not an object (a primitive), then it will cause a `TypeError`. With `Object.isExtensible()`, a non-object `target` will return false without any errors.

```js
Reflect.isExtensible(1);
// TypeError: 1 is not an object

Object.isExtensible(1);
// false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.isextensible

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isExtensible`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.isExtensible`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.isExtensible` in `core-js`
- `Reflect`
- `Object.isExtensible()`
- `handler.isExtensible()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.ownKeys()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/ownKeys · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.ownKeys()`** static method returns an array of the `target` object's own property keys.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.ownKeys()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  property1: 42,
  property2: 13,
};

const array = [];

console.log(Reflect.ownKeys(object));
// Expected output: Array ["property1", "property2"]

console.log(Reflect.ownKeys(array));
// Expected output: Array ["length"]
```

Syntax

js

```
Reflect.ownKeys(target)
```

Parameters

`target`

The target object from which to get the own keys.

Return value

An `Array` of the `target` object's own property keys, including strings and symbols. For most objects, the array will be in the order of:

1. Non-negative integer indexes in increasing numeric order (but as strings)
2. Other string keys in the order of property creation
3. Symbol keys in the order of property creation.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.ownKeys()` provides the reflective semantic of retrieving all property keys of an object. It is the only way to get all own properties – enumerable and not enumerable, strings and symbols — in one

call, without extra filtering logic. For example, `Object.getOwnPropertyNames()` takes the return value of `Reflect.ownKeys()` and filters to only string values, while `Object.getOwnPropertySymbols()` filters to only symbol values. Because normal objects implement `[[OwnPropertyKeys]]` to return all string keys before symbol keys, `Reflect.ownKeys(target)` is usually equivalent to `Object.getOwnPropertyNames(target).concat(Object.getOwnPropertySymbols(target))`. However, if the object has a custom `[[OwnPropertyKeys]]` method (such as through a proxy's `ownKeys` handler), the order of the keys may be different.

`Reflect.ownKeys()` invokes the `[[OwnPropertyKeys]]` object internal method of `target`.

Examples

Using Reflect.ownKeys()

```js
Reflect.ownKeys({ z: 3, y: 2, x: 1 }); // [ "z", "y", "x" ]
Reflect.ownKeys([]); // ["length"]

const sym = Symbol.for("comet");
const sym2 = Symbol.for("meteor");
const obj = {
  [sym]: 0,
  str: 0,
  773: 0,
  0: 0,
  [sym2]: 0,
  "-1": 0,
  8: 0,
  "second str": 0,
};
Reflect.ownKeys(obj);
// [ "0", "8", "773", "str", "-1", "second str", Symbol(comet), Symbol(meteor) ]
// Indexes in numeric order,
// strings in insertion order,
// symbols in insertion order
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.ownkeys

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
ownKeys	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.ownKeys`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.ownKeys` in `core-js`
- es-shims polyfill of `Reflect.ownKeys`
- `Reflect`
- `Object.getOwnPropertyNames()`
- `Object.getOwnPropertySymbols()`
- `handler.ownKeys()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.preventExtensions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/preventExtensions · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.preventExtensions()`** static method is like `Object.preventExtensions()`. It prevents new properties from ever being added to an object (i.e., prevents future extensions to the object).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Reflect.preventExtensions()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const object = {};

console.log(Reflect.isExtensible(object));
// Expected output: true

Reflect.preventExtensions(object);

console.log(Reflect.isExtensible(object));
// Expected output: false
```

Syntax

js

```
Reflect.preventExtensions(target)
```

Parameters

`target`

The target object on which to prevent extensions.

Return value

A `Boolean` indicating whether or not the target was successfully set to prevent extensions.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.preventExtensions()` provides the reflective semantic of preventing extensions of an object. The differences with `Object.preventExtensions()` are:

- `Reflect.preventExtensions()` throws a `TypeError` if the target is not an object, while `Object.preventExtensions()` always returns non-object targets as-is.
- `Reflect.preventExtensions()` returns a `Boolean` indicating whether or not the target was successfully set to prevent extensions, while `Object.preventExtensions()` returns the target object.

`Reflect.preventExtensions()` invokes the `[[PreventExtensions]]` object internal method of `target`.

Examples

Using Reflect.preventExtensions()

See also `Object.preventExtensions()`.

```js
// Objects are extensible by default.
const empty = {};
Reflect.isExtensible(empty); // true

// … but that can be changed.
Reflect.preventExtensions(empty);
Reflect.isExtensible(empty); // false
```

Difference with Object.preventExtensions()

If the `target` argument to this method is not an object (a primitive), then it will cause a `TypeError`. With `Object.preventExtensions()`, a non-object `target` will be returned as-is without any errors.

```js
Reflect.preventExtensions(1);
// TypeError: 1 is not an object

Object.preventExtensions(1);
// 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.preventextensions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`preventExtensions`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.preventExtensions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.preventExtensions` in `core-js`
- `Reflect`
- `Object.preventExtensions()`
- `handler.preventExtensions()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.set()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/set · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.set()`** static method is like the property accessor and assignment syntax, but as a function.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.set()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};
Reflect.set(object, "foo", 42);

console.log(object.foo);
// Expected output: 42

const array = ["duck", "duck", "duck"];
Reflect.set(array, 2, "goose");

console.log(array[2]);
// Expected output: "goose"
```

Syntax

js

```
Reflect.set(target, propertyKey, value)
Reflect.set(target, propertyKey, value, receiver)
```

Parameters

`target`

The target object on which to set the property.

`propertyKey`

The name of the property to set.

`value`

The value to set.

`receiver` Optional

The value of `this` provided for the call to the setter for `propertyKey` on `target`. If provided and `target` does not have a setter for `propertyKey`, the property will be set on `receiver` instead. Defaults

to `target`.

Return value

A `Boolean` indicating whether or not setting the property was successful.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.set()` provides the reflective semantic of a property access. That is, `Reflect.set(target, propertyKey, value, receiver)` is semantically equivalent to:

```js
target[propertyKey] = value;
```

Note that in a normal property access, `target` and `receiver` would observably be the same object.

`Reflect.set()` invokes the `[[Set]]` object internal method of `target`.

Examples

Using Reflect.set()

```js
// Object
const obj = {};
Reflect.set(obj, "prop", "value"); // true
obj.prop; // "value"

// Array
const arr = ["duck", "duck", "duck"];
Reflect.set(arr, 2, "goose"); // true
arr[2]; // "goose"

// It can truncate an array.
Reflect.set(arr, "length", 1); // true
arr; // ["duck"]

// With just one argument, propertyKey and value are "undefined".
Reflect.set(obj); // true
Reflect.getOwnPropertyDescriptor(obj, "undefined");
// { value: undefined, writable: true, enumerable: true, configurable: true }
```

Different target and receiver

When the `target` and `receiver` are different, `Reflect.set` will use the property descriptor of `target` (to find the setter or determine if the property is writable), but set the property on `receiver`.

```js
const target = {};
const receiver = {};
Reflect.set(target, "a", 2, receiver); // true
// target is {}; receiver is { a: 2 }
```

(code continues on the next page)

(code continued from the previous page)

```
const target = { a: 1 };
const receiver = {};
Reflect.set(target, "a", 2, receiver); // true
// target is { a: 1 }; receiver is { a: 2 }

const target = {
  set a(v) {
    this.b = v;
  },
};
const receiver = {};
Reflect.set(target, "a", 2, receiver); // true
// target is { a: [Setter] }; receiver is { b: 2 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.set`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`set`	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.set`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.set` in `core-js`
- `Reflect`
- Property accessors
- `handler.set()`

JavaScript › Reference › Standard built-in objects › Reflect

Reflect.setPrototypeOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Reflect/setPrototypeOf · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Reflect.setPrototypeOf()`** static method is like `Object.setPrototypeOf()` but returns a `Boolean`. It sets the prototype (i.e., the internal `[[Prototype]]` property) of a specified object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Reflect.setPrototypeOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object1 = {};

console.log(Reflect.setPrototypeOf(object1, Object.prototype));
// Expected output: true

console.log(Reflect.setPrototypeOf(object1, null));
// Expected output: true

const object2 = {};

console.log(Reflect.setPrototypeOf(Object.freeze(object2), null));
// Expected output: false
```

Syntax

js

```
Reflect.setPrototypeOf(target, prototype)
```

Parameters

`target`

The target object of which to set the prototype.

`prototype`

The object's new prototype (an object or `null`).

Return value

A `Boolean` indicating whether or not the prototype was successfully set.

Exceptions

`TypeError`

Thrown if `target` is not an object or if `prototype` is neither an object nor `null`.

Description

`Reflect.setPrototypeOf()` provides the reflective semantic of setting the prototype of an object. At the very low level, setting the prototype returns a boolean (as is the case with the proxy handler). `Object.setPrototypeOf()` provides nearly the same semantic, but it throws a `TypeError` if the status is `false` (the operation was unsuccessful), while `Reflect.setPrototypeOf()` directly returns the status.

`Reflect.setPrototypeOf()` invokes the `[[SetPrototypeOf]]` object internal method of `target`.

Examples

Using Reflect.setPrototypeOf()

```js
Reflect.setPrototypeOf({}, Object.prototype); // true

// It can change an object's [[Prototype]] to null.
Reflect.setPrototypeOf({}, null); // true

// Returns false if target is not extensible.
Reflect.setPrototypeOf(Object.freeze({}), null); // false

// Returns false if it cause a prototype chain cycle.
const target = {};
const proto = Object.create(target);
Reflect.setPrototypeOf(target, proto); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/reflection.html#sec-reflect.setprototypeof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setPrototypeOf	49	12	42	36	10	49	42	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Reflect.setPrototypeOf`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Reflect.setPrototypeOf` in `core-js`
- `Reflect`
- `Object.setPrototypeOf()`
- `handler.setPrototypeOf()`

JavaScript › Reference › Standard built-in objects

RegExp

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **RegExp** object is used for matching text with a pattern.

For an introduction to regular expressions, read the Regular expressions chapter in the JavaScript guide. For detailed information of regular expression syntax, read the regular expression reference.

Description

Literal notation and constructor

There are two ways to create a `RegExp` object: a *literal notation* and a *constructor*.

- The *literal notation* takes a pattern between two slashes, followed by optional flags, after the second slash.
- The *constructor function* takes either a string or a `RegExp` object as its first parameter and a string of optional flags as its second parameter.

The following three expressions create the same regular expression object:

```js
const re = /ab+c/i; // literal notation
// OR
const re = new RegExp("ab+c", "i"); // constructor with string pattern as first argument
// OR
const re = new RegExp(/ab+c/, "i"); // constructor with regular expression literal as first
argument
```

Before regular expressions can be used, they have to be compiled. This process allows them to perform matches more efficiently. More about the process can be found in dotnet docs.

The literal notation results in compilation of the regular expression when the expression is evaluated. On the other hand, the constructor of the `RegExp` object, `new RegExp('ab+c')`, results in runtime compilation of the regular expression.

Use a string as the first argument to the `RegExp()` constructor when you want to build the regular expression from dynamic input.

Flags in constructor

The expression `new RegExp(/ab+c/, flags)` will create a new `RegExp` using the source of the first parameter and the flags provided by the second.

When using the constructor function, the normal string escape rules (preceding special characters with `\` when included in a string) are necessary.

For example, the following are equivalent:

```js
const re = /\w+/;
// OR
const re = new RegExp("\\w+");
```

Special handling for regexes

> **Note:** Whether something is a "regex" can be duck-typed. It doesn't have to be a `RegExp`!

Some built-in methods would treat regexes specially. They decide whether `x` is a regex through multiple steps:

1. `x` must be an object (not a primitive).
2. If `x[Symbol.match]` is not `undefined`, check if it's truthy.
3. Otherwise, if `x[Symbol.match]` is `undefined`, check if `x` had been created with the `RegExp` constructor. (This step should rarely happen, since if `x` is a `RegExp` object that have not been tampered with, it should have a `Symbol.match` property.)

Note that in most cases, it would go through the `Symbol.match` check, which means:

- An actual `RegExp` object whose `Symbol.match` property's value is falsy but not `undefined` (even with everything else intact, like `exec` and `[Symbol.replace]()`) can be used as if it's not a regex.
- A non-`RegExp` object with a `Symbol.match` property will be treated as if it's a regex.

This choice was made because `[Symbol.match]()` is the most indicative property that something is intended to be used for matching. (`exec` could also be used, but because it's not a symbol property, there would be too many false positives.) The places that treat regexes specially include:

- `String.prototype.endsWith()`, `startsWith()`, and `includes()` throw a `TypeError` if the first argument is a regex.
- `String.prototype.matchAll()` and `replaceAll()` check whether the global flag is set if the first argument is a regex before invoking its `[Symbol.matchAll]()` or `[Symbol.replace]()` method.
- The `RegExp()` constructor directly returns the `pattern` argument only if `pattern` is a regex (among a few other conditions). If `pattern` is a regex, it would also interrogate `pattern`'s `source` and `flags` properties instead of coercing `pattern` to a string.

For example, `String.prototype.endsWith()` would coerce all inputs to strings, but it would throw if the argument is a regex, because it's only designed to match strings, and using a regex is likely a developer mistake.

```js
"foobar".endsWith({ toString: () => "bar" }); // true
"foobar".endsWith(/bar/); // TypeError: First argument to String.prototype.endsWith must not be a regular expression
```

You can get around the check by setting `[Symbol.match]` to a falsy value that's not `undefined`. This would mean that the regex cannot be used for `String.prototype.match()` (since without `[Symbol.match]`, `match()` would construct a new `RegExp` object with the two enclosing slashes added by `re.toString()`), but it can be used for virtually everything else.

```js
const re = /bar/g;
re[Symbol.match] = false;
"/bar/g".endsWith(re); // true
re.exec("bar"); // [ 'bar', index: 0, input: 'bar', groups: undefined ]
"bar & bar".replace(re, "foo"); // 'foo & foo'
```

Perl-like RegExp properties

Note that several of the `RegExp` properties have both long and short (Perl-like) names. Both names always refer to the same value. (Perl is the programming language from which JavaScript modeled its regular expressions.) See also deprecated `RegExp` properties.

Constructor

`RegExp()`

Creates a new `RegExp` object.

Static properties

`RegExp.$1, ..., RegExp.$9` Deprecated

Static read-only properties that contain parenthesized substring matches.

`RegExp.input ($_)` Deprecated

A static property that contains the last string against which a regular expression was successfully matched.

`RegExp.lastMatch ($&)` Deprecated

A static read-only property that contains the last matched substring.

`RegExp.lParen ($+)` Deprecated

A static read-only property that contains the last parenthesized substring match.

`RegExp.leftContext ($`)` Deprecated

A static read-only property that contains the substring preceding the most recent match.

`RegExp.rightContext ($')` Deprecated

A static read-only property that contains the substring following the most recent match.

`RegExp[Symbol.species]`

The constructor function that is used to create derived objects.

Static methods

`RegExp.escape()`

Escapes any potential regex syntax characters in a string, and returns a new string that can be safely used as a literal pattern for the `RegExp()` constructor.

Instance properties

These properties are defined on `RegExp.prototype` and shared by all `RegExp` instances.

`RegExp.prototype.constructor`

The constructor function that created the instance object. For `RegExp` instances, the initial value is the `RegExp` constructor.

`RegExp.prototype.dotAll`

Whether `.` matches newlines or not.

`RegExp.prototype.flags`

A string that contains the flags of the `RegExp` object.

`RegExp.prototype.global`
Whether to test the regular expression against all possible matches in a string, or only against the first.

`RegExp.prototype.hasIndices`
Whether the regular expression result exposes the start and end indices of captured substrings.

`RegExp.prototype.ignoreCase`
Whether to ignore case while attempting a match in a string.

`RegExp.prototype.multiline`
Whether or not to search in strings across multiple lines.

`RegExp.prototype.source`
The text of the pattern.

`RegExp.prototype.sticky`
Whether or not the search is sticky.

`RegExp.prototype.unicode`
Whether or not Unicode features are enabled.

`RegExp.prototype.unicodeSets`
Whether or not the `v` flag, an upgrade to the `u` mode, is enabled.

These properties are own properties of each `RegExp` instance.

`lastIndex`
The index at which to start the next match.

Instance methods

`RegExp.prototype.compile()` Deprecated
(Re-)compiles a regular expression during execution of a script.

`RegExp.prototype.exec()`
Executes a search for a match in its string parameter.

`RegExp.prototype.test()`
Tests for a match in its string parameter.

`RegExp.prototype.toString()`
Returns a string representing the specified object. Overrides the `Object.prototype.toString()` method.

`RegExp.prototype[Symbol.match]()`
Performs match to given string and returns match result.

`RegExp.prototype[Symbol.matchAll]()`
Returns all matches of the regular expression against a string.

`RegExp.prototype[Symbol.replace]()`
Replaces matches in given string with new substring.

`RegExp.prototype[Symbol.search]()`
Searches the match in given string and returns the index the pattern found in the string.

`RegExp.prototype[Symbol.split]()`
Splits given string into an array by separating the string into substrings.

Examples

Using a regular expression to change data format

The following script uses the `String.prototype.replace()` method to match a name in the format *first last* and output it in the format *last, first*.

In the replacement text, the script uses `$1` and `$2` to indicate the results of the corresponding matching parentheses in the regular expression pattern.

```js
const re = /(\w+)\s(\w+)/;
const str = "Maria Cruz";
const newStr = str.replace(re, "$2, $1");
console.log(newStr);
```

This displays `"Cruz, Maria"`.

Using regular expression to split lines with different line endings/ends of line/line breaks

The default line ending varies depending on the platform (Unix, Windows, etc.). The line splitting provided in this example works on all platforms.

```js
const text = "Some text\nAnd some more\r\nAnd yet\nThis is the end";
const lines = text.split(/\r?\n/);
console.log(lines); // [ 'Some text', 'And some more', 'And yet', 'This is the end' ]
```

Note that the order of the patterns in the regular expression matters.

Using regular expression on multiple lines

By default, the `.` character does not match newlines. To make it match newlines, use the `s` flag (`dotAll` mode).

```js
const s = "Please yes\nmake my day!";

s.match(/yes.*day/);
// Returns null

s.match(/yes.*day/s);
// Returns ["yes\nmake my day"]
```

Using a regular expression with the sticky flag

The `sticky` flag indicates that the regular expression performs sticky matching in the target string by attempting to match starting at `RegExp.prototype.lastIndex`.

```js
const str = "#foo#";
const regex = /foo/y;

regex.lastIndex = 1;
regex.test(str); // true
regex.lastIndex = 5;
regex.test(str); // false (lastIndex is taken into account with sticky flag)
regex.lastIndex; // 0 (reset after match failure)
```

The difference between the sticky flag and the global flag

With the sticky flag `y`, the next match has to happen at the `lastIndex` position, while with the global flag `g`, the match can happen at the `lastIndex` position or later:

```js
const re = /\d/y;
let r;
while ((r = re.exec("123 456"))) {
  console.log(r, "AND re.lastIndex", re.lastIndex);
}

// [ '1', index: 0, input: '123 456', groups: undefined ] AND re.lastIndex 1
// [ '2', index: 1, input: '123 456', groups: undefined ] AND re.lastIndex 2
// [ '3', index: 2, input: '123 456', groups: undefined ] AND re.lastIndex 3
//  … and no more match.
```

With the global flag `g`, all 6 digits would be matched, not just 3.

Regular expression and Unicode characters

`\w` and `\W` only matches ASCII based characters; for example, `a` to `z`, `A` to `Z`, `0` to `9`, and `_`.

To match characters from other languages such as Cyrillic or Hebrew, use `\uHHHH`, where `HHHH` is the character's Unicode value in hexadecimal.

This example demonstrates how one can separate out Unicode characters from a word.

```js
const text = "Образец text на русском языке";
const regex = /[\u0400-\u04ff]+/g;

const match = regex.exec(text);
console.log(match[0]); // 'Образец'
console.log(regex.lastIndex); // 7

const match2 = regex.exec(text);
console.log(match2[0]); // 'на' (did not log 'text')
console.log(regex.lastIndex); // 15

// and so on
```

The Unicode property escapes feature provides a simpler way to target particular Unicode ranges, by allowing for statements like `\p{scx=Cyrl}` (to match any Cyrillic letter), or `\p{L}/u` (to match a letter from any language).

Extracting subdomain name from URL

```js
const url = "http://xxx.example.com";
console.log(/^https?:\/\/(.+?)\./.exec(url)[1]); // 'xxx'
```

> **Note:** Instead of using regular expressions for parsing URLs, it is usually better to use the browsers built-in URL parser by using the URL API.

Building a regular expression from dynamic inputs

```js
const breakfasts = ["bacon", "eggs", "oatmeal", "toast", "cereal"];
const order = "Let me get some bacon and eggs, please";

order.match(new RegExp(`\\b(${breakfasts.join("|")})\\b`, "g"));
// Returns ['bacon', 'eggs']
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp-regular-expression-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
RegExp	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
[Symbol.match]	50	13	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0
[Symbol.matchAll]	73	79	67	60	13	73	67	52	13	5.0	73	13	1.0.0	1.0	12.0.0
[Symbol.replace]	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0
[Symbol.search]	50	13	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0
[Symbol.species]	51	13	49	38	10	51	49	41	10	5.0	51	10	1.0.0	1.0	6.5.0
[Symbol.split]	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0
RegExp() constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
compile Deprecated	1	12	1	6	3.1	18	4	10.1	2	1.0	4.4	2	1.0.0	1.0	0.10.0
Throws TypeError when used in RegExp subclass or with cross-realm mismatch. Deprecated Non-standard	**No**	**No**	148	**No**	15.6	**No**	148	**No**	15.6	**No**	**No**	15.6	**No**	**No**	**No**
dotAll	62	79	78	49	11.1	62	79	46	11.3	8.0	62	11.3	1.0.0	1.0	8.10.0
escape	136	136	134	121	18.2	136	134	90	18.2	29.0	136	18.2	1.1.22	2.3.2	24.0.0
exec	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
flags	49	79	37	39	9	49	37	41	9	5.0	49	9	1.0.0	1.0	6.0.0
global	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Prototype accessor property (ES2015)	48	12	38	35	1.3	48	38	35	1	5.0	48	1	1.0.0	1.0	6.0.0
hasIndices	90	90	88	76	15	90	88	64	15	15.0	90	15	1.0.0	1.8	16.0.0
ignoreCase	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Prototype accessor property (ES2015)	48	12	38	35	1.3	48	38	35	1	5.0	48	1	1.0.0	1.0	6.0.0
RegExp.input ($_) Deprecated	1	12	1	15	3	18	4	14	1	1.0	4.4	1	1.0.0	1.0	0.10.0
lastIndex	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
RegExp.lastMatch ($&) Deprecated	1	12	1	10.5	3	18	4	11	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`RegExp.lastParen` (`$+`) Deprecated	1	12	1	10.5	3	18	4	11	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`RegExp.leftContext` (`$``) Deprecated	1	12	1	8	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`multiline`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Prototype accessor property (ES2015)	48	12	38	35	1.3	48	38	35	1	5.0	48	1	1.0.0	1.0	6.0.0
`RegExp.$1-$9` Deprecated	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`RegExp.rightContext` (`$'`) Deprecated	1	12	1	8	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`source`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
"(?:)" for empty regexps	6	12	38	15	5	18	38	14	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0
Line breaks and slashes are escaped	73	12	38	60	6	73	38	52	6	11.0	73	6	1.0.0	1.0	12.0.0
Prototype accessor property (ES2015)	48	12	41	35	1.3	48	41	35	1	5.0	48	1	1.0.0	1.0	6.0.0
`sticky`	49	13	3	36	10	49	4	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Anchored sticky flag behavior per ES2015	49	13	44	36	10	49	44	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Prototype accessor property (ES2015)	49	13	38	36	10	49	38	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`test`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Line breaks and slashes are escaped	73	12	38	60	6	73	38	52	6	11.0	73	6	1.0.0	1.0	12.0.0
`unicode`	50	12[1]	46	37	10	50	46	37	10	5.0	50	10	1.0.0	1.0	6.0.0[2]
`unicodeSets`	112	112	116	98	17	112	116	75	17	23.0	112	17	1.0.0	1.32	20.0.0

1. Case folding is implemented in version 13
2. Case folding is implemented in version 8.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

Firefox-specific notes

Starting with Firefox 34, in the case of a capturing group with quantifiers preventing its exercise, the matched text for a capturing group is now `undefined` instead of an empty string:

```
// Firefox 33 or older
"x".replace(/x(.)?/g, (m, group) => {
  console.log(`group: ${JSON.stringify(group)}`);
});
// group: ""

// Firefox 34 or newer
"x".replace(/x(.)?/g, (m, group) => {
  console.log(`group: ${group}`);
});
// group: undefined
```

Note that due to web compatibility, `RegExp.$N` will still return an empty string instead of `undefined` (bug 1053944).

See also

- Polyfill of many modern `RegExp` features (`dotAll`, `sticky` flags, named capture groups, etc.) in `core-js`
- Regular expressions guide
- Regular expressions
- `String.prototype.match()`
- `String.prototype.replace()`
- `String.prototype.split()`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/RegExp · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`RegExp()`** constructor creates `RegExp` objects.

For an introduction to regular expressions, read the Regular Expressions chapter in the JavaScript Guide.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex1 = /\w+/;
const regex2 = new RegExp("\\w+");

console.log(regex1);
// Expected output: /\w+/

console.log(regex2);
// Expected output: /\w+/

console.log(regex1 === regex2);
// Expected output: false
```

Syntax

js

```
new RegExp(pattern)
new RegExp(pattern, flags)
RegExp(pattern)
RegExp(pattern, flags)
```

> **Note:** `RegExp()` can be called with or without `new`, but sometimes with different effects. See Return value.

Parameters

`pattern`

The text of the regular expression. This can also be another `RegExp` object.

`flags` Optional

If specified, `flags` is a string that contains the flags to add. Alternatively, if a `RegExp` object is supplied for the `pattern`, the `flags` string will replace any of that object's flags (and `lastIndex` will be reset to `0`).

`flags` may contain any combination of the following characters:

`d` (indices)

Generate indices for substring matches.

`g` (global)

Find all matches rather than stopping after the first match.

`i` (ignore case)

When matching, casing differences are ignored.

`m` (multiline)

Treat beginning and end assertions (`^` and `$`) as working over multiple lines. In other words, match the beginning or end of *each* line (delimited by `\n` or `\r`), not only the very beginning or end of the whole input string.

`s` (dotAll)

Allows `.` to match newlines.

`u` (unicode)

Treat `pattern` as a sequence of Unicode code points.

`v` (unicodeSets)

An upgrade to the `u` flag that enables set notation in character classes as well as properties of strings.

`y` (sticky)

Matches only from the index indicated by the `lastIndex` property of this regular expression in the target string. Does not attempt to match from any later indexes.

Return value

`RegExp(pattern)` returns `pattern` directly if all of the following are true:

- `RegExp()` is called without `new`;
- `pattern` is a regex;
- `pattern.constructor === RegExp` (usually meaning it's not a subclass);
- `flags` is `undefined`.

In all other cases, calling `RegExp()` with or without `new` both create a new `RegExp` object. If `pattern` is a regex, the new object's source is `pattern.source`; otherwise, its source is `pattern` coerced to a string. If the `flags` parameter is not `undefined`, the new object's `flags` is the parameter's value; otherwise, its `flags` is `pattern.flags` (if `pattern` is a regex).

Exceptions

`SyntaxError`

Thrown in one of the following cases:

- `pattern` cannot be parsed as a valid regular expression.
- `flags` contains repeated characters or any character outside of those allowed.

Examples

Literal notation and constructor

There are two ways to create a `RegExp` object: a *literal notation* and a *constructor*.

- The *literal notation* takes a pattern between two slashes, followed by optional flags, after the second slash.
- The *constructor function* takes either a string or a `RegExp` object as its first parameter and a string of optional flags as its second parameter.

The following three expressions create the same regular expression:

```js
/ab+c/i;
new RegExp(/ab+c/, "i"); // literal notation
new RegExp("ab+c", "i"); // constructor
```

Before regular expressions can be used, they have to be compiled. This process allows them to perform matches more efficiently. There are two ways to compile and get a `RegExp` object.

The literal notation results in compilation of the regular expression when the expression is evaluated. On the other hand, the constructor of the `RegExp` object, `new RegExp('ab+c')`, results in runtime compilation of the regular expression.

Use a string as the first argument to the `RegExp()` constructor when you want to build the regular expression from dynamic input.

Building a regular expression from dynamic inputs

```js
const breakfasts = ["bacon", "eggs", "oatmeal", "toast", "cereal"];
const order = "Let me get some bacon and eggs, please";

order.match(new RegExp(`\\b(${breakfasts.join("|")})\\b`, "g"));
// Returns ['bacon', 'eggs']
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RegExp()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.RegExp`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of many modern `RegExp` features (`dotAll`, `sticky` flags, named capture groups, etc.) in `core-js`
- Regular expressions guide
- `String.prototype.match()`
- `String.prototype.replace()`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.escape()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/escape · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since May 2025; may not work in older ones.

The **`RegExp.escape()`** static method escapes any potential regex syntax characters in a string, and returns a new string that can be safely used as a literal pattern for the `RegExp()` constructor.

When dynamically creating a `RegExp` with user-provided content, consider using this function to sanitize the input (unless the input is actually intended to contain regex syntax). In addition, don't try to re-implement its functionality by, for example, using `String.prototype.replaceAll()` to insert a `\` before all syntax characters. `RegExp.escape()` is designed to use escape sequences that work in many more edge cases/contexts than hand-crafted code is likely to achieve.

Syntax

```js
RegExp.escape(string)
```

Parameters

`string`

The string to escape.

Return value

A new string that can be safely used as a literal pattern for the `RegExp()` constructor. Namely, the following things in the input string are replaced:

- The first character of the string, if it's either a decimal digit (0–9) or ASCII letter (a–z, A–Z), is escaped using the `\x` character escape syntax. For example, `RegExp.escape("foo")` returns

`"\\x66oo"` (here and after, the two backslashes in a string literal denote a single backslash character). This step ensures that if this escaped string is embedded into a bigger pattern where it's immediately preceded by `\1`, `\x0`, `\u000`, etc., the leading character doesn't get interpreted as part of the escape sequence.

- Regex syntax characters, including `^`, `$`, `\`, `.`, `*`, `+`, `?`, `(`, `)`, `[`, `]`, `{`, `}`, and `|`, as well as the `/` delimiter, are escaped by inserting a `\` character before them. For example, `RegExp.escape("foo.bar")` returns `"\\x66oo\\.bar"`, and `RegExp.escape("(foo)")` returns `"\\(foo\\)"`.
- Other punctuators, including `,`, `-`, `=`, `<`, `>`, `#`, `&`, `!`, `%`, `:`, `;`, `@`, `~`, `'`, `` ` ``, and `"`, are escaped using the `\x` syntax. For example, `RegExp.escape("foo-bar")` returns `"\\x66oo\\x2dbar"`. These characters cannot be escaped by prefixing with `\` because, for example, `/foo\-bar/u` is a syntax error.
- The characters with their own character escape sequences: `\f` (U+000C FORM FEED), `\n` (U+000A LINE FEED), `\r` (U+000D CARRIAGE RETURN), `\t` (U+0009 CHARACTER TABULATION), and `\v` (U+000B LINE TABULATION), are replaced with their escape sequences. For example, `RegExp.escape("foo\nbar")` returns `"\\x66oo\\nbar"`.
- The space character is escaped as `"\\x20"`.
- Other non-ASCII line break and white space characters are replaced with one or two `\uXXXX` escape sequences representing their UTF-16 code units. For example, `RegExp.escape("foo\u2028bar")` returns `"\\x66oo\\u2028bar"`.
- Lone surrogates are replaced with their `\uXXXX` escape sequences. For example, `RegExp.escape("foo\uD800bar")` returns `"\\x66oo\\ud800bar"`.

Exceptions

`TypeError`

Thrown if `string` is not a string.

Examples

Using RegExp.escape()

The following examples demonstrate various inputs and outputs for the `RegExp.escape()` method.

```js
RegExp.escape("Buy it. use it. break it. fix it.");
// "\\x42uy\\x20it\\.\\x20use\\x20it\\.\\x20break\\x20it\\.\\x20fix\\x20it\\."
RegExp.escape("foo.bar"); // "\\x66oo\\.bar"
RegExp.escape("foo-bar"); // "\\x66oo\\x2dbar"
RegExp.escape("foo\nbar"); // "\\x66oo\\nbar"
RegExp.escape("foo\uD800bar"); // "\\x66oo\\ud800bar"
RegExp.escape("foo\u2028bar"); // "\\x66oo\\u2028bar"
```

Using RegExp.escape() with the RegExp constructor

The primary use case of `RegExp.escape()` is when you want to embed a string into a bigger regex pattern, and you want to ensure that the string is treated as a literal pattern, not as a regex syntax. Consider the following naïve example that replaces URLs:

```js
function removeDomain(text, domain) {
  return text.replace(new RegExp(`https?://${domain}(?=/)`, "g"), "");
}

const input =
  "Consider using [RegExp.escape()](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/escape) to escape special characters in a string.";
const domain = "developer.mozilla.org";
console.log(removeDomain(input, domain));
// Consider using [RegExp.escape()](/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/escape) to escape special characters in a string.
```

Inserting the `domain` above results in the regular expression literal `https?://developer.mozilla.org(?=/)`, where the "." character is a regex wildcard character. This means the string will match the string with any character in place of the ".", such as `developer-mozilla-org`. Therefore, it would incorrectly also change the following text:

```js
const input =
  "This is not an MDN link: https://developer-mozilla.org/, be careful!";
const domain = "developer.mozilla.org";
console.log(removeDomain(input, domain));
// This is not an MDN link: /, be careful!
```

To fix this, we can use `RegExp.escape()` to ensure that any user input is treated as a literal pattern:

```js
function removeDomain(text, domain) {
  return text.replace(
    new RegExp(`https?://${RegExp.escape(domain)}(?=/)`, "g"),
    "",
  );
}
```

Now this function will do exactly what we intend to, and will not transform `developer-mozilla.org` URLs.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.escape

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
escape	136	136	134	121	18.2	136	134	90	18.2	29.0	136	18.2	1.1.22	2.3.2	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.escape`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `RegExp.escape` in `core-js`
- es-shims polyfill of `RegExp.escape`
- `RegExp`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.$1, ..., RegExp.$9

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/n · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.$1, …, RegExp.$9`** static accessor properties return parenthesized substring matches.

Description

Because `$1`–`$9` are static properties of `RegExp`, you always use them as `RegExp.$1`, `RegExp.$2`, etc., rather than as properties of a `RegExp` object you created.

The values of `$1, …, $9` update whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, or if the last match does not have the corresponding capturing group, the respective property is an empty string. The set accessor of each property is `undefined`, so you cannot change the properties directly.

The number of possible parenthesized substrings is unlimited, but the `RegExp` object can only hold the first nine. You can access all parenthesized substrings through the returned array's indexes.

`$1, …, $9` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp.$n` legacy properties.

Examples

Using $n with RegExp.prototype.test()

The following script uses the `RegExp.prototype.test()` method to grab a number in a generic string.

```js
const str = "Test 24";
const number = /(\d+)/.test(str) ? RegExp.$1 : "0";
number; // "24"
```

Please note that any operation involving the usage of other regular expressions between a `re.test(str)` call and the `RegExp.$n` property, might have side effects, so that accessing these special properties should be done instantly, otherwise the result might be unexpected.

Specifications

Specification
Legacy RegExp features `https://github.com/tc39/proposal-regexp-legacy-features/#additional-properties-of-the-regexp-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`RegExp.$1-$9` Deprecated	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.n`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.input ($_)`
- `RegExp.lastMatch ($&)`
- `RegExp.lastParen ($+)`
- `RegExp.leftContext ($`)`
- `RegExp.rightContext ($')`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.input ($_)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/input · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.input`** static accessor property returns the string against which a regular expression is matched. `RegExp.$_` is an alias for this property.

Description

Because `input` is a static property of `RegExp`, you always use it as `RegExp.input` or `RegExp.$_`, rather than as a property of a `RegExp` object you created.

The value of `input` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, `input` is an empty string. You can set the value of `input`, but this does not affect other behaviors of the regex, and the value will be overwritten again when the next successful match is made.

Examples

Using input and $_

```js
const re = /hi/g;
re.test("hi there!");
RegExp.input; // "hi there!"
re.test("foo"); // new test, non-matching
RegExp.$_; // "hi there!"
re.test("hi world!"); // new test, matching
RegExp.$_; // "hi world!"
```

Specifications

Specification
Legacy RegExp features https://github.com/tc39/proposal-regexp-legacy-features/#additional-properties-of-the-regexp-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RegExp.input` ($_) Deprecated	1	12	1	15	3	18	4	14	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.input`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.lastMatch ($&)`
- `RegExp.lastParen ($+)`
- `RegExp.leftContext ($`)`
- `RegExp.rightContext ($')`
- `RegExp.$1, ..., RegExp.$9`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.lastMatch ($&)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/lastMatch · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.lastMatch`** static accessor property returns the last matched substring. `RegExp["$&"]` is an alias for this property.

Description

Because `lastMatch` is a static property of `RegExp`, you always use it as `RegExp.lastMatch` or `RegExp["$&"]`, rather than as a property of a `RegExp` object you created.

The value of `lastMatch` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, `lastMatch` is an empty string. The set accessor of `lastMatch` is `undefined`, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (`RegExp.$&`), because `&` is not a valid identifier part, so this causes a `SyntaxError`. Use the bracket notation instead.

`$&` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp["$&"]` legacy property.

Examples

Using lastMatch and $&

```js
const re = /hi/g;
re.test("hi there!");
RegExp.lastMatch; // "hi"
RegExp["$&"]; // "hi"
```

Specifications

Specification
Legacy RegExp features `https://github.com/tc39/proposal-regexp-legacy-features/#additional-properties-of-the-regexp-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RegExp.lastMatch` ($&) Deprecated	1	12	1	10.5	3	18	4	11	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.lastMatch`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.input ($_)`
- `RegExp.lastParen ($+)`
- `RegExp.leftContext ($`)`

- RegExp.rightContext ($')
- RegExp.$1, ..., RegExp.$9

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.lastParen ($+)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/lastParen · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** All RegExp static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **RegExp.lastParen** static accessor property returns the last parenthesized substring match, if any. RegExp["$+"] is an alias for this property.

Description

Because lastParen is a static property of RegExp, you always use it as RegExp.lastParen or RegExp["$+"], rather than as a property of a RegExp object you created.

The value of lastParen updates whenever a RegExp (but not a RegExp subclass) instance makes a successful match. If no matches have been made, or if the most recent regex execution contains no capturing groups, lastParen is an empty string. The set accessor of lastParen is undefined, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (RegExp.$+), because + is not a valid identifier part, so this causes a SyntaxError. Use the bracket notation instead.

Examples

Using lastParen and $+

```js
const re = /(hi)/g;
re.test("hi there!");
RegExp.lastParen; // "hi"
RegExp["$+"]; // "hi"
```

Specifications

Specification
Legacy RegExp features https://github.com/tc39/proposal-regexp-legacy-features/#additional-properties-of-the-regexp-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RegExp.lastParen` (`$+`) Deprecated	1	12	1	10.5	3	18	4	11	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.lastParen`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.input` (`$_`)
- `RegExp.lastMatch` (`$&`)
- `RegExp.leftContext` (`$``)
- `RegExp.rightContext` (`$'`)
- `RegExp.$1`, …, `RegExp.$9`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.leftContext ($`)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/leftContext · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.leftContext`** static accessor property returns the substring preceding the most recent match. `RegExp["$`"]` is an alias for this property.

Description

Because `leftContext` is a static property of `RegExp`, you always use it as `RegExp.leftContext` or `RegExp["$`"]`, rather than as a property of a `RegExp` object you created.

The value of `leftContext` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, `leftContext` is an empty string. The set accessor of `leftContext` is `undefined`, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (`RegExp.$`), because ` is not a valid identifier part, so this causes a `SyntaxError`. Use the bracket notation instead.

`$`` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp["$`"]` legacy property.

Examples

Using leftContext and $`

```js
const re = /world/g;
re.test("hello world!");
RegExp.leftContext; // "hello "
RegExp["$`"]; // "hello "
```

Specifications

Specification
Legacy RegExp features https://github.com/tc39/proposal-regexp-legacy-features/#additional-properties-of-the-regexp-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RegExp.leftContext` (`$``) Deprecated	1	12	1	8	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.leftContext`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.input ($_)`
- `RegExp.lastMatch ($&)`
- `RegExp.lastParen ($+)`
- `RegExp.rightContext ($')`
- `RegExp.$1, ..., RegExp.$9`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.rightContext ($')

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/rightContext · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.rightContext`** static accessor property returns the substring following the most recent match. `RegExp["$'"]` is an alias for this property.

Description

Because `rightContext` is a static property of `RegExp`, you always use it as `RegExp.rightContext` or `RegExp["$'"]`, rather than as a property of a `RegExp` object you created.

The value of `rightContext` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, `rightContext` is an empty string. The set accessor of `rightContext` is `undefined`, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (`RegExp.$'`), because `'` is not a valid identifier part, so this causes a `SyntaxError`. Use the bracket notation instead.

`$'` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp["$'"]` legacy property.

Examples

Using rightContext and $'

```js
const re = /hello/g;
re.test("hello world!");
RegExp.rightContext; // " world!"
RegExp["$'"]; // " world!"
```

Specifications

Specification
Legacy RegExp features https://github.com/tc39/proposal-regexp-legacy-features/#additional-properties-of-the-regexp-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RegExp.rightContext ($')` Deprecated	1	12	1	8	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.rightContext`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `RegExp.input ($_)`
- `RegExp.lastMatch ($&)`
- `RegExp.lastParen ($+)`
- `RegExp.leftContext ($`)`
- `RegExp.$1, ..., RegExp.$9`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/Symbol.species · MDN page last modified 2025-10-01

Baseline: Widely available — available across major browsers since September 2016.

The **`RegExp[Symbol.species]`** static accessor property returns the constructor used to construct copied regular expressions in certain `RegExp` methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
RegExp[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct copied `RegExp` instances.

Description

The `[Symbol.species]` accessor property returns the default constructor for `RegExp` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class RegExp {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubRegExp extends RegExp {}
SubRegExp[Symbol.species] === SubRegExp; // true
```

Some `RegExp` methods create a copy of the current regex instance before running `exec()`, so that side effects such as changes to `lastIndex` are not retained. The `[Symbol.species]` property is used to determine the constructor of the new instance. The methods that copy the current regex instance are:

- `[Symbol.matchAll]()`
- `[Symbol.split]()`

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `RegExp` constructor for `RegExp` objects:

```js
RegExp[Symbol.species]; // function RegExp()
```

Species in derived objects

In an instance of a custom `RegExp` subclass, such as `MyRegExp`, the `MyRegExp` species is the `MyRegExp` constructor. However, you might want to overwrite this, in order to return parent `RegExp` objects in your derived class methods:

```js
class MyRegExp extends RegExp {
  // Overwrite MyRegExp species to the parent RegExp constructor
  static get [Symbol.species]() {
    return RegExp;
  }
}
```

Or you can use this to observe the copying process:

```js
class MyRegExp extends RegExp {
  constructor(...args) {
    console.log("Creating a new MyRegExp instance with args:", args);
    super(...args);
  }
  static get [Symbol.species]() {
    console.log("Copying MyRegExp");
    return this;
  }
  exec(value) {
    console.log("Executing with lastIndex:", this.lastIndex);
    return super.exec(value);
  }
}

Array.from("aabbccdd".matchAll(new MyRegExp("[ac]", "g")));
// Creating a new MyRegExp instance with args: [ '[ac]', 'g' ]
// Copying MyRegExp
// Creating a new MyRegExp instance with args: [ MyRegExp /[ac]/g, 'g' ]
// Executing with lastIndex: 0
// Executing with lastIndex: 1
// Executing with lastIndex: 2
// Executing with lastIndex: 5
// Executing with lastIndex: 6
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp-%symbol.species%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.species]	51	13	49	38	10	51	49	41	10	5.0	51	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.@@species`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp`
- `Symbol.species`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.compile()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/compile · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** The `compile()` method is only specified for compatibility reasons. Using `compile()` causes the otherwise immutable regex source and flags to become mutable, which may break user expectations. You can use the `RegExp()` constructor to construct a new regular expression object instead.

The **`compile()`** method of `RegExp` instances is used to recompile a regular expression with new source and flags after the `RegExp` object has already been created.

Syntax

```
compile(pattern, flags)
```

Parameters

`pattern`

The text of the regular expression.

`flags`

Any combination of flag values.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if the `this` value is not an instance of the current realm's `RegExp` constructor. This includes a subclass of `RegExp` and the `RegExp` constructor from a different realm.

Examples

Using compile()

The following example shows how to recompile a regular expression with a new pattern and a new flag.

```js
const regexObj = /foo/gi;
regexObj.compile("new foo", "g");
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-regexp.prototype.compile`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`compile` Deprecated	1	12	1	6	3.1	18	4	10.1	2	1.0	4.4	2	1.0.0	1.0	0.10.0
Throws `TypeError` when used in `RegExp` subclass or with cross-realm mismatch. Deprecated Non-standard	**No**	**No**	148	**No**	15.6	**No**	148	**No**	15.6	**No**	**No**	15.6	**No**	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.compile`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `RegExp`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.exec()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/exec · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`exec()`** method of `RegExp` instances executes a search with this regular expression for a match in a specified string and returns a result array, or `null`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.exec()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex = /fo+/g;
const str = "table football, foosball";
let array;

while ((array = regex.exec(str)) !== null) {
  console.log(`Found ${array[0]}. Next starts at ${regex.lastIndex}.`);
  // Expected output: "Found foo. Next starts at 9."
  // Expected output: "Found foo. Next starts at 19."
}
```

Syntax

js

```
exec(str)
```

Parameters

`str`

The string against which to match the regular expression. All values are coerced to strings, so omitting it or passing `undefined` causes `exec()` to search for the string `"undefined"`, which is rarely what you want.

Return value

If the match fails, the `exec()` method returns `null`, and sets the regex's `lastIndex` to `0`.

If the match succeeds, the `exec()` method returns an array and updates the `lastIndex` property of the regular expression object. The returned array has the matched text as the first item, and then one item for each capturing group of the matched text. The array also has the following additional properties:

`index`

The 0-based index of the match in the string.

`input`

The original string that was matched against.

`groups`

A `null`-prototype object of named capturing groups, whose keys are the names, and values are the capturing groups, or `undefined` if no named capturing groups were defined. See capturing groups for more information.

`indices` Optional

This property is only present when the `d` flag is set. It is an array where each entry represents the bounds of a substring match. The index of each element in this array corresponds to the index of the respective substring match in the array returned by `exec()`. In other words, the first `indices` entry represents the entire match, the second `indices` entry represents the first capturing group, etc. Each entry itself is a two-element array, where the first number represents the match's start index, and the second number, its end index.

The `indices` array additionally has a `groups` property, which holds a `null`-prototype object of all named capturing groups. The keys are the names of the capturing groups, and each value is a two-element array, with the first number being the start index, and the second number being the end index of the capturing group. If the regular expression doesn't contain any named capturing groups, `groups` is `undefined`.

Description

JavaScript `RegExp` objects are *stateful* when they have the global or sticky flags set (e.g., `/foo/g` or `/foo/y`). They store a `lastIndex` from the previous match. Using this internally, `exec()` can be used to iterate over multiple matches in a string of text (with capture groups), as opposed to getting just the matching strings with `String.prototype.match()`.

When using `exec()`, the global flag has no effect when the sticky flag is set — the match is always sticky.

`exec()` is the primitive method of regexps. Many other regexp methods call `exec()` internally — including those called by string methods, like `[Symbol.replace]()`. While `exec()` itself is powerful (and is the most efficient), it often does not convey the intent most clearly.

- If you only care whether the regex matches a string, but not what is actually being matched, use `RegExp.prototype.test()` instead.
- If you are finding all occurrences of a global regex and you don't care about information like capturing groups, use `String.prototype.match()` instead. In addition, `String.prototype.matchAll()` helps to simplify matching multiple parts of a string (with capture groups) by allowing you to iterate over the matches.
- If you are executing a match to find its index position in the string, use the `String.prototype.search()` method instead.

`exec()` is useful for complex operations that cannot be easily achieved via any of the methods above, often when you need to manually adjust `lastIndex`. (`String.prototype.matchAll()` copies the regex, so changing `lastIndex` while iterating over `matchAll` does not affect the iteration.) For one such example, see rewinding `lastIndex`.

Examples

Using exec()

Consider the following example:

```js
// Match "quick brown" followed by "jumps", ignoring characters in between
// Remember "brown" and "jumps"
// Ignore case
const re = /quick\s(?<color>brown).+?(jumps)/dgi;
const result = re.exec("The Quick Brown Fox Jumps Over The Lazy Dog");
```

The following table shows the state of `result` after running this script:

Property	Value
`[0]`	`"Quick Brown Fox Jumps"`
`[1]`	`"Brown"`
`[2]`	`"Jumps"`
`index`	`4`
`indices`	`[[4, 25], [10, 15], [20, 25]]` `groups: { color: [10, 15 ]}`
`input`	`"The Quick Brown Fox Jumps Over The Lazy Dog"`
`groups`	`{ color: "Brown" }`

In addition, `re.lastIndex` will be set to `25`, due to this regex being global.

Finding successive matches

If your regular expression uses the `g` flag, you can use the `exec()` method multiple times to find successive matches in the same string. When you do so, the search starts at the substring of `str` specified by the regular expression's `lastIndex` property (`test()` will also advance the `lastIndex` property). Note that the `lastIndex` property will not be reset when searching a different string, it will start its search at its existing `lastIndex`.

For example, assume you have this script:

```js
const myRe = /ab*/g;
const str = "abbcdefabh";
let myArray;
while ((myArray = myRe.exec(str)) !== null) {
  let msg = `Found ${myArray[0]}. `;
  msg += `Next match starts at ${myRe.lastIndex}`;
  console.log(msg);
}
```

This script displays the following text:

```
Found abb. Next match starts at 3
Found ab. Next match starts at 9
```

> **Warning:** There are many pitfalls that can lead to this becoming an infinite loop!
>
> - Do *not* place the regular expression literal (or `RegExp` constructor) within the `while` condition — it will recreate the regex for every iteration and reset `lastIndex`.
> - Be sure that the global (`g`) flag is set, or `lastIndex` will never be advanced.
> - If the regex may match zero-length characters (e.g., `/^/gm`), increase its `lastIndex` manually each time to avoid being stuck in the same place.

You can usually replace this kind of code with `String.prototype.matchAll()` to make it less error-prone.

Using exec() with RegExp literals

You can also use `exec()` without creating a `RegExp` object explicitly:

```js
const matches = /(hello \S+)/.exec("This is a hello world!");
console.log(matches[1]);
```

This will log a message containing `'hello world!'`.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.prototype.exec

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
exec	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.exec`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `RegExp.prototype.exec` with support for named capturing groups, `dotAll`, and `sticky` in `core-js`
- Regular expressions guide
- `RegExp`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.test()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/test · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`test()`** method of `RegExp` instances executes a search with this regular expression for a match between a regular expression and a specified string. Returns `true` if there is a match; `false` otherwise.

JavaScript `RegExp` objects are **stateful** when they have the `global` or `sticky` flags set (e.g., `/foo/g` or `/foo/y`). They store a `lastIndex` from the previous match. Using this internally, `test()` can be used to iterate over multiple matches in a string of text (with capture groups).

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.test()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const str = "table football";

const regex = /fo+/;
const globalRegex = /fo+/g;

console.log(regex.test(str));
// Expected output: true

console.log(globalRegex.lastIndex);
// Expected output: 0

console.log(globalRegex.test(str));
// Expected output: true

console.log(globalRegex.lastIndex);
// Expected output: 9

console.log(globalRegex.test(str));
// Expected output: false
```

Syntax

js

```
test(str)
```

Parameters

`str`

The string against which to match the regular expression. All values are coerced to strings, so omitting it or passing `undefined` causes `test()` to search for the string `"undefined"`, which is rarely what you want.

Return value

`true` if there is a match between the regular expression and the string `str`. Otherwise, `false`.

Description

Use `test()` whenever you want to know whether a pattern is found in a string. `test()` returns a boolean, unlike the `String.prototype.search()` method (which returns the index of a match, or `-1` if not found).

To get more information (but with slower execution), use the `exec()` method. (This is similar to the `String.prototype.match()` method.)

As with `exec()` (or in combination with it), `test()` called multiple times on the same global regular expression instance will advance past the previous match.

Examples

Using test()

This example tests if "hello" is contained at the very beginning of a string, returning a boolean result.

```js
const str = "hello world!";
const result = /^hello/.test(str);

console.log(result); // true
```

The following example logs a message which depends on the success of the test:

```js
function testInput(re, str) {
  const midString = re.test(str) ? "contains" : "does not contain";
  console.log(`${str} ${midString} ${re.source}`);
}
```

Using test() on a regex with the "global" flag

When a regex has the global flag set, `test()` will advance the `lastIndex` of the regex. (`RegExp.prototype.exec()` also advances the `lastIndex` property.)

Further calls to `test(str)` will resume searching `str` starting from `lastIndex`. The `lastIndex` property will continue to increase each time `test()` returns `true`.

> **Note:** As long as `test()` returns `true`, `lastIndex` will *not* reset—even when testing a different string!

When `test()` returns `false`, the calling regex's `lastIndex` property will reset to `0`.

The following example demonstrates this behavior:

```js
const regex = /foo/g; // the "global" flag is set

// regex.lastIndex is at 0
regex.test("foo"); // true

// regex.lastIndex is now at 3
regex.test("foo"); // false

// regex.lastIndex is at 0
regex.test("barfoo"); // true

// regex.lastIndex is at 6
regex.test("foobar"); // false

// regex.lastIndex is at 0
regex.test("foobarfoo"); // true

// regex.lastIndex is at 3
regex.test("foobarfoo"); // true
```

(code continues on the next page)

(code continued from the previous page)

```
// regex.lastIndex is at 9
regex.test("foobarfoo"); // false

// regex.lastIndex is at 0
// (...and so on)
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.prototype.test

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
test	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.test`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `RegExp.prototype.test` with support for named capturing groups, `dotAll`, and `sticky` in `core-js`
- Regular expressions guide
- `RegExp`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/toString · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`toString()`** method of `RegExp` instances returns a string representing this regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.toString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(new RegExp("a+b+c"));
// Expected output: /a+b+c/

console.log(new RegExp("a+b+c").toString());
// Expected output: "/a+b+c/"

console.log(new RegExp("bar", "g").toString());
// Expected output: "/bar/g"

console.log(new RegExp("\n", "g").toString());
// Expected output: "/\n/g"

console.log(new RegExp("\\n", "g").toString());
// Expected output: "/\n/g"
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

A string representing the given object.

Description

The `RegExp` object overrides the `toString()` method of the `Object` object; it does not inherit `Object.prototype.toString()`. For `RegExp` objects, the `toString()` method returns a string representation of the regular expression.

In practice, it reads the regex's `source` and `flags` properties and returns a string in the form `/source/flags`. The `toString()` return value is guaranteed to be a parsable regex literal, although it may not be the exact same text as what was originally specified for the regex (for example, the flags may be reordered).

Examples

Using toString()

The following example displays the string value of a `RegExp` object:

js

```
const myExp = new RegExp("a+b+c");
console.log(myExp.toString()); // '/a+b+c/'

const foo = new RegExp("bar", "g");
console.log(foo.toString()); // '/bar/g'
```

Empty regular expressions and escaping

Since `toString()` accesses the `source` property, an empty regular expression returns the string `"/(?:)/"`, and line terminators such as `\n` are escaped. This makes the returned value always a valid regex literal.

```js
new RegExp().toString(); // "/(?:)/"

new RegExp("\n").toString() === "/\\n/"; // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Line breaks and slashes are escaped	73	12	38	60	6	73	38	52	6	11.0	73	6	1.0.0	1.0	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.toString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of modern `RegExp.prototype.toString` behavior in `core-js`
- `Object.prototype.toString()`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype[Symbol.match]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/Symbol.match · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`[Symbol.match]()`** method of `RegExp` instances specifies how `String.prototype.match()` should behave. In addition, its presence (or absence) can influence whether an object is regarded as a regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype[Symbol.match]()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class RegExp1 extends RegExp {
  [Symbol.match](str) {
    const result = RegExp.prototype[Symbol.match].call(this, str);
    if (result) {
      return "VALID";
    }
    return "INVALID";
  }
}

console.log("2012-07-02".match(new RegExp1("(\\d+)-(\\d+)-(\\d+)")));
// Expected output: "VALID"
```

Syntax

js

```
regexp[Symbol.match](str)
```

Parameters

`str`

A `String` that is a target of the match.

Return value

An `Array` whose contents depend on the presence or absence of the global (g) flag, or `null` if no matches are found.

- If the g flag is used, all results matching the complete regular expression will be returned, but capturing groups are not included.
- If the g flag is not used, only the first complete match and its related capturing groups are returned. In this case, `match()` will return the same result as `RegExp.prototype.exec()` (an array with some extra properties).

Description

This method exists for customizing match behavior within `RegExp` subclasses. It is called internally in `String.prototype.match()`. For example, the following two examples return the same result.

js

```
"abc".match(/a/);

/a/[Symbol.match]("abc");
```

If the regex is global (with the g flag), its `lastIndex` is first set to 0, so matching always starts from the beginning of the string, and the regex's `exec()` method is repeatedly called until `exec()` returns `null`. If the current match is an empty string, the `lastIndex` would still be advanced — if the regex is Unicode-aware, it would advance by one Unicode code point; otherwise, it advances by one UTF-16 code unit.

```js
console.log("😄".match(/(?:)/g)); // [ '', '', '' ]
console.log("😄".match(/(?:)/gu)); // [ '', '' ]
```

If the regex is not global, `exec()` would only be called once and its result becomes the return value of `[Symbol.match]()`.

The `exec()` method automatically resets `lastIndex` to 0 when the last match fails, so for global regexes with `lastIndex` starting at 0, `[Symbol.match]()` generally produces no side-effects. However, when the regex is sticky but not global, `exec()` is only called once and therefore does not reset `lastIndex` if the match was successful. In this case, each call to `match()` may return a different result.

```js
const re = /[abc]/y;
for (let i = 0; i < 5; i++) {
  console.log("abc".match(re), re.lastIndex);
}
// [ 'a' ] 1
// [ 'b' ] 2
// [ 'c' ] 3
// null 0
// [ 'a' ] 1
```

When the regex is sticky and global, it would still perform sticky matches — i.e., it would fail to match any occurrences beyond the `lastIndex`.

```js
console.log("ab-c".match(/[abc]/gy)); // [ 'a', 'b' ]
```

In addition, the `[Symbol.match]` property is used to check whether an object is a regular expression.

Examples

Direct call

This method can be used in *almost* the same way as `String.prototype.match()`, except the different `this` and the different arguments order.

```js
const re = /\d+/g;
const str = "2016-01-02";
const result = re[Symbol.match](str);
console.log(result); // ["2016", "01", "02"]
```

Using `[Symbol.match]()` in subclasses

Subclasses of `RegExp` can override the `[Symbol.match]()` method to modify the default behavior.

```js
class MyRegExp extends RegExp {
  [Symbol.match](str) {
    const result = RegExp.prototype[Symbol.match].call(this, str);
    if (!result) return null;
    return {
      group(n) {
        return result[n];
      },
```

(code continues on the next page)

(code continued from the previous page)

```
    };
  }
}

const re = new MyRegExp("(\\d+)-(\\d+)-(\\d+)");
const str = "2016-01-02";
const result = str.match(re); // String.prototype.match calls re[Symbol.match]().
console.log(result.group(1)); // 2016
console.log(result.group(2)); // 01
console.log(result.group(3)); // 02
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.prototype-%symbol.match%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.match]	50	13	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.@@match`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `RegExp.prototype[Symbol.match]` in `core-js`
- `String.prototype.match()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype[Symbol.split]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.match`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype[Symbol.matchAll]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/Symbol.matchAll · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`[Symbol.matchAll]()`** method of `RegExp` instances specifies how `String.prototype.matchAll` should behave.

Try it

▷ Interactive example on MDN: "JavaScript Demo: RegExp.prototype[Symbol.matchAll]()". It runs live in the browser there; its source code follows.

js · interactive example source

```
class MyRegExp extends RegExp {
  [Symbol.matchAll](str) {
    const result = RegExp.prototype[Symbol.matchAll].call(this, str);
    if (!result) {
      return null;
    }
    return Array.from(result);
  }
}

const re = new MyRegExp("-\\d+", "g");
console.log("2016-01-02|2019-03-07".matchAll(re));
// Expected output: Array [Array ["-01"], Array ["-02"], Array ["-03"], Array ["-07"]]
```

Syntax

js

```
regexp[Symbol.matchAll](str)
```

Parameters

`str`

A `String` that is a target of the match.

Return value

An iterable iterator object (which is not restartable) of matches. Each match is an array with the same shape as the return value of `RegExp.prototype.exec()`.

Description

This method exists for customizing the behavior of `matchAll()` in `RegExp` subclasses. It is called internally in `String.prototype.matchAll()`. For example, the following two examples return the same result.

js

```
"abc".matchAll(/a/g);

/a/g[Symbol.matchAll]("abc");
```

Like `[Symbol.split]()`, `[Symbol.matchAll]()` starts by using `[Symbol.species]` to construct a new regex, thus avoiding mutating the original regexp in any way. The constructor receives `this` and the original flags. `lastIndex` starts as the original regex's value.

```js
const regexp = /[a-c]/g;
regexp.lastIndex = 1;
const str = "abc";
Array.from(str.matchAll(regexp), (m) => `${regexp.lastIndex} ${m[0]}`);
// [ "1 b", "1 c" ]
```

If the regex is global (with the `g` flag), each time the returned iterator's `next()` method is called, the regex's `exec()` is called and the result is yielded. If the current match is an empty string, the `lastIndex` will still be advanced. If the regex has the `u` flag, it advances by one Unicode code point; otherwise, it advances by one UTF-16 code point.

```js
console.log(Array.from("😄".matchAll(/(?:)/g)));
// [ [ "" ], [ "" ], [ "" ] ]

console.log(Array.from("😄".matchAll(/(?:)/gu)));
// [ [ "" ], [ "" ] ]
```

If the regex is not global, the returned iterator yields the `exec()` result once and then completes. (The validation that the input is a global regex happens in `String.prototype.matchAll()`. `[Symbol.matchAll]()` does not validate the flags of `this`.)

When the regex is sticky and global, it will still perform sticky matches — i.e., it will not match any occurrences beyond the `lastIndex`.

```js
console.log(Array.from("ab-c".matchAll(/[abc]/gy)));
// [ [ "a" ], [ "b" ] ]
```

Examples

Direct call

This method can be used in almost the same way as `String.prototype.matchAll()`, except for the different value of `this` and the different order of arguments.

```js
const re = /\d+/g;
const str = "2016-01-02";
const result = re[Symbol.matchAll](str);

console.log(Array.from(result, (x) => x[0]));
// [ "2016", "01", "02" ]
```

Using `[Symbol.matchAll]()` in subclasses

Subclasses of `RegExp` can override the `[Symbol.matchAll]()` method to modify the default behavior.

For example, to return an `Array` instead of an iterator:

```js
class MyRegExp extends RegExp {
  [Symbol.matchAll](str) {
    const result = RegExp.prototype[Symbol.matchAll].call(this, str);
    return result ? Array.from(result) : null;
  }
}

const re = new MyRegExp("(\\d+)-(\\d+)-(\\d+)", "g");
const str = "2016-01-02|2019-03-07";
const result = str.matchAll(re);

console.log(result[0]);
// [ "2016-01-02", "2016", "01", "02" ]

console.log(result[1]);
// [ "2019-03-07", "2019", "03", "07" ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp-prototype-%symbol.matchall%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.matchAll]	73	79	67	60	13	73	67	52	13	5.0	73	13	1.0.0	1.0	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.@@matchAll`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `RegExp.prototype[Symbol.matchAll]` in `core-js`
- es-shims polyfill of `RegExp.prototype[Symbol.matchAll]`
- `String.prototype.matchAll()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype[Symbol.split]()`
- `Symbol.matchAll`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype[Symbol.replace]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/Symbol.replace · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`[Symbol.replace]()`** method of `RegExp` instances specifies how `String.prototype.replace()` and `String.prototype.replaceAll()` should behave when the regular expression is passed in as the pattern.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype[Symbol.replace]()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class RegExp1 extends RegExp {
  [Symbol.replace](str) {
    return RegExp.prototype[Symbol.replace].call(this, str, "#!@?");
  }
}

console.log("football".replace(new RegExp1("foo")));
// Expected output: "#!@?tball"
```

Syntax

js

```
regexp[Symbol.replace](str, replacement)
```

Parameters

`str`

A `String` that is a target of the replacement.

`replacement`

Can be a string or a function.

- If it's a string, it will replace the substring matched by the current regexp. A number of special replacement patterns are supported; see the Specifying a string as the replacement section of `String.prototype.replace`.
- If it's a function, it will be invoked for every match and the return value is used as the replacement text. The arguments supplied to this function are described in the Specifying a function as the replacement section of `String.prototype.replace`.

Return value

A new string, with one, some, or all matches of the pattern replaced by the specified replacement.

Description

This method exists for customizing replace behavior in `RegExp` subclasses. It is called internally in `String.prototype.replace()` and `String.prototype.replaceAll()` if the `pattern` argument is a `RegExp` object. For example, the following two examples return the same result.

```js
"abc".replace(/a/, "A");

/a/[Symbol.replace]("abc", "A");
```

If the regex is global (with the g flag), its `lastIndex` is first set to 0, so matching always starts from the beginning of the string, and the regex's `exec()` method is repeatedly called until `exec()` returns `null`. If the current match is an empty string, the `lastIndex` would still be advanced — if the regex is Unicode-aware, it would advance by one Unicode code point; otherwise, it advances by one UTF-16 code unit.

```js
console.log("😄".replace(/(?:)/g, " ")); // " \ud83d \ude04 "
console.log("😄".replace(/(?:)/gu, " ")); // " 😄 "
```

If the regex is not global, `exec()` would only be called once.

Substitution happens after all matching substrings are identified. For each successful `exec()` result, a substitution string is created based on the `replacement` argument, the process of which is described in `String.prototype.replace()`.

The `exec()` method automatically resets `lastIndex` to 0 when the last match fails, so for global regexes with `lastIndex` starting at 0, `[Symbol.replace]()` generally produces no side-effects. However, when the regex is sticky but not global, `exec()` is only called once and therefore does not reset `lastIndex` if the match was successful. In this case, each call to `replace()` may return a different result.

```js
const re = /a/y;

for (let i = 0; i < 5; i++) {
  console.log("aaa".replace(re, "b"), re.lastIndex);
}

// baa 1
// aba 2
// aab 3
// aaa 0
// baa 1
```

When the regex is sticky and global, it would still perform sticky matches — i.e., it would fail to match any occurrences beyond the `lastIndex`.

```js
console.log("aa-a".replace(/a/gy, "b")); // "bb-a"
```

Examples

Direct call

This method can be used in almost the same way as `String.prototype.replace()`, except the different `this` and the different arguments order.

```js
const re = /-/g;
const str = "2016-01-01";
const newStr = re[Symbol.replace](str, ".");
console.log(newStr); // 2016.01.01
```

Using `[Symbol.replace]()` in subclasses

Subclasses of `RegExp` can override the `[Symbol.replace]()` method to modify the default behavior.

```js
class MyRegExp extends RegExp {
  constructor(pattern, flags, count) {
    super(pattern, flags);
    this.count = count;
  }
  [Symbol.replace](str, replacement) {
    // Perform [Symbol.replace]() `count` times.
    let result = str;
    for (let i = 0; i < this.count; i++) {
      result = RegExp.prototype[Symbol.replace].call(this, result, replacement);
    }
    return result;
  }
}

const re = new MyRegExp("\\d", "", 3);
const str = "01234567";
const newStr = str.replace(re, "#"); // String.prototype.replace calls re[Symbol.replace]().
console.log(newStr); // ###34567
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.prototype-%symbol.replace%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.replace]	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.@@replace`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `RegExp.prototype[Symbol.replace]` in `core-js`
- `String.prototype.replace()`
- `String.prototype.replaceAll()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype[Symbol.split]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.replace`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype[Symbol.search]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/Symbol.search · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`[Symbol.search]()`** method of `RegExp` instances specifies how `String.prototype.search` should behave.

Try it

▷ Interactive example on MDN: "JavaScript Demo: RegExp.prototype[Symbol.search]()". It runs live in the browser there; its source code follows.

js · interactive example source

```
class RegExp1 extends RegExp {
  constructor(str) {
    super(str);
    this.pattern = str;
  }
  [Symbol.search](str) {
    return str.indexOf(this.pattern);
  }
}

console.log("table football".search(new RegExp1("foo")));
// Expected output: 6
```

Syntax

js

```
regexp[Symbol.search](str)
```

Parameters

`str`

A `String` that is a target of the search.

Return value

The index of the first match between the regular expression and the given string, or `-1` if no match was found.

Description

This method exists for customizing the search behavior in `RegExp` subclasses. It is called internally in `String.prototype.search()`. For example, the following two examples return the same result.

```js
"abc".search(/a/);

/a/[Symbol.search]("abc");
```

`[Symbol.search]()` always calls the regex's `exec()` method exactly once, and returns the `index` property of the result, or `-1` if the result is `null`. The `g` flag has no effect with this method.

This method does not copy the regular expression, unlike `[Symbol.split]()` or `[Symbol.matchAll]()`. However, unlike `[Symbol.match]()` or `[Symbol.replace]()`, it will always set `lastIndex` to 0 when execution starts and restore it to the previous value when it exits, therefore generally avoiding side effects. This means that it always returns the first match in the string even when `lastIndex` is non-zero, and sticky regexes always search strictly at the beginning of the string.

```js
const re = /[abc]/g;
re.lastIndex = 2;
console.log("abc".search(re)); // 0

const re2 = /[bc]/y;
re2.lastIndex = 1;
console.log("abc".search(re2)); // -1
console.log("abc".match(re2)); // [ 'b' ]
```

Examples

Direct call

This method can be used in almost the same way as `String.prototype.search()`, except for the different value of `this` and the different arguments order.

```js
const re = /-/g;
const str = "2016-01-02";
const result = re[Symbol.search](str);
console.log(result); // 4
```

Using `[Symbol.search]()` in subclasses

Subclasses of `RegExp` can override `[Symbol.search]()` method to modify the behavior.

```js
class MyRegExp extends RegExp {
  constructor(str) {
    super(str);
    this.pattern = str;
  }
  [Symbol.search](str) {
    return str.indexOf(this.pattern);
  }
}

const re = new MyRegExp("a+b");
const str = "ab a+b";
const result = str.search(re); // String.prototype.search calls re[Symbol.search]().
console.log(result); // 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.prototype-%symbol.search%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.search]	50	13	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.@@search`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `RegExp.prototype[Symbol.search]` in `core-js`
- `String.prototype.search()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.split]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.search`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype[Symbol.split]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/Symbol.split · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`[Symbol.split]()`** method of `RegExp` instances specifies how `String.prototype.split` should behave when the regular expression is passed in as the separator.

Try it

▷ Interactive example on MDN: "JavaScript Demo: RegExp.prototype[Symbol.split]()". It runs live in the browser there; its source code follows.

js · interactive example source

```
class RegExp1 extends RegExp {
  [Symbol.split](str, limit) {
    const result = RegExp.prototype[Symbol.split].call(this, str, limit);
    return result.map((x) => `(${x})`);
  }
}

console.log("2016-01-02".split(new RegExp1("-")));
// Expected output: Array ["(2016)", "(01)", "(02)"]

console.log("2016-01-02".split(/-/));
// Expected output: Array ["2016", "01", "02"]
```

Syntax

js

```
regexp[Symbol.split](str)
regexp[Symbol.split](str, limit)
```

Parameters

`str`

The target of the split operation.

`limit` Optional

Integer specifying a limit on the number of splits to be found. The `[Symbol.split]()` method still splits on every match of `this` RegExp pattern (or, in the Syntax above, `regexp`), until the number of split items match the `limit` or the string falls short of `this` pattern.

Return value

An `Array` containing substrings as its elements. Capturing groups are included.

Description

This method exists for customizing the behavior of `split()` in `RegExp` subclasses. It is called internally in `String.prototype.split()` when a `RegExp` is passed as the separator. For example, the following two examples return the same result.

```js
"a-b-c".split(/-/);

/-/[Symbol.split]("a-b-c");
```

Like `[Symbol.matchAll]()`, `[Symbol.split]()` starts by using `[Symbol.species]` to construct a new regex, thus avoiding mutating the original regexp in any way. The constructor receives `this` and the original flags, plus the `y` ("sticky") flag if it was not originally present. The `g` ("global") flag is irrelevant for the method's behavior. By default, due to the `RegExp()` constructor's behavior, `lastIndex` starts as 0.

If the target string is empty, and the regexp can match empty strings (for example, `/a?/`), an empty array is returned. Otherwise, if the regexp can't match an empty string, `[""]` is returned.

The regex's `exec()` method is repeatedly called, advancing the `lastIndex` each time, until it is at the end of the string. If the current match is an empty string, or if the regexp doesn't match at the current position (since it's sticky), the `lastIndex` would still be advanced — if the regex is Unicode-aware, it would advance by one Unicode code point; otherwise, it advances by one UTF-16 code unit.

```js
console.log("😄".split(/(?:)/g)); // [ '\ud83d', '\ude04' ]
console.log("😄".split(/(?:)/gu)); // [ '😄' ]
```

For each match, the substring between the last matched string's end and the current matched string's beginning is first appended to the result array. Then, the capturing groups' values are appended one-by-one. The returned array's length will never exceed the `limit` parameter, if provided, while trying to be as close as possible. Therefore, the last match and its capturing groups may not all be present in the returned array if the array is already filled.

If there was no successful match anywhere in the string, the target string is returned as-is, wrapped in an array.

Examples

Direct call

This method can be used in almost the same way as `String.prototype.split()`, except the different `this` and the different order of arguments.

```js
const re = /-/g;
const str = "2016-01-02";
const result = re[Symbol.split](str);
console.log(result); // ["2016", "01", "02"]
```

Using `[Symbol.split]()` in subclasses

Subclasses of `RegExp` can override the `[Symbol.split]()` method to modify the default behavior.

```js
class MyRegExp extends RegExp {
  [Symbol.split](str, limit) {
    const result = RegExp.prototype[Symbol.split].call(this, str, limit);
    return result.map((x) => `(${x})`);
  }
}

const re = new MyRegExp("-");
const str = "2016-01-02";
const result = str.split(re); // String.prototype.split calls re[Symbol.split]().
console.log(result); // ["(2016)", "(01)", "(02)"]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-regexp.prototype-%symbol.split%

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.split]	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.@@split`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `RegExp.prototype[Symbol.split]` in `core-js`
- `String.prototype.split()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.split`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.dotAll

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/dotAll · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2020.

The **`dotAll`** accessor property of RegExp instances returns whether or not the `s` flag is used with this regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.dotAll". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex1 = /f.o/s;

console.log(regex1.dotAll);
// Expected output: true

const regex2 = /bar/;

console.log(regex2.dotAll);
// Expected output: false
```

Description

`RegExp.prototype.dotAll` has the value `true` if the `s` flag was used; otherwise, `false`. The `s` flag indicates that the dot special character (`.`) should additionally match the following line terminator ("newline") characters in a string, which it would not match otherwise:

- U+000A LINE FEED (LF) (`\n`)
- U+000D CARRIAGE RETURN (CR) (`\r`)
- U+2028 LINE SEPARATOR
- U+2029 PARAGRAPH SEPARATOR

This effectively means the dot will match any UTF-16 code unit. However, it will *not* match characters that are outside of the Unicode Basic Multilingual Plane (BMP), also known as astral characters, which are represented as surrogate pairs and necessitate matching with two `.` patterns instead of one.

js

```
"😄".match(/(.)(.)/s);
// Array(3) [ "😄", "\ud83d", "\ude04" ]
```

The `u` (unicode) flag can be used to allow the dot to match astral characters as a single character.

js

```
"😄".match(/./su);
// Array [ "😄" ]
```

Note that a pattern such as `.*` is still capable of *consuming* astral characters as part of a larger context, even without the `u` flag.

```js
"😊".match(/.*/s);
// Array [ "😊" ]
```

Using both the `s` and `u` flags in conjunction allows the dot to match any Unicode character in a more intuitive manner.

The set accessor of `dotAll` is `undefined`. You cannot change this property directly.

Examples

Using dotAll

```js
const str1 = "bar\nexample foo example";

const regex1 = /bar.example/s;

console.log(regex1.dotAll); // true

console.log(str1.replace(regex1, "")); // foo example

const str2 = "bar\nexample foo example";

const regex2 = /bar.example/;

console.log(regex2.dotAll); // false

console.log(str2.replace(regex2, ""));
// bar
// example foo example
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.dotAll

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
dotAll	62	79	78	49	11.1	62	79	46	11.3	8.0	62	11.3	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.dotAll`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of the `dotAll` flag in `core-js`
- `RegExp.prototype.lastIndex`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.flags

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/flags · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`flags`** accessor property of `RegExp` instances returns the flags of this regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.flags". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Outputs RegExp flags in alphabetical order

console.log(/foo/gi.flags);
// Expected output: "gi"

console.log(/^bar/muy.flags);
// Expected output: "muy"
```

Description

`RegExp.prototype.flags` has a string as its value. Flags in the `flags` property are sorted alphabetically (from left to right, e.g., `"dgimsuvy"`). It actually invokes the other flag accessors (`hasIndices`, `global`, etc.) one-by-one and concatenates the results.

All built-in functions read the `flags` property instead of reading individual flag accessors.

The set accessor of `flags` is `undefined`. You cannot change this property directly.

Examples

Using flags

js

```
/foo/ig.flags; // "gi"
/^bar/myu.flags; // "muy"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.flags`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`flags`	49	79	37	39	9	49	37	41	9	5.0	49	9	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.flags`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `RegExp.prototype.flags` in `core-js`
- es-shims polyfill of `RegExp.prototype.flags`
- Advanced searching with flags in the Regular expressions guide
- `RegExp.prototype.source`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.global

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/global · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`global`** accessor property of `RegExp` instances returns whether or not the g flag is used with this regular expression.

Try it

▷ Interactive example on MDN: "JavaScript Demo: RegExp.prototype.global". It runs live in the browser there; its source code follows.

js · interactive example source

```
const regex1 = /foo/g;

console.log(regex1.global);
// Expected output: true

const regex2 = /bar/i;

console.log(regex2.global);
// Expected output: false
```

Description

`RegExp.prototype.global` has the value `true` if the g flag was used; otherwise, `false`. The g flag indicates that the regular expression should be tested against all possible matches in a string. Each call to `exec()` will update its `lastIndex` property, so that the next call to `exec()` will start at the next character.

Some methods, such as `String.prototype.matchAll()` and `String.prototype.replaceAll()`, will validate that, if the parameter is a regex, it is global. The regex's `[Symbol.match]()` and `[Symbol.replace]()` methods (called by `String.prototype.match()` and `String.prototype.replace()`) would also have different behaviors when the regex is global.

The set accessor of `global` is `undefined`. You cannot change this property directly.

Examples

Using global

```js
const globalRegex = /foo/g;

const str = "fooexamplefoo";
console.log(str.replace(globalRegex, "")); // example

const nonGlobalRegex = /foo/;
console.log(str.replace(nonGlobalRegex, "")); // examplefoo
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.global`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`global`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Prototype accessor property (ES2015)	48	12	38	35	1.3	48	38	35	1	5.0	48	1	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.global`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.hasIndices`

- RegExp.prototype.ignoreCase
- RegExp.prototype.multiline
- RegExp.prototype.source
- RegExp.prototype.sticky
- RegExp.prototype.unicode

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.hasIndices

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/hasIndices · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`hasIndices`** accessor property of RegExp instances returns whether or not the `d` flag is used with this regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.hasIndices". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex1 = /foo/d;

console.log(regex1.hasIndices);
// Expected output: true

const regex2 = /bar/;

console.log(regex2.hasIndices);
// Expected output: false
```

Description

`RegExp.prototype.hasIndices` has the value `true` if the `d` flag was used; otherwise, `false`. The `d` flag indicates that the result of a regular expression match should contain the start and end indices of the substrings of each capture group. It does not change the regex's interpretation or matching behavior in any way, but only provides additional information in the matching result.

This flag primarily affects the return value of `exec()`. If the `d` flag is present, the array returned by `exec()` has an additional `indices` property as described in the `exec()` method's return value. Because all other regex-related methods (such as `String.prototype.match()`) call `exec()` internally, they will also return the indices if the regex has the `d` flag.

The set accessor of `hasIndices` is `undefined`. You cannot change this property directly.

Examples

There's a more detailed usage example at Groups and backreferences > Using groups and match indices.

Using hasIndices

```js
const str1 = "foo bar foo";

const regex1 = /foo/dg;

console.log(regex1.hasIndices); // true

console.log(regex1.exec(str1).indices[0]); // [0, 3]
console.log(regex1.exec(str1).indices[0]); // [8, 11]

const str2 = "foo bar foo";

const regex2 = /foo/;

console.log(regex2.hasIndices); // false

console.log(regex2.exec(str2).indices); // undefined
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.hasIndices`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`hasIndices`	90	90	88	76	15	90	88	64	15	15.0	90	15	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.hasIndices`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.exec()`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.ignoreCase

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/ignoreCase · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`ignoreCase`** accessor property of `RegExp` instances returns whether or not the `i` flag is used with this regular expression.

Try it

▷ Interactive example on MDN: "JavaScript Demo: RegExp.prototype.ignoreCase". It runs live in the browser there; its source code follows.

js · interactive example source

```
const regex1 = /foo/;
const regex2 = /foo/i;

console.log(regex1.test("Football"));
// Expected output: false

console.log(regex2.ignoreCase);
// Expected output: true

console.log(regex2.test("Football"));
// Expected output: true
```

Description

`RegExp.prototype.ignoreCase` has the value `true` if the `i` flag was used; otherwise, `false`. The `i` flag indicates that case should be ignored while attempting a match in a string. Case-insensitive matching is done by mapping both the expected character set and the matched string to the same casing.

If the regex is Unicode-aware, the case mapping happens through *simple case folding* specified in `CaseFolding.txt`. The mapping always maps to a single code point, so it does not map, for example, ß (U+00DF LATIN SMALL LETTER SHARP S) to `ss` (which is *full case folding*, not *simple case folding*). It may however map code points outside the Basic Latin block to code points within it — for example, ſ (U+017F LATIN SMALL LETTER LONG S) case-folds to `s` (U+0073 LATIN SMALL LETTER S) and K (U+212A KELVIN SIGN) case-folds to `k` (U+006B LATIN SMALL LETTER K). Therefore, ſ and K can be matched by `/[a-z]/ui`.

If the regex is Unicode-unaware, case mapping uses the Unicode Default Case Conversion — the same algorithm used in `String.prototype.toUpperCase()`. This algorithm prevents code points outside the Basic Latin block to be mapped to code points within it, so ſ and K mentioned previously are not matched by `/[a-z]/i`.

Unicode-aware case folding generally folds to lower case, while Unicode-unaware case folding folds to upper case. These two are not perfect reverse operations, so there are some subtle behavior differences. For example, Ω (U+2126 OHM SIGN) and Ω (U+03A9 GREEK CAPITAL LETTER OMEGA) are both mapped by simple case folding to ω (U+03C9 GREEK SMALL LETTER OMEGA), so `"\u2126"` is matched by `/[\u03c9]/ui` and `/[\u03a9]/ui`; on the other hand, U+2126 is mapped by Default Case Conversion

to itself, while the other two both map to U+03A9, so `"\u2126"` is matched by neither `/[\u03c9]/i` nor `/[\u03a9]/i`.

The set accessor of `ignoreCase` is `undefined`. You cannot change this property directly.

Examples

Using ignoreCase

```js
const regex = /foo/i;

console.log(regex.ignoreCase); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.ignorecase

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ignoreCase`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Prototype accessor property (ES2015)	48	12	38	35	1.3	48	38	35	1	5.0	48	1	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.ignoreCase`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.multiline

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/multiline · MDN page last modified 2026-08-27

Baseline: Widely available — available across major browsers since July 2015.

The **`multiline`** accessor property of `RegExp` instances returns whether or not the `m` flag is used with this regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.multiline". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex1 = /^football/;
const regex2 = /^football/m;

console.log(regex1.multiline);
// Expected output: false

console.log(regex2.multiline);
// Expected output: true

console.log(regex1.test("rugby\nfootball"));
// Expected output: false

console.log(regex2.test("rugby\nfootball"));
// Expected output: true
```

Description

`RegExp.prototype.multiline` has the value `true` if the `m` flag was used; otherwise, `false`. The `m` flag indicates that a multiline input string should be treated as multiple lines. For example, if `m` is used, `^` and `$` change from matching at only the start or end of the entire string to the start or end of any line within the string.

> **Note:** To match the start and end of the entire string in `m` mode, use the buffer boundary assertions `\A`, `\z`, and `\Z`.

The set accessor of `multiline` is `undefined`. You cannot change this property directly.

Examples

Using multiline

js

```
const regex = /^foo/m;

console.log(regex.multiline); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.multiline`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`multiline`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Prototype accessor property (ES2015)	48	12	38	35	1.3	48	38	35	1	5.0	48	1	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.multiline`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.source

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/source · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`source`** accessor property of `RegExp` instances returns a string containing the source text of this regular expression, without the two forward slashes on both sides or any flags.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.source". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex = /fooBar/gi;

console.log(regex.source);
// Expected output: "fooBar"

console.log(new RegExp().source);
// Expected output: "(?:)"

console.log(new RegExp("\n").source === "\\n");
// Expected output: true (starting with ES5)
// Due to escaping
```

Description

Conceptually, the `source` property is the text between the two forward slashes in the regular expression literal. The language requires the returned string to be properly escaped, so that when the `source` is concatenated with a forward slash on both ends, it would form a parsable regex literal. For example, for `new RegExp("/")`, the `source` is `\\/`, because if it generates `/`, the resulting literal becomes `///`, which is a line comment. Similarly, all line terminators will be escaped because line terminator *characters* would break up the regex literal. There's no requirement for other characters, as long as the result is parsable. For empty regular expressions, the string `(?:)` is returned.

Examples

Using source

js

```
const regex = /fooBar/gi;

console.log(regex.source); // "fooBar", doesn't contain /.../ and "gi".
```

Empty regular expressions and escaping

js

```
new RegExp().source; // "(?:)"

new RegExp("\n").source === "\\n"; // true, starting with ES5
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.source

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`source`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
"(?:)" for empty regexps	6	12	38	15	5	18	38	14	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0
Line breaks and slashes are escaped	73	12	38	60	6	73	38	52	6	11.0	73	6	1.0.0	1.0	12.0.0
Prototype accessor property (ES2015)	48	12	41	35	1.3	48	41	35	1	5.0	48	1	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.source`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `RegExp.prototype.flags`

RegExp.prototype.sticky

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/sticky · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`sticky`** accessor property of `RegExp` instances returns whether or not the `y` flag is used with this regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.sticky". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const str = "table football";
const regex = /foo/y;

regex.lastIndex = 6;

console.log(regex.sticky);
// Expected output: true

console.log(regex.test(str));
// Expected output: true

console.log(regex.test(str));
// Expected output: false
```

Description

`RegExp.prototype.sticky` has the value `true` if the `y` flag was used; otherwise, `false`. The `y` flag indicates that the regex attempts to match the target string only from the index indicated by the `lastIndex` property (and unlike a global regex, does not attempt to match from any later indexes).

The set accessor of `sticky` is `undefined`. You cannot change this property directly.

For both sticky regexes and global regexes:

- They start matching at `lastIndex`.
- When the match succeeds, `lastIndex` is advanced to the end of the match.
- When `lastIndex` is out of bounds of the currently matched string, `lastIndex` is reset to 0.

However, for the `exec()` method, the behavior when matching fails is different:

- When the `exec()` method is called on a sticky regex, if the regex fails to match at `lastIndex`, the regex immediately returns `null` and resets `lastIndex` to 0.
- When the `exec()` method is called on a global regex, if the regex fails to match at `lastIndex`, it tries to match from the next character, and so on until a match is found or the end of the string is reached.

For the `exec()` method, a regex that's both sticky and global behaves the same as a sticky and non-global regex. Because `test()` is a simple wrapper around `exec()`, `test()` would ignore the global flag and perform sticky matches as well. However, due to many other methods special-casing the behavior of global regexes, the global flag is, in general, orthogonal to the sticky flag.

- `String.prototype.matchAll()` (which calls `RegExp.prototype[Symbol.matchAll]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: `matchAll()` throws; `[Symbol.matchAll]()` yields the `exec()` result exactly once, without updating the regex's `lastIndex`.
  - For `g` or `gy` regexes: returns an iterator that yields a sequence of `exec()` results.
- `String.prototype.match()` (which calls `RegExp.prototype[Symbol.match]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: returns the `exec()` result and updates the regex's `lastIndex`.
  - For `g` or `gy` regexes: returns an array of all `exec()` results.
- `String.prototype.search()` (which calls `RegExp.prototype[Symbol.search]()`): the `g` flag is always irrelevant.
  - For `y` or `gy` regexes: always returns `0` (if the very beginning of the string matches) or `-1` (if the beginning doesn't match), without updating the regex's `lastIndex` when it exits.
  - For `g` regexes: returns the index of the first match in the string, or `-1` if no match is found.
- `String.prototype.split()` (which calls `RegExp.prototype[Symbol.split]()`): `y`, `g`, and `gy` all have the same behavior.
- `String.prototype.replace()` (which calls `RegExp.prototype[Symbol.replace]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: replaces once at the current `lastIndex` and updates `lastIndex`.
  - For `g` and `gy` regexes: replaces all occurrences matched by `exec()`.
- `String.prototype.replaceAll()` (which calls `RegExp.prototype[Symbol.replace]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: `replaceAll()` throws.
  - For `g` and `gy` regexes: replaces all occurrences matched by `exec()`.

Examples

Using a regular expression with the sticky flag

```js
const str = "#foo#";
const regex = /foo/y;

regex.lastIndex = 1;
regex.test(str); // true
regex.lastIndex = 5;
regex.test(str); // false (lastIndex is taken into account with sticky flag)
regex.lastIndex; // 0 (reset after match failure)
```

Anchored sticky flag

For several versions, Firefox's SpiderMonkey engine had a bug with regard to the ^ assertion and the sticky flag which allowed expressions starting with the ^ assertion and using the sticky flag to match when they shouldn't. The bug was introduced some time after Firefox 3.6 (which had the sticky flag but not the bug) and fixed in 2015. Perhaps because of the bug, the specification specifically calls out the fact that:

> Even when the y flag is used with a pattern, ^ always matches only at the beginning of *Input*, or (if *rer*.[[Multiline]] is `true`) at the beginning of a line.

Examples of correct behavior:

```js
const regex1 = /^foo/y;
regex1.lastIndex = 2;
regex1.test("..foo"); // false - index 2 is not the beginning of the string

const regex2 = /^foo/my;
regex2.lastIndex = 2;
regex2.test("..foo"); // false - index 2 is not the beginning of the string or line
regex2.lastIndex = 2;
regex2.test(".\nfoo"); // true - index 2 is the beginning of a line
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.sticky

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
sticky	49	13	3	36	10	49	4	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
Anchored sticky flag behavior per ES2015	49	13	44	36	10	49	44	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Prototype accessor property (ES2015)	49	13	38	36	10	49	38	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.sticky`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of the `sticky` flag in `core-js`
- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.unicode`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.unicode

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/unicode · MDN page last modified 2026-08-27

Baseline: Widely available — available across major browsers since September 2016.

The **`unicode`** accessor property of `RegExp` instances returns whether or not the `u` flag is used with this regular expression.

Try it

▷ Interactive example on MDN: "JavaScript Demo: RegExp.prototype.unicode". It runs live in the browser there; its source code follows.

js · interactive example source

```
const regex1 = /\u{61}/;
const regex2 = /\u{61}/u;

console.log(regex1.unicode);
// Expected output: false

console.log(regex2.unicode);
// Expected output: true
```

Description

`RegExp.prototype.unicode` has the value `true` if the `u` flag was used; otherwise, `false`. The `u` flag enables various Unicode-related features. With the "u" flag:

- Any Unicode code point escapes (`\u{xxxx}`, `\p{UnicodePropertyValue}`) will be interpreted as such instead of identity escapes. For example `/\u{61}/u` matches `"a"`, but `/\u{61}/` (without `u` flag) matches `"u".repeat(61)`, where the `\u` is equivalent to a single `u`.
- Buffer boundary assertions (`\A`, `\z`, `\Z`) will be interpreted as such instead of identity escapes.
- Surrogate pairs will be interpreted as whole characters instead of two separate characters. For example, `/[😄]/u` would only match `"😄"` but not `"\ud83d"`.
- When `lastIndex` is automatically advanced (such as when calling `exec()`), unicode regexes advance by Unicode code points instead of UTF-16 code units.

There are other changes to the parsing behavior that prevent possible syntax mistakes (which are analogous to strict mode for regex syntax). These syntaxes are all deprecated and only kept for web compatibility, and you should not rely on them.

The set accessor of `unicode` is `undefined`. You cannot change this property directly.

Unicode-aware mode

When we refer to *Unicode-aware mode*, we mean the regex has either the `u` or the `v` flag, in which case the regex enables Unicode-related features (such as Unicode character class escape) and has much stricter syntax rules. Because `u` and `v` interpret the same regex in incompatible ways, using both flags results in a `SyntaxError`.

Similarly, a regex is *Unicode-unaware* if it has neither the `u` nor the `v` flag. In this case, the regex is interpreted as a sequence of UTF-16 code units, and there are many legacy syntaxes that do not become syntax errors.

Examples

Using the unicode property

```js
const regex = /\u{61}/u;

console.log(regex.unicode); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.unicode

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`unicode`	50	12[1]	46	37	10	50	46	37	10	5.0	50	10	1.0.0	1.0	6.0.0[2]

1. Case folding is implemented in version 13
2. Case folding is implemented in version 8.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.unicode`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`

JavaScript › Reference › Standard built-in objects › RegExp

RegExp.prototype.unicodeSets

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/unicodeSets · MDN page last modified 2026-09-18

Baseline: Widely available — available across major browsers since September 2023.

The **`unicodeSets`** accessor property of `RegExp` instances returns whether or not the v flag is used with this regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp.prototype.unicodeSets". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex1 = /[α-ω]/u;
const regex2 = /[\p{Lowercase}&&\p{Script=Greek}]/v;

console.log(regex1.unicodeSets);
// Expected output: false

console.log(regex2.unicodeSets);
// Expected output: true
```

Description

`RegExp.prototype.unicodeSets` has the value `true` if the `v` flag was used; otherwise, `false`. The `v` flag is an "upgrade" to the `u` flag that enables more Unicode-related features. ("v" is the next letter after "u" in the alphabet.) Because `u` and `v` interpret the same regex in incompatible ways, using both flags results in a `SyntaxError`. With the `v` flag, you get all features mentioned in the `u` flag description, plus:

- The `\p` escape sequence can be additionally used to match properties of strings, instead of just characters.
- The character class syntax is upgraded to allow intersection, union, and subtraction syntaxes, as well as matching multiple Unicode characters.
- The character class complement syntax `[^...]` constructs a complement class instead of negating the match result, avoiding some confusing behaviors with case-insensitive matching. For more information, see Complement classes and case-insensitive matching.

Some valid `u`-mode regexes become invalid in `v`-mode. Specifically, the character class syntax is different and some characters can no longer appear literally. For more information, see `v`-mode character class.

> **Note:** The `v` mode does not interpret grapheme clusters as single characters; they are still multiple code points. For example, `/[ᴜɴ]/v` is still able to match `"ᴜ"`.

The set accessor of `unicodeSets` is `undefined`. You cannot change this property directly.

Examples

Using the unicodeSets property

```js
const regex = /[\p{Script_Extensions=Greek}&&\p{Letter}]/v;

console.log(regex.unicodeSets); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-get-regexp.prototype.unicodesets`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`unicodeSets`	112	112	116	98	17	112	116	75	17	23.0	112	17	1.0.0	1.32	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.unicodeSets`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`
- RegExp v flag with set notation and properties of strings on v8.dev (2022)

JavaScript › Reference › Standard built-in objects › RegExp

RegExp: lastIndex

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/lastIndex · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`lastIndex`** data property of a `RegExp` instance specifies the index at which to start the next match.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp: lastIndex". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regex = /foo/g;
const str = "table football, foosball";

regex.test(str);

console.log(regex.lastIndex);
// Expected output: 9

regex.test(str);

console.log(regex.lastIndex);
// Expected output: 19
```

Value

A non-negative integer.

Property attributes of `RegExp: lastIndex`	
Writable	yes
Enumerable	no
Configurable	no

Description

This property is set only if the regular expression instance used the `g` flag to indicate a global search, or the `y` flag to indicate a sticky search. The following rules apply when `exec()` is called on a given input:

- If `lastIndex` is greater than the length of the input, `exec()` will not find a match, and `lastIndex` will be set to 0.
- If `lastIndex` is equal to or less than the length of the input, `exec()` will attempt to match the input starting from `lastIndex`.
  - If `exec()` finds a match, then `lastIndex` will be set to the position of the end of the matched string in the input.
  - If `exec()` does not find a match, then `lastIndex` will be set to 0.

Other regex-related methods, such as `RegExp.prototype.test()`, `String.prototype.match()`, `String.prototype.replace()`, etc., call `exec()` under the hood, so they have different effects on `lastIndex`. See their respective pages for details.

Examples

Using lastIndex

Consider the following sequence of statements:

```js
const re = /(hi)?/g;
```

Matches the empty string.

```js
console.log(re.exec("hi"));
console.log(re.lastIndex);
```

Returns `["hi", "hi"]` with `lastIndex` equal to 2.

```js
console.log(re.exec("hi"));
console.log(re.lastIndex);
```

Returns `["", undefined]`, an empty array whose zeroth element is the match string. In this case, the empty string because `lastIndex` was 2 (and still is 2) and `hi` has length 2.

Using lastIndex with sticky regexes

The `lastIndex` property is writable. You can set it to make the regex start its next search at a given index.

The `y` flag almost always requires setting `lastIndex`. It always matches strictly at `lastIndex` and does not attempt any later positions. This is usually useful for writing parsers, when you want to match tokens only at the current position.

```js
const stringPattern = /"[^"]*"/y;
const input = `const message = "Hello world";`;

stringPattern.lastIndex = 6;
console.log(stringPattern.exec(input)); // null

stringPattern.lastIndex = 16;
console.log(stringPattern.exec(input)); // ['"Hello world"']
```

Rewinding lastIndex

The g flag also benefits from setting `lastIndex`. One common use case is when the string is modified in the middle of a global search. In this case, we may miss a particular match if the string is shortened. We can avoid this by rewinding `lastIndex`.

```js
const mdLinkPattern = /\[[^\]]+\]\((?<link>[^()\s]+)\)/dg;

function resolveMDLink(line) {
  let match;
  let modifiedLine = line;
  while ((match = mdLinkPattern.exec(modifiedLine))) {
    const originalLink = match.groups.link;
    const resolvedLink = originalLink.replaceAll(/^files|\/index\.md$/g, "");
    modifiedLine =
      modifiedLine.slice(0, match.indices.groups.link[0]) +
      resolvedLink +
      modifiedLine.slice(match.indices.groups.link[1]);
    // Rewind the pattern to the end of the resolved link
    mdLinkPattern.lastIndex += resolvedLink.length - originalLink.length;
  }
  return modifiedLine;
}

console.log(
  resolveMDLink(
    "[`lastIndex`](files/en-
us/web/javascript/reference/global_objects/regexp/lastindex/index.md)",
  ),
); // [`lastIndex`](/en-us/web/javascript/reference/global_objects/regexp/lastindex)
console.log(
  resolveMDLink(
    "[`ServiceWorker`](files/en-us/web/api/serviceworker/index.md) and [`SharedWorker`]
(files/en-us/web/api/sharedworker/index.md)",
  ),
); // [`ServiceWorker`](/en-us/web/api/serviceworker) and [`SharedWorker`](/en-
us/web/api/sharedworker)
```

Try deleting the `mdLinkPattern.lastIndex += resolvedLink.length - originalLink.length` line and running the second example. You will find that the second link is not replaced correctly, because the `lastIndex` is already past the link's index after the string is shortened.

> **Warning:** This example is for demonstration only. To deal with Markdown, you should probably use a parsing library instead of regex.

Optimizing searching

You can optimize searching by setting `lastIndex` to a point where previous possible occurrences can be ignored. For example, instead of this:

```js
const stringPattern = /"[^"]*"/g;
const input = `const message = "Hello " + "world";`;

// Pretend we've already dealt with the previous parts of the string
let offset = 26;
const remainingInput = input.slice(offset);
const nextString = stringPattern.exec(remainingInput);
console.log(nextString[0]); // "world"
offset += nextString.index + nextString.length;
```

Consider this:

```js
stringPattern.lastIndex = offset;
const nextString = stringPattern.exec(remainingInput);
console.log(nextString[0]); // "world"
offset = stringPattern.lastIndex;
```

This is potentially more performant because we avoid string slicing.

Avoiding side effects

The side effects caused by `exec()` can be confusing, especially if the input is different for each `exec()`.

```js
const re = /foo/g;
console.log(re.test("foo bar")); // true
console.log(re.test("foo baz")); // false, because lastIndex is non-zero
```

This is even more confusing when you are hand-modifying `lastIndex`. To contain the side effects, remember to reset `lastIndex` after each input is completely processed.

```js
const re = /foo/g;
console.log(re.test("foo bar")); // true
re.lastIndex = 0;
console.log(re.test("foo baz")); // true
```

With some abstraction, you can require `lastIndex` to be set to a particular value before each `exec()` call.

```js
function createMatcher(pattern) {
  // Create a copy, so that the original regex is never updated
  const regex = new RegExp(pattern, "g");
  return (input, offset) => {
    regex.lastIndex = offset;
    return regex.exec(input);
  };
}

const matchFoo = createMatcher(/foo/);
console.log(matchFoo("foo bar", 0)[0]); // "foo"
console.log(matchFoo("foo baz", 0)[0]); // "foo"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-properties-of-regexp-instances`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`lastIndex`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.RegExp.lastIndex`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

JavaScript › Reference › Standard built-in objects

Set

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Set`** object lets you store unique values of any type, whether primitive values or object references.

Description

`Set` objects are collections of values. A value in the set **may only occur once**; it is unique in the set's collection. You can iterate through the elements of a set in insertion order. The *insertion order* corresponds to the order in which each element was inserted into the set by the `add()` method successfully (that is, there wasn't an identical element already in the set when `add()` was called).

The specification requires sets to be implemented "that, on average, provide access times that are sublinear on the number of elements in the collection". Therefore, it could be represented internally as a hash table (with O(1) lookup), a search tree (with O(log(N)) lookup), or any other data structure, as long as the complexity is better than O(N).

Value equality

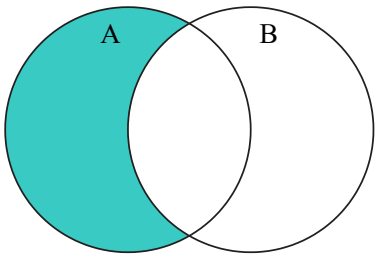
Value equality is based on the SameValueZero algorithm. (It used to use SameValue, which treated `0` and `-0` as different. Check browser compatibility.) This means `NaN` is considered the same as `NaN` (even though `NaN !== NaN`) and all other values are considered equal according to the semantics of the `===` operator. Also, for object keys, equality is based on object identity. They are compared by reference, not by value. See Using the Set object for examples.

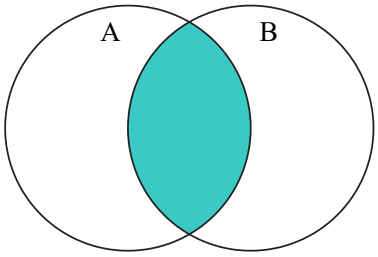
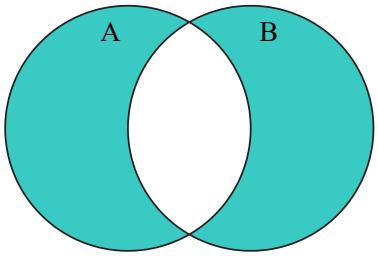
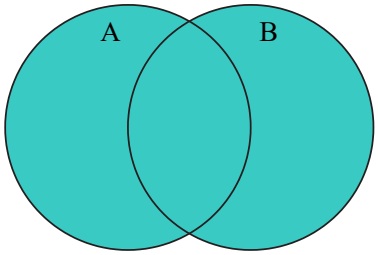
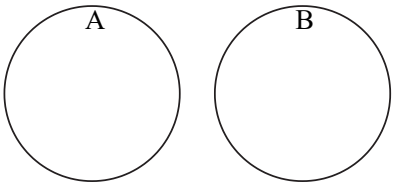
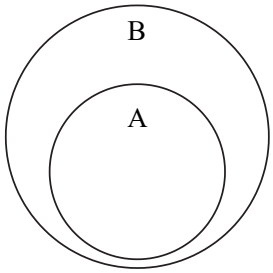
Performance

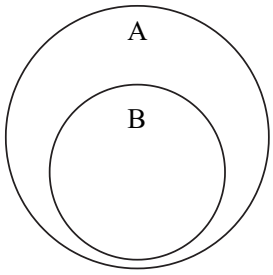
The `has` method checks if a value is in the set, using an approach that is, on average, quicker than testing most of the elements that have previously been added to the set. In particular, it is, on average, faster than the `Array.prototype.includes` method when an array has a `length` equal to a set's `size`.

Set composition

The `Set` object provides some methods that allow you to compose sets like you would with mathematical operations. These methods include:

Method	Return type	Mathematical equivalent	Venn diagram
`A.difference(B)`	`Set`	$A \setminus B$	

Method	Return type	Mathematical equivalent	Venn diagram
`A.intersection(B)`	`Set`	$A \cap B$	
`A.symmetricDifference(B)`	`Set`	$(A \setminus B) \cup (B \setminus A)$	
`A.union(B)`	`Set`	$A \cup B$	
`A.isDisjointFrom(B)`	`Boolean`	$A \cap B = \emptyset$	
`A.isSubsetOf(B)`	`Boolean`	$A \subseteq B$	

Method	Return type	Mathematical equivalent	Venn diagram
`A.isSupersetOf(B)`	`Boolean`	$A \supseteq B$	

To make them more generalizable, these methods don't just accept `Set` objects, but anything that's set-like.

Set-like objects

All set composition methods require `this` to be an actual `Set` instance, but their arguments just need to be set-like. A *set-like object* is an object that provides the following:

- A `size` property that contains a number.
- A `has()` method that takes an element and returns a boolean.
- A `keys()` method that returns an iterator of the elements in the set.

For example, `Map` objects are set-like because they also have `size`, `has()`, and `keys()`, so they behave just like sets of keys when used in set methods:

```js
const a = new Set([1, 2, 3]);
const b = new Map([
  [1, "one"],
  [2, "two"],
  [4, "four"],
]);
console.log(a.union(b)); // Set(4) {1, 2, 3, 4}
```

> **Note:** The set-like protocol invokes the `keys()` method instead of `[Symbol.iterator]()` to produce elements. This is to make maps valid set-like objects, because for maps, the iterator produces *entries* but the `has()` method takes *keys*.

Arrays are not set-like because they don't have a `has()` method or the `size` property, and their `keys()` method produces indices instead of elements. `WeakSet` objects are also not set-like because they don't have a `keys()` method.

Set-like browser APIs

Browser **`Set`-like objects** (or "setlike objects") are Web API interfaces that behave in many ways like a `Set`.

Just like `Set`, elements can be iterated in the same order that they were added to the object. `Set`-like objects and `Set` also have properties and methods that share the same name and behavior. However unlike `Set` they only allow a specific predefined type for each entry.

The allowed types are set in the specification IDL definition. For example, `GPUSupportedFeatures` is a `Set`-like object that must use strings as the key/value. This is defined in the specification IDL below:

webidl

```
interface GPUSupportedFeatures {
  readonly setlike<DOMString>;
};
```

`Set`-like objects are either read-only or read-writable (see the `readonly` keyword in the IDL above).

- Read-only `Set`-like objects have the property `size`, and the methods: `entries()`, `forEach()`, `has()`, `keys()`, `values()`, and `[Symbol.iterator]()`.
- Writeable `Set`-like objects additionally have the methods: `clear()`, `delete()`, and `add()`.

The methods and properties have the same behavior as the equivalent entities in `Set`, except for the restriction on the types of the entry.

The following are examples of read-only `Set`-like browser objects:

- `GPUSupportedFeatures`
- `XRAnchorSet`

The following are examples of writable `Set`-like browser objects:

- `CustomStateSet`
- `FontFaceSet`
- `Highlight`
- `ViewTransitionTypeSet`

Constructor

`Set()`
: Creates a new `Set` object.

Static properties

`Set[Symbol.species]`
: The constructor function that is used to create derived objects.

Instance properties

These properties are defined on `Set.prototype` and shared by all `Set` instances.

`Set.prototype.constructor`
: The constructor function that created the instance object. For `Set` instances, the initial value is the `Set` constructor.

`Set.prototype.size`
: Returns the number of values in the `Set` object.

`Set.prototype[Symbol.toStringTag]`
: The initial value of the `[Symbol.toStringTag]` property is the string `"Set"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Set.prototype.add()`

Inserts the specified value into this set, if it is not already present.

`Set.prototype.clear()`

Removes all elements from the `Set` object.

`Set.prototype.delete()`

Removes the specified value from this set, if it is in the set.

`Set.prototype.difference()`

Takes a set and returns a new set containing elements in this set but not in the given set.

`Set.prototype.entries()`

Returns a new iterator object that contains **an array of `[value, value]`** for each element in the `Set` object, in insertion order. This is similar to the `Map` object, so that each entry's *key* is the same as its *value* for a `Set`.

`Set.prototype.forEach()`

Calls `callbackFn` once for each value present in the `Set` object, in insertion order. If a `thisArg` parameter is provided, it will be used as the `this` value for each invocation of `callbackFn`.

`Set.prototype.has()`

Returns a boolean indicating whether the specified value exists in this `Set` or not.

`Set.prototype.intersection()`

Takes a set and returns a new set containing elements in both this set and the given set.

`Set.prototype.isDisjointFrom()`

Takes a set and returns a boolean indicating if this set has no elements in common with the given set.

`Set.prototype.isSubsetOf()`

Takes a set and returns a boolean indicating if all elements of this set are in the given set.

`Set.prototype.isSupersetOf()`

Takes a set and returns a boolean indicating if all elements of the given set are in this set.

`Set.prototype.keys()`

An alias for `Set.prototype.values()`.

`Set.prototype.symmetricDifference()`

Takes a set and returns a new set containing elements which are in either this set or the given set, but not in both.

`Set.prototype.union()`

Takes a set and returns a new set containing elements which are in either or both of this set and the given set.

`Set.prototype.values()`

Returns a new iterator object that yields the **values** for each element in the `Set` object in insertion order.

`Set.prototype[Symbol.iterator]()`

Returns a new iterator object that yields the **values** for each element in the `Set` object in insertion order.

Examples

Using the Set object

```js
const mySet1 = new Set();

mySet1.add(1); // Set(1) { 1 }
mySet1.add(5); // Set(2) { 1, 5 }
mySet1.add(5); // Set(2) { 1, 5 }
mySet1.add("some text"); // Set(3) { 1, 5, 'some text' }
const o = { a: 1, b: 2 };
mySet1.add(o);

mySet1.add({ a: 1, b: 2 }); // o is referencing a different object, so this is okay

mySet1.has(1); // true
mySet1.has(3); // false, since 3 has not been added to the set
mySet1.has(5); // true
mySet1.has(Math.sqrt(25)); // true
mySet1.has("Some Text".toLowerCase()); // true
mySet1.has(o); // true

mySet1.size; // 5

mySet1.delete(5); // removes 5 from the set
mySet1.has(5); // false, 5 has been removed

mySet1.size; // 4, since we just removed one value

mySet1.add(5); // Set(5) { 1, 'some text', {...}, {...}, 5 } - a previously deleted item
will be added as a new item, it will not retain its original position before deletion

console.log(mySet1); // Set(5) { 1, "some text", {…}, {…}, 5 }
```

Iterating sets

The iteration over a set visits elements in insertion order.

```js
for (const item of mySet1) {
  console.log(item);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5

for (const item of mySet1.keys()) {
  console.log(item);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5

for (const item of mySet1.values()) {
  console.log(item);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5
```

(code continues on the next page)

(code continued from the previous page)

```
// key and value are the same here
for (const [key, value] of mySet1.entries()) {
  console.log(key);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5

// Convert Set object to an Array object, with Array.from
const myArr = Array.from(mySet1); // [1, "some text", {"a": 1, "b": 2}, {"a": 1, "b": 2}, 5]

// the following will also work if run in an HTML document
mySet1.add(document.body);
mySet1.has(document.querySelector("body")); // true

// converting between Set and Array
const mySet2 = new Set([1, 2, 3, 4]);
console.log(mySet2.size); // 4
console.log([...mySet2]); // [1, 2, 3, 4]

// intersect can be simulated via
const intersection = new Set([...mySet1].filter((x) => mySet2.has(x)));

// difference can be simulated via
const difference = new Set([...mySet1].filter((x) => !mySet2.has(x)));

// Iterate set entries with forEach()
mySet2.forEach((value) => {
  console.log(value);
});
// 1
// 2
// 3
// 4
```

Implementing basic set operations

js

```
function isSuperset(set, subset) {
  for (const elem of subset) {
    if (!set.has(elem)) {
      return false;
    }
  }
  return true;
}

function union(setA, setB) {
  const _union = new Set(setA);
  for (const elem of setB) {
    _union.add(elem);
  }
  return _union;
}
```

(code continues on the next page)

(code continued from the previous page)

```
function intersection(setA, setB) {
  const _intersection = new Set();
  for (const elem of setB) {
    if (setA.has(elem)) {
      _intersection.add(elem);
    }
  }
  return _intersection;
}

function symmetricDifference(setA, setB) {
  const _difference = new Set(setA);
  for (const elem of setB) {
    if (_difference.has(elem)) {
      _difference.delete(elem);
    } else {
      _difference.add(elem);
    }
  }
  return _difference;
}

function difference(setA, setB) {
  const _difference = new Set(setA);
  for (const elem of setB) {
    _difference.delete(elem);
  }
  return _difference;
}

// Examples
const setA = new Set([1, 2, 3, 4]);
const setB = new Set([2, 3]);
const setC = new Set([3, 4, 5, 6]);

isSuperset(setA, setB); // returns true
union(setA, setC); // returns Set {1, 2, 3, 4, 5, 6}
intersection(setA, setC); // returns Set {3, 4}
symmetricDifference(setA, setC); // returns Set {1, 2, 5, 6}
difference(setA, setC); // returns Set {1, 2}
```

Relation to arrays

js

```
const myArray = ["value1", "value2", "value3"];

// Use the regular Set constructor to transform an Array into a Set
const mySet = new Set(myArray);

mySet.has("value1"); // returns true

// Use the spread syntax to transform a set into an Array.
console.log([...mySet]); // Will show you exactly the same Array as myArray
```

Remove duplicate elements from an array

```js
// Use to remove duplicate elements from an array
const numbers = [2, 13, 4, 4, 2, 13, 13, 4, 4, 5, 5, 6, 6, 7, 5, 32, 13, 4, 5];

console.log([...new Set(numbers)]); // [2, 13, 4, 5, 6, 7, 32]
```

Relation to strings

```js
// Case sensitive (set will contain "F" and "f")
new Set("Firefox"); // Set(7) [ "F", "i", "r", "e", "f", "o", "x" ]

// Duplicate omission ("f" occurs twice in the string but set will contain only one)
new Set("firefox"); // Set(6) [ "f", "i", "r", "e", "o", "x" ]
```

Use a set to ensure the uniqueness of a list of values

```js
const array = Array.from(document.querySelectorAll("[id]")).map((e) => e.id);

const set = new Set(array);
console.assert(set.size === array.length);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Set	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0
[Symbol.iterator]	43	12	36[1,2,3,4]	30	9	43	36[1,2,3,4]	30	9	4.0	43	9	1.0.0	1.0	0.12.0
[Symbol.species]	51	13	41	38	10	51	41	41	10	5.0	51	10	1.0.0	1.0	6.5.0
Set() constructor	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0
new Set(iterable)	38	12	13	25	9	38	14	25	9	3.0	38	9	1.0.0	1.0	0.12.0
new Set(null)	38	12	37	25	9	38	37	25	9	3.0	38	9	1.0.0	1.0	0.12.0
add	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0
clear	38	12	19	25	8	38	19	25	8	3.0	38	8	1.0.0	1.0	0.12.0
delete	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0
difference	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0
entries	38	12	24	25	8	38	24	25	8	3.0	38	8	1.0.0	1.0	0.12.0
forEach	38	12	25	25	8	38	25	25	8	3.0	38	8	1.0.0	1.0	0.12.0
has	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`intersection`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0
`isDisjointFrom`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0
`isSubsetOf`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0
`isSupersetOf`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0
Key equality for -0 and 0	38	12	29	25	9	38	29	25	9	3.0	38	9	1.0.0	1.0	4.0.0
`keys`	38	12	24	25	8	38	24	25	8	3.0	38	8	1.0.0	1.0	0.12.0
`size`	38	12	19[5]	25	8	38	19[6]	25	8	3.0	38	8	1.0.0	1.0	0.12.0
`symmetricDifference`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0
`union`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0
`values`	38	12	24	25	8	38	24	25	8	3.0	38	8	1.0.0	1.0	0.12.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.
5. From Firefox 13 to Firefox 18, the `size` property was implemented as a `Set.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.
6. From Firefox for Android 14 to Firefox for Android 18, the `size` property was implemented as a `Set.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Set` in `core-js`
- es-shims polyfill of `Set`
- `Map`
- `WeakMap`
- `WeakSet`

JavaScript › Reference › Standard built-in objects › Set

Set() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/Set · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Set()`** constructor creates `Set` objects.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set([1, 2, 3, 4, 5]);

console.log(set.has(1));
// Expected output: true

console.log(set.has(5));
// Expected output: true

console.log(set.has(6));
// Expected output: false
```

Syntax

js

```
new Set()
new Set(iterable)
```

> **Note:** `Set()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`iterable` Optional

If an iterable object (such as an array) is passed, all of its elements will be added to the new `Set`. If you don't specify this parameter, or its value is `null` or `undefined`, the new `Set` is empty.

Return value

A new `Set` object.

Examples

Using the `Set` object

js

```
const mySet = new Set();

mySet.add(1); // Set [ 1 ]
mySet.add(5); // Set [ 1, 5 ]
mySet.add(5); // Set [ 1, 5 ]
mySet.add("some text"); // Set [ 1, 5, 'some text' ]
const o = { a: 1, b: 2 };
mySet.add(o);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Set()` constructor	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0
`new Set(iterable)`	38	12	13	25	9	38	14	25	9	3.0	38	9	1.0.0	1.0	0.12.0
`new Set(null)`	38	12	37	25	9	38	37	25	9	3.0	38	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.Set`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Set` in `core-js`
- es-shims polyfill of `Set`
- `Set`

JavaScript › Reference › Standard built-in objects › Set

Set[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/Symbol.species · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`Set[Symbol.species]`** static accessor property is an unused accessor property specifying how to copy `Set` objects.

Syntax

```
Set[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct copied `Set` instances.

Description

The `[Symbol.species]` accessor property returns the default constructor for `Set` objects. Subclass constructors may override it to change the constructor assignment.

> **Note:** This property is currently unused by all `Set` methods.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `Set` constructor for `Set`.

```js
Set[Symbol.species]; // function Set()
```

Species in derived objects

In an instance of a custom `Set` subclass, such as `MySet`, the `MySet` species is the `MySet` constructor. However, you might want to overwrite this, in order to return parent `Set` objects in your derived class methods:

```js
class MySet extends Set {
  // Overwrite MySet species to the parent Set constructor
  static get [Symbol.species]() {
    return Set;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-get-set-%symbol.species%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.species]	51	13	41	38	10	51	41	41	10	5.0	51	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.@@species`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Set`
- `Symbol.species`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/add · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since July 2015.

The **`add()`** method of `Set` instances inserts the specified value into this set, if it is not already present.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.add()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set();

set.add(42);
set.add(42);
set.add(13);

for (const item of set) {
  console.log(item);
  // Expected output: 42
  // Expected output: 13
}
```

Syntax

js

```
add(value)
```

Parameters

`value`

The value to add to the `Set` object. Objects are compared by reference, not by value.

Return value

The `Set` object.

Examples

Using add()

js

```
const mySet = new Set();

mySet.add(1);
mySet.add(5).add("some text"); // chainable

console.log(mySet);
// Set [1, 5, "some text"]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.add`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.add`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Set`
- `Set.prototype.delete()`
- `Set.prototype.has()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.clear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/clear · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`clear()`** method of `Set` instances removes all elements from this set.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.clear()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set();
set.add(1);
set.add("foo");

console.log(set.size);
// Expected output: 2

set.clear();

console.log(set.size);
// Expected output: 0
```

Syntax

```js
clear()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Using the clear() method

```js
const mySet = new Set();
mySet.add(1);
mySet.add("foo");

console.log(mySet.size); // 2
console.log(mySet.has("foo")); // true

mySet.clear();

console.log(mySet.size); // 0
console.log(mySet.has("foo")); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.clear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`clear`	38	12	19	25	8	38	19	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.clear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Set`
- `Set.prototype.delete()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.delete()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/delete · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since July 2015.

The **`delete()`** method of `Set` instances removes the specified value from this set, if it is in the set.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.delete()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set();
set.add({ x: 10, y: 20 }).add({ x: 20, y: 30 });

// Delete any point with `x > 10`.
set.forEach((point) => {
  if (point.x > 10) {
    set.delete(point);
  }
});

console.log(set.size);
// Expected output: 1
```

Syntax

js

```
setInstance.delete(value)
```

Parameters

`value`

The value to remove from the `Set` object. Objects are compared by reference, not by value.

Return value

`true` if a value in the `Set` object has been removed successfully. `false` if the value is not found in the `Set`.

Examples

Using delete()

js

```
const mySet = new Set();
mySet.add("foo");

console.log(mySet.delete("bar")); // false; no "bar" element found to be deleted.
console.log(mySet.delete("foo")); // true; successfully removed.

console.log(mySet.has("foo")); // false; the "foo" element is no longer present.
```

Deleting an object from a set

Because objects are compared by reference, you have to delete them by checking individual properties if you don't have a reference to the original object.

```js
const setObj = new Set(); // Create a new set.

setObj.add({ x: 10, y: 20 }); // Add object in the set.

setObj.add({ x: 20, y: 30 }); // Add object in the set.

// Delete any point with `x > 10`.
setObj.forEach((point) => {
  if (point.x > 10) {
    setObj.delete(point);
  }
});
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.delete

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
delete	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.delete`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Set`
- `Set.prototype.add()`
- `Set.prototype.clear()`
- `Set.prototype.has()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.difference()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/difference · MDN page last modified 2026-10-26

Baseline 2024: Newly available — works across the latest browser versions since June 2024; may not work in older ones.

The **`difference()`** method of `Set` instances takes a set and returns a new set containing elements in this set but not in the given set.

Syntax

```js
difference(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

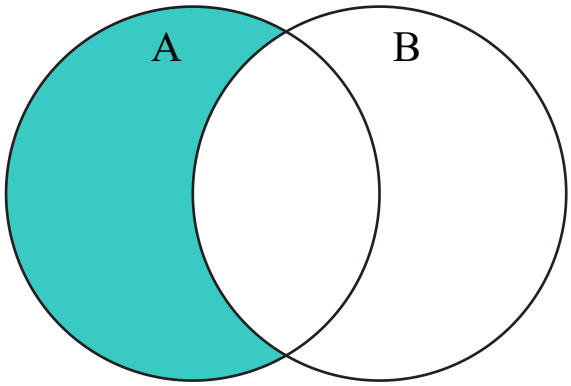
A new `Set` object containing elements in this set but not in the `other` set.

Description

In mathematical notation, *difference* is defined as:

$$A \setminus B = \{x \in A \mid x \notin B\}$$

And using Venn diagram:



`difference()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements in `this` that are not seen in `other`.
- Otherwise, it iterates over the elements in `this`, and constructs a new set with all elements `e` in `this` that cause `other.has(e)` to return a falsy value.

The order of elements in the returned set is the same as in `this`.

Examples

Using difference()

The following example computes the difference between the set of odd numbers (<10) and the set of perfect squares (<10). The result is the set of odd numbers that are not perfect squares.

```js
const odds = new Set([1, 3, 5, 7, 9]);
const squares = new Set([1, 4, 9]);
console.log(odds.difference(squares)); // Set(3) { 3, 5, 7 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.difference

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`difference`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.difference`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Set.prototype.difference` in `core-js`
- es-shims polyfill of `Set.prototype.difference`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.entries()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/entries · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`entries()`** method of `Set` instances returns a new *set iterator* object that contains **an array of `[value, value]`** for each element in this set, in insertion order. For `Set` objects there is no `key` like in `Map` objects. However, to keep the API similar to the `Map` object, each *entry* has the same value for its *key* and *value* here, so that an array `[value, value]` is returned.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.entries()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set();
set.add(42);
set.add("forty two");

const iterator = set.entries();

for (const entry of iterator) {
  console.log(entry);
  // Expected output: Array [42, 42]
  // Expected output: Array ["forty two", "forty two"]
}
```

Syntax

js

```
entries()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using entries()

```js
const mySet = new Set();
mySet.add("foobar");
mySet.add(1);
mySet.add("baz");

const setIter = mySet.entries();

console.log(setIter.next().value); // ["foobar", "foobar"]
console.log(setIter.next().value); // [1, 1]
console.log(setIter.next().value); // ["baz", "baz"]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.entries`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`entries`	38	12	24	25	8	38	24	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.entries`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Set.prototype.keys()`
- `Set.prototype.values()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.forEach()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/forEach · MDN page last modified 2026-05-21

Baseline: Widely available — available across major browsers since July 2015.

The **`forEach()`** method of `Set` instances executes a provided function once for each value in this set, in insertion order.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.forEach()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function logSetElements(value1, value2, set) {
  console.log(`s[${value1}] = ${value2}`);
}

new Set(["foo", "bar", undefined]).forEach(logSetElements);

// Expected output: "s[foo] = foo"
// Expected output: "s[bar] = bar"
// Expected output: "s[undefined] = undefined"
```

Syntax

js

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

Parameters

`callback`

A function to execute for each entry in the set. The function is called with the following arguments:

`value`

Value of each iteration.

`key`

Key of each iteration. This is always the same as `value`.

`set`

The set being iterated.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`.

Return value

None (`undefined`).

Description

The `forEach()` method executes the provided `callback` once for each value which actually exists in the `Set` object. It is not invoked for values which have been deleted. However, it is executed for values which are present but have the value `undefined`.

`callback` is invoked with **three arguments**:

- the **element value**
- the **element key**
- the **`Set` object being traversed**

There are no keys in `Set` objects, however, so the first two arguments are both **values** contained in the `Set`. This is to make it consistent with other `forEach()` methods for `Map` and `Array`.

If a `thisArg` parameter is provided to `forEach()`, it will be passed to `callback` when invoked, for use as its `this` value. Otherwise, the value `undefined` will be passed for use as its `this` value. The `this` value ultimately observable by `callback` is determined according to the usual rules for determining the `this` seen by a function.

Each value is visited once, except in the case when it was deleted and re-added before `forEach()` has finished. `callback` is not invoked for values deleted before being visited. New values added before `forEach()` has finished will be visited.

`forEach()` executes the `callback` function once for each element in the `Set` object; it does not return a value.

Examples

Logging the contents of a Set object

The following code logs a line for each element in a `Set` object:

```js
function logSetElements(value1, value2, set) {
  console.log(`s[${value1}] = ${value2}`);
}

new Set(["foo", "bar", undefined]).forEach(logSetElements);

// Logs:
// "s[foo] = foo"
// "s[bar] = bar"
// "s[undefined] = undefined"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.foreach

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`forEach`	38	12	25	25	8	38	25	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.forEach`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Array.prototype.forEach()`
- `Map.prototype.forEach()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.has()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/has · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since July 2015.

The **`has()`** method of `Set` instances returns a boolean indicating whether the specified value exists in this `Set` or not.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.has()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set([1, 2, 3, 4, 5]);

console.log(set.has(1));
// Expected output: true

console.log(set.has(5));
// Expected output: true

console.log(set.has(6));
// Expected output: false
```

Syntax

js

```
has(value)
```

Parameters

`value`

The value to test for presence in the `Set` object.

Return value

Returns `true` if the specified value exists in the `Set` object; otherwise `false`.

Examples

Using has()

```js
const mySet = new Set();
mySet.add("foo");

console.log(mySet.has("foo")); // true
console.log(mySet.has("bar")); // false

const set = new Set();
const obj = { key1: 1 };
set.add(obj);

console.log(set.has(obj)); // true
console.log(set.has({ key1: 1 })); // false, because they are different object references
console.log(set.add({ key1: 1 })); // now set contains 2 entries
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.has`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
has	38	12	13	25	8	38	14	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.has`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Set`
- `Set.prototype.add()`
- `Set.prototype.delete()`

Set.prototype.intersection()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/intersection · MDN page last modified 2026-10-26

Baseline 2024: Newly available — works across the latest browser versions since June 2024; may not work in older ones.

The **`intersection()`** method of `Set` instances takes a set and returns a new set containing elements in both this set and the given set.

Syntax

```js
intersection(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

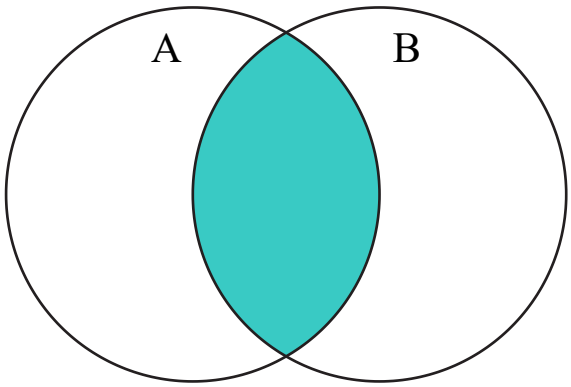
A new `Set` object containing elements in both this set and the `other` set.

Description

In mathematical notation, *intersection* is defined as:

$$A \cap B = \{x \in A \mid x \in B\}$$

And using Venn diagram:



`intersection()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements produced that are also present in `this`.
- Otherwise, it iterates over the elements in `this`, and constructs a new set with all elements `e` in `this` that cause `other.has(e)` to return a truthy value.

Because of this implementation, the efficiency of `intersection()` mostly depends on the size of the smaller set between `this` and `other` (assuming sets can be accessed in sublinear time). The order of elements in the returned set is the same as that of the smaller of `this` and `other`.

Examples

Using intersection()

The following example computes the intersection between the set of odd numbers (<10) and the set of perfect squares (<10). The result is the set of odd numbers that are perfect squares.

```js
const odds = new Set([1, 3, 5, 7, 9]);
const squares = new Set([1, 4, 9]);
console.log(odds.intersection(squares)); // Set(2) { 1, 9 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.intersection

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
intersection	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.intersection`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Set.prototype.intersection` in `core-js`
- es-shims polyfill of `Set.prototype.intersection`
- `Set.prototype.difference()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.isDisjointFrom()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/isDisjointFrom · MDN page last modified 2026-10-26

Baseline 2024: Newly available — works across the latest browser versions since June 2024; may not work in older ones.

The **`isDisjointFrom()`** method of `Set` instances takes a set and returns a boolean indicating if this set has no elements in common with the given set.

Syntax

```js
isDisjointFrom(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

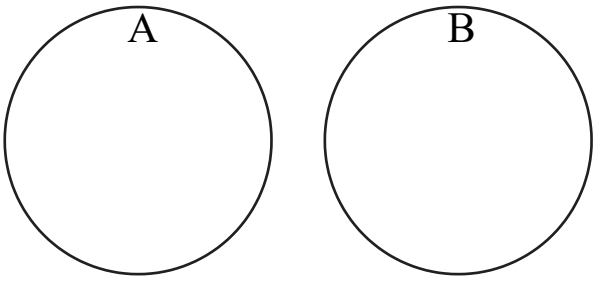
`true` if this set has no elements in common with the `other` set, and `false` otherwise.

Description

Two sets are *disjoint* if they have no elements in common. In mathematical notation:

$$A \text{ is disjoint from } B \Leftrightarrow A \cap B = \emptyset$$

And using Venn diagram:



`isDisjointFrom()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it iterates over `other` by calling its `keys()` method, and if any element in `other` is present in `this`, it returns `false` (and closes the `keys()` iterator by calling its `return()` method). Otherwise, it returns `true`.

- Otherwise, it iterates over the elements in `this`, and returns `false` if any element `e` in `this` causes `other.has(e)` to return a truthy value. Otherwise, it returns `true`.

Because of this implementation, the efficiency of `isDisjointFrom()` mostly depends on the size of the smaller set between `this` and `other` (assuming sets can be accessed in sublinear time).

Examples

Using isDisjointFrom()

The set of perfect squares (<20) is disjoint from the set of prime numbers (<20), because a perfect square is by definition decomposable into the product of two integers, while 1 is also not considered a prime number:

```js
const primes = new Set([2, 3, 5, 7, 11, 13, 17, 19]);
const squares = new Set([1, 4, 9, 16]);
console.log(primes.isDisjointFrom(squares)); // true
```

The set of perfect squares (<20) is not disjoint from the set of composite numbers (<20), because all non-1 perfect squares are by definition composite numbers:

```js
const composites = new Set([4, 6, 8, 9, 10, 12, 14, 15, 16, 18]);
const squares = new Set([1, 4, 9, 16]);
console.log(composites.isDisjointFrom(squares)); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.isdisjointfrom

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isDisjointFrom`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.isDisjointFrom`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Set.prototype.isDisjointFrom` in `core-js`
- es-shims polyfill of `Set.prototype.isDisjointFrom`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isSubsetOf()`

- Set.prototype.isSupersetOf()
- Set.prototype.symmetricDifference()
- Set.prototype.union()

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.isSubsetOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/isSubsetOf · MDN page last modified 2026-10-26

Baseline 2024: Newly available — works across the latest browser versions since June 2024; may not work in older ones.

The **`isSubsetOf()`** method of Set instances takes a set and returns a boolean indicating if all elements of this set are in the given set.

Syntax

```js
isSubsetOf(other)
```

Parameters

`other`

A Set object, or set-like object.

Return value

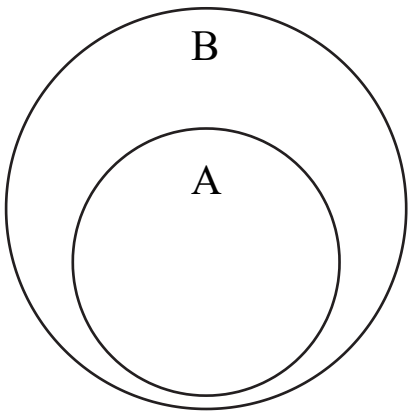
`true` if all elements in this set are also in the `other` set, and `false` otherwise.

Description

In mathematical notation, *subset* is defined as:

$$A \subseteq B \Leftrightarrow \forall x \in A,\ x \in B$$

And using Venn diagram:



> **Note:** The *subset* relationship is not *proper subset*, which means `isSubsetOf()` returns `true` if `this` and `other` contain the same elements.

`isSubsetOf()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it directly returns `false`.
- Otherwise, it iterates over the elements in `this`, and returns `false` if any element `e` in `this` causes `other.has(e)` to return a falsy value. Otherwise, it returns `true`.

Examples

Using isSubsetOf()

The set of multiples of 4 (<20) is a subset of even numbers (<20):

```js
const fours = new Set([4, 8, 12, 16]);
const evens = new Set([2, 4, 6, 8, 10, 12, 14, 16, 18]);
console.log(fours.isSubsetOf(evens)); // true
```

The set of prime numbers (<20) is not a subset of all odd numbers (<20), because 2 is prime but not odd:

```js
const primes = new Set([2, 3, 5, 7, 11, 13, 17, 19]);
const odds = new Set([3, 5, 7, 9, 11, 13, 15, 17, 19]);
console.log(primes.isSubsetOf(odds)); // false
```

Equivalent sets are subsets of each other:

```js
const set1 = new Set([1, 2, 3]);
const set2 = new Set([1, 2, 3]);
console.log(set1.isSubsetOf(set2)); // true
console.log(set2.isSubsetOf(set1)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.issubsetof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isSubsetOf`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.isSubsetOf`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Set.prototype.isSubsetOf` in `core-js`
- es-shims polyfill of `Set.prototype.isSubsetOf`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.isSupersetOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/isSupersetOf · MDN page last modified 2026-10-26

Baseline 2024: Newly available — works across the latest browser versions since June 2024; may not work in older ones.

The **`isSupersetOf()`** method of `Set` instances takes a set and returns a boolean indicating if all elements of the given set are in this set.

Syntax

```js
isSupersetOf(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

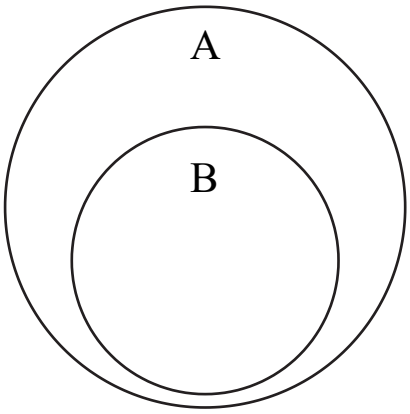
`true` if all elements in the `other` set are also in this set, and `false` otherwise.

Description

In mathematical notation, *superset* is defined as:

$$A \supseteq B \Leftrightarrow \forall x \in B,\ x \in A$$

And using Venn diagram:



> **Note:** The *superset* relationship is not *proper superset*, which means `isSupersetOf()` returns `true` if `this` and `other` contain the same elements.

`isSupersetOf()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are fewer elements in `this` than `other.size`, then it directly returns `false`.
- Otherwise, it iterates over `other` by calling its `keys()` method, and if any element in `other` is not present in `this`, it returns `false` (and closes the `keys()` iterator by calling its `return()` method). Otherwise, it returns `true`.

Examples

Using isSupersetOf()

The set of even numbers (<20) is a superset of multiples of 4 (<20):

```js
const evens = new Set([2, 4, 6, 8, 10, 12, 14, 16, 18]);
const fours = new Set([4, 8, 12, 16]);
console.log(evens.isSupersetOf(fours)); // true
```

The set of all odd numbers (<20) is not a superset of prime numbers (<20), because 2 is prime but not odd:

```js
const primes = new Set([2, 3, 5, 7, 11, 13, 17, 19]);
const odds = new Set([3, 5, 7, 9, 11, 13, 15, 17, 19]);
console.log(odds.isSupersetOf(primes)); // false
```

Equivalent sets are supersets of each other:

```js
const set1 = new Set([1, 2, 3]);
const set2 = new Set([1, 2, 3]);
console.log(set1.isSupersetOf(set2)); // true
console.log(set2.isSupersetOf(set1)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.issupersetof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isSupersetOf`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.isSupersetOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Set.prototype.isSupersetOf` in `core-js`
- es-shims polyfill of `Set.prototype.isSupersetOf`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.keys()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/keys · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`keys()`** method of `Set` instances is an alias for the `values()` method.

Syntax

```
keys()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using keys()

The `keys()` method is exactly equivalent to the `values()` method.

```js
const mySet = new Set();
mySet.add("foo");
mySet.add("bar");
mySet.add("baz");

const setIter = mySet.keys();

console.log(setIter.next().value); // "foo"
console.log(setIter.next().value); // "bar"
console.log(setIter.next().value); // "baz"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.keys`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
keys	38	12	24	25	8	38	24	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.keys`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Set.prototype.entries()`
- `Set.prototype.values()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.symmetricDifference()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/symmetricDifference · MDN page last modified 2026-10-26

Baseline 2024: Newly available — works across the latest browser versions since June 2024; may not work in older ones.

The **`symmetricDifference()`** method of `Set` instances takes a set and returns a new set containing elements which are in either this set or the given set, but not in both.

Syntax

```
symmetricDifference(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

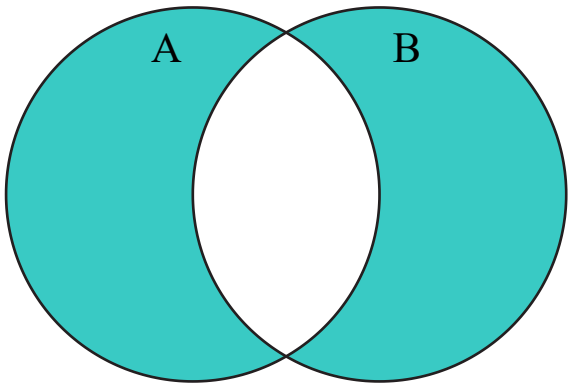
A new `Set` object containing elements which are in either this set or the `other` set, but not in both.

Description

In mathematical notation, *symmetric difference* is defined as:

$$A \ominus B = (A \setminus B) \cup (B \setminus A)$$

And using Venn diagram:



`symmetricDifference()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements in `this` that are not seen in `other`, and all elements in `other` that are not seen in `this`.

The order of elements in the returned set is first those in `this` followed by those in `other`.

Examples

Using symmetricDifference()

The following example computes the symmetric difference between the set of even numbers (<10) and the set of perfect squares (<10). The result is the set of numbers that are either even or a perfect square, but not both.

```js
const evens = new Set([2, 4, 6, 8]);
const squares = new Set([1, 4, 9]);
console.log(evens.symmetricDifference(squares)); // Set(5) { 2, 6, 8, 1, 9 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.symmetricdifference

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
symmetricDifference	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.symmetricDifference`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Set.prototype.symmetricDifference` in `core-js`
- es-shims polyfill of `Set.prototype.symmetricDifference`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.union()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.union()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/union · MDN page last modified 2026-10-26

Baseline 2024: Newly available — works across the latest browser versions since June 2024; may not work in older ones.

The **`union()`** method of `Set` instances takes a set and returns a new set containing elements which are in either or both of this set and the given set.

Syntax

```js
union(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

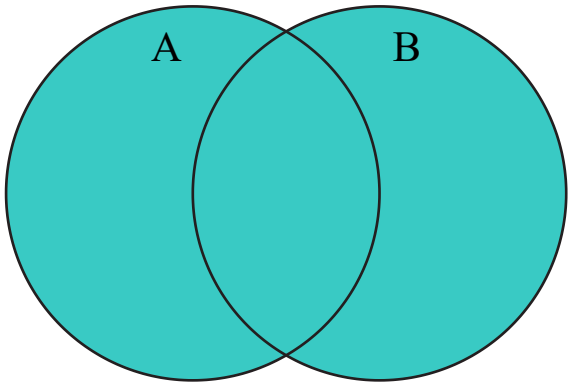
A new `Set` object containing elements which are in either or both of this set and the `other` set.

Description

In mathematical notation, *union* is defined as:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

And using Venn diagram:



`union()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements in `this`, followed by all elements in `other` that are not present in `this`.

The order of elements in the returned set is first those in `this` followed by those in `other`.

Examples

Using union()

The following example computes the union between the set of even numbers (<10) and the set of perfect squares (<10). The result is the set of numbers that are either even or a perfect square, or both.

```js
const evens = new Set([2, 4, 6, 8]);
const squares = new Set([1, 4, 9]);
console.log(evens.union(squares)); // Set(6) { 2, 4, 6, 8, 1, 9 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.union`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`union`	122	122	127	108	17	122	127	81	17	26.0	122	17	1.0.0	1.42	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.union`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Set.prototype.union` in `core-js`
- es-shims polyfill of `Set.prototype.union`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.values()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/values · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`values()`** method of `Set` instances returns a new *set iterator* object that contains the values for each element in this set in insertion order.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.values()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set();
set.add(42);
set.add("forty two");

const iterator = set.values();

console.log(iterator.next().value);
// Expected output: 42

console.log(iterator.next().value);
// Expected output: "forty two"
```

Syntax

js

```
values()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using values()

js

```
const mySet = new Set();
mySet.add("foo");
mySet.add("bar");
mySet.add("baz");

const setIter = mySet.values();

console.log(setIter.next().value); // "foo"
console.log(setIter.next().value); // "bar"
console.log(setIter.next().value); // "baz"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype.values

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`values`	38	12	24	25	8	38	24	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.values`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Set.prototype.entries()`
- `Set.prototype.keys()`

JavaScript › Reference › Standard built-in objects › Set

Set.prototype[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/Symbol.iterator · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since September 2015.

The **`[Symbol.iterator]()`** method of `Set` instances implements the iterable protocol and allows `Set` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a set iterator object that yields the values of the set in insertion order.

The initial value of this property is the same function object as the initial value of the `Set.prototype.values` property.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Set.prototype[Symbol.iterator]()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const set = new Set();

set.add(42);
set.add("forty two");

const iterator = set[Symbol.iterator]();

console.log(iterator.next().value);
// Expected output: 42

console.log(iterator.next().value);
// Expected output: "forty two"
```

Syntax

```js
set[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `Set.prototype.values()`: a new iterable iterator object that yields the values of the set.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `Set` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```js
const mySet = new Set();
mySet.add("0");
mySet.add(1);
mySet.add({});

for (const v of mySet) {
  console.log(v);
}
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```js
const mySet = new Set();
mySet.add("0");
mySet.add(1);
mySet.add({});

const setIter = mySet[Symbol.iterator]();

console.log(setIter.next().value); // "0"
console.log(setIter.next().value); // 1
console.log(setIter.next().value); // {}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-set.prototype-%symbol.iterator%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.iterator]	43	12	36[1,2,3,4]	30	9	43	36[1,2,3,4]	30	9	4.0	43	9	1.0.0	1.0	0.12.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.@@iterator`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Set`
- `Set.prototype.entries()`
- `Set.prototype.keys()`
- `Set.prototype.values()`
- `Symbol.iterator`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › Set

Set.prototype.size

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set/size · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`size`** accessor property of `Set` instances returns the number of (unique) elements in this set.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Set.prototype.size". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const set = new Set();
const object = {};

set.add(42);
set.add("forty two");
set.add("forty two");
set.add(object);

console.log(set.size);
// Expected output: 3
```

Description

The value of `size` is an integer representing how many entries the `Set` object has. A set accessor function for `size` is `undefined`; you cannot change this property.

Examples

Using size

```js
const mySet = new Set();
mySet.add(1);
mySet.add(5);
mySet.add("some text");

console.log(mySet.size); // 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-get-set.prototype.size`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`size`	38	12	19[1]	25	8	38	19[2]	25	8	3.0	38	8	1.0.0	1.0	0.12.0

1. From Firefox 13 to Firefox 18, the `size` property was implemented as a `Set.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.
2. From Firefox for Android 14 to Firefox for Android 18, the `size` property was implemented as a `Set.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Set.size`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Set`

JavaScript › Reference › Standard built-in objects

SharedArrayBuffer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer · MDN page last modified 2026-10-28

Baseline: Widely available — available across major browsers since December 2021.

The **`SharedArrayBuffer`** object is used to represent a generic raw binary data buffer, similar to the `ArrayBuffer` object, but in a way that they can be used to create views on shared memory. A `SharedArrayBuffer` is not a Transferable Object, unlike an `ArrayBuffer` which is transferable.

Description

To share memory using `SharedArrayBuffer` objects from one agent in the cluster to another (an agent is either the web page's main program or one of its web workers), `postMessage` and structured cloning is used.

The structured clone algorithm accepts `SharedArrayBuffer` objects and typed arrays mapped onto `SharedArrayBuffer` objects. In both cases, the `SharedArrayBuffer` object is transmitted to the receiver resulting in a new, private `SharedArrayBuffer` object in the receiving agent (just as for `ArrayBuffer`). However, the shared data block referenced by the two `SharedArrayBuffer` objects is the same data block, and a side effect to the block in one agent will eventually become visible in the other agent.

```js
const sab = new SharedArrayBuffer(1024);
worker.postMessage(sab);
```

Shared memory can be created and updated simultaneously in workers or the main thread. Depending on the system (the CPU, the OS, the Browser) it can take a while until the change is propagated to all contexts. To synchronize, atomic operations are needed.

`SharedArrayBuffer` objects are used by some web APIs, such as:

- `WebGLRenderingContext.bufferData()`
- `WebGLRenderingContext.bufferSubData()`
- `WebGL2RenderingContext.getBufferSubData()`

Security requirements

Shared memory and high-resolution timers were effectively disabled at the start of 2018 in light of Spectre. In 2020, a new, secure approach has been standardized to re-enable shared memory.

To use shared memory your document must be in a secure context and cross-origin isolated. You can use the `Window.crossOriginIsolated` and `WorkerGlobalScope.crossOriginIsolated` properties to check if the document is cross-origin isolated:

```js
const myWorker = new Worker("worker.js");

if (crossOriginIsolated) {
  const buffer = new SharedArrayBuffer(16);
  myWorker.postMessage(buffer);
} else {
  const buffer = new ArrayBuffer(16);
  myWorker.postMessage(buffer);
}
```

When cross-origin isolated, `postMessage()` no longer throws for `SharedArrayBuffer` objects, and shared memory across threads is therefore available.

API availability

Depending on whether the above security measures are taken, the various memory-sharing APIs have different availabilities:

- The `Atomics` object is always available.
- `SharedArrayBuffer` objects are in principle always available, but unfortunately the constructor on the global object is hidden, unless the two headers mentioned above are set, for compatibility with web content. There is hope that this restriction can be removed in the future. `WebAssembly.Memory` can still be used to get an instance.
- Unless the two headers mentioned above are set, the various `postMessage()` APIs will throw for `SharedArrayBuffer` objects. If they are set, `postMessage()` on `Window` objects and dedicated workers will function and allow for memory sharing.

WebAssembly shared memory

`WebAssembly.Memory` objects can be created with the `shared` constructor flag. When this flag is set to `true`, the constructed `Memory` object can be shared between workers via `postMessage()`, just like `SharedArrayBuffer`, and the backing `buffer` of the `Memory` object is a `SharedArrayBuffer`. Therefore, the requirements listed above for sharing a `SharedArrayBuffer` between workers also apply to sharing a `WebAssembly.Memory`.

The WebAssembly Threads proposal also defines a new set of atomic instructions. Just as `SharedArrayBuffer` and its methods are unconditionally enabled (and only sharing between threads is gated on the new headers), the WebAssembly atomic instructions are also unconditionally allowed.

Growing SharedArrayBuffers

`SharedArrayBuffer` objects can be made growable by including the `maxByteLength` option when calling the `SharedArrayBuffer()` constructor. You can query whether a `SharedArrayBuffer` is growable and what its maximum size is by accessing its `growable` and `maxByteLength` properties, respectively. You can assign a new size to a growable `SharedArrayBuffer` with a `grow()` call. New bytes are initialized to 0.

These features make growing `SharedArrayBuffers` more efficient — otherwise, you have to make a copy of the buffer with a new size. It also gives JavaScript parity with WebAssembly in this regard (Wasm linear memory can be resized with `WebAssembly.Memory.prototype.grow()`).

For security reasons, `SharedArrayBuffers` cannot be reduced in size, only grown.

Constructor

`SharedArrayBuffer()`

Creates a new `SharedArrayBuffer` object.

Static properties

`SharedArrayBuffer[Symbol.species]`

Returns the constructor used to construct return values from `SharedArrayBuffer` methods.

Instance properties

These properties are defined on `SharedArrayBuffer.prototype` and shared by all `SharedArrayBuffer` instances.

`SharedArrayBuffer.prototype.byteLength`

The size, in bytes, of the array. This is established when the array is constructed and can only be changed using the `SharedArrayBuffer.prototype.grow()` method if the `SharedArrayBuffer` is growable.

`SharedArrayBuffer.prototype.constructor`

The constructor function that created the instance object. For `SharedArrayBuffer` instances, the initial value is the `SharedArrayBuffer` constructor.

`SharedArrayBuffer.prototype.growable`

Read-only. Returns `true` if the `SharedArrayBuffer` can be grown, or `false` if not.

`SharedArrayBuffer.prototype.maxByteLength`

The read-only maximum length, in bytes, that the `SharedArrayBuffer` can be grown to. This is established when the array is constructed and cannot be changed.

`SharedArrayBuffer.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"SharedArrayBuffer"`. This property is used in `Object.prototype.toString()`.

Instance methods

`SharedArrayBuffer.prototype.grow()`

Grows the `SharedArrayBuffer` to the specified size, in bytes.

`SharedArrayBuffer.prototype.slice()`

Returns a new `SharedArrayBuffer` whose contents are a copy of this `SharedArrayBuffer`'s bytes from `begin`, inclusive, up to `end`, exclusive. If either `begin` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

Examples

Note that these examples cannot be run directly from the console or and arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Creating a new SharedArrayBuffer

```js
const sab = new SharedArrayBuffer(1024);
```

Slicing the SharedArrayBuffer

```js
sab.slice(); // SharedArrayBuffer { byteLength: 1024 }
sab.slice(2); // SharedArrayBuffer { byteLength: 1022 }
sab.slice(-2); // SharedArrayBuffer { byteLength: 2 }
sab.slice(0, 1); // SharedArrayBuffer { byteLength: 1 }
```

Using it in a WebGL buffer

```js
const canvas = document.querySelector("canvas");
const gl = canvas.getContext("webgl");
const buffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
gl.bufferData(gl.ARRAY_BUFFER, sab, gl.STATIC_DRAW);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-sharedarraybuffer-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`SharedArrayBuffer`	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0
[Symbol.species]	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0
`SharedArrayBuffer()` constructor	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0
`maxByteLength` option	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0
`byteLength`	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0
`grow`	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0
`growable`	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0
`maxByteLength`	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0
`slice`	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Agent communication and coordination guide
- `Atomics`
- `ArrayBuffer`
- JavaScript typed arrays guide
- Web Workers
- Shared Memory – a brief tutorial in the TC39 ecmascript-sharedmem proposal
- A Taste of JavaScript's New Parallel Primitives on hacks.mozilla.org (2016)
- COOP and COEP explained by the Chrome team (2020)
- `Cross-Origin-Opener-Policy`
- `Cross-Origin-Embedder-Policy`
- `Cross-Origin-Resource-Policy`

- `Window.crossOriginIsolated` and `WorkerGlobalScope.crossOriginIsolated`
- SharedArrayBuffer updates in Android Chrome 88 and Desktop Chrome 92 on developer.chrome.com (2021)

JavaScript › Reference › Standard built-in objects › SharedArrayBuffer

SharedArrayBuffer() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer/SharedArrayBuffer · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since December 2021.

> **Note:** The `SharedArrayBuffer` constructor may not always be globally available unless certain security requirements are met.

The **`SharedArrayBuffer()`** constructor creates `SharedArrayBuffer` objects.

Syntax

```js
new SharedArrayBuffer(length)
new SharedArrayBuffer(length, options)
```

> **Note:** `SharedArrayBuffer()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`length`

The size, in bytes, of the array buffer to create.

`options` Optional

An object, which can contain the following properties:

`maxByteLength` Optional

The maximum size, in bytes, that the shared array buffer can be resized to.

Return value

A new `SharedArrayBuffer` object of the specified size, with its `maxByteLength` property set to the specified `maxByteLength` if one was specified. Its contents are initialized to 0.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Basic usage

Create a buffer specifying its size in bytes.

```js
// Create a SharedArrayBuffer with a size in bytes
const buffer = new SharedArrayBuffer(8);

console.log(buffer.byteLength); // 8
```

Always use the new operator to create a SharedArrayBuffer

`SharedArrayBuffer` constructors are required to be constructed with a `new` operator. Calling a `SharedArrayBuffer` constructor as a function without `new` will throw a `TypeError`.

js · ✗ bad example

```js
const sab = SharedArrayBuffer(1024);
// TypeError: calling a builtin SharedArrayBuffer constructor
// without new is forbidden
```

js · ✓ good example

```js
const sab = new SharedArrayBuffer(1024);
```

Growing a growable SharedArrayBuffer

In this example, we create an 8-byte buffer that is growable to a max length of 16 bytes, then `grow()` it to 12 bytes:

```js
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

buffer.grow(12);
```

> **Note:** It is recommended that `maxByteLength` is set to the smallest value possible for your use case. It should never exceed `1073741824` (1GB), to reduce the risk of out-of-memory errors.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-sharedarraybuffer-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`SharedArrayBuffer()` constructor	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0
`maxByteLength` option	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer.SharedArrayBuffer`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Atomics`
- `ArrayBuffer`
- JavaScript typed arrays guide

JavaScript › Reference › Standard built-in objects › SharedArrayBuffer

SharedArrayBuffer[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer/Symbol.species · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since December 2021.

The **`SharedArrayBuffer[Symbol.species]`** static accessor property returns the constructor used to construct return values from `SharedArrayBuffer` methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```js
SharedArrayBuffer[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from array buffer methods that create new array buffer.

Description

The `[Symbol.species]` accessor property returns the default constructor for `SharedArrayBuffer` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```js
// Hypothetical underlying implementation for illustration
class SharedArrayBuffer {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```js
class SubArrayBuffer extends SharedArrayBuffer {}
SubArrayBuffer[Symbol.species] === SubArrayBuffer; // true
```

When calling array buffer methods that do not mutate the existing array but return a new array buffer instance (for example, `slice()`), the array's `constructor[Symbol.species]` will be accessed. The

returned constructor will be used to construct the return value of the array buffer method.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `SharedArrayBuffer` constructor for `SharedArrayBuffer`.

```js
SharedArrayBuffer[Symbol.species]; // function SharedArrayBuffer()
```

Species in derived objects

In an instance of a custom `SharedArrayBuffer` subclass, such as `MySharedArrayBuffer`, the `MySharedArrayBuffer` species is the `MySharedArrayBuffer` constructor. However, you might want to overwrite this, in order to return parent `SharedArrayBuffer` objects in your derived class methods:

```js
class MySharedArrayBuffer extends SharedArrayBuffer {
  // Overwrite MySharedArrayBuffer species to the parent SharedArrayBuffer constructor
  static get [Symbol.species]() {
    return SharedArrayBuffer;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-sharedarraybuffer-%symbol.species%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.species]	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer.@@species`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `SharedArrayBuffer`
- `Symbol.species`

JavaScript › Reference › Standard built-in objects › SharedArrayBuffer

SharedArrayBuffer.prototype.grow()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer/grow · MDN page last modified 2026-02-10

Baseline 2024: Newly available — works across the latest browser versions since July 2024; may not work in older ones.

The **`grow()`** method of `SharedArrayBuffer` instances grows the `SharedArrayBuffer` to the specified size, in bytes.

Syntax

```js
grow(newLength)
```

Parameters

`newLength`

The new length, in bytes, to resize the `SharedArrayBuffer` to.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if the `SharedArrayBuffer` is not growable.

`RangeError`

Thrown if `newLength` is larger than the `maxByteLength` of the `SharedArrayBuffer` or smaller than the `byteLength`.

Description

The `grow()` method grows a `SharedArrayBuffer` to the size specified by the `newLength` parameter, provided that the `SharedArrayBuffer` is growable and the new size is less than or equal to the `maxByteLength` of the `SharedArrayBuffer`. New bytes are initialized to 0.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using grow()

In this example, we create a 8-byte buffer that is growable to a max length of 16 bytes, then check its `growable` property, growing it if `growable` returns `true`:

```js
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

if (buffer.growable) {
  console.log("SAB is growable!");
  buffer.grow(12);
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-sharedarraybuffer.prototype.grow

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
grow	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer.grow`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `SharedArrayBuffer`
- `SharedArrayBuffer.prototype.growable`
- `SharedArrayBuffer.prototype.maxByteLength`

JavaScript › Reference › Standard built-in objects › SharedArrayBuffer

SharedArrayBuffer.prototype.slice()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer/slice · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`slice()`** method of `SharedArrayBuffer` instances returns a new `SharedArrayBuffer` whose contents are a copy of this `SharedArrayBuffer`'s bytes from `start`, inclusive, up to `end`, exclusive. If either `start` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

Syntax

```js
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

- Negative index counts back from the end of the buffer — if `-buffer.length <= start < 0`, `start + buffer.length` is used.
- If `start < -buffer.length` or `start` is omitted, `0` is used.
- If `start >= buffer.length`, an empty buffer is returned.

`end` Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

- Negative index counts back from the end of the buffer — if `-buffer.length <= end < 0`, `end + buffer.length` is used.
- If `end < -buffer.length`, `0` is used.
- If `end >= buffer.length` or `end` is omitted or `undefined`, `buffer.length` is used, causing all elements until the end to be extracted.
- If `end` implies a position before or at the position that `start` implies, an empty buffer is returned.

Return value

A new `SharedArrayBuffer` containing the extracted elements.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using slice()

```js
// Create a SharedArrayBuffer with a size in bytes
const buffer = new SharedArrayBuffer(16);
const int32View = new Int32Array(buffer);
int32View[1] = 42;
// Produces Int32Array [0, 42, 0, 0]

const sliced = new Int32Array(buffer.slice(4, 12));
console.log(sliced); // Int32Array [42, 0]
```

Using different start and end values

```js
const sab = new SharedArrayBuffer(1024);
sab.slice(); // SharedArrayBuffer { byteLength: 1024 }
sab.slice(2); // SharedArrayBuffer { byteLength: 1022 }
sab.slice(-2); // SharedArrayBuffer { byteLength: 2 }
sab.slice(0, 1); // SharedArrayBuffer { byteLength: 1 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-sharedarraybuffer.prototype.slice

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
slice	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer.slice`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `SharedArrayBuffer`
- `ArrayBuffer.prototype.slice()`

JavaScript › Reference › Standard built-in objects › SharedArrayBuffer

SharedArrayBuffer.prototype.byteLength

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer/byteLength · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`byteLength`** accessor property of `SharedArrayBuffer` instances returns the length (in bytes) of this `SharedArrayBuffer`.

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the shared array is constructed and cannot be changed.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using byteLength

```js
const sab = new SharedArrayBuffer(1024);
sab.byteLength; // 1024
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-get-sharedarraybuffer.prototype.bytelength`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`byteLength`	68	79	79	55	15.2	89	79	63	15.2	15.0	**No**	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer.byteLength`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `SharedArrayBuffer`

JavaScript › Reference › Standard built-in objects › SharedArrayBuffer

SharedArrayBuffer.prototype.growable

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer/growable · MDN page last modified 2026-02-10

Baseline 2024: Newly available — works across the latest browser versions since July 2024; may not work in older ones.

The **`growable`** accessor property of `SharedArrayBuffer` instances returns whether this `SharedArrayBuffer` can be grow or not.

Description

The `growable` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed. If a `maxByteLength` option was set in the constructor, `growable` will return `true`; if not, it will return `false`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using growable

In this example, we create a 8-byte buffer that is growable to a max length of 16 bytes, then check its `growable` property, growing it if `growable` returns `true`:

```js
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

if (buffer.growable) {
  console.log("SAB is growable!");
  buffer.grow(12);
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-get-sharedarraybuffer.prototype.growable

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
growable	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer.growable`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `SharedArrayBuffer`
- `SharedArrayBuffer.prototype.grow()`
- `SharedArrayBuffer.prototype.maxByteLength`

SharedArrayBuffer.prototype.maxByteLength

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer/maxByteLength · MDN page last modified 2026-02-10

Baseline 2024: Newly available — works across the latest browser versions since July 2024; may not work in older ones.

The **`maxByteLength`** accessor property of `SharedArrayBuffer` instances returns the maximum length (in bytes) that this `SharedArrayBuffer` can be grown to.

Description

The `maxByteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the shared array is constructed, set via the `maxByteLength` option of the `SharedArrayBuffer()` constructor, and cannot be changed.

If this `SharedArrayBuffer` was constructed without specifying a `maxByteLength` value, this property returns a value equal to the value of the `SharedArrayBuffer`'s `byteLength`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using maxByteLength

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then return its `maxByteLength`:

```js
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

buffer.maxByteLength; // 16
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-get-sharedarraybuffer.prototype.maxbytelength`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`maxByteLength`	111	111	128	97	16.4	111	128	75	16.4	22.0	**No**	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SharedArrayBuffer.maxByteLength`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `SharedArrayBuffer`

JavaScript › Reference › Standard built-in objects

String

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since July 2015.

The **`String`** object is used to represent and manipulate a sequence of characters.

Description

Strings are useful for holding data that can be represented in text form. Some of the most-used operations on strings are to check their `length`, to build and concatenate them using the + and += string operators, checking for the existence or location of substrings with the `indexOf()` method, or extracting substrings with the `substring()` method.

Creating strings

Strings can be created as primitives, from string literals, or as objects, using the `String()` constructor:

```js
const string1 = "A string primitive";
const string2 = 'Also a string primitive';
const string3 = `Yet another string primitive`;
```

```js
const string4 = new String("A String object");
```

String primitives and string objects share many behaviors, but have other important differences and caveats. See "String primitives and String objects" below.

String literals can be specified using single or double quotes, which are treated identically, or using the backtick character `` ` ``. This last form specifies a template literal: with this form you can interpolate expressions. For more information on the syntax of string literals, see lexical grammar.

Character access

There are two ways to access an individual character in a string. The first is the `charAt()` method:

```js
"cat".charAt(1); // gives value "a"
```

The other way is to treat the string as an array-like object, where individual characters correspond to a numerical index:

```js
"cat"[1]; // gives value "a"
```

When using bracket notation for character access, attempting to delete or assign a value to these properties will not succeed. The properties involved are neither writable nor configurable. (See `Object.defineProperty()` for more information.)

Comparing strings

Use the less-than and greater-than operators to compare strings:

```js
const a = "a";
const b = "b";
if (a < b) {
  // true
  console.log(`${a} is less than ${b}`);
} else if (a > b) {
  console.log(`${a} is greater than ${b}`);
} else {
  console.log(`${a} and ${b} are equal.`);
}
```

Note that all comparison operators, including `===` and `==`, compare strings case-sensitively. A common way to compare strings case-insensitively is to convert both to the same case (upper or lower) before comparing them.

```js
function areEqualCaseInsensitive(str1, str2) {
  return str1.toUpperCase() === str2.toUpperCase();
}
```

The choice of whether to transform by `toUpperCase()` or `toLowerCase()` is mostly arbitrary, and neither one is fully robust when extending beyond the Latin alphabet. For example, the German lowercase letter ß and `ss` are both transformed to `SS` by `toUpperCase()`, while the Turkish letter `ı` would be falsely reported as unequal to `I` by `toLowerCase()` unless specifically using `toLocaleLowerCase("tr")`.

```js
const areEqualInUpperCase = (str1, str2) =>
  str1.toUpperCase() === str2.toUpperCase();
const areEqualInLowerCase = (str1, str2) =>
  str1.toLowerCase() === str2.toLowerCase();

areEqualInUpperCase("ß", "ss"); // true; should be false
areEqualInLowerCase("ı", "I"); // false; should be true
```

A locale-aware and robust solution for testing case-insensitive equality is to use the `Intl.Collator` API or the string's `localeCompare()` method — they share the same interface — with the `sensitivity` option set to `"accent"` or `"base"`.

```js
const areEqual = (str1, str2, locale = "en-US") =>
  str1.localeCompare(str2, locale, { sensitivity: "accent" }) === 0;

areEqual("ß", "ss", "de"); // false
areEqual("ı", "I", "tr"); // true
```

The `localeCompare()` method enables string comparison in a similar fashion as `strcmp()` — it allows sorting strings in a locale-aware manner.

String primitives and String objects

Note that JavaScript distinguishes between `String` objects and primitive string values. (The same is true of `Boolean` and `Numbers`.)

String literals (denoted by double or single quotes) and strings returned from `String` calls in a non-constructor context (that is, called without using the `new` keyword) are primitive strings. In contexts where a method is to be invoked on a primitive string or a property lookup occurs, JavaScript will

automatically wrap the string primitive and call the method or perform the property lookup on the wrapper object instead.

```js
const strPrim = "foo"; // A literal is a string primitive
const strPrim2 = String(1); // Coerced into the string primitive "1"
const strPrim3 = String(true); // Coerced into the string primitive "true"
const strObj = new String(strPrim); // String with new returns a string wrapper object.

console.log(typeof strPrim); // "string"
console.log(typeof strPrim2); // "string"
console.log(typeof strPrim3); // "string"
console.log(typeof strObj); // "object"
```

> **Warning:** You should rarely find yourself using `String` as a constructor.

String primitives and `String` objects also give different results when using `eval()`. Primitives passed to `eval` are treated as source code; `String` objects are treated as all other objects are, by returning the object. For example:

```js
const s1 = "2 + 2"; // creates a string primitive
const s2 = new String("2 + 2"); // creates a String object
console.log(eval(s1)); // returns the number 4
console.log(eval(s2)); // returns the string "2 + 2"
```

For these reasons, the code may break when it encounters `String` objects when it expects a primitive string instead, although generally, authors need not worry about the distinction.

A `String` object can always be converted to its primitive counterpart with the `valueOf()` method.

```js
console.log(eval(s2.valueOf())); // returns the number 4
```

String coercion

Many built-in operations that expect strings first coerce their arguments to strings (which is largely why `String` objects behave similarly to string primitives). The operation can be summarized as follows:

- Strings are returned as-is.
- `undefined` turns into `"undefined"`.
- `null` turns into `"null"`.
- `true` turns into `"true"`; `false` turns into `"false"`.
- Numbers are converted with the same algorithm as `toString(10)`.
- BigInts are converted with the same algorithm as `toString(10)`.
- Symbols throw a `TypeError`.
- Objects are first converted to a primitive by calling its `[Symbol.toPrimitive]()` (with `"string"` as hint), `toString()`, and `valueOf()` methods, in that order. The resulting primitive is then converted to a string.

There are several ways to achieve nearly the same effect in JavaScript.

- Template literal: `` `${x}` `` does exactly the string coercion steps explained above for the embedded expression.

- The `String()` function: `String(x)` uses the same algorithm to convert `x`, except that Symbols don't throw a `TypeError`, but return `"Symbol(description)"`, where `description` is the description of the Symbol.
- Using the + operator: `"" + x` coerces its operand to a *primitive* instead of a *string*, and, for some objects, has entirely different behaviors from normal string coercion. See its reference page for more details.

Depending on your use case, you may want to use `` `${x}` `` (to mimic built-in behavior) or `String(x)` (to handle symbol values without throwing an error), but you should not use `"" + x`.

UTF-16 characters, Unicode code points, and grapheme clusters

Strings are represented fundamentally as sequences of UTF-16 code units. In UTF-16 encoding, every code unit is exact 16 bits long. This means there are a maximum of 2^{16}, or 65536 possible characters representable as single UTF-16 code units. This character set is called the basic multilingual plane (BMP), and includes the most common characters like the Latin, Greek, Cyrillic alphabets, as well as many East Asian characters. Each code unit can be written in a string with `\u` followed by exactly four hex digits.

However, the entire Unicode character set is much, much bigger than 65536. The extra characters are stored in UTF-16 as *surrogate pairs*, which are pairs of 16-bit code units that represent a single character. To avoid ambiguity, the two parts of the pair must be between `0xD800` and `0xDFFF`, and these code units are not used to encode single-code-unit characters. (More precisely, leading surrogates, also called high-surrogate code units, have values between `0xD800` and `0xDBFF`, inclusive, while trailing surrogates, also called low-surrogate code units, have values between `0xDC00` and `0xDFFF`, inclusive.) Each Unicode character, comprised of one or two UTF-16 code units, is also called a *Unicode code point*. Each Unicode code point can be written in a string with `\u{xxxxxx}` where `xxxxxx` represents 1–6 hex digits.

A "lone surrogate" is a 16-bit code unit satisfying one of the descriptions below:

- It is in the range `0xD800–0xDBFF`, inclusive (i.e., is a leading surrogate), but it is the last code unit in the string, or the next code unit is not a trailing surrogate.
- It is in the range `0xDC00–0xDFFF`, inclusive (i.e., is a trailing surrogate), but it is the first code unit in the string, or the previous code unit is not a leading surrogate.

Lone surrogates do not represent any Unicode character. Although most JavaScript built-in methods handle them correctly because they all work based on UTF-16 code units, lone surrogates are often not valid values when interacting with other systems — for example, `encodeURI()` will throw a `URIError` for lone surrogates, because URI encoding uses UTF-8 encoding, which does not have any encoding for lone surrogates. Strings not containing any lone surrogates are called *well-formed* strings, and are safe to be used with functions that do not deal with UTF-16 (such as `encodeURI()` or `TextEncoder`). You can check if a string is well-formed with the `isWellFormed()` method, or sanitize lone surrogates with the `toWellFormed()` method.

On top of Unicode characters, there are certain sequences of Unicode characters that should be treated as one visual unit, known as a *grapheme cluster*. The most common case is emojis: many emojis that have a range of variations are actually formed by multiple emojis, usually joined by the <ZWJ> (`U+200D`) character.

You must be careful which level of characters you are iterating on. For example, `split("")` will split by UTF-16 code units and will separate surrogate pairs. String indexes also refer to the index of each UTF-

16 code unit. On the other hand, `[Symbol.iterator]()` iterates by Unicode code points. Iterating through grapheme clusters will require some custom code.

```js
"😄".split(""); // ['\ud83d', '\ude04']; splits into two lone surrogates

// "Backhand Index Pointing Right: Dark Skin Tone"
[..."👉🏿"]; // ['👉', '🏿']
// splits into the basic "Backhand Index Pointing Right" emoji and
// the "Dark skin tone" emoji

// "Family: Man, Boy"
[..."👨‍👦"]; // [ '👨', '', '👦' ]
// splits into the "Man" and "Boy" emoji, joined by a ZWJ

// The United Nations flag
[..."🇺🇳"]; // [ '🇺', '🇳' ]
// splits into two "region indicator" letters "U" and "N".
// All flag emojis are formed by joining two region indicator letters
```

Constructor

`String()`

Creates `String` objects. When called as a function, it returns primitive values of type String.

Static methods

`String.fromCharCode()`

Returns a string created by using the specified sequence of Unicode values.

`String.fromCodePoint()`

Returns a string created by using the specified sequence of code points.

`String.raw()`

Returns a string created from a raw template string.

Instance properties

These properties are defined on `String.prototype` and shared by all `String` instances.

`String.prototype.constructor`

The constructor function that created the instance object. For `String` instances, the initial value is the `String` constructor.

These properties are own properties of each `String` instance.

`length`

Reflects the `length` of the string. Read-only.

Instance methods

`String.prototype.at()`

Returns the character (exactly one UTF-16 code unit) at the specified `index`. Accepts negative integers, which count back from the last string character.

`String.prototype.charAt()`

Returns the character (exactly one UTF-16 code unit) at the specified `index`.

`String.prototype.charCodeAt()`

Returns a number that is the UTF-16 code unit value at the given `index`.

`String.prototype.codePointAt()`

Returns a nonnegative integer Number that is the code point value of the UTF-16 encoded code point starting at the specified `pos`.

`String.prototype.concat()`

Combines the text of two (or more) strings and returns a new string.

`String.prototype.endsWith()`

Determines whether a string ends with the characters of the string `searchString`.

`String.prototype.includes()`

Determines whether the calling string contains `searchString`.

`String.prototype.indexOf()`

Returns the index within this string of the first occurrence of `searchValue`, or `-1` if not found.

`String.prototype.isWellFormed()`

Returns a boolean indicating whether this string contains any lone surrogates.

`String.prototype.lastIndexOf()`

Returns the index within this string of the last occurrence of `searchValue`, or `-1` if not found.

`String.prototype.localeCompare()`

Returns a number indicating whether the reference string `compareString` comes before, after, or is equivalent to the given string in sort order.

`String.prototype.match()`

Used to match regular expression `regexp` against a string.

`String.prototype.matchAll()`

Returns an iterator of all `regexp`'s matches.

`String.prototype.normalize()`

Returns the Unicode Normalization Form of the calling string value.

`String.prototype.padEnd()`

Pads the current string from the end with a given string and returns a new string of the length `targetLength`.

`String.prototype.padStart()`

Pads the current string from the start with a given string and returns a new string of the length `targetLength`.

`String.prototype.repeat()`

Returns a string consisting of the elements of the object repeated `count` times.

`String.prototype.replace()`

Used to replace occurrences of `searchFor` using `replaceWith`. `searchFor` may be a string or Regular Expression, and `replaceWith` may be a string or function.

`String.prototype.replaceAll()`

Used to replace all occurrences of `searchFor` using `replaceWith`. `searchFor` may be a string or Regular Expression, and `replaceWith` may be a string or function.

`String.prototype.search()`

Search for a match between a regular expression `regexp` and the calling string.

`String.prototype.slice()`

Extracts a section of a string and returns a new string.

`String.prototype.split()`

Returns an array of strings populated by splitting the calling string at occurrences of the substring `sep`.

`String.prototype.startsWith()`

Determines whether the calling string begins with the characters of string `searchString`.

`String.prototype.substr()` Deprecated

Returns a portion of the string, starting at the specified index and extending for a given number of characters afterwards.

`String.prototype.substring()`

Returns a new string containing characters of the calling string from (or between) the specified index (or indices).

`String.prototype.toLocaleLowerCase()`

The characters within a string are converted to lowercase while respecting the current locale.

For most languages, this will return the same as `toLowerCase()`.

`String.prototype.toLocaleUpperCase()`

The characters within a string are converted to uppercase while respecting the current locale.

For most languages, this will return the same as `toUpperCase()`.

`String.prototype.toLowerCase()`

Returns the calling string value converted to lowercase.

`String.prototype.toString()`

Returns a string representing the specified object. Overrides the `Object.prototype.toString()` method.

`String.prototype.toUpperCase()`

Returns the calling string value converted to uppercase.

`String.prototype.toWellFormed()`

Returns a string where all lone surrogates of this string are replaced with the Unicode replacement character U+FFFD.

`String.prototype.trim()`

Trims whitespace from the beginning and end of the string.

`String.prototype.trimEnd()`

Trims whitespace from the end of the string.

`String.prototype.trimStart()`

Trims whitespace from the beginning of the string.

`String.prototype.valueOf()`

Returns the primitive value of the specified object. Overrides the `Object.prototype.valueOf()` method.

`String.prototype[Symbol.iterator]()`

Returns a new iterator object that iterates over the code points of a String value, returning each code point as a String value.

HTML wrapper methods

> **Warning:** Deprecated. Avoid these methods.
>
> They are of limited use, as they are based on a very old HTML standard and provide only a subset of the currently available HTML tags and attributes. Many of them create deprecated or non-standard markup today. In addition, they do string concatenation without any validation or sanitation, which makes them a potential security threat when directly inserted using `innerHTML`. Use DOM APIs such as `document.createElement()` instead.

`String.prototype.anchor()` Deprecated

`<a name="name">` (hypertext target)

`String.prototype.big()` Deprecated

`<big>`

`String.prototype.blink()` Deprecated

`<blink>`

`String.prototype.bold()` Deprecated

`<b>`

`String.prototype.fixed()` Deprecated

`<tt>`

`String.prototype.fontcolor()` Deprecated

`<font color="color">`

`String.prototype.fontsize()` Deprecated

`<font size="size">`

`String.prototype.italics()` Deprecated

`<i>`

`String.prototype.link()` Deprecated

`<a href="url">` (link to URL)

`String.prototype.small()` Deprecated

`<small>`

`String.prototype.strike()` Deprecated

`<strike>`

`String.prototype.sub()` Deprecated

`<sub>`

`String.prototype.sup()` Deprecated

`<sup>`

Note that these methods do not check if the string itself contains HTML tags, so it's possible to create invalid HTML:

```js
"</b>".bold(); // <b></b></b>
```

The only escaping they do is to replace " in the attribute value (for `anchor()`, `fontcolor()`, `fontsize()`, and `link()`) with `"`.

```js
"foo".anchor('"Hello"'); // <a name="&quot;Hello&quot;">foo</a>
```

Examples

String conversion

The `String()` function is a more reliable way of converting values to strings than calling the `toString()` method of the value, as the former works when used on `null` and `undefined`. For example:

```js
// You cannot access properties on null or undefined

const nullVar = null;
nullVar.toString(); // TypeError: Cannot read properties of null
String(nullVar); // "null"

const undefinedVar = undefined;
undefinedVar.toString(); // TypeError: Cannot read properties of undefined
String(undefinedVar); // "undefined"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`String`	1	12	1	3	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
[Symbol.iterator]	38	12	36[1,2,3,4]	25	9	38	36[1,2,3,4]	25	9	3.0	38	9	1.0.0	1.0	0.12.0
`String()` constructor	1	12	1	3	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`anchor` Deprecated	1	12	1[5]	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`at`	92	92	90	78	15.4	92	90	65	15.4	16.0	92	15.4	1.0.0	1.12	16.6.0
`big` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`blink` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`bold` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`charAt`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`charCodeAt`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`codePointAt`	41	12	29	28	9	41	29	28	9	4.0	41	9	1.0.0	1.0	4.0.0
`concat`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`endsWith`	41	12	17	28	9	36	17	24	9	3.0	37	9	1.0.0	1.0	4.0.0
`fixed` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`fontcolor` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`fontsize` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`fromCharCode`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`fromCodePoint`	41	12	29	28	9	41	29	28	9	4.0	41	9	1.0.0	1.0	4.0.0
`includes`	41	12	40[6]	28	9	41	40[6]	28	9	4.0	41	9	1.0.0	1.0	4.0.0
`indexOf`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`isWellFormed`	111	111	119	97	16.4	111	119	75	16.4	22.0	111	16.4	1.0.0	1.32	20.0.0
`italics` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`lastIndexOf`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`length`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`link` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`localeCompare`	1	12	1	7	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`locales` parameter	24	12	29	15	10	26	56	**No**	10	1.5	**No**	10	1.0.0	1.8[7,8]	13.0.0[9,10]
`options` parameter	24	12	29	15	10	26	56	**No**	10	1.5	**No**	10	1.0.0	1.0	0.12.0
`match`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`matchAll`	73	79	67	60	13	73	67	52	13	11.0	73	13	1.0.0	1.0	12.0.0
`normalize`	34	12	31	21	10	34	31	21	10	2.0	37	10	1.0.0	1.0	0.12.0
`padEnd`	57	15	48	44	10	57	48	43	10	7.0	57	10	1.0.0	1.0	8.0.0
`padStart`	57	15	48	44	10	57	48	43	10	7.0	57	10	1.0.0	1.0	8.0.0
`raw`	41	12	34	28	9	41	34	28	9	4.0	41	9	1.0.0	1.0	4.0.0
`repeat`	41	12	24	28	9	36	24	28	9	3.0	41	9	1.0.0	1.0	4.0.0
`replace`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`replaceAll`	85	85	77	71	13.1	85	79	60	13.4	14.0	85	13.4	1.0.0	1.2	15.0.0
`search`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`slice`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`small` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`split`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`startsWith`	41	12	17	28	9	36	17	24	9	3.0	37	9	1.0.0	1.0	4.0.0
`strike` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`sub` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`substr` Deprecated	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`substring`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`sup` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toLocaleLowerCase`	1	12	1	4	1.3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`locales` parameter	58	12	55	45	10	58	55	43	10	7.0	58	10	1.0.0	1.8[7,8]	13.0.0[9,10]

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleUpperCase`	1	12	1	4	1.3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`locales` parameter	58	12	55	45	10	58	55	42	10	7.0	58	10	1.0.0	1.8[7,8]	13.0.0[9,10]
`toLowerCase`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toUpperCase`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toWellFormed`	111	111	119	97	16.4	111	119	75	16.4	22.0	111	16.4	1.0.0	1.32	20.0.0
`trim`	4	12	3.5	10.5	5	18	4	11	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`trimEnd`	66	79	61	53	12	66	61	47	12	9.0	66	12	1.0.0	1.0	10.0.0
`trimLeft` Deprecated	4	12	3.5	15	5	18	4	14	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0
`trimRight` Deprecated	4	12	3.5	15	5	18	4	14	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0
`trimStart`	66	79	61	53	12	66	61	47	12	9.0	66	12	1.0.0	1.0	10.0.0
Unicode code point escapes \u{xxxxxx}	1	12	40	4	1	18	40	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`valueOf`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.
5. Starting with version 17, the quotation mark (") is replaced by its HTML reference character (") in strings supplied for the `name` parameter.
6. Also from version 18–48: uses the non-standard name `contains`.
7. Also from version 1.0–1.8: partial support.
8. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
9. Also from version 0.12.0–13.0.0: partial support.
10. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Numbers and strings guide
- RegExp

JavaScript › Reference › Standard built-in objects › String

String() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/String · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`String()`** constructor creates `String` objects. When called as a function, it returns primitive values of type String.

Syntax

```js
new String(thing)
String(thing)
```

> **Note:** `String()` can be called with or without `new`, but with different effects. See Return value.

Parameters

`thing`

Anything to be converted to a string.

Return value

When `String()` is called as a function (without `new`), it returns `value` coerced to a string primitive. Specially, Symbol values are converted to `"Symbol(description)"`, where `description` is the description of the Symbol, instead of throwing.

When `String()` is called as a constructor (with `new`), it coerces `value` to a string primitive (without special symbol handling) and returns a wrapping `String` object, which is **not** a primitive.

> **Warning:** You should rarely find yourself using `String` as a constructor.

Examples

String constructor and String function

String function and String constructor produce different results:

```js
const a = new String("Hello world"); // a === "Hello world" is false
const b = String("Hello world"); // b === "Hello world" is true
a instanceof String; // is true
b instanceof String; // is false
typeof a; // "object"
typeof b; // "string"
```

Here, the function produces a string (the primitive type) as promised. However, the constructor produces an instance of the type String (an object wrapper) and that's why you rarely want to use the String constructor at all.

Using String() to stringify a symbol

`String()` is the only case where a symbol can be converted to a string without throwing, because it's very explicit.

js · ✗ bad example

```
const sym = Symbol("example");
`${sym}`; // TypeError: Cannot convert a Symbol value to a string
"" + sym; // TypeError: Cannot convert a Symbol value to a string
"".concat(sym); // TypeError: Cannot convert a Symbol value to a string
```

js · ✓ good example

```
const sym = Symbol("example");
String(sym); // "Symbol(example)"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`String()` constructor	1	12	1	3	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.String`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Numbers and strings guide

JavaScript › Reference › Standard built-in objects › String

String.fromCharCode()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/fromCharCode · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`String.fromCharCode()`** static method returns a string created from the specified sequence of UTF-16 code units.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.fromCharCode()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(String.fromCharCode(189, 43, 190, 61));
// Expected output: "½+¾="
```

Syntax

js

```
String.fromCharCode()
String.fromCharCode(num1)
String.fromCharCode(num1, num2)
String.fromCharCode(num1, num2, /* …, */ numN)
```

Parameters

num1, ..., numN

A number between 0 and 65535 (0xFFFF) representing a UTF-16 code unit. Numbers greater than 0xFFFF are truncated to the last 16 bits. No validity checks are performed.

Return value

A string of length N consisting of the N specified UTF-16 code units.

Description

Because fromCharCode() is a static method of String, you always use it as String.fromCharCode(), rather than as a method of a String value you created.

Unicode code points range from 0 to 1114111 (0x10FFFF). charCodeAt() always returns a value that is less than 65536, because the higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, in order to produce a full character with value greater than 65535, it is necessary to provide two code units (as if manipulating a string with two characters). For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Because fromCharCode() only works with 16-bit values (same as the \u escape sequence), a surrogate pair is required in order to return a supplementary character. For example, both String.fromCharCode(0xd83c, 0xdf03) and "\ud83c\udf03" return code point U+1F303 "Night with Stars". While there is a mathematical relationship between the supplementary code point value (e.g., 0x1f303) and both surrogate values that represent it (e.g., 0xd83c and 0xdf03), it does require an extra step to either calculate or look up the surrogate pair values every time a supplementary code point is to be used. For this reason, it's more convenient to use String.fromCodePoint(), which allows for returning supplementary characters based on their actual code point value. For example, String.fromCodePoint(0x1f303) returns code point U+1F303 "Night with Stars".

Examples

Using fromCharCode()

BMP characters, in UTF-16, use a single code unit:

```js
String.fromCharCode(65, 66, 67); // returns "ABC"
String.fromCharCode(0x2014); // returns "—"
String.fromCharCode(0x12014); // also returns "—"; the digit 1 is truncated and ignored
String.fromCharCode(8212); // also returns "—"; 8212 is the decimal form of 0x2014
```

Supplementary characters, in UTF-16, require two code units (i.e., a surrogate pair):

```js
String.fromCharCode(0xd83c, 0xdf03); // Code Point U+1F303 "Night with
String.fromCharCode(55356, 57091); // Stars" === "\uD83C\uDF03"

String.fromCharCode(0xd834, 0xdf06, 0x61, 0xd834, 0xdf07); // "\uD834\uDF06a\uD834\uDF07"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.fromcharcode`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fromCharCode`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.fromCharCode`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.fromCodePoint()`
- `String.prototype.charAt()`
- `String.prototype.charCodeAt()`
- `String.prototype.codePointAt()`

JavaScript › Reference › Standard built-in objects › String

String.fromCodePoint()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/fromCodePoint · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`String.fromCodePoint()`** static method returns a string created from the specified sequence of code points.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.fromCodePoint()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(String.fromCodePoint(9731, 9733, 9842, 0x2f804));
// Expected output: "☃★♲你"
```

Syntax

js

```
String.fromCodePoint()
String.fromCodePoint(num1)
String.fromCodePoint(num1, num2)
String.fromCodePoint(num1, num2, /* …, */ numN)
```

Parameters

num1, ..., numN

An integer between 0 and 0x10FFFF (inclusive) representing a Unicode code point.

Return value

A string created by using the specified sequence of code points.

Exceptions

RangeError

Thrown if numN is not an integer, is less than 0, or is greater than 0x10FFFF after being converted to a number.

Description

Because fromCodePoint() is a static method of String, you always use it as String.fromCodePoint(), rather than as a method of a String value you created.

Unicode code points range from 0 to 1114111 (0x10FFFF). In UTF-16, each string index is a code unit with value 0 – 65535. Higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, fromCodePoint() may return a string whose length (in UTF-16 code units) is larger than the number of arguments passed. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Examples

Using fromCodePoint()

Valid input:

```js
String.fromCodePoint(42); // "*"
String.fromCodePoint(65, 90); // "AZ"
String.fromCodePoint(0x404); // "\u0404" === "Є"
String.fromCodePoint(0x2f804); // "\uD87E\uDC04"
String.fromCodePoint(194564); // "\uD87E\uDC04"
String.fromCodePoint(0x1d306, 0x61, 0x1d307); // "\uD834\uDF06a\uD834\uDF07"
```

Invalid input:

```js
String.fromCodePoint("_"); // RangeError
String.fromCodePoint(Infinity); // RangeError
String.fromCodePoint(-1); // RangeError
String.fromCodePoint(3.14); // RangeError
String.fromCodePoint(3e-2); // RangeError
String.fromCodePoint(NaN); // RangeError
```

Compared to fromCharCode()

String.fromCharCode() cannot return supplementary characters (i.e., code points 0x010000 – 0x10FFFF) by specifying their code point. Instead, it requires the UTF-16 surrogate pair in order to return a supplementary character:

```js
String.fromCharCode(0xd83c, 0xdf03); // Code Point U+1F303 "Night with
String.fromCharCode(55356, 57091); // Stars" === "\uD83C\uDF03"
```

`String.fromCodePoint()`, on the other hand, can return 4-byte supplementary characters, as well as the more common 2-byte BMP characters, by specifying their code point (which is equivalent to the UTF-32 code unit):

```js
String.fromCodePoint(0x1f303); // or 127747 in decimal
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.fromcodepoint`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fromCodePoint`	41	12	29	28	9	41	29	28	9	4.0	41	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.fromCodePoint`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.fromCodePoint` in `core-js`
- es-shims polyfill of `String.fromCodePoint`
- `String.fromCharCode()`
- `String.prototype.charAt()`
- `String.prototype.codePointAt()`
- `String.prototype.charCodeAt()`

JavaScript › Reference › Standard built-in objects › String

String.raw()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/raw · MDN page last modified 2026-10-26
Baseline: Widely available — available across major browsers since September 2015.

The **`String.raw()`** static method is a tag function of template literals. This is similar to the `r` prefix in Python, or the `@` prefix in C# for string literals. It's used to get the raw string form of template literals — that is, substitutions (e.g., `${foo}`) are processed, but escape sequences (e.g., `\n`) are not.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.raw()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create a variable that uses a Windows
// path without escaping the backslashes:
const filePath = String.raw`C:\Development\profile\about.html`;

console.log(`The file was uploaded from: ${filePath}`);
// Expected output: "The file was uploaded from: C:\Development\profile\about.html"
```

Syntax

js

```
String.raw(strings)
String.raw(strings, sub1)
String.raw(strings, sub1, sub2)
String.raw(strings, sub1, sub2, /* …, */ subN)

String.raw`templateString`
```

Parameters

`strings`

Well-formed template literal array object, like `{ raw: ['foo', 'bar', 'baz'] }`. Should be an object with a `raw` property whose value is an array-like object of strings.

`sub1, ..., subN`

Contains substitution values.

`templateString`

A template literal, optionally with substitutions (`${...}`).

Return value

The raw string form of a given template literal.

Exceptions

`TypeError`

Thrown if the first argument doesn't have a `raw` property, or the `raw` property is `undefined` or `null`.

Description

In most cases, `String.raw()` is used with template literals. The first syntax mentioned above is only rarely used, because the JavaScript engine will call this with proper arguments for you, (just like with other tag functions).

`String.raw()` is the only built-in template literal tag. It has close semantics to an untagged literal since it concatenates all arguments and returns a string. You can even re-implement it with normal JavaScript code.

> **Warning:** You should not use `String.raw` directly as an "identity" tag. See Building an identity tag for how to implement this.

If `String.raw()` is called with an object whose `raw` property doesn't have a `length` property or a non-positive `length`, it returns an empty string `""`. If `substitutions.length < strings.raw.length - 1` (i.e., there are not enough substitutions to fill the placeholders — which can't happen in a well-formed tagged template literal), the rest of the placeholders are filled with empty strings.

Examples

Using String.raw()

```js
String.raw`Hi\n${2 + 3}!`;
// 'Hi\\n5!', the character after 'Hi'
// is not a newline character,
// '\' and 'n' are two characters.

String.raw`Hi\u000A!`;
// 'Hi\\u000A!', same here, this time we will get the
// \, u, 0, 0, 0, A, 6 characters.
// All kinds of escape characters will be ineffective
// and backslashes will be present in the output string.
// You can confirm this by checking the .length property
// of the string.

const name = "Bob";
String.raw`Hi\n${name}!`;
// 'Hi\\nBob!', substitutions are processed.
```

Raw strings containing template literal syntax

`String.raw` is a function, so it cannot circumvent basic template literal syntax such as backticks as delimiters and `${` for substitutions. If you want to include these characters in the output string, you need to escape them with backslashes. However, since `String.raw` outputs raw strings, the backslashes will be preserved in the output.

```js
String.raw`Hi \${name}!`;
// 'Hi \\${name}!', the dollar sign is escaped; there's no interpolation.
// However, the backslash is still present in the output string.

String.raw`This is a backtick: \``;
// 'This is a backtick: \\`', the backslash is still present.

String.raw`A trailing backslash: \\`;
// 'A trailing backslash: \\\\', both backslashes are present.
// If you use a single backslash at the end, it escapes the ending backtick,
// causing subsequent code to be included in the string.
```

To work around this, you can use a substitution to insert these characters.

```js
String.raw`Hi ${"$"}{name}!`;
// 'Hi ${name}!', the substitution inserts a single dollar sign.
String.raw`This is a backtick: ${"`"}`;
// 'This is a backtick: `', the substitution inserts a single backtick.
String.raw`A trailing backslash: ${"\\"}`;
// 'A trailing backslash: \\', the substitution inserts a single backslash.
```

This approach works for `String.raw` because it just concatenates the raw strings and the substitutions. In general, unfortunately, there's no way for a template literal tag to receive a `raw` string that contains unescaped template literal syntax.

```js
function tag(strings) {
  console.log(strings.raw[0]); // This will never contain unescaped `${` or backticks
}
```

Using String.raw with RegExp

Combining a `String.raw` template literal with the `RegExp()` constructor allows you to create regular expressions with dynamic parts (which is not possible with regex literals) without double-escaping (`\\`) regular expression escape sequences (which is not possible with normal string literals). This is also valuable in strings that contain a lot of slashes, such as file paths or URLs.

```js
// A String.raw template allows a fairly readable regular expression matching a URL:
const reRawTemplate = new RegExp(
  String.raw`https://developer\.mozilla\.org/en-US/docs/Web/JavaScript/Reference/`,
);

// The same thing with a regexp literal looks like this, with \/ for
// each forward slash:
const reRegexpLiteral =
  /https:\/\/developer\.mozilla\.org\/en-US\/docs\/Web\/JavaScript\/Reference\//;

// And the same thing written with the RegExp constructor and a
// traditional string literal, with \\. for each period:
const reStringLiteral = new RegExp(
  "https://developer\\.mozilla\\.org/en-US/docs/Web/JavaScript/Reference/",
);

// String.raw also allows dynamic parts to be included
function makeURLRegExp(path) {
  return new RegExp(String.raw`https://developer\.mozilla\.org/${path}`);
}

const reDynamic = makeURLRegExp("en-US/docs/Web/JavaScript/Reference/");
const reWildcard = makeURLRegExp(".*");
```

Building an identity tag

Many tools give special treatment to literals tagged by a particular name.

```js
// Some formatters will format this literal's content as HTML
const doc = html`<!doctype html>
  <html lang="en-US">
    <head>
      <title>Hello</title>
    </head>
    <body>
      <h1>Hello world!</h1>
    </body>
  </html>`;
```

One might naïvely implement the `html` tag as:

```js
const html = String.raw;
```

This, in fact, works for the case above. However, because `String.raw` would concatenate the *raw* string literals instead of the "cooked" ones, escape sequences would not be processed.

```js
const doc = html`<canvas>\n</canvas>`;
// "<canvas>\\n</canvas>"
```

This may not be what you want for a "true identity" tag, where the tag is purely for markup and doesn't change the literal's value. In this case, you can create a custom tag and pass the "cooked" (i.e., escape sequences are processed) literal array to `String.raw`, pretending they are raw strings.

```js
const html = (strings, ...values) => String.raw({ raw: strings }, ...values);
// Some formatters will format this literal's content as HTML
const doc = html`<canvas>\n</canvas>`;
// "<canvas>\n</canvas>"; the "\n" becomes a line break
```

Notice the first argument is an object with a `raw` property, whose value is an array-like object (with a `length` property and integer indexes) representing the separated strings in the template literal. The rest of the arguments are the substitutions. Since the `raw` value can be any array-like object, it can even be a string! For example, `'test'` is treated as `['t', 'e', 's', 't']`. The following is equivalent to `` `t${0}e${1}s${2}t` ``:

```js
String.raw({ raw: "test" }, 0, 1, 2); // 't0e1s2t'
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.raw

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`raw`	41	12	34	28	9	41	34	28	9	4.0	41	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.raw`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.raw` in `core-js`
- es-shims polyfill of `String.raw`
- Template literals

- String
- Lexical grammar

JavaScript › Reference › Standard built-in objects › String

String.prototype.anchor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/anchor · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **anchor()** method of String values creates a string that embeds this string in an <a> element with a name (str).

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as document.createElement() instead.
>
> The HTML specification no longer allows the <a> element to have a name attribute, so this method doesn't even create valid markup.

Syntax

```js
anchor(name)
```

Parameters

name

A string representing a name value to put into the generated <a name="..."> start tag.

Return value

A string beginning with an start tag (double quotes in name are replaced with "), then the text str, and then an end tag.

Examples

Using anchor()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.anchor("hello");
```

This will create the following HTML:

```html
<a name="hello">Hello, world</a>
```

> **Warning:** This markup is invalid, because name is no longer a valid attribute of the <a> element.

Instead of using anchor() and creating HTML text directly, you should use DOM APIs such as document.createElement(). For example:

```js
const contentString = "Hello, world";
const elem = document.createElement("a");
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.anchor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
anchor Deprecated	1	12	1[1]	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

1. Starting with version 17, the quotation mark (") is replaced by its HTML reference character (") in strings supplied for the `name` parameter.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.anchor`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.anchor` in `core-js`
- es-shims polyfill of `String.prototype.anchor`
- HTML wrapper methods
- `<a>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.at()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/at · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since March 2022.

The **`at()`** method of `String` values takes an integer value and returns a new `String` consisting of the single UTF-16 code unit located at the specified offset. This method allows for positive and negative integers. Negative integers count back from the last string character.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.at()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const sentence = "The quick brown fox jumps over the lazy dog.";

let index = 5;

console.log(`An index of ${index} returns the character ${sentence.at(index)}`);
// Expected output: "An index of 5 returns the character u"

index = -4;

console.log(`An index of ${index} returns the character ${sentence.at(index)}`);
// Expected output: "An index of -4 returns the character d"
```

Syntax

js

```
at(index)
```

Parameters

`index`

The index (position) of the string character to be returned. Supports relative indexing from the end of the string when passed a negative index; i.e., if a negative number is used, the character returned will be found by counting back from the end of the string.

Return value

A `String` consisting of the single UTF-16 code unit located at the specified position. Returns `undefined` if the given index cannot be found.

Examples

Return the last character of a string

The following example provides a function which returns the last character found in a specified string.

js

```
// A function which returns the last character of a given string
function returnLast(str) {
  return str.at(-1);
}

let invoiceRef = "my-invoice01";

console.log(returnLast(invoiceRef)); // '1'

invoiceRef = "my-invoice02";

console.log(returnLast(invoiceRef)); // '2'
```

Comparing methods

Here we compare different ways to select the penultimate (last but one) character of a `String`. While all below methods are valid, it highlights the succinctness and readability of the `at()` method.

js

```
const myString = "Every green bus drives fast.";

// Using length property and charAt() method
const lengthWay = myString.charAt(myString.length - 2);
console.log(lengthWay); // 't'

// Using slice() method
const sliceWay = myString.slice(-2, -1);
console.log(sliceWay); // 't'

// Using at() method
const atWay = myString.at(-2);
console.log(atWay); // 't'
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.at

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
at	92	92	90	78	15.4	92	90	65	15.4	16.0	92	15.4	1.0.0	1.12	16.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.at`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.at` in `core-js`
- es-shims polyfill of `String.prototype.at`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.charCodeAt()`
- `String.prototype.codePointAt()`
- `String.prototype.split()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.big()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/big · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`big()`** method of `String` values creates a string that embeds this string in a `<big>` element (`<big>str</big>`), which causes this string to be displayed in a big font.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `big()`, the `<big>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```js
big()
```

Parameters

None.

Return value

A string beginning with a `<big>` start tag, then the text `str`, and then a `</big>` end tag.

Examples

Using big()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.big();
```

This will create the following HTML:

```html
<big>Hello, world</big>
```

> **Warning:** This markup is invalid, because `big` is no longer a valid element.

Instead of using `big()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `font-size` through the `element.style` attribute:

```js
document.getElementById("yourElemId").style.fontSize = "2em";
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.big`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`big` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.big`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.big` in `core-js`
- es-shims polyfill of `String.prototype.big`
- HTML wrapper methods
- `<big>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.blink()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/blink · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`blink()`** method of `String` values creates a string that embeds this string in a `<blink>` element (`<blink>str</blink>`), which used to cause a string to blink in old browsers.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `blink()`, the `<blink>` element itself is removed from modern browsers, and blinking text is frowned upon by several accessibility standards. Avoid using the element in any way.

Syntax

```js
blink()
```

Parameters

None.

Return value

A string beginning with a `<blink>` start tag, then the text `str`, and then a `</blink>` end tag.

Examples

Using blink()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.blink();
```

This will create the following HTML:

```html
<blink>Hello, world</blink>
```

> **Warning:** This markup is invalid, because `blink` is no longer a valid element.

You should avoid blinking elements altogether.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.blink`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`blink` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.blink`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.blink` in `core-js`
- es-shims polyfill of `String.prototype.blink`
- HTML wrapper methods

JavaScript › Reference › Standard built-in objects › String

String.prototype.bold()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/bold · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`bold()`** method of `String` values creates a string that embeds this string in a `<b>` element (`<b>str</b>`), which causes this string to be displayed as bold.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```js
bold()
```

Parameters

None.

Return value

A string beginning with a `<b>` start tag, then the text `str`, and then a `</b>` end tag.

Examples

Using bold()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.bold();
```

This will create the following HTML:

```html
<b>Hello, world</b>
```

Instead of using `bold()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```js
const contentString = "Hello, world";
const elem = document.createElement("b");
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.bold`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`bold` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.bold`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.bold` in `core-js`
- es-shims polyfill of `String.prototype.bold`
- HTML wrapper methods
- `<b>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.charAt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/charAt · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`charAt()`** method of `String` values returns a new string consisting of the single UTF-16 code unit at the given index.

`charAt()` always indexes the string as a sequence of UTF-16 code units, so it may return lone surrogates. To get the full Unicode code point at the given index, use `String.prototype.codePointAt()` and `String.fromCodePoint()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.charAt()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const sentence = "The quick brown fox jumps over the lazy dog.";

const index = 4;

console.log(`The character at index ${index} is ${sentence.charAt(index)}`);
// Expected output: "The character at index 4 is q"
```

Syntax

```js
charAt(index)
```

Parameters

`index`

Zero-based index of the character to be returned. Converted to an integer — `undefined` is converted to 0.

Return value

A string representing the character (exactly one UTF-16 code unit) at the specified `index`. If `index` is out of the range of `0` – `str.length - 1`, `charAt()` returns an empty string.

Description

Characters in a string are indexed from left to right. The index of the first character is `0`, and the index of the last character in a string called `str` is `str.length - 1`.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). `charAt()` always returns a character whose value is less than `65536`, because the higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, in order to get a full character with value greater than `65535`, it is necessary to retrieve not only `charAt(i)`, but also `charAt(i + 1)` (as if manipulating a string with two characters), or to use `codePointAt(i)` and `String.fromCodePoint()` instead. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

`charAt()` is very similar to using bracket notation to access a character at the specified index. The main differences are:

- `charAt()` attempts to convert `index` to an integer, while bracket notation does not, and directly uses `index` as a property name.
- `charAt()` returns an empty string if `index` is out of range, while bracket notation returns `undefined`.

Examples

Using charAt()

The following example displays characters at different locations in the string `"Brave new world"`:

```js
const anyString = "Brave new world";
console.log(`The character at index 0   is '${anyString.charAt()}'`);
// No index was provided, used 0 as default

console.log(`The character at index 0   is '${anyString.charAt(0)}'`);
console.log(`The character at index 1   is '${anyString.charAt(1)}'`);
console.log(`The character at index 2   is '${anyString.charAt(2)}'`);
console.log(`The character at index 3   is '${anyString.charAt(3)}'`);
console.log(`The character at index 4   is '${anyString.charAt(4)}'`);
console.log(`The character at index 999 is '${anyString.charAt(999)}'`);
```

These lines display the following:

```
The character at index 0   is 'B'

The character at index 0   is 'B'
The character at index 1   is 'r'
The character at index 2   is 'a'
The character at index 3   is 'v'
The character at index 4   is 'e'
The character at index 999 is ''
```

`charAt()` may return lone surrogates, which are not valid Unicode characters.

```js
const str = "吉吼";
console.log(str.charAt(0)); // "\ud842", which is not a valid Unicode character
console.log(str.charAt(1)); // "\udfb7", which is not a valid Unicode character
```

To get the full Unicode code point at the given index, use an indexing method that splits by Unicode code points, such as `String.prototype.codePointAt()` and spreading strings into an array of Unicode code points.

```js
const str = "吉吼";
console.log(String.fromCodePoint(str.codePointAt(0))); // "吉"
console.log([...str][0]); // "吉"
```

> **Note:** Avoid re-implementing the solutions above using `charAt()`. The detection of lone surrogates and their pairing is complex, and built-in APIs may be more performant as they directly use the internal representation of the string. Install a polyfill for the APIs mentioned above if necessary.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.charat`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`charAt`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.charAt`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.charCodeAt()`

- String.prototype.codePointAt()
- String.prototype.split()
- String.fromCodePoint()
- JavaScript has a Unicode problem by Mathias Bynens (2013)

JavaScript › Reference › Standard built-in objects › String

String.prototype.charCodeAt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/charCodeAt · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **charCodeAt()** method of String values returns an integer between 0 and 65535 representing the UTF-16 code unit at the given index.

charCodeAt() always indexes the string as a sequence of UTF-16 code units, so it may return lone surrogates. To get the full Unicode code point at the given index, use String.prototype.codePointAt().

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.charCodeAt()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const sentence = "The quick brown fox jumps over the lazy dog.";

const index = 4;

console.log(
  `Character code ${sentence.charCodeAt(index)} is equal to ${sentence.charAt(
    index,
  )}`,
);
// Expected output: "Character code 113 is equal to q"
```

Syntax

js

```
charCodeAt(index)
```

Parameters

index

Zero-based index of the character to be returned. Converted to an integer — undefined is converted to 0.

Return value

An integer between 0 and 65535 representing the UTF-16 code unit value of the character at the specified index. If index is out of range of 0 – str.length - 1, charCodeAt() returns NaN.

Description

Characters in a string are indexed from left to right. The index of the first character is `0`, and the index of the last character in a string called `str` is `str.length - 1`.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). `charCodeAt()` always returns a value that is less than `65536`, because the higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, in order to get a full character with value greater than `65535`, it is necessary to retrieve not only `charCodeAt(i)`, but also `charCodeAt(i + 1)` (as if manipulating a string with two characters), or to use `codePointAt(i)` instead. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Examples

Using charCodeAt()

The following example returns `65`, the Unicode value for A.

```js
"ABC".charCodeAt(0); // returns 65
```

`charCodeAt()` may return lone surrogates, which are not valid Unicode characters.

```js
const str = "吉吼";
console.log(str.charCodeAt(0)); // 55362, or d842, which is not a valid Unicode character
console.log(str.charCodeAt(1)); // 57271, or dfb7, which is not a valid Unicode character
```

To get the full Unicode code point at the given index, use `String.prototype.codePointAt()`.

```js
const str = "吉吼";
console.log(str.codePointAt(0)); // 134071
```

> **Note:** Avoid re-implementing `codePointAt()` using `charCodeAt()`. The translation from UTF-16 surrogates to Unicode code points is complex, and `codePointAt()` may be more performant as it directly uses the internal representation of the string. Install a polyfill for `codePointAt()` if necessary.

Below is a possible algorithm to convert a pair of UTF-16 code units into a Unicode code point, adapted from the Unicode FAQ:

```js
// constants
const LEAD_OFFSET = 0xd800 - (0x10000 >> 10);
const SURROGATE_OFFSET = 0x10000 - (0xd800 << 10) - 0xdc00;

function utf16ToUnicode(lead, trail) {
  return (lead << 10) + trail + SURROGATE_OFFSET;
}
function unicodeToUTF16(codePoint) {
  const lead = LEAD_OFFSET + (codePoint >> 10);
  const trail = 0xdc00 + (codePoint & 0x3ff);
  return [lead, trail];
}
```

(code continues on the next page)

(code continued from the previous page)

```
const str = "吉";
console.log(utf16ToUnicode(str.charCodeAt(0), str.charCodeAt(1))); // 134071
console.log(str.codePointAt(0)); // 134071
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.charcodeat

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
charCodeAt	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.charCodeAt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.fromCharCode()`
- `String.prototype.charAt()`
- `String.fromCodePoint()`
- `String.prototype.codePointAt()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.codePointAt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/codePointAt · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`codePointAt()`** method of `String` values returns a non-negative integer that is the Unicode code point value of the character starting at the given index. Note that the index is still based on UTF-16 code units, not Unicode code points.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.codePointAt()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const icons = "🐞★🛆";

console.log(icons.codePointAt(1));
// Expected output: "9733"
```

Syntax

```js
codePointAt(index)
```

Parameters

`index`

Zero-based index of the character to be returned. Converted to an integer — `undefined` is converted to 0.

Return value

A non-negative integer representing the code point value of the character at the given `index`.

- If `index` is out of the range of `0` – `str.length - 1`, `codePointAt()` returns `undefined`.
- If the element at `index` is a UTF-16 leading surrogate, returns the code point of the surrogate *pair*.
- If the element at `index` is a UTF-16 trailing surrogate, returns *only* the trailing surrogate code unit.

Description

Characters in a string are indexed from left to right. The index of the first character is `0`, and the index of the last character in a string called `str` is `str.length - 1`.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). In UTF-16, each string index is a code unit with value `0` – `65535`. Higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, `codePointAt()` returns a code point that may span two string indices. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Examples

Using codePointAt()

```js
"ABC".codePointAt(0); // 65
"ABC".codePointAt(0).toString(16); // 41

"😍".codePointAt(0); // 128525
"\ud83d\ude0d".codePointAt(0); // 128525
"\ud83d\ude0d".codePointAt(0).toString(16); // 1f60d

"😍".codePointAt(1); // 56845
"\ud83d\ude0d".codePointAt(1); // 56845
"\ud83d\ude0d".codePointAt(1).toString(16); // de0d

"ABC".codePointAt(42); // undefined
```

Looping with codePointAt()

Because using string indices for looping causes the same code point to be visited twice (once for the leading surrogate, once for the trailing surrogate), and the second time `codePointAt()` returns *only* the trailing surrogate, it's better to avoid looping by index.

js · ✗ bad example

```
const str = "\ud83d\udc0e\ud83d\udc71\u2764";

for (let i = 0; i < str.length; i++) {
  console.log(str.codePointAt(i).toString(16));
}
// '1f40e', 'dc0e', '1f471', 'dc71', '2764'
```

Instead, use a `for...of` statement or spread the string, both of which invoke the string's `[Symbol.iterator]()`, which iterates by code points. Then, use `codePointAt(0)` to get the code point of each element.

js

```
for (const codePoint of str) {
  console.log(codePoint.codePointAt(0).toString(16));
}
// '1f40e', '1f471', '2764'

[...str].map((cp) => cp.codePointAt(0).toString(16));
// ['1f40e', '1f471', '2764']
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.codepointat

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
codePointAt	41	12	29	28	9	41	29	28	9	4.0	41	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.codePointAt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.codePointAt` in `core-js`
- es-shims polyfill of `String.prototype.codePointAt`
- `String.fromCodePoint()`
- `String.fromCharCode()`
- `String.prototype.charCodeAt()`
- `String.prototype.charAt()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.concat()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/concat · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`concat()`** method of `String` values concatenates the string arguments to this string and returns a new string.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.concat()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const str1 = "Hello";
const str2 = "World";

console.log(str1.concat(" ", str2));
// Expected output: "Hello World"

console.log(str2.concat(", ", str1));
// Expected output: "World, Hello"
```

Syntax

js

```
concat(str1)
concat(str1, str2)
concat(str1, str2, /* …, */ strN)
```

Parameters

`str1, ..., strN`

One or more strings to concatenate to `str`. Though technically permitted, calling `String.prototype.concat()` with no arguments is a useless operation, because it does not result in observable copying (like `Array.prototype.concat()`), since strings are immutable. It should only happen if you are spreading an array of strings as arguments, and that array happens to be empty.

Return value

A new string containing the combined text of the strings provided.

Description

The `concat()` function concatenates the string arguments to the calling string and returns a new string.

If the arguments are not of the type string, they are converted to string values before concatenating.

The `concat()` method is very similar to the addition/string concatenation operators (`+`, `+=`), except that `concat()` coerces its arguments directly to strings, while addition coerces its operands to primitives first. For more information, see the reference page for the `+` operator.

Examples

Using concat()

The following example combines strings into a new string.

```js
const hello = "Hello, ";
console.log(hello.concat("Kevin", ". Have a nice day."));
// Hello, Kevin. Have a nice day.

const greetList = ["Hello", " ", "Venkat", "!"];
"".concat(...greetList); // "Hello Venkat!"

"".concat({}); // "[object Object]"
"".concat([]); // ""
"".concat(null); // "null"
"".concat(true); // "true"
"".concat(4, 5); // "45"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.concat

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
concat	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.concat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Array.prototype.concat()`
- Addition (+)

JavaScript › Reference › Standard built-in objects › String

String.prototype.endsWith()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/endsWith · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`endsWith()`** method of String values determines whether a string ends with the characters of this string, returning `true` or `false` as appropriate.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.endsWith()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const str1 = "Cats are the best!";

console.log(str1.endsWith("best!"));
// Expected output: true

console.log(str1.endsWith("best", 17));
// Expected output: true

const str2 = "Is this a question?";

console.log(str2.endsWith("question"));
// Expected output: false
```

Syntax

js

```
endsWith(searchString)
endsWith(searchString, endPosition)
```

Parameters

`searchString`

The characters to be searched for at the end of `str`. Cannot be a regex. All values that are not regexes are coerced to strings, so omitting it or passing `undefined` causes `endsWith()` to search for the string `"undefined"`, which is rarely what you want.

`endPosition` Optional

The end position at which `searchString` is expected to be found (the index of `searchString`'s last character plus 1). Defaults to `str.length`.

Return value

`true` if the given characters are found at the end of the string, including when `searchString` is an empty string; otherwise, **`false`**.

Exceptions

`TypeError`

Thrown if `searchString` is a regex.

Description

This method lets you determine whether or not a string ends with another string. This method is case-sensitive.

Examples

Using endsWith()

```js
const str = "To be, or not to be, that is the question.";

console.log(str.endsWith("question.")); // true
console.log(str.endsWith("to be")); // false
console.log(str.endsWith("to be", 19)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.endswith

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
endsWith	41	12	17	28	9	36	17	24	9	3.0	37	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.endsWith`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.endsWith` in `core-js`
- es-shims polyfill of `String.prototype.endsWith`
- `String.prototype.startsWith()`
- `String.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.fixed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/fixed · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`fixed()`** method of `String` values creates a string that embeds this string in a `<tt>` element (`<tt>str</tt>`), which causes this string to be displayed in a fixed-width font.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `fixed()`, the `<tt>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```js
fixed()
```

Parameters

None.

Return value

A string beginning with a `<tt>` start tag, then the text `str`, and then a `</tt>` end tag.

Examples

Using fixed()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.fixed();
```

This will create the following HTML:

```html
<tt>Hello, world</tt>
```

> **Warning:** This markup is invalid, because `tt` is no longer a valid element.

Instead of using `fixed()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `font-family` through the `element.style` attribute:

```js
document.getElementById("yourElemId").style.fontFamily = "monospace";
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.fixed`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fixed` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.fixed`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.fixed` in `core-js`
- es-shims polyfill of `String.prototype.fixed`
- HTML wrapper methods
- `<tt>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.fontcolor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/fontcolor · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`fontcolor()`** method of `String` values creates a string that embeds this string in a `<font>` element (`<font color="...">str</font>`), which causes this string to be displayed in the specified font color.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `fontcolor()`, the `<font>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```js
fontcolor(color)
```

Parameters

`color`

A string expressing the color as a hexadecimal RGB triplet or as a string literal. String literals for color names are listed in the CSS color reference.

Return value

A string beginning with a `<font color="color">` start tag (double quotes in `color` are replaced with `"`), then the text `str`, and then a `</font>` end tag.

Description

The `fontcolor()` method itself simply joins the string parts together without any validation or normalization. However, to create valid `<font>` elements, if you express color as a hexadecimal RGB triplet, you must use the format `rrggbb`. For example, the hexadecimal RGB values for salmon are red=FA, green=80, and blue=72, so the RGB triplet for salmon is `"FA8072"`.

Examples

Using fontcolor()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.fontcolor("red");
```

This will create the following HTML:

```html
<font color="red">Hello, world</font>
```

> **Warning:** This markup is invalid, because `font` is no longer a valid element.

Instead of using `fontcolor()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `color` through the `element.style` attribute:

```js
document.getElementById("yourElemId").style.color = "red";
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.fontcolor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fontcolor` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.fontcolor`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.fontcolor` in `core-js`
- es-shims polyfill of `String.prototype.fontcolor`
- HTML wrapper methods
- `<font>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.fontsize()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/fontsize · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`fontsize()`** method of `String` values creates a string that embeds this string in a `<font>` element (`<font size="...">str</font>`), which causes this string to be displayed in the specified font size.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `fontsize()`, the `<font>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```
fontsize(size)
```

Parameters

`size`

An integer between 1 and 7, or a string representing a signed integer between 1 and 7.

Return value

A string beginning with a `<font size="size">` start tag (double quotes in `size` are replaced with `"`), then the text `str`, and then a `</font>` end tag.

Description

The `fontsize()` method itself simply joins the string parts together without any validation or normalization. However, to create valid `<font>` elements, When you specify size as an integer, you set the font size of `str` to one of the 7 defined sizes. You can specify `size` as a string such as `"-2"` or `"+3"` to adjust the font size of `str` relative to 3, the default value.

Examples

Using fontsize()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.fontsize(7);
```

This will create the following HTML:

```html
<font size="7">Hello, world</font>
```

> **Warning:** This markup is invalid, because `font` is no longer a valid element.

Instead of using `fontsize()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `font-size` through the `element.style` attribute:

```js
document.getElementById("yourElemId").style.fontSize = "7pt";
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.fontsize

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fontsize` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.fontsize`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.fontsize` in `core-js`
- es-shims polyfill of `String.prototype.fontsize`
- HTML wrapper methods
- `<font>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.includes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/includes · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`includes()`** method of `String` values performs a case-sensitive search to determine whether a given string may be found within this string, returning `true` or `false` as appropriate.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.includes()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const sentence = "The quick brown fox jumps over the lazy dog.";

const word = "fox";

console.log(
  `The word "${word}" ${
    sentence.includes(word) ? "is" : "is not"
  } in the sentence`,
);
// Expected output: "The word "fox" is in the sentence"
```

Syntax

js

```
includes(searchString)
includes(searchString, position)
```

Parameters

`searchString`

A string to be searched for within `str`. Cannot be a regex. All values that are not regexes are coerced to strings, so omitting it or passing `undefined` causes `includes()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

The position within the string at which to begin searching for `searchString`. (Defaults to `0`.)

Return value

`true` if the search string is found anywhere within the given string, including when `searchString` is an empty string; otherwise, **`false`**.

Exceptions

`TypeError`

Thrown if `searchString` is a regex.

Description

This method lets you determine whether or not a string includes another string.

Case-sensitivity

The `includes()` method is case sensitive. For example, the following expression returns `false`:

```js
"Blue Whale".includes("blue"); // returns false
```

You can work around this constraint by transforming both the original string and the search string to all lowercase:

```js
"Blue Whale".toLowerCase().includes("blue"); // returns true
```

Examples

Using includes()

```js
const str = "To be, or not to be, that is the question.";

console.log(str.includes("To be")); // true
console.log(str.includes("question")); // true
console.log(str.includes("nonexistent")); // false
console.log(str.includes("To be", 1)); // false
console.log(str.includes("TO BE")); // false
console.log(str.includes("")); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.includes`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`includes`	41	12	40[1]	28	9	41	40[1]	28	9	4.0	41	9	1.0.0	1.0	4.0.0

1. Also from version 18–48: uses the non-standard name `contains`.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.includes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.includes` in `core-js`
- es-shims polyfill of `String.prototype.includes`
- `Array.prototype.includes()`
- `TypedArray.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.startsWith()`
- `String.prototype.endsWith()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.indexOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/indexOf · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`indexOf()`** method of `String` values searches this string and returns the index of the first occurrence of the specified substring. It takes an optional starting position and returns the first occurrence of the specified substring at an index greater than or equal to the specified number.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.indexOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

const searchTerm = "dog";
const indexOfFirst = paragraph.indexOf(searchTerm);

console.log(`The index of the first "${searchTerm}" is ${indexOfFirst}`);
// Expected output: "The index of the first "dog" is 15"

console.log(
  `The index of the second "${searchTerm}" is ${paragraph.indexOf(
    searchTerm,
    indexOfFirst + 1,
  )}`,
);
// Expected output: "The index of the second "dog" is 38"
```

Syntax

js

```
indexOf(searchString)
indexOf(searchString, position)
```

Parameters

`searchString`

Substring to search for. All values are coerced to strings, so omitting it or passing `undefined` causes `indexOf()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

The method returns the index of the first occurrence of the specified substring at a position greater than or equal to `position`, which defaults to `0`. If `position` is greater than the length of the calling string, the method doesn't search the calling string at all. If `position` is less than zero, the method behaves as it would if `position` were `0`.

- `'hello world hello'.indexOf('o', -5)` returns `4` — because it causes the method to behave as if the second argument were `0`, and the first occurrence of `o` at a position greater or equal to `0` is at position `4`.
- `'hello world hello'.indexOf('world', 12)` returns `-1` — because, while it's true the substring `world` occurs at index `6`, that position is not greater than or equal to `12`.
- `'hello world hello'.indexOf('o', 99)` returns `-1` — because `99` is greater than the length of `hello world hello`, which causes the method to not search the string at all.

Return value

The index of the first occurrence of `searchString` found, or `-1` if not found.

Return value when using an empty search string

Searching for an empty search string produces strange results. With no second argument, or with a second argument whose value is less than the calling string's length, the return value is the same as the value of the second argument:

```js
"hello world".indexOf(""); // returns 0
"hello world".indexOf("", 0); // returns 0
"hello world".indexOf("", 3); // returns 3
"hello world".indexOf("", 8); // returns 8
```

However, with a second argument whose value is greater than or equal to the string's length, the return value is the string's length:

```js
"hello world".indexOf("", 11); // returns 11
"hello world".indexOf("", 13); // returns 11
"hello world".indexOf("", 22); // returns 11
```

In the former instance, the method behaves as if it found an empty string just after the position specified in the second argument. In the latter instance, the method behaves as if it found an empty string at the end of the calling string.

Description

Strings are zero-indexed: The index of a string's first character is `0`, and the index of a string's last character is the length of the string minus 1.

```js
"Blue Whale".indexOf("Blue"); // returns  0
"Blue Whale".indexOf("Wale"); // returns -1
"Blue Whale".indexOf("Whale", 0); // returns  5
"Blue Whale".indexOf("Whale", 5); // returns  5
"Blue Whale".indexOf("Whale", 7); // returns -1
"Blue Whale".indexOf(""); // returns  0
"Blue Whale".indexOf("", 9); // returns  9
"Blue Whale".indexOf("", 10); // returns 10
"Blue Whale".indexOf("", 11); // returns 10
```

The `indexOf()` method is case sensitive. For example, the following expression returns `-1`:

```js
"Blue Whale".indexOf("blue"); // returns -1
```

Checking occurrences

When checking if a specific substring occurs within a string, the correct way to check is test whether the return value is `-1`:

```js
"Blue Whale".indexOf("Blue") !== -1; // true; found 'Blue' in 'Blue Whale'
"Blue Whale".indexOf("Wale") !== -1; // false; no 'Wale' in 'Blue Whale'
```

Examples

Using indexOf()

The following example uses `indexOf()` to locate substrings in the string `"Brave new world"`.

```js
const str = "Brave new world";

console.log(str.indexOf("w")); // 8
console.log(str.indexOf("new")); // 6
```

indexOf() and case-sensitivity

The following example defines two string variables.

The variables contain the same string, except that the second string contains uppercase letters. The first `console.log()` method displays `19`. But because the `indexOf()` method is case sensitive, the string `"cheddar"` is not found in `myCapString`, so the second `console.log()` method displays `-1`.

```js
const myString = "brie, pepper jack, cheddar";
const myCapString = "Brie, Pepper Jack, Cheddar";

console.log(myString.indexOf("cheddar")); // 19
console.log(myCapString.indexOf("cheddar")); // -1
```

Using indexOf() to count occurrences of a letter in a string

The following example sets `count` to the number of occurrences of the letter `e` in the string `str`:

```js
const str = "To be, or not to be, that is the question.";
let count = 0;
let position = str.indexOf("e");

while (position !== -1) {
  count++;
  position = str.indexOf("e", position + 1);
}

console.log(count); // 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.indexof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
indexOf	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.indexOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.charAt()`
- `String.prototype.lastIndexOf()`
- `String.prototype.includes()`
- `String.prototype.split()`
- `Array.prototype.indexOf()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.isWellFormed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/isWellFormed · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since October 2023.

The **`isWellFormed()`** method of `String` values returns a boolean indicating whether this string contains any lone surrogates.

Syntax

```js
isWellFormed()
```

Parameters

None.

Return value

Returns `true` if this string does not contain any lone surrogates, `false` otherwise.

Description

Strings in JavaScript are UTF-16 encoded. UTF-16 encoding has the concept of *surrogate pairs*, which is introduced in detail in the UTF-16 characters, Unicode code points, and grapheme clusters section.

`isWellFormed()` allows you to test whether a string is well-formed (i.e., does not contain any lone surrogates). Compared to a custom implementation, `isWellFormed()` is more efficient, as engines can directly access the internal representation of strings. If you need to convert a string to a well-formed string, use the `toWellFormed()` method. `isWellFormed()` allows you to handle ill-formed strings differently from well-formed strings, such as throwing an error or marking it as invalid.

Examples

Using isWellFormed()

```js
const strings = [
  // Lone leading surrogate
  "ab\uD800",
  "ab\uD800c",
  // Lone trailing surrogate
  "\uDFFFab",
  "c\uDFFFab",
  // Well-formed
  "abc",
  "ab\uD83D\uDE04c",
];

for (const str of strings) {
  console.log(str.isWellFormed());
}
// Logs:
// false
// false
// false
// false
// true
// true
```

Avoiding errors in encodeURI()

`encodeURI` throws an error if the string passed is not well-formed. This can be avoided by using `isWellFormed()` to test the string before passing it to `encodeURI()`.

```js
const illFormed = "https://example.com/search?q=\uD800";

try {
  encodeURI(illFormed);
} catch (e) {
  console.log(e); // URIError: URI malformed
}

if (illFormed.isWellFormed()) {
  console.log(encodeURI(illFormed));
} else {
  console.warn("Ill-formed strings encountered."); // Ill-formed strings encountered.
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.iswellformed

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
isWellFormed	111	111	119	97	16.4	111	119	75	16.4	22.0	111	16.4	1.0.0	1.32	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.isWellFormed`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.isWellFormed` in `core-js`
- es-shims polyfill of `String.prototype.isWellFormed`
- `String.prototype.toWellFormed()`
- `String.prototype.normalize()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.italics()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/italics · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`italics()`** method of `String` values creates a string that embeds this string in an `<i>` element (`<i>str</i>`), which causes this string to be displayed as italic.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
italics()
```

Parameters

None.

Return value

A string beginning with an `<i>` start tag, then the text `str`, and then an `</i>` end tag.

Examples

Using italics()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.italics();
```

This will create the following HTML:

```
<i>Hello, world</i>
```

Instead of using `italics()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("i");
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.italics

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`italics` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.italics`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.italics` in `core-js`
- es-shims polyfill of `String.prototype.italics`
- HTML wrapper methods
- `<i>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.lastIndexOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/lastIndexOf · MDN page last modified 2025-10-01

Baseline: Widely available — available across major browsers since July 2015.

The **`lastIndexOf()`** method of `String` values searches this string and returns the index of the last occurrence of the specified substring. It takes an optional starting position and returns the last occurrence of the specified substring at an index less than or equal to the specified number.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.lastIndexOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

const searchTerm = "dog";

console.log(
  `Index of the last "${searchTerm}" is ${paragraph.lastIndexOf(searchTerm)}`,
);
// Expected output: "Index of the last "dog" is 38"
```

Syntax

js

```
lastIndexOf(searchString)
lastIndexOf(searchString, position)
```

Parameters

`searchString`

Substring to search for. All values are coerced to strings, so omitting it or passing `undefined` causes `lastIndexOf()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

The method returns the index of the last occurrence of the specified substring at a position less than or equal to `position`, which defaults to `Infinity`. If `position` is greater than the length of the calling string, the method searches the entire string. If `position` is less than `0`, the behavior is the same as for `0` — that is, the method looks for the specified substring only at index `0`.

- `'hello world hello'.lastIndexOf('world', 4)` returns `-1` — because, while the substring `world` does occurs at index `6`, that position is not less than or equal to `4`.
- `'hello world hello'.lastIndexOf('hello', 99)` returns `12` — because the last occurrence of `hello` at a position less than or equal to `99` is at position `12`.
- `'hello world hello'.lastIndexOf('hello', 0)` and `'hello world hello'.lastIndexOf('hello', -5)` both return `0` — because both cause the method to only look for `hello` at index `0`.

Return value

The index of the last occurrence of `searchString` found, or `-1` if not found.

Description

Strings are zero-indexed: The index of a string's first character is `0`, and the index of a string's last character is the length of the string minus 1.

```js
"canal".lastIndexOf("a"); // returns 3
"canal".lastIndexOf("a", 2); // returns 1
"canal".lastIndexOf("a", 0); // returns -1
"canal".lastIndexOf("x"); // returns -1
"canal".lastIndexOf("c", -5); // returns 0
"canal".lastIndexOf("c", 0); // returns 0
"canal".lastIndexOf(""); // returns 5
"canal".lastIndexOf("", 2); // returns 2
```

Case-sensitivity

The `lastIndexOf()` method is case sensitive. For example, the following expression returns `-1`:

```js
"Blue Whale, Killer Whale".lastIndexOf("blue"); // returns -1
```

Examples

Using indexOf() and lastIndexOf()

The following example uses `indexOf()` and `lastIndexOf()` to locate values in the string `"Brave, Brave New World"`.

```js
const anyString = "Brave, Brave New World";

console.log(anyString.indexOf("Brave")); // 0
console.log(anyString.lastIndexOf("Brave")); // 7
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.lastindexof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`lastIndexOf`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.lastIndexOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.charAt()`
- `String.prototype.indexOf()`
- `String.prototype.split()`
- `Array.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.link()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/link · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`link()`** method of `String` values creates a string that embeds this string in an `<a>` element (`<a href="...">str</a>`), to be used as a hypertext link to another URL.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
link(url)
```

Parameters

`url`

Any string that specifies the `href` attribute of the `<a>` element; it should be a valid URL (relative or absolute), with any `&` characters escaped as `&`.

Return value

A string beginning with an `<a href="url">` start tag (double quotes in `url` are replaced with `"`), then the text `str`, and then an `</a>` end tag.

Examples

Using link()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "MDN Web Docs";

document.body.innerHTML = contentString.link("https://developer.mozilla.org/");
```

This will create the following HTML:

```html
<a href="https://developer.mozilla.org/">MDN Web Docs</a>
```

Instead of using `link()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```js
const contentString = "MDN Web Docs";
const elem = document.createElement("a");
elem.href = "https://developer.mozilla.org/";
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.link

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`link` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.link`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.link` in `core-js`
- es-shims polyfill of `String.prototype.link`
- HTML wrapper methods
- `<a>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.localeCompare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/localeCompare · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since July 2015.

The **`localeCompare()`** method of `String` values returns a number indicating whether this string comes before, or after, or is the same as the given string in sort order. In implementations with `Intl.Collator` API support, this method delegates to `Intl.Collator`.

When comparing large numbers of strings, such as in sorting large arrays, it is better to create an `Intl.Collator` object and use the function provided by its `compare()` method.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.localeCompare()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const a = "réservé"; // With accents, lowercase
const b = "RESERVE"; // No accents, uppercase

console.log(a.localeCompare(b));
// Expected output: 1
console.log(a.localeCompare(b, "en", { sensitivity: "base" }));
// Expected output: 0
```

Syntax

js

```
localeCompare(compareString)
localeCompare(compareString, locales)
localeCompare(compareString, locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.Collator` API, these parameters correspond exactly to the `Intl.Collator()` constructor's parameters. Implementations without `Intl.Collator` support are asked to ignore both parameters, making the comparison result returned entirely implementation-dependent — it's only required to be *consistent*.

`compareString`

The string against which the `referenceStr` is compared. All values are coerced to strings, so omitting it or passing `undefined` causes `localeCompare()` to compare against the string `"undefined"`, which is rarely what you want.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.Collator()` constructor.

In implementations without `Intl.Collator` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.Collator()` constructor.

In implementations without `Intl.Collator` support, this parameter is ignored.

See the `Intl.Collator()` constructor for details on the `locales` and `options` parameters and how to use them.

Return value

A **negative** number if `referenceStr` occurs before `compareString`; **positive** if the `referenceStr` occurs after `compareString`; `0` if they are equivalent.

In implementations with `Intl.Collator`, this is equivalent to `new Intl.Collator(locales, options).compare(referenceStr, compareString)`.

Description

Returns an integer indicating whether the `referenceStr` comes before, after or is equivalent to the `compareString`.

- Negative when the `referenceStr` occurs before `compareString`
- Positive when the `referenceStr` occurs after `compareString`
- Returns `0` if they are equivalent

> **Warning:** Do not rely on exact return values of `-1` or `1`!
>
> Negative and positive integer results vary between browsers (as well as between browser versions) because the ECMAScript specification only mandates negative and positive values. Some browsers may return `-2` or `2`, or even some other negative or positive value.

Examples

Using localeCompare()

```js
// The letter "a" is before "c" yielding a negative value
"a".localeCompare("c"); // -2 or -1 (or some other negative value)

// Alphabetically the word "check" comes after "against" yielding a positive value
"check".localeCompare("against"); // 2 or 1 (or some other positive value)

// "a" and "a" are equivalent yielding a neutral value of zero
"a".localeCompare("a"); // 0
```

Sort an array

`localeCompare()` enables case-insensitive sorting for an array.

```js
const items = ["réservé", "Premier", "Cliché", "communiqué", "café", "Adieu"];
items.sort((a, b) => a.localeCompare(b, "fr", { ignorePunctuation: true }));
// ['Adieu', 'café', 'Cliché', 'communiqué', 'Premier', 'réservé']
```

Check browser support for extended arguments

The `locales` and `options` arguments are not supported in all browsers yet.

To check whether an implementation supports them, use the `"i"` argument (a requirement that illegal language tags are rejected) and look for a `RangeError` exception:

```js
function localeCompareSupportsLocales() {
  try {
    "foo".localeCompare("bar", "i");
  } catch (e) {
    return e.name === "RangeError";
  }
  return false;
}
```

Using locales

The results provided by `localeCompare()` vary between languages. In order to get the sort order of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
console.log("ä".localeCompare("z", "de")); // a negative value: in German, ä sorts before z
console.log("ä".localeCompare("z", "sv")); // a positive value: in Swedish, ä sorts after z
```

Using options

The results provided by `localeCompare()` can be customized using the `options` argument:

```js
// in German, ä has a as the base letter
console.log("ä".localeCompare("a", "de", { sensitivity: "base" })); // 0

// in Swedish, ä and a are separate base letters
console.log("ä".localeCompare("a", "sv", { sensitivity: "base" })); // a positive value
```

Numeric sorting

```js
// by default, "2" > "10"
console.log("2".localeCompare("10")); // 1

// numeric using options:
console.log("2".localeCompare("10", undefined, { numeric: true })); // -1

// numeric using locales tag:
console.log("2".localeCompare("10", "en-u-kn-true")); // -1
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.localecompare`
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sup-String.prototype.localeCompare`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`localeCompare`	1	12	1	7	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`locales` parameter	24	12	29	15	10	26	56	**No**	10	1.5	**No**	10	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	12	29	15	10	26	56	**No**	10	1.5	**No**	10	1.0.0	1.0	0.12.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.localeCompare`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Collator`

JavaScript › Reference › Standard built-in objects › String

String.prototype.match()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/match · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`match()`** method of `String` values retrieves the result of matching this string against a regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.match()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const paragraph = "The quick brown fox jumps over the lazy dog. It barked.";
const regex = /[A-Z]/g;
const found = paragraph.match(regex);

console.log(found);
// Expected output: Array ["T", "I"]
```

Syntax

js

```
match(regexp)
```

Parameters

`regexp`

A regular expression object, or any object that has a `Symbol.match` method.

If `regexp` is not a `RegExp` object and does not have a `Symbol.match` method, it is implicitly converted to a `RegExp` by using `new RegExp(regexp)`.

If you don't give any parameter and use the `match()` method directly, you will get an `Array` with an empty string: `[""]`, because this is equivalent to `match(/(?:)/)`.

Return value

An `Array` whose contents depend on the presence or absence of the global (g) flag, or `null` if no matches are found.

- If the g flag is used, all results matching the complete regular expression will be returned, but capturing groups are not included.
- If the g flag is not used, only the first complete match and its related capturing groups are returned. In this case, `match()` will return the same result as `RegExp.prototype.exec()` (an array with some extra properties).

Description

The implementation of `String.prototype.match` doesn't do much other than calling the `Symbol.match` method of the argument with the string as the first parameter. The actual implementation comes from `RegExp.prototype[Symbol.match]()`.

- If you need to know if a string matches a regular expression `RegExp`, use `RegExp.prototype.test()`.
- If you only want the first match found, you might want to use `RegExp.prototype.exec()` instead.
- If you want to obtain capture groups and the global flag is set, you need to use `RegExp.prototype.exec()` or `String.prototype.matchAll()` instead.

For more information about the semantics of `match()` when a regex is passed, see `RegExp.prototype[Symbol.match]()`.

Examples

Using match()

In the following example, `match()` is used to find `"Chapter"` followed by one or more numeric characters followed by a decimal point and numeric character zero or more times.

The regular expression includes the `i` flag so that upper/lower case differences will be ignored.

```js
const str = "For more information, see Chapter 3.4.5.1";
const re = /see (chapter \d+(\.\d)*)/i;
const found = str.match(re);

console.log(found);
// [
//   'see Chapter 3.4.5.1',
//   'Chapter 3.4.5.1',
//   '.1',
//   index: 22,
//   input: 'For more information, see Chapter 3.4.5.1',
//   groups: undefined
// ]
```

In the match result above, `'see Chapter 3.4.5.1'` is the whole match. `'Chapter 3.4.5.1'` was captured by `(chapter \d+(\.\d)*)`. `'.1'` was the last value captured by `(\.\d)`. The `index` property (`22`) is the zero-based index of the whole match. The `input` property is the original string that was parsed.

Using global and ignoreCase flags with match()

The following example demonstrates the use of the global flag and ignore-case flag with `match()`. All letters `A` through `E` and `a` through `e` are returned, each its own element in the array.

```js
const str = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
const regexp = /[a-e]/gi;
const matches = str.match(regexp);

console.log(matches);
// ['A', 'B', 'C', 'D', 'E', 'a', 'b', 'c', 'd', 'e']
```

> **Note:** See also `String.prototype.matchAll()` and Advanced searching with flags.

Using named capturing groups

In browsers which support named capturing groups, the following code captures `"fox"` or `"cat"` into a group named `animal`:

```js
const paragraph = "The quick brown fox jumps over the lazy dog. It barked.";

const capturingRegex = /(?<animal>fox|cat) jumps over/;
const found = paragraph.match(capturingRegex);
console.log(found.groups); // {animal: "fox"}
```

Using match() with no parameter

```js
const str = "Nothing will come of nothing.";

str.match(); // returns [""]
```

Using match() with a non-RegExp implementing `[Symbol.match]()`

If an object has a `Symbol.match` method, it can be used as a custom matcher. The return value of `Symbol.match` becomes the return value of `match()`.

```js
const str = "Hmm, this is interesting.";

str.match({
  [Symbol.match](str) {
    return ["Yes, it's interesting."];
  },
}); // returns ["Yes, it's interesting."]
```

A non-RegExp as the parameter

When the `regexp` parameter is a string or a number, it is implicitly converted to a `RegExp` by using `new RegExp(regexp)`.

```js
const str1 =
  "All numbers except NaN satisfy <= Infinity and >= -Infinity in JavaScript.";
const str2 =
  "My grandfather is 65 years old and My grandmother is 63 years old.";
const str3 = "The contract was declared null and void.";
str1.match("number"); // "number" is a string. returns ["number"]
str1.match(NaN); // the type of NaN is the number. returns ["NaN"]
str1.match(Infinity); // the type of Infinity is the number. returns ["Infinity"]
str1.match(-Infinity); // returns ["-Infinity"]
str2.match(65); // returns ["65"]
str3.match(null); // returns ["null"]
```

This may have unexpected results if special characters are not properly escaped.

```js
console.log("123".match("1.3")); // [ "123" ]
```

This is a match because `.` in a regex matches any character. In order to make it only match specifically a dot character, you need to escape the input.

```js
console.log("123".match("1\\.3")); // null
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.match

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`match`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.match`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.match` in `core-js` with fixes and implementation of modern behavior like `Symbol.match` support
- Regular expressions guide
- `String.prototype.matchAll()`
- `RegExp`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.matchAll()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/matchAll · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`matchAll()`** method of `String` values returns an iterator of all results matching this string against a regular expression, including capturing groups.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.matchAll()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regexp = /t(e)(st(\d?))/g;
const str = "test1test2";

const array = [...str.matchAll(regexp)];

console.log(array[0]);
// Expected output: Array ["test1", "e", "st1", "1"]

console.log(array[1]);
// Expected output: Array ["test2", "e", "st2", "2"]
```

Syntax

```js
matchAll(regexp)
```

Parameters

`regexp`

A regular expression object, or any object that has a `Symbol.matchAll` method.

If `regexp` is not a `RegExp` object and does not have a `Symbol.matchAll` method, it is implicitly converted to a `RegExp` by using `new RegExp(regexp, 'g')`.

If `regexp` is a regex, then it must have the global (`g`) flag set, or a `TypeError` is thrown.

Return value

An iterable iterator object (which is not restartable) of matches or an empty iterator if no matches are found. Each value yielded by the iterator is an array with the same shape as the return value of `RegExp.prototype.exec()`.

Exceptions

`TypeError`

Thrown if the `regexp` is a regex that does not have the global (`g`) flag set (its `flags` property does not contain `"g"`).

Description

The implementation of `String.prototype.matchAll` doesn't do much other than calling the `Symbol.matchAll` method of the argument with the string as the first parameter (apart from the extra input validation that the regex is global). The actual implementation comes from `RegExp.prototype[Symbol.matchAll]()`.

Examples

Regexp.prototype.exec() and matchAll()

Without `matchAll()`, it's possible to use calls to `regexp.exec()` (and regexes with the g flag) in a loop to obtain all the matches:

```js
const regexp = /foo[a-z]*/g;
const str = "table football, foosball";
let match;

while ((match = regexp.exec(str)) !== null) {
  console.log(
    `Found ${match[0]} start=${match.index} end=${regexp.lastIndex}.`,
  );
}
// Found football start=6 end=14.
// Found foosball start=16 end=24.
```

With `matchAll()` available, you can avoid the `while` loop and `exec` with g. Instead, you get an iterator to use with the more convenient `for...of`, array spreading, or `Array.from()` constructs:

```js
const regexp = /foo[a-z]*/g;
const str = "table football, foosball";
const matches = str.matchAll(regexp);

for (const match of matches) {
  console.log(
    `Found ${match[0]} start=${match.index} end=${
      match.index + match[0].length
    }.`,
  );
}
// Found football start=6 end=14.
// Found foosball start=16 end=24.

// matches iterator is exhausted after the for...of iteration
// Call matchAll again to create a new iterator
Array.from(str.matchAll(regexp), (m) => m[0]);
// [ "football", "foosball" ]
```

`matchAll` will throw an exception if the g flag is missing.

```js
const regexp = /[a-c]/;
const str = "abc";
str.matchAll(regexp);
// TypeError
```

`matchAll` internally makes a clone of the `regexp` — so, unlike `regexp.exec()`, `lastIndex` does not change as the string is scanned.

```js
const regexp = /[a-c]/g;
regexp.lastIndex = 1;
const str = "abc";
Array.from(str.matchAll(regexp), (m) => `${regexp.lastIndex} ${m[0]}`);
// [ "1 b", "1 c" ]
```

However, this means that unlike using `regexp.exec()` in a loop, you can't mutate `lastIndex` to make the regex advance or rewind.

Better access to capturing groups (than String.prototype.match())

Another compelling reason for `matchAll` is the improved access to capture groups.

Capture groups are ignored when using `match()` with the global g flag:

```js
const regexp = /t(e)(st(\d?))/g;
const str = "test1test2";

str.match(regexp); // ['test1', 'test2']
```

Using `matchAll`, you can access capture groups easily:

```js
const array = [...str.matchAll(regexp)];

array[0];
// ['test1', 'e', 'st1', '1', index: 0, input: 'test1test2', length: 4]
array[1];
// ['test2', 'e', 'st2', '2', index: 5, input: 'test1test2', length: 4]
```

Using matchAll() with a non-RegExp implementing `[Symbol.matchAll]()`

If an object has a `Symbol.matchAll` method, it can be used as a custom matcher. The return value of `Symbol.matchAll` becomes the return value of `matchAll()`.

```js
const str = "Hmm, this is interesting.";

str.matchAll({
  [Symbol.matchAll](str) {
    return [["Yes, it's interesting."]];
  },
}); // returns [["Yes, it's interesting."]]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.matchall

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`matchAll`	73	79	67	60	13	73	67	52	13	11.0	73	13	1.0.0	1.0	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.matchAll`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.matchAll` in `core-js`
- es-shims polyfill of `String.prototype.matchAll`
- Regular expressions guide
- Groups and backreferences guide
- `String.prototype.match()`
- `RegExp`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `RegExp.prototype[Symbol.matchAll]()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.normalize()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/normalize · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`normalize()`** method of `String` values returns the Unicode Normalization Form of this string.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.normalize()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const name1 = "\u0041\u006d\u00e9\u006c\u0069\u0065";
const name2 = "\u0041\u006d\u0065\u0301\u006c\u0069\u0065";

console.log(`${name1}, ${name2}`);
// Expected output: "Amélie, Amélie"
console.log(name1 === name2);
// Expected output: false
console.log(name1.length === name2.length);
// Expected output: false

const name1NFC = name1.normalize("NFC");
const name2NFC = name2.normalize("NFC");

console.log(`${name1NFC}, ${name2NFC}`);
// Expected output: "Amélie, Amélie"
console.log(name1NFC === name2NFC);
// Expected output: true
console.log(name1NFC.length === name2NFC.length);
// Expected output: true
```

Syntax

js

```
normalize()
normalize(form)
```

Parameters

`form` Optional

One of `"NFC"`, `"NFD"`, `"NFKC"`, or `"NFKD"`, specifying the Unicode Normalization Form. If omitted or `undefined`, `"NFC"` is used.

These values have the following meanings:

`"NFC"`

Canonical Decomposition, followed by Canonical Composition.

`"NFD"`

Canonical Decomposition.

`"NFKC"`

Compatibility Decomposition, followed by Canonical Composition.

`"NFKD"`

Compatibility Decomposition.

Return value

A string containing the Unicode Normalization Form of the given string.

Exceptions

`RangeError`

Thrown if `form` isn't one of the values specified above.

Description

Unicode assigns a unique numerical value, called a *code point*, to each character. For example, the code point for `"A"` is given as U+0041. However, sometimes more than one code point, or sequence of code points, can represent the same abstract character — the character `"ñ"` for example can be represented by either of:

- The single code point U+00F1.
- The code point for `"n"` (U+006E) followed by the code point for the combining tilde (U+0303).

```js
const string1 = "\u00F1";
const string2 = "\u006E\u0303";

console.log(string1); // ñ
console.log(string2); // ñ
```

However, since the code points are different, string comparison will not treat them as equal. And since the number of code points in each version is different, they even have different lengths.

```js
const string1 = "\u00F1"; // ñ
const string2 = "\u006E\u0303"; // ñ

console.log(string1 === string2); // false
console.log(string1.length); // 1
console.log(string2.length); // 2
```

The `normalize()` method helps solve this problem by converting a string into a normalized form common for all sequences of code points that represent the same characters. There are two main normalization forms, one based on **canonical equivalence** and the other based on **compatibility**.

Canonical equivalence normalization

In Unicode, two sequences of code points have canonical equivalence if they represent the same abstract characters, and should always have the same visual appearance and behavior (for example, they should always be sorted in the same way).

You can use `normalize()` using the `"NFD"` or `"NFC"` arguments to produce a form of the string that will be the same for all canonically equivalent strings. In the example below we normalize two representations of the character `"ñ"`:

```js
let string1 = "\u00F1"; // ñ
let string2 = "\u006E\u0303"; // ñ

string1 = string1.normalize("NFD");
string2 = string2.normalize("NFD");

console.log(string1 === string2); // true
console.log(string1.length); // 2
console.log(string2.length); // 2
```

Composed and decomposed forms

Note that the length of the normalized form under `"NFD"` is `2`. That's because `"NFD"` gives you the **decomposed** version of the canonical form, in which single code points are split into multiple combining ones. The decomposed canonical form for `"ñ"` is `"\u006E\u0303"`.

You can specify `"NFC"` to get the **composed** canonical form, in which multiple code points are replaced with single code points where possible. The composed canonical form for `"ñ"` is `"\u00F1"`:

```js
let string1 = "\u00F1"; // ñ
let string2 = "\u006E\u0303"; // ñ

string1 = string1.normalize("NFC");
string2 = string2.normalize("NFC");

console.log(string1 === string2); // true
console.log(string1.length); // 1
console.log(string2.length); // 1
console.log(string2.codePointAt(0).toString(16)); // f1
```

Compatibility normalization

In Unicode, two sequences of code points are compatible if they represent the same abstract characters, and should be treated alike in some — but not necessarily all — applications.

All canonically equivalent sequences are also compatible, but not vice versa.

For example:

- the code point U+FB00 represents the ligature `"ﬀ"`. It is compatible with two consecutive U+0066 code points (`"ff"`).
- the code point U+24B9 represents the symbol `"Ⓓ"`. It is compatible with the U+0044 code point (`"D"`).

In some respects (such as sorting) they should be treated as equivalent—and in some (such as visual appearance) they should not, so they are not canonically equivalent.

You can use `normalize()` using the `"NFKD"` or `"NFKC"` arguments to produce a form of the string that will be the same for all compatible strings:

```js
let string1 = "\uFB00";
let string2 = "\u0066\u0066";
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(string1); // ff
console.log(string2); // ff
console.log(string1 === string2); // false
console.log(string1.length); // 1
console.log(string2.length); // 2

string1 = string1.normalize("NFKD");
string2 = string2.normalize("NFKD");

console.log(string1); // ff <- visual appearance changed
console.log(string2); // ff
console.log(string1 === string2); // true
console.log(string1.length); // 2
console.log(string2.length); // 2
```

When applying compatibility normalization it's important to consider what you intend to do with the strings, since the normalized form may not be appropriate for all applications. In the example above the normalization is appropriate for search, because it enables a user to find the string by searching for `"f"`. But it may not be appropriate for display, because the visual representation is different.

As with canonical normalization, you can ask for decomposed or composed compatible forms by passing `"NFKD"` or `"NFKC"`, respectively.

Examples

Using normalize()

js

```
// Initial string

// U+1E9B: LATIN SMALL LETTER LONG S WITH DOT ABOVE
// U+0323: COMBINING DOT BELOW
const str = "\u1E9B\u0323";

// Canonically-composed form (NFC)

// U+1E9B: LATIN SMALL LETTER LONG S WITH DOT ABOVE
// U+0323: COMBINING DOT BELOW
str.normalize("NFC"); // '\u1E9B\u0323'
str.normalize(); // same as above

// Canonically-decomposed form (NFD)

// U+017F: LATIN SMALL LETTER LONG S
// U+0323: COMBINING DOT BELOW
// U+0307: COMBINING DOT ABOVE
str.normalize("NFD"); // '\u017F\u0323\u0307'

// Compatibly-composed (NFKC)

// U+1E69: LATIN SMALL LETTER S WITH DOT BELOW AND DOT ABOVE
str.normalize("NFKC"); // '\u1E69'

// Compatibly-decomposed (NFKD)
```

(code continues on the next page)

(code continued from the previous page)

```
// U+0073: LATIN SMALL LETTER S
// U+0323: COMBINING DOT BELOW
// U+0307: COMBINING DOT ABOVE
str.normalize("NFKD"); // '\u0073\u0323\u0307'
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.normalize`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`normalize`	34	12	31	21	10	34	31	21	10	2.0	37	10	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.normalize`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Unicode Standard Annex #15, Unicode Normalization Forms
- Unicode equivalence on Wikipedia

JavaScript › Reference › Standard built-in objects › String

String.prototype.padEnd()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/padEnd · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since April 2017.

The **`padEnd()`** method of `String` values pads this string with a given string (repeated and/or truncated, if needed) so that the resulting string has a given length. The padding is applied from the end of this string.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.padEnd()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const str1 = "Breaded Mushrooms";

console.log(str1.padEnd(25, "."));
// Expected output: "Breaded Mushrooms........"

const str2 = "200";

console.log(str2.padEnd(5));
// Expected output: "200  "
```

Syntax

```js
padEnd(targetLength)
padEnd(targetLength, padString)
```

Parameters

`targetLength`

The length of the resulting string once the current `str` has been padded. If the value is less than or equal to `str.length`, then `str` is returned as-is.

`padString` Optional

The string to pad the current `str` with. If `padString` is too long to stay within `targetLength`, it will be truncated from the end. The default value is the space character (U+0020).

Return value

A `String` of the specified `targetLength` with `padString` applied at the end.

Examples

Using String.prototype.padEnd()

```js
"abc".padEnd(10); // "abc       "
"abc".padEnd(10, "foo"); // "abcfoofoof"
"abc".padEnd(6, "123456"); // "abc123"
"abc".padEnd(1); // "abc"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.padend

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
padEnd	57	15	48	44	10	57	48	43	10	7.0	57	10	1.0.0	1.0	8.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.padEnd`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.padEnd` in `core-js`
- es-shims polyfill of `String.prototype.padEnd`

- String.prototype.padStart()

JavaScript › Reference › Standard built-in objects › String

String.prototype.padStart()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/padStart · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since April 2017.

The **padStart()** method of String values pads this string with a given string (repeated and/or truncated, if needed) so that the resulting string has a given length. The padding is applied from the start of this string.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.padStart()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const str = "5";

console.log(str.padStart(2, "0"));
// Expected output: "05"

const fullNumber = "2034399002125581";
const last4Digits = fullNumber.slice(-4);
const maskedNumber = last4Digits.padStart(fullNumber.length, "*");

console.log(maskedNumber);
// Expected output: "************5581"
```

Syntax

js

```
padStart(targetLength)
padStart(targetLength, padString)
```

Parameters

targetLength

The length of the resulting string once the current str has been padded. If the value is less than or equal to str.length, then str is returned as-is.

padString Optional

The string to pad the current str with. If padString is too long to stay within targetLength, it will be truncated from the end. The default value is the space character (U+0020).

Return value

A String of the specified targetLength with padString applied at the start.

Examples

Using String.prototype.padStart()

```js
"abc".padStart(10); // "       abc"
"abc".padStart(10, "foo"); // "foofoofabc"
"abc".padStart(6, "123465"); // "123abc"
"abc".padStart(8, "0"); // "00000abc"
"abc".padStart(1); // "abc"
```

Fixed width string number conversion

```js
// JavaScript version of: (unsigned)
// printf "%0*d" width num
function leftFillNum(num, targetLength) {
  return num.toString().padStart(targetLength, "0");
}

const num = 123;
console.log(leftFillNum(num, 5)); // "00123"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.padstart

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
padStart	57	15	48	44	10	57	48	43	10	7.0	57	10	1.0.0	1.0	8.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.padStart`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.padStart` in `core-js`
- es-shims polyfill of `String.prototype.padStart`
- `String.prototype.padEnd()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.repeat()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/repeat · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`repeat()`** method of `String` values constructs and returns a new string which contains the specified number of copies of this string, concatenated together.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.repeat()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const mood = "Happy! ";

console.log(`I feel ${mood.repeat(3)}`);
// Expected output: "I feel Happy! Happy! Happy! "
```

Syntax

js

```
repeat(count)
```

Parameters

`count`

An integer between `0` and `Infinity`, indicating the number of times to repeat the string.

Return value

A new string containing the specified number of copies of the given string.

Exceptions

`RangeError`

Thrown if `count` is negative or if `count` overflows maximum string length.

Examples

Using repeat()

js

```
"abc".repeat(-1); // RangeError
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (count will be converted to integer)
"abc".repeat(1 / 0); // RangeError

({ toString: () => "abc", repeat: String.prototype.repeat }).repeat(2);
// 'abcabc' (repeat() is a generic method)
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.repeat`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`repeat`	41	12	24	28	9	36	24	28	9	3.0	41	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.repeat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.repeat` in `core-js`
- es-shims polyfill of `String.prototype.repeat`
- `String.prototype.concat()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.replace()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/replace · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`replace()`** method of `String` values returns a new string with one, some, or all matches of a `pattern` replaced by a `replacement`. The `pattern` can be a string or a `RegExp`, and the `replacement` can be a string or a function called for each match. If `pattern` is a string, only the first occurrence will be replaced. The original string is left unchanged.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.replace()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const paragraph = "This dog's name is just Dog! Yes, that is the name.";

console.log(paragraph.replace("name", "nickname"));
// Expected output: "This dog's nickname is just Dog! Yes, that is the name."

console.log(paragraph.replace(/\bis\b/, "was"));
// Expected output: "This dog's name was just Dog! Yes, that is the name."

console.log(paragraph.replace(/\bis\b/g, "was"));
// Expected output: "This dog's name was just Dog! Yes, that was the name."
```

Syntax

```js
replace(pattern, replacement)
```

Parameters

`pattern`

Can be a string or an object with a `Symbol.replace` method — the typical example being a regular expression. Any value that doesn't have the `Symbol.replace` method will be coerced to a string.

`replacement`

Can be a string or a function.

- If it's a string, it will replace the substring matched by `pattern`. A number of special replacement patterns are supported; see the Specifying a string as the replacement section below.
- If it's a function, it will be invoked for every match and its return value is used as the replacement text. The arguments supplied to this function are described in the Specifying a function as the replacement section below.

Return value

A new string, with one, some, or all matches of the pattern replaced by the specified replacement.

Description

This method does not mutate the string value it's called on. It returns a new string.

A string pattern will only be replaced once. To perform a global search and replace, use a regular expression with the `g` flag, or use `replaceAll()` instead.

If `pattern` is an object with a `Symbol.replace` method (including `RegExp` objects), that method is called with the target string and `replacement` as arguments. Its return value becomes the return value of `replace()`. In this case the behavior of `replace()` is entirely encoded by the `[Symbol.replace]()` method — for example, any mention of "capturing groups" in the description below is actually functionality provided by `RegExp.prototype[Symbol.replace]()`.

If the `pattern` is an empty string, the replacement is prepended to the start of the string.

```js
"xxx".replace("", "_"); // "_xxx"
```

A regexp with the `g` flag is the only case where `replace()` replaces more than once. For more information about how regex properties (especially the sticky flag) interact with `replace()`, see `RegExp.prototype[Symbol.replace]()`.

Specifying a string as the replacement

The replacement string can include the following special replacement patterns:

Pattern	Inserts
`$$`	Inserts a "`$`".
`$&`	Inserts the matched substring.
`$``	Inserts the portion of the string that precedes the matched substring.

Pattern	Inserts
`$'`	Inserts the portion of the string that follows the matched substring.
`$n`	Inserts the nth (1-indexed) capturing group where `n` is a positive integer less than 100.
`$<Name>`	Inserts the named capturing group where `Name` is the group name.

`$n` and `$<Name>` are only available if the `pattern` argument is a RegExp object. If the `pattern` is a string, or if the corresponding capturing group isn't present in the regex, then the pattern will be replaced as a literal. If the group is present but isn't matched (because it's part of a disjunction), it will be replaced with an empty string.

```js
"foo".replace(/(f)/, "$2");
// "$2oo"; the regex doesn't have the second group

"foo".replace("f", "$1");
// "$1oo"; the pattern is a string, so it doesn't have any groups

"foo".replace(/(f)|(g)/, "$2");
// "oo"; the second group exists but isn't matched
```

Specifying a function as the replacement

You can specify a function as the second parameter. In this case, the function will be invoked after the match has been performed. The function's result (return value) will be used as the replacement string.

> **Note:** The above-mentioned special replacement patterns do *not* apply for strings returned from the replacer function.

The function has the following signature:

```js
function replacer(match, p1, p2, /* …, */ pN, offset, string, groups) {
  return replacement;
}
```

The arguments to the function are as follows:

`match`

The matched substring. (Corresponds to `$&` above.)

`p1, p2, ..., pN`

The nth string found by a capture group (including named capturing groups), provided the first argument to `replace()` is a RegExp object. (Corresponds to `$1`, `$2`, etc. above.) For example, if the `pattern` is `/(\a+)(\b+)/`, then `p1` is the match for `\a+`, and `p2` is the match for `\b+`. If the group is part of a disjunction (e.g., `"abc".replace(/(a)|(b)/, replacer)`), the unmatched alternative will be `undefined`.

`offset`

The offset of the matched substring within the whole string being examined. For example, if the whole string was `'abcd'`, and the matched substring was `'bc'`, then this argument will be `1`.

`string`

The whole string being examined.

groups

An object whose keys are the used group names, and whose values are the matched portions (`undefined` if not matched). Only present if the `pattern` contains at least one named capturing group.

The exact number of arguments depends on whether the first argument is a `RegExp` object — and, if so, how many capture groups it has.

The following example will set `newString` to `'abc - 12345 - #$*%'`:

```js
function replacer(match, p1, p2, p3, offset, string) {
  // p1 is non-digits, p2 digits, and p3 non-alphanumerics
  return [p1, p2, p3].join(" - ");
}
const newString = "abc12345#$*%".replace(/(\D*)(\d*)(\W*)/, replacer);
console.log(newString); // abc - 12345 - #$*%
```

The function will be invoked multiple times for each full match to be replaced if the regular expression in the first parameter is global.

Examples

Defining the regular expression in replace()

In the following example, the regular expression is defined in `replace()` and includes the ignore case flag.

```js
const str = "Twas the night before Xmas...";
const newStr = str.replace(/xmas/i, "Christmas");
console.log(newStr); // Twas the night before Christmas...
```

This logs `'Twas the night before Christmas...'`.

> **Note:** See the regular expression guide for more explanations about regular expressions.

Using the global and ignoreCase flags with replace()

Global replace can only be done with a regular expression. In the following example, the regular expression includes the global and ignore case flags which permits `replace()` to replace each occurrence of `'apples'` in the string with `'oranges'`.

```js
const re = /apples/gi;
const str = "Apples are round, and apples are juicy.";
const newStr = str.replace(re, "oranges");
console.log(newStr); // oranges are round, and oranges are juicy.
```

This logs `'oranges are round, and oranges are juicy'`.

Switching words in a string

The following script switches the words in the string. For the replacement text, the script uses capturing groups and the `$1` and `$2` replacement patterns.

```js
const re = /(\w+)\s(\w+)/;
const str = "Maria Cruz";
const newStr = str.replace(re, "$2, $1");
console.log(newStr); // Cruz, Maria
```

This logs `'Cruz, Maria'`.

Using an inline function that modifies the matched characters

In this example, all occurrences of capital letters in the string are converted to lower case, and a hyphen is inserted just before the match location. The important thing here is that additional operations are needed on the matched item before it is given back as a replacement.

The replacement function accepts the matched snippet as its parameter, and uses it to transform the case and concatenate the hyphen before returning.

```js
function styleHyphenFormat(propertyName) {
  function upperToHyphenLower(match, offset, string) {
    return (offset > 0 ? "-" : "") + match.toLowerCase();
  }
  return propertyName.replace(/[A-Z]/g, upperToHyphenLower);
}
```

Given `styleHyphenFormat('borderTop')`, this returns `'border-top'`.

Because we want to further transform the *result* of the match before the final substitution is made, we must use a function. This forces the evaluation of the match prior to the `toLowerCase()` method. If we had tried to do this using the match without a function, the `toLowerCase()` would have no effect.

js · ✗ bad example

```
// Won't work
const newString = propertyName.replace(/[A-Z]/g, "-" + "$&".toLowerCase());
```

This is because `'$&'.toLowerCase()` would first be evaluated as a string literal (resulting in the same `'$&'`) before using the characters as a pattern.

Replacing a Fahrenheit degree with its Celsius equivalent

The following example replaces a Fahrenheit degree with its equivalent Celsius degree. The Fahrenheit degree should be a number ending with `"F"`. The function returns the Celsius number ending with `"C"`. For example, if the input number is `"212F"`, the function returns `"100C"`. If the number is `"0F"`, the function returns `"-17.77777777777778C"`.

The regular expression `test` checks for any number that ends with `F`. The number of Fahrenheit degrees is accessible to the function through its second parameter, `p1`. The function sets the Celsius number based on the number of Fahrenheit degrees passed in a string to the `f2c()` function. `f2c()` then returns the Celsius number. This function approximates Perl's `s///e` flag.

```js
function f2c(x) {
  function convert(str, p1, offset, s) {
    return `${((p1 - 32) * 5) / 9}C`;
  }
  const s = String(x);
  const test = /(-?\d+(?:\.\d*)?)F\b/g;
  return s.replace(test, convert);
}
```

Making a generic replacer

Suppose we want to create a replacer that appends the offset data to every matched string. Because the replacer function already receives the `offset` parameter, it will be trivial if the regex is statically known.

```js
"abcd".replace(/(bc)/, (match, p1, offset) => `${match} (${offset}) `);
// "abc (1) d"
```

However, this replacer would be hard to generalize if we want it to work with any regex pattern. The replacer is *variadic* — the number of arguments it receives depends on the number of capturing groups present. We can use rest parameters, but it would also collect `offset`, `string`, etc. into the array. The fact that `groups` may or may not be passed depending on the identity of the regex would also make it hard to generically know which argument corresponds to the `offset`.

js · ✗ bad example

```js
function addOffset(match, ...args) {
  const offset = args.at(-2);
  return `${match} (${offset}) `;
}

console.log("abcd".replace(/(bc)/, addOffset)); // "abc (1) d"
console.log("abcd".replace(/(?<group>bc)/, addOffset)); // "abc (abcd) d"
```

The `addOffset` example above doesn't work when the regex contains a named group, because in this case `args.at(-2)` would be the `string` instead of the `offset`.

Instead, you need to extract the last few arguments based on type, because `groups` is an object while `string` is a string.

```js
function addOffset(match, ...args) {
  const hasNamedGroups = typeof args.at(-1) === "object";
  const offset = hasNamedGroups ? args.at(-3) : args.at(-2);
  return `${match} (${offset}) `;
}

console.log("abcd".replace(/(bc)/, addOffset)); // "abc (1) d"
console.log("abcd".replace(/(?<group>bc)/, addOffset)); // "abc (1) d"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.replace

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`replace`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.replace`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.replace` in `core-js` with fixes and implementation of modern behavior like `Symbol.replace` support
- Regular expressions guide
- `String.prototype.replaceAll()`
- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.replace`
- `RegExp.prototype[Symbol.replace]()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.replaceAll()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/replaceAll · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since August 2020.

The **`replaceAll()`** method of `String` values returns a new string with all matches of a `pattern` replaced by a `replacement`. The `pattern` can be a string or a `RegExp`, and the `replacement` can be a string or a function to be called for each match. The original string is left unchanged.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.replaceAll()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const paragraph = "This dog's name is just Dog! Yes, that is the name.";

console.log(paragraph.replaceAll("name", "nickname"));
// Expected output: "This dog's nickname is just Dog! Yes, that is the nickname."

console.log(paragraph.replaceAll(/\bis\b/g, "was"));
// Expected output: "This dog's name was just Dog! Yes, that was the name."
```

Syntax

```js
replaceAll(pattern, replacement)
```

Parameters

`pattern`

Can be a string or an object with a `Symbol.replace` method — the typical example being a regular expression. Any value that doesn't have the `Symbol.replace` method will be coerced to a string.

If `pattern` is a regex, then it must have the global (`g`) flag set, or a `TypeError` is thrown.

`replacement`

Can be a string or a function. The replacement has the same semantics as that of `String.prototype.replace()`.

Return value

A new string, with all matches of a pattern replaced by a replacement.

Exceptions

`TypeError`

Thrown if the `pattern` is a regex that does not have the global (`g`) flag set (its `flags` property does not contain `"g"`).

Description

This method does not mutate the string value it's called on. It returns a new string.

Unlike `replace()`, this method replaces all occurrences of a string, not just the first one. While it is also possible to use `replace()` with a global regex dynamically constructed with `RegExp()` to replace all instances of a string, this can have unintended consequences if the string contains special characters that have meaning in regular expressions (which might happen if the replacement string comes from user input). While you can mitigate this case using `RegExp.escape()` to make the regular expression string into a literal pattern, it is simpler to pass the string to `replaceAll()` directly, without converting it to a regex.

```js
function unsafeRedactName(text, name) {
  return text.replace(new RegExp(name, "g"), "[REDACTED]");
}
function semiSafeRedactName(text, name) {
  return text.replaceAll(name, "[REDACTED]");
}
function superSafeRedactName(text, name) {
  // only match at word boundaries
  return text.replaceAll(
    new RegExp(`\\b${RegExp.escape(name)}\\b`, "g"),
    "[REDACTED]",
  );
}
```

(code continues on the next page)

(code continued from the previous page)

```
let report =
  "A hacker called ha.*er used special characters in their name to breach the system.";

console.log(unsafeRedactName(report, "ha.*er")); // "A [REDACTED]s in their name to breach
the system."
console.log(semiSafeRedactName(report, "ha.*er")); // "A hacker called [REDACTED] used
special characters in their name to breach the system."

report = "A hacker called acke breached the system.";

console.log(semiSafeRedactName(report, "acke")); // "A h[REDACTED]r called [REDACTED]
breached the system."
console.log(superSafeRedactName(report, "acke")); // "A hacker called [REDACTED] breached
the system."
```

If `pattern` is an object with a `Symbol.replace` method (including `RegExp` objects), that method is called with the target string and `replacement` as arguments. Its return value becomes the return value of `replaceAll()`. In this case the behavior of `replaceAll()` is entirely encoded by the `[Symbol.replace]()` method, and therefore will have the same result as `replace()` (apart from the extra input validation that the regex is global).

If the `pattern` is an empty string, the replacement will be inserted in between every UTF-16 code unit, similar to `split()` behavior.

js

```
"xxx".replaceAll("", "_"); // "_x_x_x_"
```

For more information about how regex properties (especially the sticky flag) interact with `replaceAll()`, see `RegExp.prototype[Symbol.replace]()`.

Examples

Using replaceAll()

js

```
"aabbcc".replaceAll("b", ".");
// 'aa..cc'
```

Non-global regex throws

When using a regular expression search value, it must be global. This won't work:

js · ✗ bad example

```
"aabbcc".replaceAll(/b/, ".");
// TypeError: replaceAll must be called with a global RegExp
```

This will work:

js · ✓ good example

```
"aabbcc".replaceAll(/b/g, ".");
("aa..cc");
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.replaceall`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`replaceAll`	85	85	77	71	13.1	85	79	60	13.4	14.0	85	13.4	1.0.0	1.2	15.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.replaceAll`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.replaceAll` in `core-js`
- es-shims polyfill of `String.prototype.replaceAll`
- Regular expressions guide
- `String.prototype.replace()`
- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.search()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/search · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`search()`** method of `String` values executes a search for a match between a regular expression and this string, returning the index of the first match in the string.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.search()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

// Anything not a word character, whitespace or apostrophe
const regex = /[^\w\s']/g;

console.log(paragraph.search(regex));
// Expected output: 41

console.log(paragraph[paragraph.search(regex)]);
// Expected output: "!"
```

Syntax

js

```
search(regexp)
```

Parameters

`regexp`

A regular expression object, or any object that has a `Symbol.search` method.

If `regexp` is not a `RegExp` object and does not have a `Symbol.search` method, it is implicitly converted to a `RegExp` by using `new RegExp(regexp)`.

Return value

The index of the first match between the regular expression and the given string, or `-1` if no match was found.

Description

The implementation of `String.prototype.search()` doesn't do much other than calling the `Symbol.search` method of the argument with the string as the first parameter. The actual implementation comes from `RegExp.prototype[Symbol.search]()`.

The `g` flag of `regexp` has no effect on the `search()` result, and the search always happens as if the regex's `lastIndex` is 0. For more information on the behavior of `search()`, see `RegExp.prototype[Symbol.search]()`.

When you want to know whether a pattern is found, and *also* know its index within a string, use `search()`.

- If you only want to know if it exists, use the `RegExp.prototype.test()` method, which returns a boolean.
- If you need the content of the matched text, use `String.prototype.match()` or `RegExp.prototype.exec()`.

Examples

Using search()

The following example searches a string with two different regex objects to show a successful search (positive value) vs. an unsuccessful search (`-1`).

js

```
const str = "hey JudE";
const re = /[A-Z]/;
const reDot = /[.]/;
console.log(str.search(re)); // returns 4, which is the index of the first capital letter
"J"
console.log(str.search(reDot)); // returns -1 cannot find '.' dot punctuation
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.search

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
search	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.search`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.search` in `core-js` with fixes and implementation of modern behavior like `Symbol.search` support
- Regular expressions guide
- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype[Symbol.search]()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.slice()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/slice · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`slice()`** method of `String` values extracts a section of this string and returns it as a new string, without modifying the original string.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.slice()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const str = "The quick brown fox jumps over the lazy dog.";

console.log(str.slice(31));
// Expected output: "the lazy dog."

console.log(str.slice(4, 19));
// Expected output: "quick brown fox"

console.log(str.slice(-4));
// Expected output: "dog."

console.log(str.slice(-9, -5));
// Expected output: "lazy"
```

Syntax

js

```
slice(indexStart)
slice(indexStart, indexEnd)
```

Parameters

`indexStart`

The index of the first character to include in the returned substring.

`indexEnd` Optional

The index of the first character to exclude from the returned substring.

Return value

A new string containing the extracted section of the string.

Description

`slice()` extracts the text from one string and returns a new string.

`slice()` extracts up to but not including `indexEnd`. For example, `str.slice(4, 8)` extracts the fifth character through the eighth character (characters indexed `4`, `5`, `6`, and `7`):

```
              indexStart        indexEnd
                  ↓               ↓
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
```

(code continues on the next page)

(code continued from the previous page)

```
| T | h | e |   | m | i | r | r | o | r |

                m   i   r   r
               ———————————
                    ↑
                  Result
```

- If `indexStart >= str.length`, an empty string is returned.
- If `indexStart < 0`, the index is counted from the end of the string. More formally, in this case, the substring starts at `max(indexStart + str.length, 0)`.
- If `indexStart` is omitted, `undefined`, or cannot be converted to a number, it's treated as `0`.
- If `indexEnd` is omitted or `undefined`, or if `indexEnd >= str.length`, `slice()` extracts to the end of the string.
- If `indexEnd < 0`, the index is counted from the end of the string. More formally, in this case, the substring ends at `max(indexEnd + str.length, 0)`.
- If `indexEnd <= indexStart` after normalizing negative values (i.e., `indexEnd` represents a character that's before `indexStart`), an empty string is returned.

Examples

Using slice() to create a new string

The following example uses `slice()` to create a new string.

```js
const str1 = "The morning is upon us."; // The length of str1 is 23.
const str2 = str1.slice(1, 8);
const str3 = str1.slice(4, -2);
const str4 = str1.slice(12);
const str5 = str1.slice(30);
console.log(str2); // he morn
console.log(str3); // morning is upon u
console.log(str4); // is upon us.
console.log(str5); // ""
```

Using slice() with negative indexes

The following example uses `slice()` with negative indexes.

```js
const str = "The morning is upon us.";
str.slice(-3); // 'us.'
str.slice(-3, -1); // 'us'
str.slice(0, -1); // 'The morning is upon us'
str.slice(4, -1); // 'morning is upon us'
```

This example counts backwards from the end of the string by `11` to find the start index and forwards from the start of the string by `16` to find the end index.

```js
console.log(str.slice(-11, 16)); // "is u"
```

Here it counts forwards from the start by `11` to find the start index and backwards from the end by `7` to find the end index.

```js
console.log(str.slice(11, -7)); // " is u"
```

These arguments count backwards from the end by `5` to find the start index and backwards from the end by `1` to find the end index.

```js
console.log(str.slice(-5, -1)); // "n us"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.slice`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`slice`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.slice`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `String.prototype.substr()`
- `String.prototype.substring()`
- `Array.prototype.slice()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.small()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/small · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`small()`** method of `String` values creates a string that embeds this string in a `<small>` element (`<small>str</small>`), which causes this string to be displayed in a small font.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```js
small()
```

Parameters

None.

Return value

A string beginning with a `<small>` start tag, then the text `str`, and then a `</small>` end tag.

Examples

Using small()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.small();
```

This will create the following HTML:

```html
<small>Hello, world</small>
```

Instead of using `small()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```js
const contentString = "Hello, world";
const elem = document.createElement("small");
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.small

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`small` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.small`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.small` in `core-js`
- es-shims polyfill of `String.prototype.small`
- HTML wrapper methods
- `<small>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.split()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/split · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`split()`** method of `String` values takes a pattern and divides this string into an ordered list of substrings by searching for the pattern, puts these substrings into an array, and returns the array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.split()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const str = "The quick brown fox jumps over the lazy dog.";

const words = str.split(" ");
console.log(words[3]);
// Expected output: "fox"

const chars = str.split("");
console.log(chars[8]);
// Expected output: "k"

const strCopy = str.split();
console.log(strCopy);
// Expected output: Array ["The quick brown fox jumps over the lazy dog."]
```

Syntax

js

```
split(separator)
split(separator, limit)
```

Parameters

`separator`

The pattern describing where each split should occur. Can be `undefined`, a string, or an object with a `Symbol.split` method — the typical example being a regular expression. Omitting `separator` or passing `undefined` causes `split()` to return an array with the calling string as a single element. All values that are not `undefined` or objects with a `[Symbol.split]()` method are coerced to strings.

`limit` Optional

A non-negative integer specifying a limit on the number of substrings to be included in the array. If provided, splits the string at each occurrence of the specified `separator`, but stops when `limit` entries have been placed in the array. Any leftover text is not included in the array at all.

- The array may contain fewer entries than `limit` if the end of the string is reached before the limit is reached.
- If `limit` is `0`, `[]` is returned.

Return value

If `separator` is a string, an `Array` of strings is returned, split at each point where the `separator` occurs in the given string.

If `separator` is a regex, the returned `Array` also contains the captured groups for each separator match; see below for details. The capturing groups may be unmatched, in which case they are `undefined` in the array.

If `separator` has a custom `[Symbol.split]()` method, its return value is directly returned.

Description

If `separator` is a non-empty string, the target string is split by all matches of the `separator` without including `separator` in the results. For example, a string containing tab separated values (TSV) could be parsed by passing a tab character as the separator, like `myString.split("\t")`. If `separator` contains multiple characters, that entire character sequence must be found in order to split. If `separator` appears at the beginning (or end) of the string, it still has the effect of splitting, resulting in an empty (i.e., zero length) string appearing at the first (or last) position of the returned array. If `separator` does not occur in `str`, the returned array contains one element consisting of the entire string.

If `separator` is an empty string (`""`), `str` is converted to an array of each of its UTF-16 "characters", without empty strings on either ends of the resulting string.

> **Note:** `"".split("")` is therefore the only way to produce an empty array when a string is passed as `separator` and `limit` is not `0`.

> **Warning:** When the empty string (`""`) is used as a separator, the string is **not** split by *user-perceived characters* (grapheme clusters) or unicode characters (code points), but by UTF-16 code units. This destroys surrogate pairs. See "How do you get a string to a character array in JavaScript?" on Stack Overflow.

If `separator` is a regexp that matches empty strings, whether the match is split by UTF-16 code units or Unicode code points depends on if the regex is Unicode-aware.

```js
"😄😄".split(/(?:)/); // [ "\ud83d", "\ude04", "\ud83d", "\ude04" ]
"😄😄".split(/(?:)/u); // [ "😄", "😄" ]
```

If `separator` is a regular expression with capturing groups, then each time `separator` matches, the captured groups (including any `undefined` results) are spliced into the output array. This behavior is specified by the regexp's `Symbol.split` method.

If `separator` is an object with a `Symbol.split` method, that method is called with the target string and `limit` as arguments, and `this` set to the object. Its return value becomes the return value of `split`.

Any other value will be coerced to a string before being used as separator.

Examples

Using split()

When the string is empty and a non-empty separator is specified, `split()` returns `[""]`. If the string and separator are both empty strings, an empty array is returned.

```js
const emptyString = "";

// string is empty and separator is non-empty
console.log(emptyString.split("a"));
// [""]

// string and separator are both empty strings
console.log(emptyString.split(emptyString));
// []
```

The following example defines a function that splits a string into an array of strings using `separator`. After splitting the string, the function logs messages indicating the original string (before the split), the separator used, the number of elements in the array, and the individual array elements.

```js
function splitString(stringToSplit, separator) {
  const arrayOfStrings = stringToSplit.split(separator);

  console.log("The original string is:", stringToSplit);
  console.log("The separator is:", separator);
  console.log(
    "The array has",
    arrayOfStrings.length,
    "elements:",
    arrayOfStrings.join(" / "),
  );
}

const tempestString = "Oh brave new world that has such people in it.";
const monthString = "Jan,Feb,Mar,Apr,May,Jun,Jul,Aug,Sep,Oct,Nov,Dec";

const space = " ";
const comma = ",";

splitString(tempestString, space);
splitString(tempestString);
splitString(monthString, comma);
```

This example produces the following output:

```
The original string is: "Oh brave new world that has such people in it."
The separator is: " "
The array has 10 elements: Oh / brave / new / world / that / has / such / people / in / it.

The original string is: "Oh brave new world that has such people in it."
The separator is: "undefined"
The array has 1 elements: Oh brave new world that has such people in it.

The original string is: "Jan,Feb,Mar,Apr,May,Jun,Jul,Aug,Sep,Oct,Nov,Dec"
```

(code continues on the next page)

(code continued from the previous page)

```
The separator is: ","
The array has 12 elements: Jan / Feb / Mar / Apr / May / Jun / Jul / Aug / Sep / Oct / Nov /
Dec
```

Removing spaces from a string

In the following example, `split()` looks for zero or more spaces, followed by a semicolon, followed by zero or more spaces—and, when found, removes the spaces and the semicolon from the string. `nameList` is the array returned as a result of `split()`.

js

```
const names = "Harry Trump ;Fred Barney; Helen Rigby ; Bill Abel ;Chris Hand ";

console.log(names);

const re = /\s*(?:;|$)\s*/;
const nameList = names.split(re);

console.log(nameList);
```

This logs two lines; the first line logs the original string, and the second line logs the resulting array.

```
Harry Trump ;Fred Barney; Helen Rigby ; Bill Abel ;Chris Hand
[ "Harry Trump", "Fred Barney", "Helen Rigby", "Bill Abel", "Chris Hand", "" ]
```

Returning a limited number of splits

In the following example, `split()` looks for spaces in a string and returns the first 3 splits that it finds.

js

```
const myString = "Hello World. How are you doing?";
const splits = myString.split(" ", 3);

console.log(splits); // [ "Hello", "World.", "How" ]
```

Splitting with a RegExp to include parts of the separator in the result

If `separator` is a regular expression that contains capturing parentheses `( )`, matched results are included in the array.

js

```
const myString = "Hello 1 word. Sentence number 2.";
const splits = myString.split(/(\d)/);

console.log(splits);
// [ "Hello ", "1", " word. Sentence number ", "2", "." ]
```

> **Note:** `\d` matches the character class for digits between 0 and 9.

Using a custom splitter

An object with a `Symbol.split` method can be used as a splitter with custom behavior.

The following example splits a string using an internal state consisting of an incrementing number:

```js
const splitByNumber = {
  [Symbol.split](str) {
    let num = 1;
    let pos = 0;
    const result = [];
    while (pos < str.length) {
      const matchPos = str.indexOf(num, pos);
      if (matchPos === -1) {
        result.push(str.substring(pos));
        break;
      }
      result.push(str.substring(pos, matchPos));
      pos = matchPos + String(num).length;
      num++;
    }
    return result;
  },
};

const myString = "a1bc2c5d3e4f";
console.log(myString.split(splitByNumber)); // [ "a", "bc", "c5d", "e", "f" ]
```

The following example uses an internal state to enforce certain behavior, and to ensure a "valid" result is produced.

```js
const DELIMITER = ";";

// Split the commands, but remove any invalid or unnecessary values.
const splitCommands = {
  [Symbol.split](str, lim) {
    const results = [];
    const state = {
      on: false,
      brightness: {
        current: 2,
        min: 1,
        max: 3,
      },
    };
    let pos = 0;
    let matchPos = str.indexOf(DELIMITER, pos);

    while (matchPos !== -1) {
      const subString = str.slice(pos, matchPos).trim();

      switch (subString) {
        case "light on":
          // If the `on` state is already true, do nothing.
          if (!state.on) {
            state.on = true;
            results.push(subString);
          }
          break;
```

(code continues on the next page)

(code continued from the previous page)

```
        case "light off":
          // If the `on` state is already false, do nothing.
          if (state.on) {
            state.on = false;
            results.push(subString);
          }
          break;

        case "brightness up":
          // Enforce a brightness maximum.
          if (state.brightness.current < state.brightness.max) {
            state.brightness.current += 1;
            results.push(subString);
          }
          break;

        case "brightness down":
          // Enforce a brightness minimum.
          if (state.brightness.current > state.brightness.min) {
            state.brightness.current -= 1;
            results.push(subString);
          }
          break;
      }

      if (results.length === lim) {
        break;
      }

      pos = matchPos + DELIMITER.length;
      matchPos = str.indexOf(DELIMITER, pos);
    }

    // If we broke early due to reaching the split `lim`, don't add the remaining commands.
    if (results.length < lim) {
      results.push(str.slice(pos).trim());
    }

    return results;
  },
};

const commands =
  "light on; brightness up; brightness up; brightness up; light on; brightness down;
brightness down; light off";
console.log(commands.split(splitCommands, 3)); // ["light on", "brightness up", "brightness
down"]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.split

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`split`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.split`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.split` in `core-js` with fixes and implementation of modern behavior like `Symbol.split` support
- es-shims polyfill of `String.prototype.split`
- Regular expressions guide
- `String.prototype.charAt()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `Array.prototype.join()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.startsWith()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/startsWith · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`startsWith()`** method of `String` values determines whether this string begins with the characters of a specified string, returning `true` or `false` as appropriate.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.startsWith()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const str = "Saturday night plans";

console.log(str.startsWith("Sat"));
// Expected output: true

console.log(str.startsWith("Sat", 3));
// Expected output: false
```

Syntax

```js
startsWith(searchString)
startsWith(searchString, position)
```

Parameters

`searchString`

The characters to be searched for at the start of this string. Cannot be a regex. All values that are not regexes are coerced to strings, so omitting it or passing `undefined` causes `startsWith()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

The start position at which `searchString` is expected to be found (the index of `searchString`'s first character). Defaults to `0`.

Return value

`true` if the given characters are found at the beginning of the string, including when `searchString` is an empty string; otherwise, **`false`**.

Exceptions

`TypeError`

Thrown if `searchString` is a regex.

Description

This method lets you determine whether or not a string begins with another string. This method is case-sensitive.

Examples

Using startsWith()

```js
const str = "To be, or not to be, that is the question.";

console.log(str.startsWith("To be")); // true
console.log(str.startsWith("not to be")); // false
console.log(str.startsWith("not to be", 10)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.startswith

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`startsWith`	41	12	17	28	9	36	17	24	9	3.0	37	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.startsWith`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.startsWith` in `core-js`
- es-shims polyfill of `String.prototype.startsWith`
- `String.prototype.endsWith()`
- `String.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.strike()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/strike · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`strike()`** method of `String` values creates a string that embeds this string in a `<strike>` element (`<strike>str</strike>`), which causes this string to be displayed as struck-out text.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `strike()`, the `<strike>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use the `<del>` for deleted content or the `<s>` for content that is no longer accurate or no longer relevant instead.

Syntax

```js
strike()
```

Parameters

None.

Return value

A string beginning with a `<strike>` start tag, then the text `str`, and then a `</strike>` end tag.

Examples

Using strike()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.strike();
```

This will create the following HTML:

```html
<strike>Hello, world</strike>
```

> **Warning:** This markup is invalid, because `strike` is no longer a valid element.

Instead of using `strike()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```js
const contentString = "Hello, world";
const elem = document.createElement("s");
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.strike`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`strike` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.strike`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.strike` in `core-js`
- es-shims polyfill of `String.prototype.strike`
- HTML wrapper methods
- `<strike>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.sub()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/sub · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`sub()`** method of `String` values creates a string that embeds this string in a `<sub>` element (`<sub>str</sub>`), which causes this string to be displayed as subscript.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```js
sub()
```

Parameters

None.

Return value

A string beginning with a `<sub>` start tag, then the text `str`, and then a `</sub>` end tag.

Examples

Using sub()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.sub();
```

This will create the following HTML:

```html
<sub>Hello, world</sub>
```

Instead of using `sub()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```js
const contentString = "Hello, world";
const elem = document.createElement("sub");
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.sub`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`sub` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.sub`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.sub` in `core-js`
- es-shims polyfill of `String.prototype.sub`
- HTML wrapper methods
- `<sub>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.substr()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/substr · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`substr()`** method of `String` values returns a portion of this string, starting at the specified index and extending for a given number of characters afterwards.

> **Note:** `substr()` is not part of the main ECMAScript specification — it's defined in Annex B: Additional ECMAScript Features for Web Browsers, which is normative optional for non-browser runtimes. Therefore, people are advised to use the standard `String.prototype.substring()` and `String.prototype.slice()` methods instead to make their code maximally cross-platform friendly. The `String.prototype.substring()` page has some comparisons between the three methods.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.substr()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const str = "Mozilla";

console.log(str.substr(1, 2));
// Expected output: "oz"

console.log(str.substr(2));
// Expected output: "zilla"
```

Syntax

js

```
substr(start)
substr(start, length)
```

Parameters

`start`

The index of the first character to include in the returned substring.

`length` Optional

The number of characters to extract.

Return value

A new string containing the specified part of the given string.

Description

A string's `substr()` method extracts `length` characters from the string, counting from the `start` index.

- If `start >= str.length`, an empty string is returned.
- If `start < 0`, the index starts counting from the end of the string. More formally, in this case the substring starts at `max(start + str.length, 0)`.
- If `start` is omitted or `undefined`, it's treated as `0`.
- If `length` is omitted or `undefined`, or if `start + length >= str.length`, `substr()` extracts characters to the end of the string.
- If `length < 0`, an empty string is returned.
- For both `start` and `length`, `NaN` is treated as `0`.

Although you are encouraged to avoid using `substr()`, there is no trivial way to migrate `substr()` to either `slice()` or `substring()` in legacy code without essentially writing a polyfill for `substr()`. For example, `str.substr(a, l)`, `str.slice(a, a + l)`, and `str.substring(a, a + l)` all have different results when `str = "01234", a = 1, l = -2` — `substr()` returns an empty string, `slice()` returns `"123"`, while `substring()` returns `"0"`. The actual refactoring path depends on the knowledge of the range of `a` and `l`.

Examples

Using substr()

```js
const string = "Mozilla";

console.log(string.substr(0, 1)); // 'M'
console.log(string.substr(1, 0)); // ''
console.log(string.substr(-1, 1)); // 'a'
console.log(string.substr(1, -1)); // ''
console.log(string.substr(-3)); // 'lla'
console.log(string.substr(1)); // 'ozilla'
console.log(string.substr(-20, 2)); // 'Mo'
console.log(string.substr(20, 2)); // ''
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.substr

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
substr Deprecated	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.substr`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.substr` in `core-js`
- es-shims polyfill of `String.prototype.substr`
- `String.prototype.slice()`
- `String.prototype.substring()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.substring()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/substring · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`substring()`** method of `String` values returns the part of this string from the start index up to and excluding the end index, or to the end of the string if no end index is supplied.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.substring()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const str = "Mozilla";

console.log(str.substring(1, 3));
// Expected output: "oz"

console.log(str.substring(2));
// Expected output: "zilla"
```

Syntax

js

```
substring(indexStart)
substring(indexStart, indexEnd)
```

Parameters

`indexStart`

The index of the first character to include in the returned substring.

`indexEnd` Optional

The index of the first character to exclude from the returned substring.

Return value

A new string containing the specified part of the given string.

Description

`substring()` extracts characters from `indexStart` up to *but not including* `indexEnd`. In particular:

- If `indexEnd` is omitted or `undefined`, `substring()` extracts characters to the end of the string.
- If `indexStart` is equal to `indexEnd`, `substring()` returns an empty string.
- If `indexStart` is greater than `indexEnd`, then the effect of `substring()` is as if the two arguments were swapped; see example below.

Any argument value that is less than `0` or greater than `str.length` is treated as if it were `0` and `str.length`, respectively.

Any argument value that is `NaN` is treated as if it were `0`.

Examples

Using substring()

The following example uses `substring()` to display characters from the string `"Mozilla"`:

```js
const anyString = "Mozilla";

console.log(anyString.substring(0, 1)); // "M"
console.log(anyString.substring(1, 0)); // "M"

console.log(anyString.substring(0, 6)); // "Mozill"

console.log(anyString.substring(4)); // "lla"
console.log(anyString.substring(4, 7)); // "lla"
console.log(anyString.substring(7, 4)); // "lla"

console.log(anyString.substring(0, 7)); // "Mozilla"
console.log(anyString.substring(0, 10)); // "Mozilla"
```

Using substring() with length property

The following example uses the `substring()` method and `length` property to extract the last characters of a particular string. This method may be easier to remember, given that you don't need to know the starting and ending indices as you would in the above examples.

```js
const text = "Mozilla";

// Takes 4 last characters of string
console.log(text.substring(text.length - 4)); // prints "illa"

// Takes 5 last characters of string
console.log(text.substring(text.length - 5)); // prints "zilla"
```

The difference between substring() and substr()

There are subtle differences between the `substring()` and `substr()` methods, so you should be careful not to get them confused.

- The two parameters of `substr()` are `start` and `length`, while for `substring()`, they are `start` and `end`.
- `substr()`'s `start` index will wrap to the end of the string if it is negative, while `substring()` will clamp it to `0`.
- Negative lengths in `substr()` are treated as zero, while `substring()` will swap the two indexes if `end` is less than `start`.

Furthermore, `substr()` is considered a *legacy feature in ECMAScript*, so it is best to avoid using it if possible.

```js
const text = "Mozilla";
console.log(text.substring(2, 5)); // "zil"
console.log(text.substr(2, 3)); // "zil"
```

Differences between substring() and slice()

The `substring()` and `slice()` methods are almost identical, but there are a couple of subtle differences between the two, especially in the way negative arguments are dealt with.

The `substring()` method swaps its two arguments if `indexStart` is greater than `indexEnd`, meaning that a string is still returned. The `slice()` method returns an empty string if this is the case.

```js
const text = "Mozilla";
console.log(text.substring(5, 2)); // "zil"
console.log(text.slice(5, 2)); // ""
```

If either or both of the arguments are negative or `NaN`, the `substring()` method treats them as if they were `0`.

```js
console.log(text.substring(-5, 2)); // "Mo"
console.log(text.substring(-5, -2)); // ""
```

`slice()` also treats `NaN` arguments as `0`, but when it is given negative values it counts backwards from the end of the string to find the indexes.

```js
console.log(text.slice(-5, 2)); // ""
console.log(text.slice(-5, -2)); // "zil"
```

See the `slice()` page for more examples with negative numbers.

Replacing a substring within a string

The following example replaces a substring within a string. It will replace both individual characters and substrings. The function call at the end of the example creates a string `Brave New Web` from the original string `Brave New World`.

```js
// Replaces oldS with newS in the string fullS
function replaceString(oldS, newS, fullS) {
  for (let i = 0; i < fullS.length; ++i) {
    if (fullS.substring(i, i + oldS.length) === oldS) {
      fullS =
        fullS.substring(0, i) +
        newS +
        fullS.substring(i + oldS.length, fullS.length);
    }
  }
  return fullS;
}

replaceString("World", "Web", "Brave New World");
```

Note that this can result in an infinite loop if `oldS` is itself a substring of `newS` — for example, if you attempted to replace `"World"` with `"OtherWorld"` here.

A better method for replacing strings is as follows:

```js
function replaceString(oldS, newS, fullS) {
  return fullS.split(oldS).join(newS);
}
```

The code above serves as an example for substring operations. If you need to replace substrings, most of the time you will want to use `String.prototype.replace()`.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.substring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`substring`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.substring`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.substr()`
- `String.prototype.slice()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.sup()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/sup · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`sup()`** method of `String` values creates a string that embeds this string in a `<sup>` element (`<sup>str</sup>`), which causes this string to be displayed as superscript.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```js
sup()
```

Parameters

None.

Return value

A string beginning with a `<sup>` start tag, then the text `str`, and then a `</sup>` end tag.

Examples

Using sup()

The code below creates an HTML string and then replaces the document's body with it:

```js
const contentString = "Hello, world";

document.body.innerHTML = contentString.sup();
```

This will create the following HTML:

```html
<sup>Hello, world</sup>
```

Instead of using `sup()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```js
const contentString = "Hello, world";
const elem = document.createElement("sup");
elem.innerText = contentString;
document.body.appendChild(elem);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-string.prototype.sup

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
sup Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.sup`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.sup` in `core-js`
- es-shims polyfill of `String.prototype.sup`
- HTML wrapper methods
- `<sup>`

JavaScript › Reference › Standard built-in objects › String

String.prototype.toLocaleLowerCase()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/toLocaleLowerCase · MDN page last modified 2026-10-09

Baseline: Widely available — available across major browsers since July 2015.

The **`toLocaleLowerCase()`** method of `String` values returns this string converted to lower case, according to any locale-specific case mappings.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.toLocaleLowerCase()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const dotted = "İstanbul";

console.log(`EN-US: ${dotted.toLocaleLowerCase("en-US")}`);
// Expected output: "EN-US: istanbul"

console.log(`TR: ${dotted.toLocaleLowerCase("tr")}`);
// Expected output: "TR: istanbul"
```

Syntax

js

```
toLocaleLowerCase()
toLocaleLowerCase(locales)
```

Parameters

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Indicates the locale to be used to convert to lower case according to any locale-specific case mappings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

Unlike other methods that use the `locales` argument, `toLocaleLowerCase()` does not allow locale matching. Therefore, after checking the validity of the `locales` argument, `toLocaleLowerCase()` always uses the first locale in the list (or the default locale if the list is empty), even if this locale is not supported by the implementation.

Return value

A new string representing the calling string converted to lower case, according to any locale-specific case mappings.

Description

The `toLocaleLowerCase()` method returns the value of the string converted to lower case according to any locale-specific case mappings. `toLocaleLowerCase()` does not affect the value of the string itself. In most cases, this will produce the same result as `toLowerCase()`, but for some locales, such as Turkish, whose case mappings do not follow the default case mappings in Unicode, there may be a different result.

Examples

Using toLocaleLowerCase()

```js
"ALPHABET".toLocaleLowerCase(); // 'alphabet'

"\u0130".toLocaleLowerCase("tr") === "i"; // true
"\u0130".toLocaleLowerCase("en-US") === "i"; // false

const locales = ["tr", "TR", "tr-TR", "tr-u-co-search", "tr-x-turkish"];
"\u0130".toLocaleLowerCase(locales) === "i"; // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.tolocalelowercase`
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sup-string.prototype.tolocalelowercase`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleLowerCase`	1	12	1	4	1.3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`locales` parameter	58	12	55	45	10	58	55	43	10	7.0	58	10	1.0.0	1.8[1,2]	13.0.0[3,4]

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.toLocaleLowerCase`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- String.prototype.toLocaleUpperCase()
- String.prototype.toLowerCase()
- String.prototype.toUpperCase()

JavaScript › Reference › Standard built-in objects › String

String.prototype.toLocaleUpperCase()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/toLocaleUpperCase · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since July 2015.

The **`toLocaleUpperCase()`** method of String values returns this string converted to upper case, according to any locale-specific case mappings.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.toLocaleUpperCase()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const city = "istanbul";

console.log(city.toLocaleUpperCase("en-US"));
// Expected output: "ISTANBUL"

console.log(city.toLocaleUpperCase("TR"));
// Expected output: "İSTANBUL"
```

Syntax

js

```
toLocaleUpperCase()
toLocaleUpperCase(locales)
```

Parameters

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Indicates the locale to be used to convert to upper case according to any locale-specific case mappings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

Unlike other methods that use the `locales` argument, `toLocaleUpperCase()` does not allow locale matching. Therefore, after checking the validity of the `locales` argument, `toLocaleUpperCase()` always uses the first locale in the list (or the default locale if the list is empty), even if this locale is not supported by the implementation.

Return value

A new string representing the calling string converted to upper case, according to any locale-specific case mappings.

Description

The `toLocaleUpperCase()` method returns the value of the string converted to upper case according to any locale-specific case mappings. `toLocaleUpperCase()` does not affect the value of the string itself. In most cases, this will produce the same result as `toUpperCase()`, but for some locales, such as Turkish, whose case mappings do not follow the default case mappings in Unicode, there may be a different result.

Also notice that conversion is not necessarily a 1:1 character mapping, as some characters might result in two (or even more) characters when transformed to upper-case. Therefore the length of the result string can differ from the input length. This also implies that the conversion is not stable, so for example the following can return `false`: `x.toLocaleLowerCase() === x.toLocaleUpperCase().toLocaleLowerCase()`

Examples

Using toLocaleUpperCase()

```js
"alphabet".toLocaleUpperCase(); // 'ALPHABET'

"Gesäß".toLocaleUpperCase(); // 'GESÄSS'

"i\u0307".toLocaleUpperCase("lt-LT"); // 'I'

const locales = ["lt", "LT", "lt-LT", "lt-u-co-phonebk", "lt-x-lietuva"];
"i\u0307".toLocaleUpperCase(locales); // 'I'
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.tolocaleuppercase
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sup-string.prototype.tolocaleuppercase

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleUpperCase`	1	12	1	4	1.3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`locales` parameter	58	12	55	45	10	58	55	42	10	7.0	58	10	1.0.0	1.8[1,2]	13.0.0[3,4]

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.toLocaleUpperCase`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLowerCase()`
- `String.prototype.toUpperCase()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.toLowerCase()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/toLowerCase · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toLowerCase()`** method of `String` values returns this string converted to lower case.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.toLowerCase()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const sentence = "The quick brown fox jumps over the lazy dog.";

console.log(sentence.toLowerCase());
// Expected output: "the quick brown fox jumps over the lazy dog."
```

Syntax

js

```
toLowerCase()
```

Parameters

None.

Return value

A new string representing the calling string converted to lower case.

Description

The `toLowerCase()` method returns the value of the string converted to lower case. `toLowerCase()` does not affect the value of the string `str` itself.

Examples

Using `toLowerCase()`

js

```
console.log("ALPHABET".toLowerCase()); // 'alphabet'
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.tolowercase`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLowerCase`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.toLowerCase`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLocaleUpperCase()`
- `String.prototype.toUpperCase()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/toString · MDN page last modified 2026-01-25

Baseline: Widely available — available across major browsers since July 2015.

The **`toString()`** method of `String` values returns this string value.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.toString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const stringObj = new String("foo");

console.log(stringObj);
// Expected output: String { "foo" }

console.log(stringObj.toString());
// Expected output: "foo"
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

A string representing the specified string value.

Description

The `String` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `String` values, the `toString` method returns the string itself (if it's a primitive) or the string that the `String` object wraps. It has the exact same implementation as `String.prototype.valueOf()`.

The `toString()` method requires its `this` value to be a `String` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to string values.

Because `String` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `String` *object* is used in a context expecting a string, such as in a template literal. However, string *primitive* values do not consult the `toString()` method to be coerced to strings — since they are already strings, no conversion is performed.

```js
String.prototype.toString = () => "Overridden";
console.log(`${"foo"}`); // "foo"
console.log(`${new String("foo")}`); // "Overridden"
```

Examples

Using toString()

The following example displays the string value of a `String` object:

```js
const x = new String("Hello world");

console.log(x.toString()); // "Hello world"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.valueOf()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.toUpperCase()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/toUpperCase · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toUpperCase()`** method of `String` values returns this string converted to uppercase.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.toUpperCase()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const sentence = "The quick brown fox jumps over the lazy dog.";

console.log(sentence.toUpperCase());
// Expected output: "THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG."
```

Syntax

js

```
toUpperCase()
```

Parameters

None.

Return value

A new string representing the calling string converted to upper case.

Description

The `toUpperCase()` method returns the value of the string converted to uppercase. This method does not affect the value of the string itself since JavaScript strings are immutable.

Examples

Basic usage

```js
console.log("alphabet".toUpperCase()); // 'ALPHABET'
```

Conversion of non-string `this` values to strings

This method will convert any non-string value to a string, when you set its `this` to a value that is not a string:

```js
const a = String.prototype.toUpperCase.call({
  toString() {
    return "abcdef";
  },
});

const b = String.prototype.toUpperCase.call(true);

// prints out 'ABCDEF TRUE'.
console.log(a, b);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.touppercase

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toUpperCase	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.toUpperCase`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLocaleUpperCase()`
- `String.prototype.toLowerCase()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.toWellFormed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/toWellFormed · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since October 2023.

The **`toWellFormed()`** method of `String` values returns a string where all lone surrogates of this string are replaced with the Unicode replacement character U+FFFD.

Syntax

```
toWellFormed()
```

Parameters

None.

Return value

A new string that is a copy of this string, with all lone surrogates replaced with the Unicode replacement character U+FFFD. If `str` is well formed, a new string is still returned (essentially a copy of `str`).

Description

Strings in JavaScript are UTF-16 encoded. UTF-16 encoding has the concept of *surrogate pairs*, which is introduced in detail in the UTF-16 characters, Unicode code points, and grapheme clusters section.

`toWellFormed()` iterates through the code units of this string, and replaces any lone surrogates with the Unicode replacement character U+FFFD `�`. This ensures that the returned string is well-formed and can be used in functions that expect well-formed strings, such as `encodeURI`. Compared to a custom implementation, `toWellFormed()` is more efficient, as engines can directly access the internal representation of strings.

When ill-formed strings are used in certain contexts, such as `TextEncoder`, they are automatically converted to well-formed strings using the same replacement character. When lone surrogates are rendered, they are also rendered as the replacement character (a diamond with a question mark inside).

Examples

Using toWellFormed()

```
const strings = [
  // Lone leading surrogate
  "ab\uD800",
  "ab\uD800c",
  // Lone trailing surrogate
  "\uDFFFab",
  "c\uDFFFab",
  // Well-formed
```

(code continues on the next page)

(code continued from the previous page)

```
  "abc",
  "ab\uD83D\uDE04c",
];

for (const str of strings) {
  console.log(str.toWellFormed());
}
// Logs:
// "ab"
// "ab"
// "ab"
// "c"
// "abc"
// "ab😄c"
```

Avoiding errors in encodeURI()

encodeURI throws an error if the string passed is not well-formed. This can be avoided by using `toWellFormed()` to convert the string to a well-formed string first.

js

```
const illFormed = "https://example.com/search?q=\uD800";

try {
  encodeURI(illFormed);
} catch (e) {
  console.log(e); // URIError: URI malformed
}

console.log(encodeURI(illFormed.toWellFormed())); // "https://example.com/search?
q=%EF%BF%BD"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.towellformed

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toWellFormed	111	111	119	97	16.4	111	119	75	16.4	22.0	111	16.4	1.0.0	1.32	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.toWellFormed`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.toWellFormed` in `core-js`
- es-shims polyfill of `String.prototype.toWellFormed`

- `String.prototype.isWellFormed()`
- `String.prototype.normalize()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.trim()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/trim · MDN page last modified 2026-09-07

Baseline: Widely available — available across major browsers since July 2015.

The **`trim()`** method of `String` values removes whitespace from both ends of this string and returns a new string, without modifying the original string.

To return a new string with whitespace trimmed from just one end, use `trimStart()` or `trimEnd()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String.prototype.trim()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trim());
// Expected output: "Hello world!";
```

Syntax

js

```
trim()
```

Parameters

None.

Return value

A new string representing `str` stripped of whitespace from both its beginning and end. Whitespace is defined as white space characters plus line terminators.

If neither the beginning nor the end of `str` has any whitespace, a new string is still returned (essentially a copy of `str`).

Examples

Using trim()

The following example trims whitespace from both ends of `str`.

js

```
const str = "   foo  ";
console.log(str.trim()); // 'foo'
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.trim

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`trim`	4	12	3.5	10.5	5	18	4	11	5	1.0	4.4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.trim`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.trimStart()`
- `String.prototype.trimEnd()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.trimEnd()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/trimEnd · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`trimEnd()`** method of `String` values removes whitespace from the end of this string and returns a new string, without modifying the original string. `trimRight()` is an alias of this method.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.trimEnd()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trimEnd());
// Expected output: "   Hello world!";
```

Syntax

js

```
trimEnd()

trimRight()
```

Parameters

None.

Return value

A new string representing `str` stripped of whitespace from its end (right side). Whitespace is defined as white space characters plus line terminators.

If the end of `str` has no whitespace, a new string is still returned (essentially a copy of `str`).

Aliasing

After `trim()` was standardized, engines also implemented the non-standard method `trimRight`. However, for consistency with `padEnd()`, when the method got standardized, its name was chosen as `trimEnd`. For web compatibility reasons, `trimRight` remains as an alias to `trimEnd`, and they refer to the exact same function object. In some engines this means:

```js
String.prototype.trimRight.name === "trimEnd";
```

Examples

Using trimEnd()

The following example trims whitespace from the end of `str`, but not from its start.

```js
let str = "   foo  ";

console.log(str.length); // 8

str = str.trimEnd();
console.log(str.length); // 6
console.log(str); // '   foo'
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.trimend

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`trimEnd`	66	79	61	53	12	66	61	47	12	9.0	66	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.trimEnd`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `String.prototype.trimEnd` in `core-js`
- es-shims polyfill of `String.prototype.trimEnd`
- `String.prototype.trim()`
- `String.prototype.trimStart()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.trimStart()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/trimStart · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`trimStart()`** method of `String` values removes whitespace from the beginning of this string and returns a new string, without modifying the original string. `trimLeft()` is an alias of this method.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.trimStart()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trimStart());
// Expected output: "Hello world!   ";
```

Syntax

js

```
trimStart()

trimLeft()
```

Parameters

None.

Return value

A new string representing `str` stripped of whitespace from its beginning (left side). Whitespace is defined as white space characters plus line terminators.

If the beginning of `str` has no whitespace, a new string is still returned (essentially a copy of `str`).

Aliasing

After `trim()` was standardized, engines also implemented the non-standard method `trimLeft`. However, for consistency with `padStart()`, when the method got standardized, its name was chosen as `trimStart`. For web compatibility reasons, `trimLeft` remains as an alias to `trimStart`, and they refer to the exact same function object. In some engines this means:

```js
String.prototype.trimLeft.name === "trimStart";
```

Examples

Using trimStart()

The following example trims whitespace from the start of `str`, but not from its end.

```js
let str = "   foo  ";

console.log(str.length); // 8

str = str.trimStart();
console.log(str.length); // 5
console.log(str); // 'foo  '
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.trimstart

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`trimStart`	66	79	61	53	12	66	61	47	12	9.0	66	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.trimStart`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype.trimStart` in `core-js`
- es-shims polyfill of `String.prototype.trimStart`
- `String.prototype.trim()`
- `String.prototype.trimEnd()`

JavaScript › Reference › Standard built-in objects › String

String.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/valueOf · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`valueOf()`** method of `String` values returns this string value.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype.valueOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const stringObj = new String("foo");

console.log(stringObj);
// Expected output: String { "foo" }

console.log(stringObj.valueOf());
// Expected output: "foo"
```

Syntax

js

```
valueOf()
```

Parameters

None.

Return value

A string representing the primitive value of a given `String` object.

Description

The `valueOf()` method of `String` returns the primitive value of a `String` object as a string data type. This value is equivalent to `String.prototype.toString()`.

This method is usually called internally by JavaScript and not explicitly in code.

Examples

Using `valueOf()`

js

```
const x = new String("Hello world");
console.log(x.valueOf()); // 'Hello world'
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype.valueof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `String.prototype.toString()`
- `Object.prototype.valueOf()`

JavaScript › Reference › Standard built-in objects › String

String.prototype[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/Symbol.iterator · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`[Symbol.iterator]()`** method of `String` values implements the iterable protocol and allows strings to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a string iterator object that yields the Unicode code points of the string value as individual strings.

Try it

▷ Interactive example on MDN: "JavaScript Demo: String.prototype[Symbol.iterator]()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const str = "The quick red fox jumped over the lazy dog's back.";

const iterator = str[Symbol.iterator]();
let theChar = iterator.next();

while (!theChar.done && theChar.value !== " ") {
  console.log(theChar.value);
  theChar = iterator.next();
  // Expected output: "T"
  //                  "h"
  //                  "e"
}
```

Syntax

```js
string[Symbol.iterator]()
```

Parameters

None.

Return value

A new iterable iterator object that yields the Unicode code points of the string value as individual strings.

Description

Strings are iterated by Unicode code points. This means grapheme clusters will be split, but surrogate pairs will be preserved.

```js
// "Backhand Index Pointing Right: Dark Skin Tone"
[..."👉🏿"]; // ['👉', '🏿']
// splits into the basic "Backhand Index Pointing Right" emoji and
// the "Dark skin tone" emoji

// "Family: Man, Boy"
[..."👨‍👦"]; // [ '👨', '‍', '👦' ]
// splits into the "Man" and "Boy" emoji, joined by a ZWJ
```

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes strings iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```js
const str = "A\uD835\uDC68B\uD835\uDC69C\uD835\uDC6A";

for (const v of str) {
  console.log(v);
}
// "A"
// "\uD835\uDC68"
// "B"
// "\uD835\uDC69"
// "C"
// "\uD835\uDC6A"
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```js
const str = "A\uD835\uDC68";

const strIter = str[Symbol.iterator]();

console.log(strIter.next().value); // "A"
console.log(strIter.next().value); // "\uD835\uDC68"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-string.prototype-%symbol.iterator%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.iterator]	38	12	36[1,2,3,4]	25	9	38	36[1,2,3,4]	25	9	3.0	38	9	1.0.0	1.0	0.12.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.@@iterator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `String.prototype[Symbol.iterator]` in `core-js`
- Numbers and strings guide
- `Symbol.iterator`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › String

String: length

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/length · MDN page last modified 2026-09-02

Baseline: Widely available — available across major browsers since July 2015.

The **`length`** data property of a `String` value contains the length of the string in UTF-16 code units.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: String: length". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const str = "Life, the universe and everything. Answer:";

console.log(`${str} ${str.length}`);
// Expected output: "Life, the universe and everything. Answer: 42"
```

Value

A non-negative integer.

Property attributes of `String: length`	
Writable	no
Enumerable	no
Configurable	no

Description

This property returns the number of code units in the string. JavaScript uses UTF-16 encoding, where each Unicode character may be encoded as one or two code units, so it's possible for the value returned by `length` to not match the actual number of Unicode characters in the string. For common scripts like Latin, Cyrillic, wellknown CJK characters, etc., this should not be an issue, but if you are working with certain scripts, such as emojis, mathematical symbols, or obscure Chinese characters, you may need to account for the difference between code units and characters.

The language specification requires strings to have a maximum length of 2^{53} - 1 elements, which is the upper limit for precise integers. However, a string with this length needs 16384TiB of storage, which cannot fit in any reasonable device's memory, so implementations tend to lower the threshold, which allows the string's length to be conveniently stored in a 32-bit integer.

- In V8 (used by Chrome and Node), the maximum length is 2^{29} - 24 (~1GiB). On 32-bit systems, the maximum length is 2^{28} - 16 (~512MiB).
- In Firefox, the maximum length is 2^{30} - 2 (~2GiB). Before Firefox 65, the maximum length was 2^{28} - 1 (~512MiB).
- In Safari, the maximum length is 2^{31} - 1 (~4GiB).

> **Note:** All memory sizes above assume strings are stored in UTF-16 encoding in memory, which may be an overestimate. The actual space used by a string is subject to JavaScript engine optimizations, and all major engines use a more compact internal representation, such as one-byte storage, for strings containing only Latin-1 characters.

If you are working with large strings in other encodings (such as UTF-8 files or blobs), note that when you load the data into a JS string, the encoding always becomes UTF-16. The size of the string may be different from the size of the source file.

```js
const str1 = "a".repeat(2 ** 29 - 24); // Success
const str2 = "a".repeat(2 ** 29 - 23); // RangeError: Invalid string length

const buffer = new Uint8Array(2 ** 29 - 24).fill("a".codePointAt(0)); // This buffer is
512MiB in size
const str = new TextDecoder().decode(buffer); // This string is 1GiB in size unless
optimized
```

For an empty string, `length` is 0.

The static property `String.length` is unrelated to the length of strings. It's the arity of the `String` function (loosely, the number of formal parameters it has), which is 1.

Since `length` counts code units instead of characters, if you want to get the number of characters, you can first split the string with its iterator, which iterates by characters:

```js
function getCharacterLength(str) {
  // The string iterator that is used here iterates over characters,
  // not mere code units
  return [...str].length;
}

console.log(getCharacterLength("A\uD87E\uDC04Z")); // 3
```

If you want to count characters by *grapheme clusters*, use `Intl.Segmenter`. You can first pass the string you want to split to the `segment()` method, and then iterate over the returned `Segments` object to get the length:

```js
function getGraphemeCount(str) {
  const segmenter = new Intl.Segmenter("en-US", { granularity: "grapheme" });
  // The Segments object iterator that is used here iterates over characters in grapheme
clusters,
  // which may consist of multiple Unicode characters
  return [...segmenter.segment(str)].length;
}

console.log(getGraphemeCount("👩‍👩‍👦‍👦")); // 1
```

Examples

Basic usage

```js
const x = "Mozilla";
const empty = "";

console.log(`${x} is ${x.length} code units long`);
// Mozilla is 7 code units long

console.log(`The empty string has a length of ${empty.length}`);
// The empty string has a length of 0
```

Strings with length not equal to the number of characters

```js
const emoji = "😉";
console.log(emoji.length); // 2
console.log([...emoji].length); // 1
const adlam = "𞤢𞤤𞤭𞤲";
console.log(adlam.length); // 8
console.log([...adlam].length); // 4
const formula = "∀𝑥∈ℝ,𝑥²≥0";
console.log(formula.length); // 11
console.log([...formula].length); // 9
```

Assigning to length

Because string is a primitive, attempting to assign a value to a string's `length` property has no observable effect, and will throw in strict mode.

```js
const myString = "bluebells";

myString.length = 4;
console.log(myString); // "bluebells"
console.log(myString.length); // 9
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/text-processing.html#sec-properties-of-string-instances-length

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`length`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.String.length`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Array: length`

JavaScript › Reference › Standard built-in objects

SuppressedError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SuppressedError · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`SuppressedError`** object represents an error generated while handing another error. It is generated during resource disposal using `using` or `await using`.

Compared to `AggregateError`, `SuppressedError` represents an error that happened during the handling of another error, while `AggregateError` represents a list of unrelated errors. It is possible, though, for a `SuppressedError` to contain a chain of suppressed errors (`e.suppressed.suppressed.suppressed...`). It is also semantically different from `cause` because the error is not *caused* by another error, but *happens when* handling another error.

`SuppressedError` is a subclass of `Error`.

Constructor

`SuppressedError()`
: Creates a new `SuppressedError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `SuppressedError.prototype` and shared by all `SuppressedError` instances.

`SuppressedError.prototype.constructor`
: The constructor function that created the instance object. For `SuppressedError` instances, the initial value is the `SuppressedError` constructor.

`SuppressedError.prototype.name`
: Represents the name for the type of error. For `SuppressedError.prototype.name`, the initial value is `"SuppressedError"`.

> **Note:** `SuppressedError` never has the `cause` property, because the semantics of `cause` overlaps with `suppressed`.

These properties are own properties of each `SuppressedError` instance.

`error`
: A reference to the error that results in the suppression.

`suppressed`
: A reference to the error that is suppressed by `error`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a SuppressedError

A `SuppressedError` is thrown when an error occurs during resource disposal. Throwing an error causes scope cleanup, and each disposer during the cleanup can throw its own error. All these errors are collected into a chain of `SuppressedError` instances, with the original error as the `suppressed` property and the new error thrown by the next disposer as the `error` property.

```js
try {
  using resource1 = {
    [Symbol.dispose]() {
      throw new Error("resource1 disposal failed");
    },
  };
  using resource2 = {
    [Symbol.dispose]() {
      throw new Error("resource2 disposal failed");
    },
  };
  throw new TypeError("Original error");
} catch (e) {
  console.log(e instanceof SuppressedError); // true
  console.log(e.message); // "An error was suppressed during disposal"
  console.log(e.name); // "SuppressedError"
  console.log(e.error); // Error: resource1 disposal failed
  console.log(e.suppressed); // SuppressedError: An error was suppressed during disposal
  console.log(e.suppressed.error); // Error: resource2 disposal failed
  console.log(e.suppressed.suppressed); // TypeError: Original error
}
```

The chain looks like this:

```
     SuppressedError --suppressed--> SuppressedError --suppressed--> TypeError
            |                               |
          error                           error
            |                               |
            v                               v
resource1 disposal failed       resource2 disposal failed
 (Disposal happens later)       (Disposal happens earlier)
```

Creating a SuppressedError

```js
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e instanceof SuppressedError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "SuppressedError"
  console.log(e.error); // Error: "New error"
  console.log(e.suppressed); // Error: "Original error"
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-suppressederror-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`SuppressedError`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0
`SuppressedError`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0
`error`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0
`suppressed`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SuppressedError`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `SuppressedError` in `core-js`
- `Error`
- `using`
- `await using`
- `DisposableStack`
- `AsyncDisposableStack`

JavaScript › Reference › Standard built-in objects › SuppressedError

SuppressedError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SuppressedError/SuppressedError · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`SuppressedError()`** constructor creates `SuppressedError` objects.

Syntax

```js
new SuppressedError(error, suppressed)
new SuppressedError(error, suppressed, message)

SuppressedError(error, suppressed)
SuppressedError(error, suppressed, message)
```

> **Note:** `SuppressedError()` can be called with or without `new`. Both create a new `SuppressedError` instance.

Parameters

`error`

The new error that results in the suppression of `suppressed`.

`suppressed`

The error that was originally thrown and is now suppressed.

`message` Optional

An optional human-readable description of the aggregate error.

> **Note:** `SuppressedError()` does not accept `options` like `Error()` and other subclasses do, because the semantics of `cause` overlaps with `suppressed`.

Examples

Creating a SuppressedError

```js
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e.suppressed); // Error: "Original error"
  console.log(e.error); // Error: "New error"
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-suppressederror-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`SuppressedError`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SuppressedError.SuppressedError`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `SuppressedError` in `core-js`

JavaScript › Reference › Standard built-in objects › SuppressedError

SuppressedError: error

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SuppressedError/error · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`error`** data property of a `SuppressedError` instance contains a reference to the error that results in the suppression.

Value

Any value. Like `cause`, you cannot assume it's an `Error` instance, although it usually is the case.

Property attributes of `SuppressedError: error`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using error

```js
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e.error); // Error: "New error"
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-suppressederror`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`error`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SuppressedError.error`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `SuppressedError` in `core-js`
- Control flow and error handling guide
- `SuppressedError`
- `Error: cause`

JavaScript › Reference › Standard built-in objects › SuppressedError

SuppressedError: suppressed

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SuppressedError/suppressed · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`suppressed`** data property of a `SuppressedError` instance contains a reference to the original error that got suppressed because a new error was generated while handling it.

Value

Any value. Like `cause`, you cannot assume it's an `Error` instance, although it usually is the case.

Property attributes of `SuppressedError: suppressed`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using suppressed

js

```
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e.suppressed); // Error: "Original error"
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-suppressederror`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`suppressed`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SuppressedError.suppressed`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `SuppressedError` in `core-js`
- Control flow and error handling guide
- `SuppressedError`
- `Error: cause`

JavaScript › Reference › Standard built-in objects

Symbol

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

`Symbol` is a built-in object whose constructor returns a `symbol` primitive — also called a **Symbol value** or just a **Symbol** — that's guaranteed to be unique. Symbols are often used to add unique property keys to an object that won't collide with keys any other code might add to the object, and which are hidden from any mechanisms other code will typically use to access the object. That enables a form of weak encapsulation, or a weak form of information hiding.

Every `Symbol()` call is guaranteed to return a unique Symbol. Every `Symbol.for("key")` call will always return the same Symbol for a given value of `"key"`. When `Symbol.for("key")` is called, if a Symbol with the given key can be found in the global Symbol registry, that Symbol is returned. Otherwise, a new Symbol is created, added to the global Symbol registry under the given key, and returned.

Description

To create a new primitive Symbol, you write `Symbol()` with an optional string as its description:

```js
const sym1 = Symbol();
const sym2 = Symbol("foo");
const sym3 = Symbol("foo");
```

The above code creates three new Symbols. Note that `Symbol("foo")` does not coerce the string `"foo"` into a Symbol. It creates a new Symbol each time:

```js
Symbol("foo") === Symbol("foo"); // false
```

The following syntax with the `new` operator will throw a `TypeError`:

js · ✗ bad example

```
const sym = new Symbol(); // TypeError
```

This prevents authors from creating an explicit `Symbol` wrapper object instead of a new Symbol value and might be surprising as creating explicit wrapper objects around primitive data types is generally possible (for example, `new Boolean`, `new String` and `new Number`).

If you really want to create a `Symbol` wrapper object, you can use the `Object()` function:

```js
const sym = Symbol("foo");
typeof sym; // "symbol"
const symObj = Object(sym);
typeof symObj; // "object"
```

Because symbols are the only primitive data type that has reference identity (that is, you cannot create the same symbol twice), they behave like objects in some way. For example, they are garbage collectable and can therefore be stored in `WeakMap`, `WeakSet`, `WeakRef`, and `FinalizationRegistry` objects.

Shared Symbols in the global Symbol registry

The above syntax using the `Symbol()` function will create a Symbol whose value remains unique throughout the lifetime of the program. To create Symbols available across files and even across realms (each of which has its own global scope), use the methods `Symbol.for()` and `Symbol.keyFor()` to set and retrieve Symbols from the global Symbol registry.

Note that the "global Symbol registry" is only a fictitious concept and may not correspond to any internal data structure in the JavaScript engine — and even if such a registry exists, its content is not available to the JavaScript code, except through the `for()` and `keyFor()` methods.

The method `Symbol.for(tokenString)` takes a string key and returns a symbol value from the registry, while `Symbol.keyFor(symbolValue)` takes a symbol value and returns the string key corresponding to it. Each is the other's inverse, so the following is `true`:

```js
Symbol.keyFor(Symbol.for("tokenString")) === "tokenString"; // true
```

Because registered symbols can be arbitrarily created anywhere, they behave almost exactly like the strings they wrap. Therefore, they are not guaranteed to be unique and are not garbage collectable. Therefore, registered symbols are disallowed in `WeakMap`, `WeakSet`, `WeakRef`, and `FinalizationRegistry` objects.

Well-known Symbols

All static properties of the `Symbol` constructor are Symbols themselves, whose values are constant across realms. They are known as *well-known Symbols*, and their purpose is to serve as "protocols" for certain built-in JavaScript operations, allowing users to customize the language's behavior. For example, if a constructor function has a method with `Symbol.hasInstance` as its name, this method will encode its behavior with the `instanceof` operator.

Prior to well-known Symbols, JavaScript used normal properties to implement certain built-in operations. For example, the `JSON.stringify` function will attempt to call each object's `toJSON()` method, and the `String` function will call the object's `toString()` and `valueOf()` methods. However, as more operations are added to the language, designating each operation a "magic property" can break backward compatibility and make the language's behavior harder to reason with. Well-known Symbols allow the customizations to be "invisible" from normal code, which typically only read string properties.

> **Note:** The spec used to use the notation `@@<symbol-name>` to denote well-known symbols. For example, `Symbol.hasInstance` was written as `@@hasInstance`, and the `Array.prototype[Symbol.iterator]()` method would be called `Array.prototype[@@iterator]()`. This notation is no longer used in the spec, but you may still see it in older documentation or discussions.

Well-known symbols do not have the concept of garbage collectability, because they come in a fixed set and are unique throughout the lifetime of the program, similar to intrinsic objects such as `Array.prototype`, so they are also allowed in `WeakMap`, `WeakSet`, `WeakRef`, and `FinalizationRegistry` objects.

Finding Symbol properties on objects

The method `Object.getOwnPropertySymbols()` returns an array of Symbols and lets you find Symbol properties on a given object. Note that every object is initialized with no own Symbol properties, so that this array will be empty unless you've set Symbol properties on the object.

Constructor

`Symbol()`

Returns primitive values of type Symbol. Throws an error when called with `new`.

Static properties

The static properties are all well-known Symbols. In these Symbols' descriptions, we will use language like "`Symbol.hasInstance` is a method determining…", but bear in mind that this is referring to the semantic of an object's method having this Symbol as the method name (because well-known Symbols act as "protocols"), not describing the value of the Symbol itself.

`Symbol.asyncDispose`

A method that disposes resources of the object asynchronously when the object goes out of scope. Used by the `await using` declaration.

`Symbol.asyncIterator`

A method that returns the default AsyncIterator for an object. Used by `for await...of`.

`Symbol.dispose`

A method that disposes resources of the object when the object goes out of scope. Used by the `using` declaration.

`Symbol.hasInstance`

A method determining if a constructor object recognizes an object as its instance. Used by `instanceof`.

`Symbol.isConcatSpreadable`

A Boolean value indicating if an object should be flattened to its array elements. Used by `Array.prototype.concat()`.

`Symbol.iterator`

A method returning the default iterator for an object. Used by `for...of`.

`Symbol.match`

A method that matches against a string, also used to determine if an object may be used as a regular expression. Used by `String.prototype.match()`.

`Symbol.matchAll`

A method that returns an iterator, that yields matches of the regular expression against a string. Used by `String.prototype.matchAll()`.

`Symbol.replace`

A method that replaces matched substrings of a string. Used by `String.prototype.replace()`.

`Symbol.search`

A method that returns the index within a string that matches the regular expression. Used by `String.prototype.search()`.

`Symbol.species`

A constructor function that is used to create derived objects.

`Symbol.split`

A method that splits a string at the indices that match a regular expression. Used by `String.prototype.split()`.

`Symbol.toPrimitive`

A method converting an object to a primitive value.

`Symbol.toStringTag`

A string value used for the default description of an object. Used by `Object.prototype.toString()`.

`Symbol.unscopables`

An object value of whose own and inherited property names are excluded from the `with` environment bindings of the associated object.

Static methods

`Symbol.for()`

Searches for existing registered Symbols in the global Symbol registry with the given `key` and returns it if found. Otherwise a new Symbol gets created and registered with `key`.

`Symbol.keyFor()`

Retrieves a shared Symbol key from the global Symbol registry for the given Symbol.

Instance properties

These properties are defined on `Symbol.prototype` and shared by all `Symbol` instances.

`Symbol.prototype.constructor`

The constructor function that created the instance object. For `Symbol` instances, the initial value is the `Symbol` constructor.

`Symbol.prototype.description`

A read-only string containing the description of the Symbol.

`Symbol.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Symbol"`. This property is used in `Object.prototype.toString()`. However, because `Symbol` also has its own `toString()` method, this property is not used unless you call `Object.prototype.toString.call()` with a symbol as `thisArg`.

Instance methods

`Symbol.prototype.toString()`

Returns a string containing the description of the Symbol. Overrides the `Object.prototype.toString()` method.

`Symbol.prototype.valueOf()`

Returns the Symbol. Overrides the `Object.prototype.valueOf()` method.

`Symbol.prototype[Symbol.toPrimitive]()`

Returns the Symbol.

Examples

Using the typeof operator with Symbols

The `typeof` operator can help you to identify Symbols.

```js
typeof Symbol() === "symbol";
typeof Symbol("foo") === "symbol";
typeof Symbol.iterator === "symbol";
```

Symbol type conversions

Some things to note when working with type conversion of Symbols.

- When trying to convert a Symbol to a number, a `TypeError` will be thrown (e.g., `+sym` or `sym | 0`).
- When using loose equality, `Object(sym) == sym` returns `true`.
- `Symbol("foo") + "bar"` throws a `TypeError` (can't convert Symbol to string). This prevents you from silently creating a new string property name from a Symbol, for example.
- The "safer" `String(sym)` conversion works like a call to `Symbol.prototype.toString()` with Symbols, but note that `new String(sym)` will throw.

Symbols and for...in iteration

Symbols are not enumerable in `for...in` iterations. In addition, `Object.getOwnPropertyNames()` will not return Symbol object properties, however, you can use `Object.getOwnPropertySymbols()` to get these.

```js
const obj = {};

obj[Symbol("a")] = "a";
obj[Symbol.for("b")] = "b";
obj["c"] = "c";
obj.d = "d";

for (const i in obj) {
  console.log(i);
}
// "c" "d"
```

Symbols and JSON.stringify()

Symbol-keyed properties will be completely ignored when using `JSON.stringify()`:

```js
JSON.stringify({ [Symbol("foo")]: "foo" });
// '{}'
```

For more details, see `JSON.stringify()`.

Symbol wrapper objects as property keys

When a Symbol wrapper object is used as a property key, this object will be coerced to its wrapped Symbol:

```js
const sym = Symbol("foo");
const obj = { [sym]: 1 };
obj[sym]; // 1
obj[Object(sym)]; // still 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Symbol`	38	12[1]	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	0.12.0
[Symbol.toPrimitive]	47	15	44	34	10	47	44	34	10	5.0	47	10	1.0.0	1.0	6.0.0
`Symbol()` constructor	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	0.12.0
`asyncDispose`	127	127	141	113	Prev.	127	141	84	**No**	28.0	127	**No**	1.0.23	1.37	24.0.0 2,3,4,5
`asyncIterator`	63	79	57	50	11.1	63	57	46	11.3	8.0	63	11.3	1.0.0	1.0	10.0.0
`description`	70	79	63	57	12.1 6, 7	70	63	49	12.2 8, 9	10.0	70	12.2 8, 9	1.0.0	1.0	11.0.0
`dispose`	125	125	141	111	Prev.	125	141	83	**No**	27.0	125	**No**	1.0.23	1.37	24.0.0 2,3,4,5
`for`	40	12	36	27	9	40	36	27	9	4.0	40	9	1.0.0	1.0	0.12.0
`hasInstance`	50	15	50	37	10	50	50	37	10	5.0	50	10	1.0.0	1.0	6.5.0
`isConcatSpreadable`	48	15	48	35	10	48	48	35	10	5.0	48	10	1.0.0	1.0	6.0.0
`iterator`	43	12	36	30	10	43	36	30	10	4.0	43	10	1.0.0	1.0	0.12.0
`keyFor`	40	12	36	27	9	40	36	27	9	4.0	40	9	1.0.0	1.0	0.12.0
`match`	50	79	40	37	10	50	40	37	10	5.0	50	10	1.0.0	1.0	6.0.0
`matchAll`	73	79	67	60	13	73	67	52	13	11.0	73	13	1.0.0	1.0	12.0.0
`replace`	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0
`search`	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0
`species`	51	13	41	38	10	51	41	41	10	5.0	51	10	1.0.0	1.0	6.5.0
`split`	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0
`toPrimitive`	47	15	44	34	10	47	44	34	10	5.0	47	10	1.0.0	1.0	6.0.0
`toString`	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	0.12.0
`toStringTag`	49	15	51	36	10	49	51	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`toStringTag` available on all DOM prototype objects	50	79	78	37	14	50	79	37	14	5.0	50	14	?	1.0	**No**
`unscopables`	38	12	48	25	9	38	48	25	9	3.0	38	9	1.0.0	1.0	0.12.0
`valueOf`	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	0.12.0

1. Edge 12 included Symbol properties in `JSON.stringify()` output.
2. Also from version 20.4.0–21.0.0: partial support.
3. Also from version 20.4.0–21.0.0: Only available for `fs` and `stream` resources.
4. Also from version 18.18.0–19.0.0: partial support.
5. Also from version 18.18.0–19.0.0: Only available for `fs` and `stream` resources.
6. Also from version 12–12.1: partial support.
7. Also from version 12–12.1: No support for an undefined description.
8. Also from version 12–12.2: partial support.
9. Also from version 12–12.2: No support for an undefined description.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol` in `core-js`
- `typeof`
- JavaScript data types and data structures
- ES6 In Depth: Symbols on hacks.mozilla.org (2015)

JavaScript › Reference › Standard built-in objects › Symbol

Symbol() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/Symbol · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`Symbol()`** function returns primitive values of type Symbol.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const symbol1 = Symbol();
const symbol2 = Symbol(42);
const symbol3 = Symbol("foo");

console.log(typeof symbol1);
// Expected output: "symbol"

console.log(symbol2 === 42);
// Expected output: false

console.log(symbol3.toString());
// Expected output: "Symbol(foo)"

console.log(Symbol("foo") === Symbol("foo"));
// Expected output: false
```

Syntax

js

```
Symbol()
Symbol(description)
```

> **Note:** `Symbol()` can only be called without `new`. Attempting to construct it with `new` throws a `TypeError`.

Parameters

`description` Optional

A string. A description of the symbol which can be used for debugging but not to access the symbol itself.

Examples

Creating symbols

To create a new primitive symbol, you write `Symbol()` with an optional string as its description:

```js
const sym1 = Symbol();
const sym2 = Symbol("foo");
const sym3 = Symbol("foo");
```

The above code creates three new symbols. Note that `Symbol("foo")` does not coerce the string `"foo"` into a symbol. It creates a new symbol each time:

```js
Symbol("foo") === Symbol("foo"); // false
```

new Symbol()

The following syntax with the `new` operator will throw a `TypeError`:

```js · ✗ bad example
const sym = new Symbol(); // TypeError
```

This prevents authors from creating an explicit `Symbol` wrapper object instead of a new symbol value and might be surprising as creating explicit wrapper objects around primitive data types is generally possible (for example, `new Boolean`, `new String` and `new Number`).

If you really want to create a `Symbol` wrapper object, you can use the `Object()` function:

```js
const sym = Symbol("foo");
const symObj = Object(sym);
typeof sym; // "symbol"
typeof symObj; // "object"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Symbol()` constructor	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.Symbol`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Symbol` in `core-js`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.for()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/for · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2015.

The **`Symbol.for()`** static method searches for existing symbols in a runtime-wide symbol registry with the given key and returns it if found. Otherwise a new symbol gets created in the global symbol registry with this key.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.for()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Symbol.for("bar") === Symbol.for("bar"));
// Expected output: true

console.log(Symbol("bar") === Symbol("bar"));
// Expected output: false

const symbol1 = Symbol.for("foo");

console.log(symbol1.toString());
// Expected output: "Symbol(foo)"
```

Syntax

js

```
Symbol.for(key)
```

Parameters

key

String, required. The key for the symbol (and also used for the description of the symbol).

Return value

An existing symbol with the given key if found; otherwise, a new symbol is created and returned.

Description

In contrast to `Symbol()`, the `Symbol.for()` function creates a symbol available in a global symbol registry list. `Symbol.for()` does also not necessarily create a new symbol on every call, but checks first if a symbol with the given `key` is already present in the registry. In that case, that symbol is returned. If no symbol with the given key is found, `Symbol.for()` will create a new global symbol.

Examples

Using Symbol.for()

```js
Symbol.for("foo"); // create a new global symbol
Symbol.for("foo"); // retrieve the already created symbol

// Same global symbol, but not locally
Symbol.for("bar") === Symbol.for("bar"); // true
Symbol("bar") === Symbol("bar"); // false

// The key is also used as the description
const sym = Symbol.for("mario");
sym.toString(); // "Symbol(mario)"
```

To avoid name clashes with your global symbol keys and other (library code) global symbols, it might be a good idea to prefix your symbols:

```js
Symbol.for("mdn.foo");
Symbol.for("mdn.bar");
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.for

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
for	40	12	36	27	9	40	36	27	9	4.0	40	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.for`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Symbol.keyFor()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.keyFor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/keyFor · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2015.

The **`Symbol.keyFor()`** static method retrieves a shared symbol key from the global symbol registry for the given symbol.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.keyFor()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const globalSym = Symbol.for("foo"); // Global symbol

console.log(Symbol.keyFor(globalSym));
// Expected output: "foo"

const localSym = Symbol(); // Local symbol

console.log(Symbol.keyFor(localSym));
// Expected output: undefined

console.log(Symbol.keyFor(Symbol.iterator));
// Expected output: undefined
```

Syntax

js

```
Symbol.keyFor(sym)
```

Parameters

`sym`

Symbol, required. The symbol to find a key for.

Return value

A string representing the key for the given symbol if one is found on the global registry; otherwise, `undefined`.

Examples

Using keyFor()

```js
const globalSym = Symbol.for("foo"); // create a new global symbol
Symbol.keyFor(globalSym); // "foo"

const localSym = Symbol();
Symbol.keyFor(localSym); // undefined

// well-known symbols are not symbols registered
// in the global symbol registry
Symbol.keyFor(Symbol.iterator); // undefined
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.keyfor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
keyFor	40	12	36	27	9	40	36	27	9	4.0	40	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.keyFor`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Symbol.for()`

Symbol.asyncDispose

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/asyncDispose · MDN page last modified 2026-11-04

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Symbol.asyncDispose`** static data property represents the well-known symbol `Symbol.asyncDispose`. The `await using` declaration looks up this symbol on the variable initializer for the method to call when the variable goes out of scope.

Value

The well-known symbol `Symbol.asyncDispose`.

Property attributes of `Symbol.asyncDispose`	
Writable	no
Enumerable	no

Property attributes of `Symbol.asyncDispose`	
Configurable	no

Description

An object is async disposable if it has the `[Symbol.asyncDispose]()` method. The method is expected to have the following semantics:

- Invoking this method notifies the AsyncDisposable object that the caller does not intend to continue to use this object. This method should perform any necessary logic to explicit clean up the resource including, but not limited to, file system handles, streams, host objects, etc.
- This method can return a promise, which will be awaited before continuing.
- When an exception is thrown from this method, it typically means that the resource could not be explicitly freed. An AsyncDisposable object is not considered "disposed" until the resulting Promise has been fulfilled.
- If called more than once on the same object, the function should not throw an exception. However, this requirement is not enforced.

Examples

User defined async disposables

`[Symbol.asyncDispose]` allows the creation of custom async disposables. See the `await using` reference for more information.

```js
class Disposable {
  #fileHandle;
  #disposed;

  constructor(handle) {
    this.#disposed = false;
    this.#fileHandle = handle;
  }

  async [Symbol.asyncDispose]() {
    await this.#fileHandle.close();
    this.#disposed = true;
  }

  get isDisposed() {
    return this.#disposed;
  }
}

const resource = new Disposable(await fs.open("my-file.txt", "r"));
{
  await using resourceUsed = resource;
  console.log(resource.isDisposed); // false
}
console.log(resource.isDisposed); // true
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-well-known-symbols`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`asyncDispose`	127	127	141	113	Prev.	127	141	84	**No**	28.0	127	**No**	1.0.23	1.37	24.0.0 [1,2,3,4]

1. Also from version 20.4.0–21.0.0: partial support.
2. Also from version 20.4.0–21.0.0: Only available for `fs` and `stream` resources.
3. Also from version 18.18.0–19.0.0: partial support.
4. Also from version 18.18.0–19.0.0: Only available for `fs` and `stream` resources.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.asyncDispose`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.asyncDispose` in `core-js`
- JavaScript resource management
- `Symbol.dispose`
- `await using`
- `AsyncDisposableStack.prototype[Symbol.asyncDispose]()`
- `AsyncIterator.prototype[Symbol.asyncDispose]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.asyncIterator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/asyncIterator · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since January 2020.

The **`Symbol.asyncIterator`** static data property represents the well-known symbol `Symbol.asyncIterator`. The async iterable protocol looks up this symbol for the method that returns the async iterator for an object. In order for an object to be async iterable, it must have a `[Symbol.asyncIterator]` key.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.asyncIterator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const delayedResponses = {
  delays: [500, 1300, 3500],
```

(code continues on the next page)

(code continued from the previous page)

```
  wait(delay) {
    return new Promise((resolve) => {
      setTimeout(resolve, delay);
    });
  },

  async *[Symbol.asyncIterator]() {
    for (const delay of this.delays) {
      await this.wait(delay);
      yield `Delayed response for ${delay} milliseconds`;
    }
  },
};

(async () => {
  for await (const response of delayedResponses) {
    console.log(response);
  }
})();

// Expected output: "Delayed response for 500 milliseconds"
// Expected output: "Delayed response for 1300 milliseconds"
// Expected output: "Delayed response for 3500 milliseconds"
```

Value

The well-known symbol `Symbol.asyncIterator`.

Property attributes of `Symbol.asyncIterator`	
Writable	no
Enumerable	no
Configurable	no

Examples

User-defined async iterables

You can define your own async iterable by setting the `[Symbol.asyncIterator]()` property on an object.

js

```
const myAsyncIterable = {
  async *[Symbol.asyncIterator]() {
    yield "hello";
    yield "async";
    yield "iteration!";
  },
};
```

(code continues on the next page)

(code continued from the previous page)

```
(async () => {
  for await (const x of myAsyncIterable) {
    console.log(x);
  }
})();
// Logs:
// "hello"
// "async"
// "iteration!"
```

When creating an API, remember that async iterables are designed to represent something *iterable* — like a stream of data or a list —, not to completely replace callbacks and events in most situations.

Built-in async iterables

There is no object in the core JavaScript language that is async iterable. Some web APIs, such as `ReadableStream`, have the `Symbol.asyncIterator` method set by default.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.asynciterator`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`asyncIterator`	63	79	57	50	11.1	63	57	46	11.3	8.0	63	11.3	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.asyncIterator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Iteration protocols
- for await...of

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.dispose

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/dispose · MDN page last modified 2026-11-04

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Symbol.dispose`** static data property represents the well-known symbol `Symbol.dispose`. The `using` declaration looks up this symbol on the variable initializer for the method to call when the variable goes out of scope.

Value

The well-known symbol `Symbol.dispose`.

Property attributes of `Symbol.dispose`	
Writable	no
Enumerable	no
Configurable	no

Description

An object is disposable if it has the `[Symbol.dispose]()` method. The method is expected to have the following semantics:

- Invoking this method notifies the Disposable object that the caller does not intend to continue to use this object. This method should perform any necessary logic to explicit clean up the resource including, but not limited to, file system handles, streams, host objects, etc.
- When an exception is thrown from this method, it typically means that the resource could not be explicitly freed.
- If called more than once on the same object, the function should not throw an exception. However, this requirement is not enforced.

This method should not return a promise, as promises returned by `[Symbol.dispose]()` are not awaited by `await using`. To declare async disposables, use `Symbol.asyncDispose`.

Examples

User defined disposables

`[Symbol.dispose]` allows the creation of custom disposables. See the `using` reference for more information.

```js
class Disposable {
  #disposed;
  constructor() {
    this.#disposed = false;
  }

  [Symbol.dispose]() {
    this.#disposed = true;
  }

  get isDisposed() {
    return this.#disposed;
  }
}

const resource = new Disposable();
{
  using resourceUsed = resource;
  console.log(resource.isDisposed); // false
}
console.log(resource.isDisposed); // true
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-well-known-symbols`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`dispose`	125	125	141	111	Prev.	125	141	83	**No**	27.0	125	**No**	1.0.23	1.37	24.0.0 [1,2,3,4]

1. Also from version 20.4.0–21.0.0: partial support.
2. Also from version 20.4.0–21.0.0: Only available for `fs` and `stream` resources.
3. Also from version 18.18.0–19.0.0: partial support.
4. Also from version 18.18.0–19.0.0: Only available for `fs` and `stream` resources.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.dispose`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.dispose` in `core-js`
- JavaScript resource management
- `Symbol.asyncDispose`
- `using`
- `DisposableStack.prototype[Symbol.dispose]()`
- `Iterator.prototype[Symbol.dispose]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.hasInstance

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/hasInstance · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since April 2017.

The **`Symbol.hasInstance`** static data property represents the well-known symbol `Symbol.hasInstance`. The `instanceof` operator looks up this symbol on its right-hand operand for the method used to determine if the constructor object recognizes an object as its instance.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.hasInstance". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class Array1 {
  static [Symbol.hasInstance](instance) {
    return Array.isArray(instance);
  }
}

console.log([] instanceof Array1);
// Expected output: true
```

Value

The well-known symbol `Symbol.hasInstance`.

Property attributes of `Symbol.hasInstance`	
Writable	no
Enumerable	no
Configurable	no

Description

The `instanceof` operator uses the following algorithm to calculate the return value of `object instanceof constructor`:

1. If `constructor` has a `[Symbol.hasInstance]()` method, then call it with `object` as the first argument and return the result, coerced to a boolean. Throw a `TypeError` if `constructor` is not an object, or if `constructor[Symbol.hasInstance]` is not one of `null`, `undefined`, or a function.
2. Otherwise, if `constructor` doesn't have a `[Symbol.hasInstance]()` method (`constructor[Symbol.hasInstance]` is `null` or `undefined`), then determine the result using the same algorithm as `Function.prototype[Symbol.hasInstance]()`. Throw a `TypeError` if `constructor` is not a function.

Because all functions inherit from `Function.prototype` by default, most of the time, the `Function.prototype[Symbol.hasInstance]()` method specifies the behavior of `instanceof` when the right-hand side is a function.

Examples

Custom instanceof behavior

You could implement your custom `instanceof` behavior like this, for example:

js

```
class MyArray {
  static [Symbol.hasInstance](instance) {
    return Array.isArray(instance);
  }
}
console.log([] instanceof MyArray); // true
```

```js
function MyArray() {}
Object.defineProperty(MyArray, Symbol.hasInstance, {
  value(instance) {
    return Array.isArray(instance);
  },
});
console.log([] instanceof MyArray); // true
```

Checking the instance of an object

Just in the same manner at which you can check if an object is an instance of a class using the `instanceof` keyword, we can also use `Symbol.hasInstance` for such checks.

```js
class Animal {
  constructor() {}
}

const cat = new Animal();

console.log(Animal[Symbol.hasInstance](cat)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.hasinstance

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`hasInstance`	50	15	50	37	10	50	50	37	10	5.0	50	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.hasInstance`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `instanceof`
- `Function.prototype[Symbol.hasInstance]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.isConcatSpreadable

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/isConcatSpreadable · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since April 2017.

The **`Symbol.isConcatSpreadable`** static data property represents the well-known symbol `Symbol.isConcatSpreadable`. The `Array.prototype.concat()` method looks up this symbol on each object being concatenated to determine if it should be treated as an array-like object and flattened to its array elements.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Symbol.isConcatSpreadable". It runs live in the browser there; its source code follows.

js · interactive example source

```
const alpha = ["a", "b", "c"];
const numeric = [1, 2, 3];
let alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric);
// Expected output: Array ["a", "b", "c", 1, 2, 3]

numeric[Symbol.isConcatSpreadable] = false;
alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric);
// Expected output: Array ["a", "b", "c", Array [1, 2, 3]]
```

Value

The well-known symbol `Symbol.isConcatSpreadable`.

Property attributes of `Symbol.isConcatSpreadable`	
Writable	no
Enumerable	no
Configurable	no

Description

The `[Symbol.isConcatSpreadable]` property can be defined as an own or inherited property and its value is a boolean. It can control behavior for arrays and array-like objects:

- For array objects, the default behavior is to spread (flatten) elements. `Symbol.isConcatSpreadable` can avoid flattening in these cases.
- For array-like objects, the default behavior is no spreading or flattening. `Symbol.isConcatSpreadable` can force flattening in these cases.

Examples

Arrays

By default, `Array.prototype.concat()` spreads (flattens) arrays into its result:

```js
const alpha = ["a", "b", "c"];
const numeric = [1, 2, 3];

const alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric); // Result: ['a', 'b', 'c', 1, 2, 3]
```

When setting `Symbol.isConcatSpreadable` to `false`, you can disable the default behavior:

```js
const alpha = ["a", "b", "c"];
const numeric = [1, 2, 3];

numeric[Symbol.isConcatSpreadable] = false;
const alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric); // Result: ['a', 'b', 'c', [1, 2, 3] ]
```

Array-like objects

For array-like objects, the default is to not spread. `Symbol.isConcatSpreadable` needs to be set to `true` in order to get a flattened array:

```js
const x = [1, 2, 3];

const fakeArray = {
  [Symbol.isConcatSpreadable]: true,
  length: 2,
  0: "hello",
  1: "world",
};

x.concat(fakeArray); // [1, 2, 3, "hello", "world"]
```

> **Note:** The `length` property is used to control the number of object properties to be added. In the above example, `length:2` indicates that two properties have to be added.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.isconcatspreadable

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
isConcatSpreadable	48	15	48	35	10	48	48	35	10	5.0	48	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.isConcatSpreadable`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.isConcatSpreadable` in `core-js`
- `Array.prototype.concat()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.iterator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/iterator · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`Symbol.iterator`** static data property represents the well-known symbol `Symbol.iterator`. The iterable protocol looks up this symbol for the method that returns the iterator for an object. In order for an object to be iterable, it must have a `[Symbol.iterator]` key.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.iterator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const iterable = {};

iterable[Symbol.iterator] = function* () {
  yield 1;
  yield 2;
  yield 3;
};

console.log([...iterable]);
// Expected output: Array [1, 2, 3]
```

Value

The well-known symbol `Symbol.iterator`.

Property attributes of `Symbol.iterator`	
Writable	no
Enumerable	no
Configurable	no

Description

Whenever an object needs to be iterated (such as at the beginning of a `for...of` loop), its `[Symbol.iterator]()` method is called with no arguments, and the returned **iterator** is used to obtain the values to be iterated.

Some built-in types have a default iteration behavior, while other types (such as `Object`) do not. Some built-in types with a `[Symbol.iterator]()` method are:

- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype[Symbol.iterator]()`
- `String.prototype[Symbol.iterator]()`
- `Map.prototype[Symbol.iterator]()`
- `Set.prototype[Symbol.iterator]()`

See also Iteration protocols for more information.

Examples

User-defined iterables

We can make our own iterables like this:

```js
const myIterable = {};
myIterable[Symbol.iterator] = function* () {
  yield 1;
  yield 2;
  yield 3;
};
[...myIterable]; // [1, 2, 3]
```

Or iterables can be defined directly inside a class or object using a computed property:

```js
class Foo {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
}

const someObj = {
  *[Symbol.iterator]() {
    yield "a";
    yield "b";
  },
};

console.log(...new Foo()); // 1, 2, 3
console.log(...someObj); // 'a', 'b'
```

Non-well-formed iterables

If an iterable's `[Symbol.iterator]()` method does not return an iterator object, then it is a non-well-formed iterable. Using it as such is likely to result in runtime exceptions or buggy behavior:

js · ✗ bad example

```
const nonWellFormedIterable = {};
nonWellFormedIterable[Symbol.iterator] = () => 1;
[...nonWellFormedIterable]; // TypeError: [Symbol.iterator]() returned a non-object value
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.iterator`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`iterator`	43	12	36	30	10	43	36	30	10	4.0	43	10	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.iterator`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.iterator` in `core-js`
- Iteration protocols
- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype[Symbol.iterator]()`
- `String.prototype[Symbol.iterator]()`
- `Map.prototype[Symbol.iterator]()`
- `Set.prototype[Symbol.iterator]()`
- `arguments[Symbol.iterator]()`
- `Segments.prototype[Symbol.iterator]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.match

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/match · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`Symbol.match`** static data property represents the well-known symbol `Symbol.match`. The `String.prototype.match()` method looks up this symbol on its first argument for the method used to match an input string against the current object. This symbol is also used to determine if an object should be treated as a regex.

For more information, see `RegExp.prototype[Symbol.match]()` and `String.prototype.match()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.match". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regexp = /foo/;
// console.log('/foo/'.startsWith(regexp));
// Expected output (Chrome): Error: First argument to String.prototype.startsWith must not
be a regular expression
// Expected output (Firefox): Error: Invalid type: first can't be a Regular Expression
// Expected output (Safari): Error: Argument to String.prototype.startsWith cannot be a
RegExp

regexp[Symbol.match] = false;

console.log("/foo/".startsWith(regexp));
// Expected output: true

console.log("/baz/".endsWith(regexp));
// Expected output: false
```

Value

The well-known symbol `Symbol.match`.

Property attributes of `Symbol.match`	
Writable	no
Enumerable	no
Configurable	no

Description

This function is also used to identify if objects have the behavior of regular expressions. For example, the methods `String.prototype.startsWith()`, `String.prototype.endsWith()` and `String.prototype.includes()`, check if their first argument is a regular expression and will throw a `TypeError` if they are. Now, if the `match` symbol is set to `false` (or a Falsy value except `undefined`), it indicates that the object is not intended to be used as a regular expression object.

Examples

Marking a RegExp as not a regex

The following code will throw a TypeError:

```js
"/bar/".startsWith(/bar/);

// Throws TypeError, as /bar/ is a regular expression
// and Symbol.match is not modified.
```

However, if you set `Symbol.match` to `false`, the object will be considered as not a regular expression object. The methods `startsWith` and `endsWith` won't throw a `TypeError` as a consequence.

```js
const re = /foo/;
re[Symbol.match] = false;
"/foo/".startsWith(re); // true
"/baz/".endsWith(re); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.match

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
match	50	79	40	37	10	50	40	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.match`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Symbol.match` in `core-js`
- `Symbol.matchAll`
- `Symbol.replace`
- `Symbol.search`
- `Symbol.split`
- `String.prototype.match()`
- `RegExp.prototype[Symbol.match]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.matchAll

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/matchAll · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`Symbol.matchAll`** static data property represents the well-known symbol `Symbol.matchAll`. The `String.prototype.matchAll()` method looks up this symbol on its first argument for the method that returns an iterator, that yields matches of the current object against a string.

For more information, see `RegExp.prototype[Symbol.matchAll]()` and `String.prototype.matchAll()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.matchAll". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const re = /\d+/g;
const str = "2016-01-02|2019-03-07";
const result = re[Symbol.matchAll](str);

console.log(Array.from(result, (x) => x[0]));
// Expected output: Array ["2016", "01", "02", "2019", "03", "07"]
```

Value

The well-known symbol `Symbol.matchAll`.

Property attributes of `Symbol.matchAll`	
Writable	no
Enumerable	no
Configurable	no

Examples

Using Symbol.matchAll

js

```
const str = "2016-01-02|2019-03-07";

const numbers = {
  *[Symbol.matchAll](str) {
    for (const n of str.matchAll(/\d+/g)) yield n[0];
  },
};

console.log(Array.from(str.matchAll(numbers)));
// ["2016", "01", "02", "2019", "03", "07"]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.matchall`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`matchAll`	73	79	67	60	13	73	67	52	13	11.0	73	13	1.0.0	1.0	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.matchAll`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Symbol.matchAll` in `core-js`
- es-shims polyfill of `Symbol.matchAll`
- `Symbol.match`
- `Symbol.replace`
- `Symbol.search`
- `Symbol.split`
- `String.prototype.matchAll()`
- `RegExp.prototype[Symbol.matchAll]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.replace

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/replace · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`Symbol.replace`** static data property represents the well-known symbol `Symbol.replace`. The `String.prototype.replace()` and `String.prototype.replaceAll()` methods look up this symbol on their first argument for the method that replaces substrings matched by the current object.

For more information, see `RegExp.prototype[Symbol.replace]()`, `String.prototype.replace()`, and `String.prototype.replaceAll()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.replace". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class Replace1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.replace](string) {
    return `s/${string}/${this.value}/g`;
  }
}

console.log("foo".replace(new Replace1("bar")));
// Expected output: "s/foo/bar/g"
```

Value

The well-known symbol `Symbol.replace`.

Property attributes of `Symbol.replace`	
Writable	no
Enumerable	no
Configurable	no

Examples

Using Symbol.replace

js

```
class CustomReplacer {
  constructor(value) {
    this.value = value;
  }
  [Symbol.replace](string) {
    return string.replace(this.value, "#!@?");
  }
}

console.log("football".replace(new CustomReplacer("foo"))); // "#!@?tball"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.replace`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`replace`	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.replace`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.replace` in `core-js`
- `Symbol.match`
- `Symbol.matchAll`
- `Symbol.search`
- `Symbol.split`
- `String.prototype.replace()`
- `String.prototype.replaceAll()`
- `RegExp.prototype[Symbol.replace]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.search

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/search · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`Symbol.search`** static data property represents the well-known symbol `Symbol.search`. The `String.prototype.search()` method looks up this symbol on its first argument for the method that returns the index within a string that matches the current object.

For more information, see `RegExp.prototype[Symbol.search]()` and `String.prototype.search()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.search". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class Search1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.search](string) {
    return string.indexOf(this.value);
  }
}

console.log("foobar".search(new Search1("bar")));
// Expected output: 3
```

Value

The well-known symbol `Symbol.search`.

Property attributes of `Symbol.search`	
Writable	no
Enumerable	no
Configurable	no

Examples

Custom string search

```js
class CaseInsensitiveSearch {
  constructor(value) {
    this.value = value.toLowerCase();
  }
  [Symbol.search](string) {
    return string.toLowerCase().indexOf(this.value);
  }
}

console.log("foobar".search(new CaseInsensitiveSearch("BaR"))); // 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.search`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`search`	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.search`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Symbol.search` in `core-js`
- `Symbol.match`
- `Symbol.matchAll`
- `Symbol.replace`
- `Symbol.split`

- `String.prototype.search()`
- `RegExp.prototype[Symbol.search]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.species

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/species · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`Symbol.species`** static data property represents the well-known symbol `Symbol.species`. Methods that create copies of an object may look up this symbol on the object for the constructor function to use when creating the copy.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.species". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class Array1 extends Array {
  static get [Symbol.species]() {
    return Array;
  }
}

const a = new Array1(1, 2, 3);
const mapped = a.map((x) => x * x);

console.log(mapped instanceof Array1);
// Expected output: false

console.log(mapped instanceof Array);
// Expected output: true
```

Value

The well-known symbol `Symbol.species`.

Property attributes of `Symbol.species`	
Writable	no
Enumerable	no
Configurable	no

Description

The `[Symbol.species]` accessor property allows subclasses to override the default constructor for objects. This specifies a protocol about how instances should be copied. For example, when you use copying methods of arrays, such as `map()`, the `map()` method uses

`instance.constructor[Symbol.species]` to get the constructor for constructing the new array. For more information, see subclassing built-ins.

All built-in implementations of `[Symbol.species]` return the `this` value, which is the current instance's constructor. This allows copying methods to create instances of derived classes rather than the base class — for example, `map()` will return an array of the same type as the original array.

Examples

Using species

You might want to return `Array` objects in your derived array class `MyArray`. For example, when using methods such as `map()` that return the default constructor, you want these methods to return a parent `Array` object, instead of the `MyArray` object. The `species` symbol lets you do this:

```js
class MyArray extends Array {
  // Overwrite species to the parent Array constructor
  static get [Symbol.species]() {
    return Array;
  }
}
const a = new MyArray(1, 2, 3);
const mapped = a.map((x) => x * x);

console.log(mapped instanceof MyArray); // false
console.log(mapped instanceof Array); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.species

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
species	51	13	41	38	10	51	41	41	10	5.0	51	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.species`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Array[Symbol.species]`
- `ArrayBuffer[Symbol.species]`
- `Map[Symbol.species]`
- `Promise[Symbol.species]`

- RegExp[Symbol.species]
- Set[Symbol.species]
- TypedArray[Symbol.species]

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.split

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/split · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`Symbol.split`** static data property represents the well-known symbol `Symbol.split`. The `String.prototype.split()` method looks up this symbol on its first argument for the method that splits a string at the indices that match the current object.

For more information, see `RegExp.prototype[Symbol.split]()` and `String.prototype.split()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.split". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class Split1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.split](string) {
    const index = string.indexOf(this.value);
    return `${this.value}${string.substring(0, index)}/${string.substring(
      index + this.value.length,
    )}`;
  }
}

console.log("foobar".split(new Split1("foo")));
// Expected output: "foo/bar"
```

Value

The well-known symbol `Symbol.split`.

Property attributes of `Symbol.split`	
Writable	no
Enumerable	no
Configurable	no

Examples

Custom reverse split

```js
class ReverseSplit {
  [Symbol.split](string) {
    const array = string.split(" ");
    return array.reverse();
  }
}

console.log("Another one bites the dust".split(new ReverseSplit()));
// [ "dust", "the", "bites", "one", "Another" ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.split

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
split	50	79	49	37	10	50	49	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.split`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.split` in `core-js`
- `Symbol.match`
- `Symbol.matchAll`
- `Symbol.replace`
- `Symbol.search`
- `String.prototype.split()`
- `RegExp.prototype[Symbol.split]()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.toPrimitive

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/toPrimitive · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since April 2017.

The **`Symbol.toPrimitive`** static data property represents the well-known symbol `Symbol.toPrimitive`. All type coercion algorithms look up this symbol on objects for the method that accepts a preferred type and returns a primitive representation of the object, before falling back to using the object's `valueOf()` and `toString()` methods.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.toPrimitive". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return 42;
    }
    return null;
  },
};

console.log(+object);
// Expected output: 42
```

Value

The well-known symbol `Symbol.toPrimitive`.

Property attributes of `Symbol.toPrimitive`	
Writable	no
Enumerable	no
Configurable	no

Description

With the help of the `Symbol.toPrimitive` property (used as a function value), an object can be converted to a primitive value. The function is called with a string argument `hint`, which specifies the preferred type of the result primitive value. The `hint` argument can be one of `"number"`, `"string"`, and `"default"`.

The `"number"` hint is used by numeric coercion algorithms. The `"string"` hint is used by the string coercion algorithm. The `"default"` hint is used by the primitive coercion algorithm. The `hint` only acts as a weak signal of preference, and the implementation is free to ignore it (as `Symbol.prototype[Symbol.toPrimitive]()` does). The language does not enforce alignment between the `hint` and the result type, although `[Symbol.toPrimitive]()` must return a primitive, or a `TypeError` is thrown.

Objects without the `[Symbol.toPrimitive]` property are converted to primitives by calling the `valueOf()` and `toString()` methods in different orders, which is explained in more detail in the type coercion section. `[Symbol.toPrimitive]()` allows full control over the primitive conversion process. For example, `Date.prototype[Symbol.toPrimitive]()` treats `"default"` as if it's `"string"` and calls `toString()` instead of `valueOf()`. `Symbol.prototype[Symbol.toPrimitive]()` ignores the hint and always returns a symbol, which means even in string contexts, `Symbol.prototype.toString()` won't be called, and `Symbol` objects must always be explicitly converted to strings through `String()`.

Examples

Modifying primitive values converted from an object

Following example describes how `Symbol.toPrimitive` property can modify the primitive value converted from an object.

js

```
// An object without Symbol.toPrimitive property.
const obj1 = {};
console.log(+obj1); // NaN
console.log(`${obj1}`); // "[object Object]"
console.log(obj1 + ""); // "[object Object]"

// An object with Symbol.toPrimitive property.
const obj2 = {
  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return 10;
    }
    if (hint === "string") {
      return "hello";
    }
    return true;
  },
};
console.log(+obj2); // 10        – hint is "number"
console.log(`${obj2}`); // "hello"   – hint is "string"
console.log(obj2 + ""); // "true"    – hint is "default"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.toprimitive

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPrimitive`	47	15	44	34	10	47	44	34	10	5.0	47	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.toPrimitive`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.toPrimitive` in `core-js`
- `Date.prototype[Symbol.toPrimitive]()`
- `Symbol.prototype[Symbol.toPrimitive]()`
- `Object.prototype.toString()`
- `Object.prototype.valueOf()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.toStringTag

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/toStringTag · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since April 2017.

The **`Symbol.toStringTag`** static data property represents the well-known symbol `Symbol.toStringTag`. `Object.prototype.toString()` looks up this symbol on the `this` value for the property containing a string that represents the type of the object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.toStringTag". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class ValidatorClass {
  get [Symbol.toStringTag]() {
    return "Validator";
  }
}

console.log(Object.prototype.toString.call(new ValidatorClass()));
// Expected output: "[object Validator]"
```

Value

The well-known symbol `Symbol.toStringTag`.

Property attributes of `Symbol.toStringTag`	
Writable	no
Enumerable	no
Configurable	no

Examples

Default tags

Some values do not have `Symbol.toStringTag`, but have special `toString()` representations. For a complete list, see `Object.prototype.toString()`.

```js
Object.prototype.toString.call("foo"); // "[object String]"
Object.prototype.toString.call([1, 2]); // "[object Array]"
Object.prototype.toString.call(3); // "[object Number]"
Object.prototype.toString.call(true); // "[object Boolean]"
Object.prototype.toString.call(undefined); // "[object Undefined]"
Object.prototype.toString.call(null); // "[object Null]"
// … and more
```

Built-in toStringTag symbols

Most built-in objects provide their own `[Symbol.toStringTag]` property. Almost all built-in objects' `[Symbol.toStringTag]` property is not writable, not enumerable, and configurable; the exception is `Iterator`, which is writable for compatibility reasons.

For constructor objects like `Promise`, the property is installed on `Constructor.prototype`, so that all instances of the constructor inherit `[Symbol.toStringTag]` and can be stringified. For non-constructor objects like `Math` and `JSON`, the property is installed as a static property, so that the namespace object itself can be stringified. Sometimes, the constructor also provides its own `toString` method (for example, `Intl.Locale`), in which case the `[Symbol.toStringTag]` property is only used when you explicitly call `Object.prototype.toString` on it.

```js
Object.prototype.toString.call(new Map()); // "[object Map]"
Object.prototype.toString.call(function* () {}); // "[object GeneratorFunction]"
Object.prototype.toString.call(Promise.resolve()); // "[object Promise]"
// … and more
```

Custom tag with toStringTag

When creating your own class, JavaScript defaults to the "Object" tag:

```js
class ValidatorClass {}

Object.prototype.toString.call(new ValidatorClass()); // "[object Object]"
```

Now, with the help of `toStringTag`, you are able to set your own custom tag:

```js
class ValidatorClass {
  get [Symbol.toStringTag]() {
    return "Validator";
  }
}

Object.prototype.toString.call(new ValidatorClass()); // "[object Validator]"
```

toStringTag available on all DOM prototype objects

Due to a WebIDL spec change in mid-2020, browsers are adding a `Symbol.toStringTag` property to all DOM prototype objects. For example, to access the `Symbol.toStringTag` property on `HTMLButtonElement`:

```js
const test = document.createElement("button");
test.toString(); // "[object HTMLButtonElement]"
test[Symbol.toStringTag]; // "HTMLButtonElement"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.tostringtag

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toStringTag`	49	15	51	36	10	49	51	36	10	5.0	49	10	1.0.0	1.0	6.0.0
`toStringTag` available on all DOM prototype objects	50	79	78	37	14	50	79	37	14	5.0	50	14	?	1.0	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.toStringTag`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.toStringTag` in `core-js`
- `Object.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.unscopables

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/unscopables · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since August 2016.

The **`Symbol.unscopables`** static data property represents the well-known symbol `Symbol.unscopables`. The `with` statement looks up this symbol on the scope object for a property containing a collection of properties that should not become bindings within the `with` environment.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.unscopables". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  foo: 42,
};

object[Symbol.unscopables] = {
  foo: true,
};

with (object) {
  console.log(foo);
  // Expected output: Error: foo is not defined
}
```

Value

The well-known symbol `Symbol.unscopables`.

Property attributes of `Symbol.unscopables`	
Writable	no
Enumerable	no
Configurable	no

Description

The `[Symbol.unscopables]` symbol (accessed via `Symbol.unscopables`) can be defined on any object to exclude property names from being exposed as lexical variables in `with` environment bindings. Note that when using strict mode, `with` statements are not available, and this symbol is likely not needed.

Setting a property of the `[Symbol.unscopables]` object to `true` (or any truthy value) will make the corresponding property of the `with` scope object *unscopable* and therefore won't be introduced to the `with` body scope. Setting a property to `false` (or any falsy value) will make it *scopable* and thus appear as lexical scope variables.

When deciding whether `x` is unscopable, the entire prototype chain of the `[Symbol.unscopables]` property is looked up for a property called `x`. This means if you declared `[Symbol.unscopables]` as a plain object, `Object.prototype` properties like `toString` would become unscopable as well, which may cause backward incompatibility for legacy code assuming those properties are normally scoped (see an example below). You are advised to make your custom `[Symbol.unscopables]` property have `null` as its prototype, like `Array.prototype[Symbol.unscopables]` does.

This protocol is also utilized by DOM APIs, such as `Element.prototype.append()`.

Examples

Scoping in with statements

The following code works fine in ES5 and below. However, in ECMAScript 2015, the `Array.prototype.values()` method was introduced. That means that inside a `with` environment,

"values" would now be the `Array.prototype.values()` method and not the variable outside the `with` statement.

```js
var values = [];

with (values) {
  // If [Symbol.unscopables] did not exist, values would become
  // Array.prototype.values starting with ECMAScript 2015.
  // And an error would have occurred.
  values.push("something");
}
```

The code containing `with (values)` caused some websites to malfunction in Firefox when `Array.prototype.values()` was added (Firefox Bug 883914). Furthermore, this implies that any future array method addition may be breaking if it implicitly changes the `with` scope. Therefore, the `[Symbol.unscopables]` symbol was introduced and implemented on `Array` as `Array.prototype[Symbol.unscopables]` to prevent some of the Array methods being scoped into the `with` statement.

Unscopables in objects

You can also set `[Symbol.unscopables]` for your own objects.

```js
const obj = {
  foo: 1,
  bar: 2,
  baz: 3,
};

obj[Symbol.unscopables] = {
  // Make the object have `null` prototype to prevent
  // `Object.prototype` methods from being unscopable
  __proto__: null,
  // `foo` will be scopable
  foo: false,
  // `bar` will be unscopable
  bar: true,
  // `baz` is omitted; because `undefined` is falsy, it is also scopable (default)
};

with (obj) {
  console.log(foo); // 1
  console.log(bar); // ReferenceError: bar is not defined
  console.log(baz); // 3
}
```

Avoid using a non-null-prototype object as `[Symbol.unscopables]`

Declaring `[Symbol.unscopables]` as a plain object without eliminating its prototype may cause subtle bugs. Consider the following code working before `[Symbol.unscopables]`:

```js
const character = {
  name: "Yoda",
  toString: function () {
    return "Use with statements, you must not";
  },
};

with (character) {
  console.log(name + ' says: "' + toString() + '"'); // Yoda says: "Use with statements, you
must not"
}
```

To preserve backward compatibility, you decided to add a `[Symbol.unscopables]` property when adding more properties to `character`. You may naïvely do it like:

js · ✗ bad example

```js
const character = {
  name: "Yoda",
  toString: function () {
    return "Use with statements, you must not";
  },
  student: "Luke",
  [Symbol.unscopables]: {
    // Make `student` unscopable
    student: true,
  },
};
```

However, the code above now breaks:

```js
with (character) {
  console.log(name + ' says: "' + toString() + '"'); // Yoda says: "[object Undefined]"
}
```

This is because when looking up `character[Symbol.unscopables].toString`, it returns `Object.prototype.toString()`, which is a truthy value, thus making the `toString()` call in the `with()` statement reference `globalThis.toString()` instead — and because it's called without a `this`, `this` is `undefined`, making it return `[object Undefined]`.

Even when the method is not overridden by `character`, making it unscopable will change the value of `this`.

```js
const proto = {};
const obj = { __proto__: proto };

with (proto) {
  console.log(isPrototypeOf(obj)); // true; `isPrototypeOf` is scoped and `this` is `proto`
}

proto[Symbol.unscopables] = {};

with (proto) {
  console.log(isPrototypeOf(obj)); // TypeError: Cannot convert undefined or null to object
  // `isPrototypeOf` is unscoped and `this` is undefined
}
```

To fix this, always make sure `[Symbol.unscopables]` only contains properties you wish to be unscopable, without `Object.prototype` properties.

js · ✓ good example

```
const character = {
  name: "Yoda",
  toString: function () {
    return "Use with statements, you must not";
  },
  student: "Luke",
  [Symbol.unscopables]: {
    // Make the object have `null` prototype to prevent
    // `Object.prototype` methods from being unscopable
    __proto__: null,
    // Make `student` unscopable
    student: true,
  },
};
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.unscopables`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`unscopables`	38	12	48	25	9	38	48	25	9	3.0	38	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.unscopables`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Array.prototype[Symbol.unscopables]`
- `with`
- `Element.prototype.append()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/toString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2015.

The **`toString()`** method of `Symbol` values returns a string representing this symbol value.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Symbol.prototype.toString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(Symbol("desc").toString());
// Expected output: "Symbol(desc)"

console.log(Symbol.iterator.toString());
// Expected output: "Symbol(Symbol.iterator)

console.log(Symbol.for("foo").toString());
// Expected output: "Symbol(foo)"

// console.log(Symbol('foo') + 'bar');
// Expected output: Error: Can't convert symbol to string
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

A string representing the specified symbol value.

Description

The `Symbol` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `Symbol` values, the `toString` method returns a descriptive string in the form `"Symbol(description)"`, where `description` is the symbol's description.

The `toString()` method requires its `this` value to be a `Symbol` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to symbol values.

Because `Symbol` has a `[Symbol.toPrimitive]()` method, that method always takes priority over `toString()` when a `Symbol` object is coerced to a string. However, because `Symbol.prototype[Symbol.toPrimitive]()` returns a symbol primitive, and symbol primitives throw a `TypeError` when implicitly converted to a string, the `toString()` method is never implicitly called by the language. To stringify a symbol, you must explicitly call its `toString()` method or use the `String()` function.

Examples

Using toString()

```js
Symbol("desc").toString(); // "Symbol(desc)"

// well-known symbols
Symbol.iterator.toString(); // "Symbol(Symbol.iterator)"

// global symbols
Symbol.for("foo").toString(); // "Symbol(foo)"
```

Implicitly calling toString()

The only way to make JavaScript implicitly call `toString()` instead of `[Symbol.toPrimitive]()` on a symbol wrapper object is by deleting the `[Symbol.toPrimitive]()` method first.

> **Warning:** You should not do this in practice. Never mutate built-in objects unless you know exactly what you're doing.

```js
delete Symbol.prototype[Symbol.toPrimitive];
console.log(`${Object(Symbol("foo"))}`); // "Symbol(foo)"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.toString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Object.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/valueOf · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since September 2015.

The **`valueOf()`** method of `Symbol` values returns this symbol value.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Symbol.prototype.valueOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const symbol = Symbol("foo");

console.log(typeof Object(symbol));
// Expected output: "object"

console.log(typeof Object(symbol).valueOf());
// Expected output: "symbol"
```

Syntax

js

```
valueOf()
```

Parameters

None.

Return value

The primitive value of the specified `Symbol` object.

Description

The `valueOf()` method of `Symbol` returns the primitive value of a Symbol object as a Symbol data type.

JavaScript calls the `valueOf()` method to convert an object to a primitive value. You rarely need to invoke the `valueOf()` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

Examples

Using valueOf()

js

```
const sym = Symbol("example");
sym === sym.valueOf(); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.prototype.valueof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
valueOf	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Object.prototype.valueOf()`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.prototype[Symbol.toPrimitive]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/Symbol.toPrimitive · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2017.

The **`[Symbol.toPrimitive]()`** method of `Symbol` values returns this symbol value.

Syntax

```js
symbolValue[Symbol.toPrimitive](hint)
```

Parameters

`hint`

A string value indicating the primitive value to return. The value is ignored.

Return value

The primitive value of the specified `Symbol` object.

Description

The `[Symbol.toPrimitive]()` method of `Symbol` returns the primitive value of a Symbol object as a Symbol data type. The `hint` argument is not used.

JavaScript calls the `[Symbol.toPrimitive]()` method to convert an object to a primitive value. You rarely need to invoke the `[Symbol.toPrimitive]()` method yourself; JavaScript automatically invokes it

when encountering an object where a primitive value is expected.

Examples

Using `[Symbol.toPrimitive]()`

```js
const sym = Symbol("example");
sym === sym[Symbol.toPrimitive](); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.prototype-%symbol.toprimitive%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.toPrimitive]	47	15	44	34	10	47	44	34	10	5.0	47	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.@@toPrimitive`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Symbol.toPrimitive`

JavaScript › Reference › Standard built-in objects › Symbol

Symbol.prototype.description

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol/description · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`description`** accessor property of `Symbol` values returns a string containing the description of this symbol, or `undefined` if the symbol has no description.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Symbol.prototype.description". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Symbol("desc").description);
// Expected output: "desc"

console.log(Symbol.iterator.description);
// Expected output: "Symbol.iterator"

console.log(Symbol.for("foo").description);
// Expected output: "foo"

console.log(`${Symbol("foo").description}bar`);
// Expected output: "foobar"
```

Description

`Symbol` objects can be created with an optional description which can be used for debugging but not to access the symbol itself. The `Symbol.prototype.description` property can be used to read that description. It is different to `Symbol.prototype.toString()` as it does not contain the enclosing `"Symbol()"` string. See the examples.

Examples

Using description

js

```
Symbol("desc").toString(); // "Symbol(desc)"
Symbol("desc").description; // "desc"
Symbol("").description; // ""
Symbol().description; // undefined

// well-known symbols
Symbol.iterator.toString(); // "Symbol(Symbol.iterator)"
Symbol.iterator.description; // "Symbol.iterator"

// global symbols
Symbol.for("foo").toString(); // "Symbol(foo)"
Symbol.for("foo").description; // "foo"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-symbol.prototype.description`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`description`	70	79	63	57	12.1[1]	70	63	49	12.2[3]	10.0	70	12.2[3]	1.0.0	1.0	11.0.0

1. Also from version 12–12.1: partial support.
2. Also from version 12–12.1: No support for an undefined description.
3. Also from version 12–12.2: partial support.
4. Also from version 12–12.2: No support for an undefined description.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Symbol.description`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Symbol.prototype.description` in `core-js`
- es-shims polyfill of `Symbol.prototype.description`
- `Symbol.prototype.toString()`

JavaScript › Reference › Standard built-in objects

SyntaxError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SyntaxError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`SyntaxError`** object represents an error when trying to interpret syntactically invalid code. It is thrown when the JavaScript engine encounters tokens or token order that does not conform to the syntax of the language when parsing code.

`SyntaxError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`SyntaxError` is a subclass of `Error`.

Constructor

`SyntaxError()`
: Creates a new `SyntaxError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `SyntaxError.prototype` and shared by all `SyntaxError` instances.

`SyntaxError.prototype.constructor`
: The constructor function that created the instance object. For `SyntaxError` instances, the initial value is the `SyntaxError` constructor.

`SyntaxError.prototype.name`
: Represents the name for the type of error. For `SyntaxError.prototype.name`, the initial value is `"SyntaxError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a SyntaxError

```js
try {
  eval("hoo bar");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message);
  console.log(e.name); // "SyntaxError"
  console.log(e.stack); // Stack of the error
}
```

Creating a SyntaxError

```js
try {
  throw new SyntaxError("Hello");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "SyntaxError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-native-error-types-used-in-this-standard-syntaxerror

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
SyntaxError	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
SyntaxError() constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
SyntaxError is serializable	77	79	103[1,2,3]	64	**No**	77	103[1,2,3]	55	**No**	12.0	77	**No**	?	**No**	**No**

1. Version 103 serializable properties: `name`, `message`, `cause`, `fileName`, `lineNumber` and `columnNumber`.
2. Version 104 also serializes `stack` in the main thread (`window.postMessage()` and `structuredClone()`).
3. Version 110 also serializes `stack` in workers (`worker.postMessage()` and `structuredClone()`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SyntaxError`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Error`

JavaScript › Reference › Standard built-in objects › SyntaxError

SyntaxError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SyntaxError/SyntaxError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`SyntaxError()`** constructor creates `SyntaxError` objects.

Syntax

```js
new SyntaxError()
new SyntaxError(message)
new SyntaxError(message, options)
new SyntaxError(message, fileName)
new SyntaxError(message, fileName, lineNumber)

SyntaxError()
SyntaxError(message)
SyntaxError(message, options)
SyntaxError(message, fileName)
SyntaxError(message, fileName, lineNumber)
```

> **Note:** `SyntaxError()` can be called with or without `new`. Both create a new `SyntaxError` instance.

Parameters

`message` Optional

Human-readable description of the error

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The name of the file containing the code that caused the exception

`lineNumber` Optional Non-standard

The line number of the code that caused the exception

Examples

Catching a SyntaxError

```js
try {
  eval("hoo bar");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message);
  console.log(e.name); // "SyntaxError"
  console.log(e.stack); // Stack of the error
}
```

Creating a SyntaxError

```js
try {
  throw new SyntaxError("Hello");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "SyntaxError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-nativeerror-constructors

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`SyntaxError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.SyntaxError.SyntaxError`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`